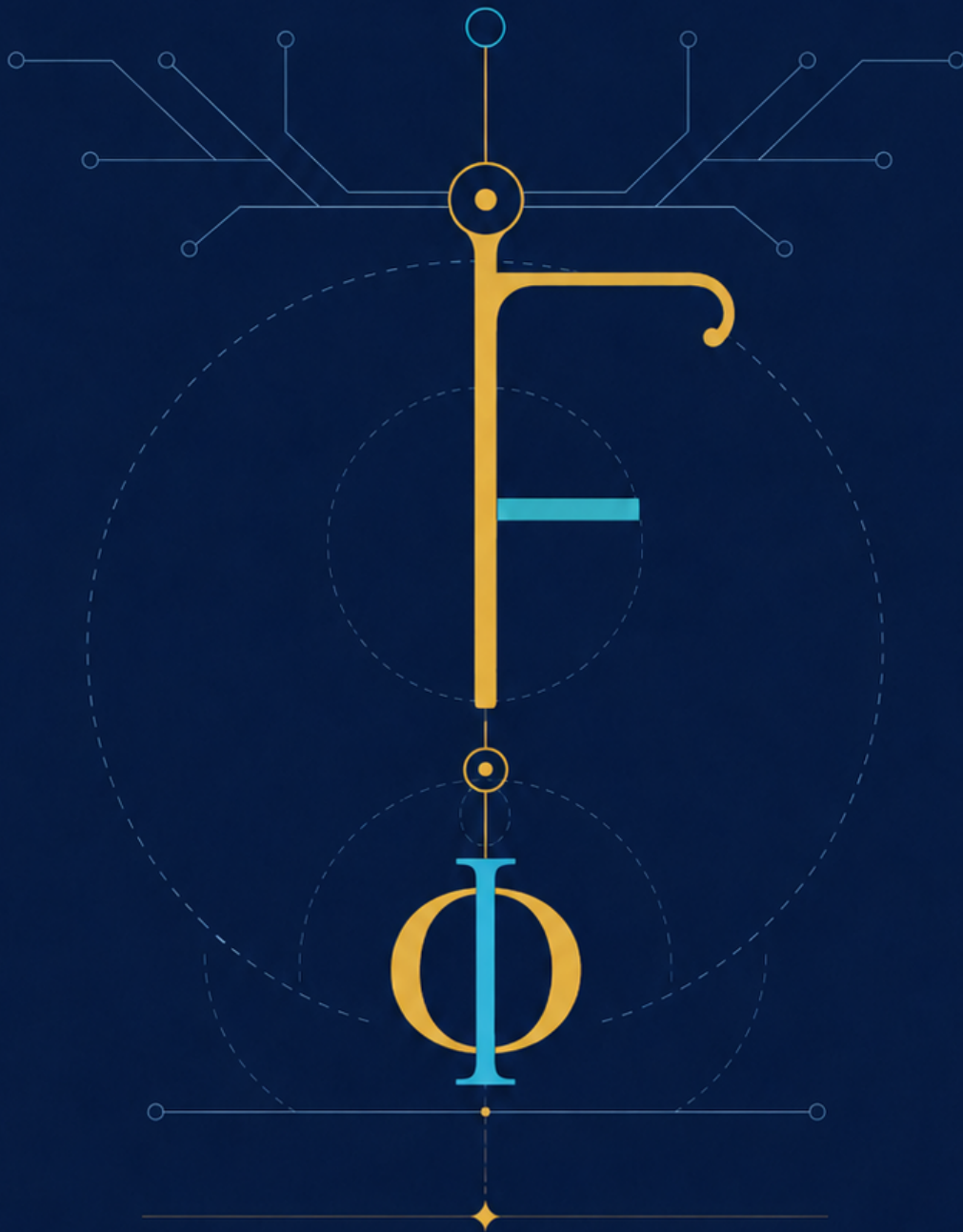


FO λ ANG

LANGUAGE REFERENCE MANUAL



Syntax, Semantics, Types, and Core Language Rules

Kameswara Rao Mithipati



Folang is a general-purpose programming language designed to be **expressive, consistent, and extensible**, merging functional fluency with object-centric abstractions.

Normative Status

This document is the authoritative and normative definition of the FoLang programming language.

All FoLang syntax, semantics, type-system rules, name-resolution rules, execution behavior, and other language requirements are defined by this document.

Every addition, correction, clarification, modification, or extension to the FoLang language must be recorded in this document before it becomes part of the language.

A compiler or other language implementation may use any internal architecture, parser, intermediate representation, runtime, or backend. However, to be identified as a conforming FoLang implementation, its externally observable behavior must adhere to the requirements defined in this document.

When an implementation conflicts with this specification, the specification governs unless the discrepancy is formally accepted and incorporated as a specification revision.

Implementation-specific extensions must be clearly identified as extensions and must not be represented as standard FoLang features unless they have been incorporated into this specification.

Audience and Scope

This document is the normative FoLang language and implementation reference. Its primary audience is frontend/compiler implementers, backend/code-generator implementers, tooling implementers, runtime/library implementers where language contracts apply, and conformance-test authors.

The document defines accepted source forms, semantic relationships, validation requirements, diagnostics, frontend obligations, backend-observable contracts, conformance requirements, and areas intentionally left implementation-defined. It is not a getting-started guide or a tutorial for learning FoLang. Examples are retained when they establish, distinguish, or test normative behavior; pedagogical progression and programmer-oriented introductory material are outside the purpose of this reference.

A backend or tool may use any internal representation or optimization permitted by this specification, but implementation techniques do not redefine FoLang syntax, type relationships, member relationships, or observable semantics.

Grammar, Semantics, and Examples

The lexical and syntactic grammar defined by this specification is the authoritative definition of the FoLang source forms accepted by the current language profile. Normative semantic rules define the meaning of those forms and any additional validity constraints that are checked after parsing.

Examples are **illustrative only**. The presence or absence of an example does not enable, disable, reserve, or otherwise modify a grammar production, token, declaration form, operator, or semantic rule. Likewise, an inventory or explanatory table does not create new source syntax unless the specification explicitly defines the corresponding grammar or identifies the spelling as an explicitly reserved future form.

For parser and grammar generation, the governing classification is therefore deterministic:

```
active lexical/syntactic grammar
```

```
-> accepted and parsed according to the applicable grammar rules
```

```
explicitly identified reserved/future form
```

```
-> recognized as language-owned where its reserved spelling is defined
```

```
-> rejected with an unsupported-feature diagnostic in the current  
profile
```

```
all other source text
```

```
-> ordinary lexical or syntax error when it does not match the active  
grammar
```

Language Evolution and Compatibility

The alpha period permits experimentation, consolidation, implementation, renaming, syntax changes, and removal. Before version 1.0, a proposed structural feature becomes part of the active language only when a specification revision explicitly incorporates it into the current lexical/syntactic grammar and defines its applicable normative semantics. A proposed or future feature that is mentioned only descriptively does not become active grammar.

Nothing is carried into version 1.0 as implicitly reserved or unsupported merely because it appeared in an example, table, inventory, or design discussion. A spelling is reserved while unsupported only when this specification explicitly identifies that spelling or form as reserved/future syntax.

At version 1.0, the **structural FoLang language surface** closes permanently. After version 1.0, no later major or minor release introduces new core grammar forms, hard/contextual keywords, declaration kinds, operator spellings or fixities, or built-in `@co.*` metadata-form names. Existing structural constructs retain the externally observable semantics defined by the 1.0 specification, except for corrections that restore already-stated intent.

The standard-package rule is narrower and separate. After version 1.0, the `co.*` **package/subpackage hierarchy** is frozen, while ordinary declarations inside those existing packages may evolve through later language-provided `.fo1enc` artifacts that expose the standard package contexts. Adding or updating an ordinary type, unit-level function, method, module, algorithm, data structure, or other declaration inside an existing package is package-API evolution and does not create new grammar or a new package path.

FoLang remains extensible after 1.0 through extension mechanisms already defined by the 1.0 language, including third-party libraries, user-defined annotations and decorators, macros, custom operators, native integration, including FFI, dynamic-runtime facilities, backend integration points, and other explicitly defined extension mechanisms. Frontend, backend, runtime, optimizer, intermediate representation, diagnostics, compilation strategy, memory management, code generation, performance, and hardware support may evolve provided externally observable FoLang semantics remain conforming.

A post-1.0 correction may fix an implementation/specification discrepancy or remove an internal contradiction when doing so restores the already-stated intent of an existing feature. A correction does not introduce a new grammar form, keyword, declaration kind, operator grammar, metadata-form name, or previously unavailable structural capability.

Lexical Profile and Statement Termination

The consolidated FoLang EBNF referenced by [Appendix A](#) is the formal lexical and syntactic grammar. The rules below restate source-level constraints needed by parser/frontend implementations and conformance tests without duplicating the complete EBNF in this document.

Source Encoding and Identifiers

FoLang source text is UTF-8. A U+FEFF byte-order mark is permitted only as the first code point of a source file; U+FEFF anywhere else is an error.

Ordinary FoLang identifiers are ASCII-only. An identifier begins with an ASCII letter, may continue with ASCII letters or decimal digits, and may contain `_` only between two non-empty alphanumeric segments. An identifier cannot begin or end with `_`, cannot contain consecutive underscores, and cannot be the single spelling `_`. The single `_` is a contextual language token whose meanings are defined by the applicable wildcard/discard, filename-derived declaration, parameterized-type-placeholder, and refinement-predicate rules.

After its character sequence is recognized, an ordinary identifier is checked against the reserved-word table. Hard-reserved words are emitted as reserved tokens rather than identifiers. The contextual keyword `forall` is reclassified only in its defined parser context.

Examples:

name	valid
myVar2	valid
v1_hr	valid
a_b_c	valid
123hr	invalid: identifier cannot begin with a digit

<code>_x</code>	invalid: identifier cannot begin with underscore
<code>_1</code>	invalid: identifier cannot begin with underscore
<code>a_</code>	invalid: trailing underscore
<code>a__b</code>	invalid: consecutive underscores
<code>_</code>	contextual token, not an ordinary identifier

Control labels use a separate apostrophe-prefixed lexical form such as `'outer'`. They are not ordinary identifiers. `'outer:'` declares a structured control label and `'outer'` references it; see [Labels and Named Blocks](#). A character literal remains distinct because it has a closing apostrophe, for example `'c'`.

Numeric Literals

FoLang supports the integer and floating literal families defined by the consolidated EBNF. Numeric digit separators are not part of the current profile, so forms such as `1'000`, `0x1'a`, and `0b1011'0010` are invalid.

A numeric sign is not part of an ordinary numeric literal token. In expressions, unary `+` or `-` is parsed as a prefix operator. Pattern syntax separately permits a leading `+` or `-` before an integer or floating literal.

Integer literals support binary, octal, decimal, and hexadecimal forms together with the suffixes admitted by the grammar. Floating literals support decimal and hexadecimal forms. A radix-point form requires at least one digit on each side of the point: `1.0` and `0.10` are valid, while `1.` and `.10` are invalid. Scientific notation without a radix point, such as `1e5`, remains valid. A backend-conditional floating suffix is accepted only when the selected backend/compiler contract supports the corresponding representation.

Comments, Whitespace, and Line Breaks

FoLang supports `//` line comments and non-nesting `/* ... */` block comments. A line comment ends before the next CR or LF terminator. A block comment ends at the first following `*/`; an inner `/*` does not nest.

Comments are recognized before ordinary symbolic-run scanning. Spaces, horizontal tabs, form-feed characters, line breaks, and comments are token separators and are discarded between tokens. A line break never terminates a FoLang statement; FoLang has no automatic semicolon insertion.

Statement and Expression Termination

FoLang uses explicit termination:

simple statement or expression statement	-> terminated by ;
direct block/body	-> terminated by its closing }
braced expression/literal	-> } closes the expression,
then ; closes its statement	

A semicolon is required after simple declarations, assignments, compound assignments, calls used as statements, `this =>` callable-result statements, `this ^=>` enclosing-callable-result statements, expression-bodied `let` function-pattern clauses, object/collection construction expressions used in a statement, type-alias declarations containing generic instantiations, literal expression statements, forward declarations, and other simple declaration forms.

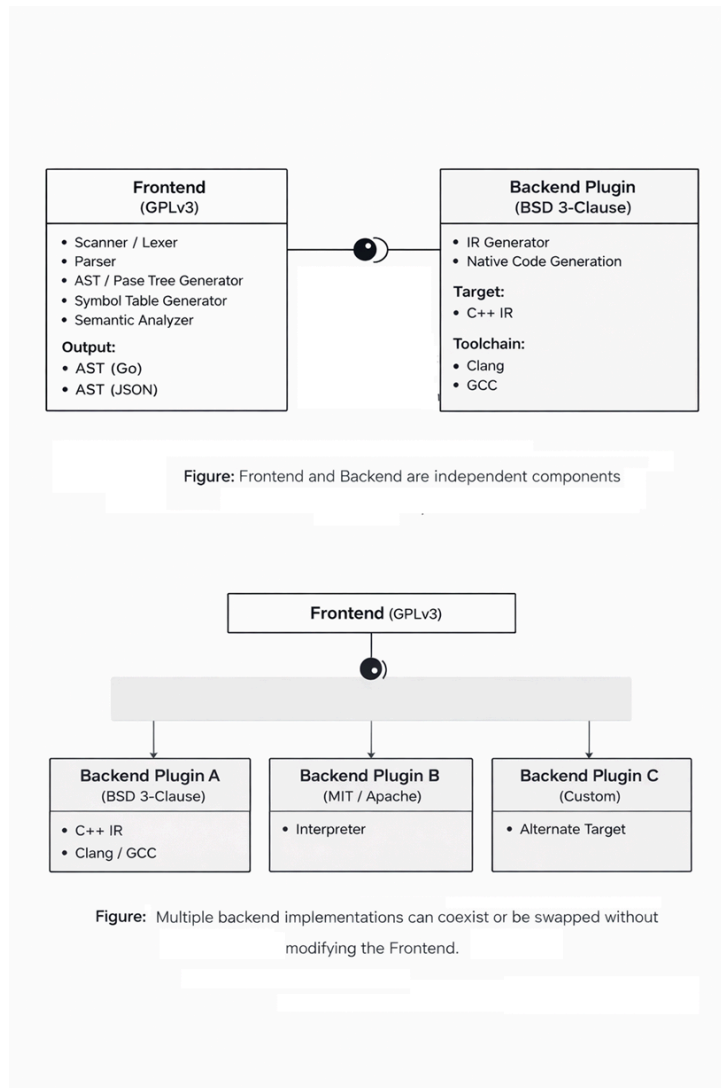
A direct declaration body, function/method body, or block-bodied `let` function-pattern clause terminates at its closing `}` and must not be followed by `;`.

A braced **expression** is different from a direct block/body. Object construction and typed map/collection values still require the enclosing statement's semicolon:

```
emp := Employee{id: 1, name: "Rao"};
this => Employee{id: 1};
StringIntMap co.type =
    co.Map(co.string, co.int);
cfg := StringIntMap{"a": 1, "b": 2};
```

Built-in directives, annotations, pragmas, and decorators are independently delimited metadata applications and do not acquire a trailing semicolon merely because they appear on their own source line.

Design Overview



FoLang follows a deliberately different approach from conventional programming language designs. The system is structured to ensure **clear separation of concerns**, **license isolation**, and **extensibility** through well-defined integration boundaries.

Compiler and Backend

1. Frontend

The Frontend is responsible for source-level analysis and semantic processing of FoLang programs.

Frontend Components

- Scanner / Lexer
- Parser

- AST / Parse Tree Generator
- Symbol Table Generator
- Installed Standard-Package Loader
- `.fo1enc` Artifact and Symbol Loader
- Semantic Analyzer

Implementation

- Implemented in **Go**
- Uses Go structures internally for AST, symbol-table, and semantic processing.
- The externally consumable frontend output is serialized according to the selected backend's interchange contract.
- The frontend writes that artifact beneath the reserved project-root `build/` domain.
- The backend-supplied contract identifies the supported FoLang/plugin protocol version, HIR schema version, and wire format.
- The backend installs/provides this interchange-contract file in the same installation directory as the FoLang compiler executable.
- The frontend reads that installed backend contract to determine the interchange representation required by the selected backend.
- Canonical `build/` presence, ownership, and source-discovery rules are defined in [Project Layout](#).

Named Diagnostic Registry

Named diagnostics use stable, case-sensitive identifiers shared by the compiler, IDE tooling, conformance tests, and serialized diagnostic output. Implementations may add explanatory text, source locations, and internal numeric identifiers, but they must preserve the registered name.

The registry groups related normative rejection rules under reusable names. The compiler emits the most specific applicable diagnostic and places the particular declaration kind, symbol, metadata form, type, relationship, or effect in its structured details. It must not create a different public name merely to restate the same rule for another declaration kind. Secondary errors caused solely by one primary failure should be suppressed or reported as related notes rather than

independent cascades. All entries currently have severity `Error` and prevent successful compilation. They are frontend diagnostics, not runtime `co.error` values or effects.

Diagnostic name	Severity	Phase	Normative use
<code>Unclassified</code>	Error	Diagnostic migration	A legacy diagnostic has not yet been assigned a more specific stable name
<code>UnsupportedFeature</code>	Error	Profile validation	Source uses a recognized feature or reserved future form that the active language profile does not support
<code>UnsupportedBackendFeature</code>	Error	Backend compatibility	Source requires a representation or facility not supported by the selected backend contract
<code>InvalidIdentifier</code>	Error	Lexing	An identifier violates FoLang spelling, underscore, reserved-word, or encoding rules
<code>InvalidLiteral</code>	Error	Lexing/parsing	A scalar literal token violates its normative lexical form
<code>UnexpectedToken</code>	Error	Parsing	A token cannot occur at the current grammar position
<code>ExpectedToken</code>	Error	Parsing	A required delimiter, binder, separator, operand, or other token is absent
<code>InvalidSyntax</code>	Error	Parsing	Source violates a grammar rule for which no more specific registered

			parser diagnostic applies
<code>UnknownOperator</code>	Error	Operator-aware scanning	A complete symbolic run is neither built in nor registered in the active operator table
<code>InvalidProjectLayout</code>	Error	Project validation	A required root, component tree, fixed surface, filename role, or project-kind layout rule is violated
<code>InvalidSourcePlacement</code>	Error	Source-structure validation	A file, declaration, unit, block form, directive, pragma, or component surface appears in a forbidden source context
<code>InvalidDeclarationForm</code>	Error	Declaration validation	A declaration has an unsupported head, nesting, body, filename-derived name, classifier combination, or structural shape
<code>ReservedPackageShadowing</code>	Error	Project validation	Project or dependency content attempts to shadow the installed <code>co.*</code> package identity
<code>InvalidImport</code>	Error	Import resolution	An import has an invalid form, target, alias, visibility, package/component/library role, or source context
<code>InvalidDependency</code>	Error	Dependency analysis	A dependency crosses a forbidden project, component,

			capability, or packaged-library direction
DependencyCycle	Error	Dependency analysis	Effective package, component-surface, or standalone-library dependencies form a cycle
CapabilityViolation	Error	Capability analysis	Source accesses or enables a facility outside its permitted application, native, dynamic-runtime, or other capability domain
ExportBoundaryViolation	Error	Boundary analysis	A public/projected/packaged surface exposes a forbidden, inaccessible, private, internal, native, or representation-reachable type
DuplicateDeclaration	Error	Symbol analysis	Two variables, types, members, aliases, fields, variants, metadata keys, or other non-callable declarations normalize to one symbol in the same namespace
DuplicateCallableSignature	Error	Symbol/overload analysis	Two callables normalize to the same owner, receiver category, name, and parameter signature, including declarations differing only by result type

UnresolvedSymbol	Error	Name resolution	A required package, type, value, callable, label, declaration reference, or other symbol cannot be resolved
AmbiguousSymbol	Error	Name resolution	More than one symbol remains after the applicable qualification and resolution rules
InaccessibleSymbol 1	Error	Access analysis	A resolved declaration exists but is not accessible from the requesting context
InvalidReceiver	Error	Receiver analysis	A value/type/companion/extension receiver is absent, has the wrong category, or does not match its required owner
UnusedImport	Error	Usage analysis	An import contributes no live resolved use under the strict import-liveness rules
UnusedSymbol	Error	Usage analysis	A declaration subject to strict liveness is not reachable or used as required by its declaration category
TypeMismatch	Error	Type checking	Actual and required canonical types, result positions, receiver types, or callable shapes are incompatible

<code>ConstraintViolation</code>	Error	Constraint checking	A value or type violates a refinement predicate, subtype/supertype set, bound, variance, capability, or other declared constraint
<code>InvalidAssignment</code>	Error	Type/assignment analysis	An assignment target, source, binding operator, mutability rule, or value-transfer relationship is invalid
<code>Uninitialized</code>	Error	Flow analysis	A refinement/dependent value is read where a valid assignment is not proven on every reachable path
<code>InvalidNoneUse</code>	Error	Type/flow analysis	<code>co.const.none</code> , <code>isNone()</code> , or missing-result none substitution is used with a refinement/dependent position that does not admit none
<code>InvalidDependentIndex</code>	Error	Dependent-type analysis	A dependent index has an invalid form, resolution, evaluability, sign, or permitted source
<code>DependentTypeMismatch</code>	Error	Dependent-type analysis	Dependent constructors or corresponding normalized indices do not match where equality is required
<code>InvalidConstruction</code>	Error	Construction analysis	A struct/class/object construction or initialization path

			cannot produce a valid readable value
<code>MutationNotAllowed</code>	Error	Mutability analysis	Source attempts a statically detectable mutation through an immutable, constant, literal, or otherwise non-mutable root
<code>OverloadNotAllowed</code>	Error	Callable validation	A declaration category or callable identity is not permitted to introduce the requested overload
<code>NoApplicableOverload</code>	Error	Overload resolution	No visible overload accepts the resolved receiver and argument types
<code>AmbiguousOverload</code>	Error	Overload resolution	Multiple incomparable most-specific overload candidates remain applicable
<code>InvalidReturn</code>	Error	Callable validation	A return count, type, or required result production is invalid
<code>MissingImplementation</code>	Error	Implementation analysis	A required callable body, interface method, abstract/virtual slot, runtime binding, or other implementation is absent
<code>InvalidForwardDeclaration</code>	Error	Declaration validation	A bodyless declaration has invalid classification, mapping, signature, or forward-declaration metadata

InvalidGenericDeclaration	Error	Generic analysis	Generic markers, rank, placement, declaration-head form, classifier combination, or parameter structure is invalid
InvalidGenericMapping	Error	Generic analysis	Generic mapping rows are absent where required, conflicting, cyclic, structurally incompatible, or leave a required marker unresolved
GenericResolutionFailure	Error	Generic analysis	Generic arguments or result markers cannot be resolved to one consistent permitted assignment
SignatureConformanceFailure	Error	Contract conformance	A module, instance, handler, interface implementation, or other declaration fails its required structural signature
InvalidAssociatedTypeBinding	Error	Contract conformance	An associated-type binding is missing, unknown, fixed incompatibly, or violates generic arity, bounds, variance, or referenced signature requirements
InvalidRelationship	Error	OOP validation	An interface/class/mixin/trait relationship has an invalid kind, target, count, selector,

			accessibility, or canonical identity
<code>InheritedMemberConflict</code>	Error	OOP validation	Inherited state or behavior conflicts and the normative merge/selection rules cannot resolve it automatically
<code>InvalidOverride</code>	Error	OOP validation	An override target, source category, accessibility, sealing rule, or exact signature does not satisfy override rules
<code>InvalidImplementation</code>	Error	OOP/contract validation	An explicit implementation mapping selects the wrong source, target, method, accessibility, or signature
<code>InvalidLifecycleDeclaration</code>	Error	Lifecycle validation	<code>@@new</code> , <code>@@init</code> , <code>::</code> invocation, lifecycle metadata, placement, accessibility, or overload/override use violates lifecycle rules
<code>InvalidMemberPolicy</code>	Error	Member-policy validation	A field or member carries a forbidden constant, immutable, shared, locking, <code>CopyOnWrite</code> , static/class, or related policy
<code>UnresolvedMetadataForm</code>	Error	Metadata resolution	A user-defined annotation/decorator name cannot be resolved to a valid metadata declaration

<code>InvalidMetadataPlacement</code>	Error	Metadata validation	An annotation, decorator, directive, or pragma appears in a source location where that metadata category is forbidden
<code>InvalidMetadataTarget</code>	Error	Metadata validation	A resolved metadata form does not permit the declaration or expression kind to which it is attached
<code>MissingMetadataField</code>	Error	Metadata validation	A required field of a recognized metadata form is absent
<code>InvalidMetadataField</code>	Error	Metadata validation	A metadata field is unknown, duplicated, uses an invalid binder, or is forbidden for the selected form/context
<code>InvalidMetadataValue</code>	Error	Metadata validation	A metadata value has the wrong type, cardinality, range, enumeration value, declaration category, or compile-time status
<code>ConflictingMetadata</code>	Error	Metadata/classification validation	Two or more individually valid metadata forms cannot legally classify or govern the same declaration together
<code>InvalidOperatorDeclaration</code>	Error	Operator registration	A custom operator declaration has invalid spelling, fixity, precedence, associativity, arity, attributes, ownership, or component placement

<code>InvalidOperatorOverload</code>	Error	Operator overload validation	An operator implementation violates operand ownership, receiver, signature, arity/fixity, genericity, accessibility, or result-independent identity rules
<code>InvalidEffectDeclaration</code>	Error	Effect analysis	<code>@co.dap.effects</code> , emitted error types, or effect metadata placement violates the effect declaration rules
<code>InvalidEffectPolicy</code>	Error	Effect analysis	A call-site <code>@co.dap.onEffect</code> record has an invalid effect key, structure, placement, or ordinary/execution-model policy shape
<code>InvalidEffectHandler</code>	Error	Effect analysis	A handler list is empty, duplicated, inaccessible, non-module, signature-incompatible, or illegally contains local <code>@co.dap.onEffect</code> handling
<code>InvalidEffectResolution</code>	Error	Effect analysis	A resolution is missing, duplicated, unknown, contextually forbidden, or incompatible with the enclosing callable results
<code>InvalidRetryPolicy</code>	Error	Effect analysis	Retry configuration has an invalid attempt

			count, exhaustion action, placement, or structure
<code>InvalidExecutionModel</code>	Error	Execution-model validation	An execution-model declaration or invocation violates its result, effects, submission, boundary, or call-site handling rules

The registry name is the stable machine-readable identifier. A diagnostic instance additionally carries at least its severity, compiler phase, human message, primary source span, zero or more related spans, and structured details. Parser recovery may emit `UnexpectedToken`, `ExpectedToken`, or the fallback `InvalidSyntax`; once a valid AST construct exists, semantic analysis should prefer its more specific registered diagnostic.

Examples of the stable name plus specific message are:

```
error[DuplicateDeclaration]: variable 'count' is already declared in this
scope
error[OverloadNotAllowed]: this callable category does not permit
overloading
error[InvalidMetadataPlacement]: @co.ddap.import is valid only in a file
preamble
```

License

- GNU General Public License v3 (GPLv3)

2. Backend

The Backend is responsible for transforming validated frontend output into executable artifacts.

The default backend must be downloaded or built separately. It is not bundled with the frontend binary.

Backend Components

- Intermediate Representation (IR) Generator
- Native Binary Executable Generation

Implementation

A backend may be implemented in any language. It consumes the validated frontend artifact from the project-root `build/` directory. The artifact encoding is selected by the backend-supplied interchange contract.

During backend installation, that interchange-contract file is placed in the same installation directory as the FoLang compiler executable. The frontend reads the installed contract to determine the compatible FoLang/plugin protocol version, HIR schema version, and wire format required by the selected backend. The exact artifact basename and format-specific schema/version may therefore vary by backend contract, but the artifact location is always beneath `<project-root>/build/`.

Runtime-operation handlers

Backend-neutral HIR may contain a resolved runtime-operation identifier originating from an exported standard declaration marked with `@co.dap.implementation`. The identifier states the required semantic operation, not its target-language spelling or implementation. Each backend owns an internal mapping from supported `co.runtime.operation.*` identifiers to backend handlers.

```
co.runtime.operation.out.println
  -> reference C++ HIR backend handler
  -> generated call to the reference runtime implementation
```

The reference C++ HIR backend may generate a call to a function declared in a separately maintained C++ header and implemented in a separately compiled C++ source/runtime library. Other backends may lower the same operation to different runtime calls, VM instructions, imports, or target-specific code. Backend headers, implementation filenames, source fragments, and native symbol spellings do not appear in the backend-independent FoLang declaration.

The backend interchange contract must identify the runtime-operation contract version or otherwise provide an equivalent compatibility check. Before code generation, the selected backend must reject any required operation for which it has no compatible handler. A backend handler is selected by the resolved operation ID carried in HIR, not by appending arbitrary target-language text obtained from source annotations.

Default / Reference Backend

The default FoLang backend is also the **reference backend implementation**. Its purpose is to provide an executable, inspectable example of how the FoLang specification can be implemented and to give backend implementers a concrete behavioural baseline for conformance testing.

The written FoLang specification remains normative. The reference backend demonstrates the required externally observable semantics, but its internal algorithms, allocation strategy, memory-management choices, data structures, optimization level, and performance characteristics are not themselves language requirements unless this specification explicitly says otherwise. If an implementation defect in the reference backend conflicts with the written specification, the written specification takes precedence.

A third-party backend may use a completely different runtime architecture or memory model and may optimize semantics differently, provided that FoLang programs observe behaviour conforming to this specification. Backend implementations may validate their behaviour against the reference backend and the FoLang conformance tests where applicable.

- Backend orchestration is implemented in **Go**
- Code generation target is **C++**
- Uses **Clang** or **GCC** to generate native binaries from generated C++ IR

License

Third-party backends may use their own licensing terms and implementation choices. The default backend uses the following license. **Default backend is not part of the complete compiler binary and is separate;** it must be downloaded or built separately.

- BSD 3-Clause License ***

Frontend Output Contract

Installed Backend Interchange Contract

The selected backend may supply this contract to tell the frontend which FoLang/plugin protocol, HIR schema, and wire format it accepts. During backend installation, the contract file is placed in the same installation directory as the FoLang compiler executable, and the frontend reads it from that installation location. Every frontend build also embeds a complete default backend contract. If `backend-conf.json` is absent, the frontend uses protobuf and that frontend release's embedded current protocol, HIR-schema, and runtime-operations versions; these values are fixed for that binary, not resolved dynamically. A present contract replaces those defaults and must be complete, well-formed, and supported, or compilation fails.

```
{
  "protocol":          "folang-plugin/1.0",
  "hir_schema":        "folang-hir/1",
  "wire":              "protobuf",
  "runtime_operations": "folang-runtime-operations/1"
}
```

The frontend has one fixed **location** contract and a backend-selected **encoding** contract:

```
<project-root>/build/
  └─ <frontend-artifact>    encoding/version selected by backend
contract
```

Rules:

- the frontend/backend interchange artifact is written beneath the reserved root-level `build/` domain;
- each frontend build embeds default protocol, HIR-schema, wire-format, and runtime-operations versions; an absent installed contract selects those fixed defaults, while a present contract must supply every field and be well-formed and supported;

- that contract file resides in the same installation directory as the FoLang compiler executable;
- `wire="protobuf"` in the example above requests Protocol Buffers; the current protobuf representation is a provisional, language-neutral `Value / Struct / List` artifact tree carrying the logical `foLang-hir/1` model, not yet a typed protobuf definition of every HIR node and interface, and a typed schema with node messages and interface `oneof` declarations is required before treating it as the stable third-party-backend schema; another supported contract may request a different compatible wire format/version;
- `runtime_operations` identifies the backend-neutral runtime-operation contract understood by the selected backend;
- backends consume the validated frontend artifact from `build/`;
- canonical filesystem rules for `build/` are defined only in [Project Layout](#).

Licensing Summary

Layer	Responsibility	Implementation	License
Frontend	Parsing and semantic analysis	Go	GPLv3
Backend (default)	IR processing and native code generation	Go (orchestration) + C++ (target)	BSD 3-Clause

The copyrightable material in the [FoLang Language Definition and Documentation](#), including its syntax, grammar, and semantic-rule descriptions, is licensed separately under [CC BY 4.0](#). ***

3. Capability Security Model

FoLang’s compiler ships with all language features compiled in but **native capabilities are disabled by default**. The compiler has no hardcoded keys — capability configuration happens entirely at install time. This moves authorization from source code (developer-controlled) to the compiler installation (organization-controlled).

Capability Domains and Install-Time Gates

Domain	Features	Default State
<code>application</code>	Ordinary FoLang language/application capabilities, including macros, templates, concurrency/control abstractions, <code>co.http</code> , core declarations such as <code>co.List</code> , <code>co.encoding</code> , <code>co.crypto</code> , etc.	✓ enabled
<code>dynamicvmrt</code>	application capabilities plus the defined dynamic-runtime / <code>co.meta</code> facilities	explicit source capability domain; toolchain policy may restrict availability
<code>native</code>	Raw pointers/references/addresses, pointer arithmetic, <code>@co.dap.native</code> , <code>co.native</code> , <code>co.sys.unsafe</code> , MMIO, heap allocators, low-level platform/runtime implementation, <code>co.sys.ffi</code> , extern declarations/types, foreign symbols, calling conventions, linkage, C/native ABI work, and ABI-compatible <code>cstruct</code> values	🔒 disabled by default — requires install-time configuration

`packaged` is an exposure model using application capabilities, not a separate privileged capability domain.

Variables

```
// typed
name co.string = "Rao";
age  co.int    = 30;
```

```
// inferred static type – := declares a new binding and errors if already
declared
name := "Rao";
age  := 30;

// dynamic binding – ::= declares and initializes a new dynamically typed
binding
value ::= 30;
value = "Rao"; // valid: the binding may later hold a different concrete
runtime type

// define, infer, and assign if not defined; otherwise assign a new value
name ?= "Kumar";
```

`co.string` and `co.int` are built-in data types. See [Builtin Data Types](#).

`=`, `:=`, `::=`, and `?=` are built-in assignment/definition spellings. See [Builtin Operators](#).

Uninitialized Values and `co.const.none`

FoLang’s uniform object model permits ordinary declared types to contain the uninitialized value represented by `co.const.none`. Refinement-type and dependent-type values are the deliberate non-`none` exceptions described below. An ordinary storage location declared without an initializer begins in the **none** state:

```
x co.int;
y co.int;
s co.string;

x.isNone(); // co.const.true
y.isNone(); // co.const.true
s.isNone(); // co.const.true
```

The `none` state is not a singleton object and does not have stable observable identity. The runtime may reuse, rematerialize, or otherwise represent `none` on each access. Two accesses may expose the same internal representation or different representations; neither result is guaranteed by the language.

Consequently, equality, inequality, identity, ordering, and hashing involving a none operand are unspecified. These expressions must not be used to test whether a value is initialized:

```
x == x;           // unspecified while x is none
x == y;           // unspecified
x == co.const.none; // unspecified
co.const.none == co.const.none; // unspecified
```

The only reliable none test is the intrinsic object operation `isNone()`:

```
x.isNone(); // always co.const.true while x is none

x = 10;
x.isNone(); // co.const.false
```

`isNone()` is available on every ordinary FoLang object/value that admits none, returns exactly `co.const.true` or `co.const.false`, and cannot be overridden to disguise the none state. It is not available on a statically resolved refinement-type or dependent-type value. Assigning `co.const.none` explicitly resets an ordinary destination to the none state:

```
x = co.const.none;
x.isNone(); // co.const.true
```

A none value may be stored in an ordinary local, field, collection element, parameter, or return position. It cannot be stored in any position whose resolved static type is a refinement type or dependent type. For types that admit none, passing, returning, or assigning none transfers the none state; it does not promise preservation of a none object's identity. The omitted/unprovided state of an optional argument remains a separate concept and is tested through its `omitted` flag rather than `isNone()`.

Except for `isNone()` and constructs that explicitly define none behavior, an operation that observes a none value has unspecified semantic output. A backend may implement none with tags, initialization bits, placeholder objects, null representations, or another strategy, but it must not expose unrelated raw host-memory contents or turn none access into backend undefined behavior.

When `@co.dap.onEffect(... resolution=continue)` consumes an effect from a value-producing call, every missing result position that admits none receives `co.const.none`. Execution then continues after the decorated call or its containing assignment:

```
customer Customer =
  @co.dap.onEffect(
    co.NotFound={resolution=continue}
  )
  loadCustomer(id);

customer.isNone(); // co.const.true when NotFound was continued
```

The substitution supplies a result state only. It does not undo mutation, I/O, or any other side effect completed before the callee emitted the effect.

Non-None Refinement and Dependent Values

A value whose resolved static type is declared using `co.refinementType`, or is a concrete dependent type such as `Vector(3)`, cannot contain `co.const.none`. This rule applies to locals, fields, parameters, results, collection elements, and every other typed storage or transfer position. It applies to dependent-type values, not merely to the `co.dependentType` meta-level result kind used while a type-valued ordinary function constructs such a type.

A declaration without an initializer remains syntactically legal. Unlike an ordinary declaration, however, it creates a **definitely uninitialized** binding and does not place a readable `co.const.none` value into that binding. Every value-producing access must be preceded on every reachable control-flow path by a successful assignment whose value satisfies the declared type. Otherwise, the compiler reports the canonical diagnostic `Uninitialized` from the [Named Diagnostic Registry](#).

Value-producing access includes reading the value, passing it as an argument, returning it, applying an operator, accessing one of its members, indexing through it, interpolating it, or using it as the read side of a compound assignment. Naming the binding solely as the destination of a simple assignment is not a read.

```
positiveInt co.refinementType =
  (co.int).where(_ > 0);
```

```

count positiveInt;      // valid declaration; count has no readable value
print(count);           // compile-time error: Uninitialized
count.isNone();         // compile-time error: isNone is invalid for
positiveInt
count = co.const.none;  // compile-time error: positiveInt cannot contain
none

count = 10;             // valid assignment and refinement validation
print(count);           // valid: count is definitely initialized

values Vector(3);       // valid declaration; no readable value yet
values.isNone();         // compile-time error: invalid for Vector(3)
dotProduct(values, values); // compile-time error: Uninitialized
values = Vector(3){1, 2, 3};
dotProduct(values, values); // valid

```

Definite initialization is flow-sensitive. Assignment in only one conditional branch is insufficient when another reachable branch can reach the access without assigning the binding. Function parameters are initialized on entry.

For aggregates, each construction path is checked independently:

Aggregate path	Required proof
Struct literal or function returning a struct	Every refinement/dependent field is supplied before the constructed value is returned or read
Class lifecycle	<code>@@new</code> may produce <code>co.uninit</code> , but every successful <code>@@init</code> path must initialize each refinement/dependent field before the instance becomes readable
Lifecycle overloads	Each <code>@@init</code> overload is checked separately; assignments proved for another overload do not count

```

// Limits.fol
_ co.struct = {
  minimum positiveInt;
}

// limits.unit.fol
_ co.unit = {
  demonstrate()->() = {

```



```

        valid Limits = Limits{minimum: 1}; // valid
        invalid Limits = Limits{};         // compile-time error:
Uninitialized
    }
}

```

Multiple `@@new` overloads do not merge initialization proofs. Allocation and field initialization remain separate: an uninitialized class value cannot be read, and only a completed `@@init` path establishes a readable instance.

Any construct that would place `co.const.none` into a missing result position is invalid when that position has refinement or dependent type. This includes effect resolutions such as `continue`, `return`, or `return_with_error`, and execution-model failure conversion, whenever their missing-result substitution would target such a position. The caller must instead choose control flow that does not require that result, provide a valid initialized value, or model the outcome explicitly with a suitable variant type.

Variant-Based Absence as a Recommended API Practice

FoLang does not require a plain result type to be replaced merely because `resolution=continue` can leave that result in the universal none state. The preceding `Customer` declaration is therefore valid, and the compiler does not require a variant type or a preceding `isNone()` test before later use. This is a deliberate permissive rule rather than a static definite-presence guarantee.

For APIs where absence is an expected domain outcome, a closed variant-based type is nevertheless the recommended design. It makes the expected outcomes visible without turning ordinary absence into an effect:

```

CustomerLookup co.type =
    co.variants(
        Found(Customer),
        NotFound
    );

lookupCustomer(id co.int)->(CustomerLookup) = {
    ...
}

```

`CustomerLookup.NotFound` is an ordinary initialized variant value. It is not `co.const.none`, and it has the normal identity, matching, and behavior of its declared variant. The universal none state remains available to ordinary storage types independently of whether a type is variant-based; refinement and dependent values remain the non-none exceptions.

The recommended distinction is therefore:

Situation	Recommended representation
Expected alternative such as found/not-found or accepted/rejected	Closed <code>co.variants(...)</code> result
Recoverable operational failure requiring caller policy	Typed effect
Caller intentionally consumes an effect and can tolerate a missing result	<code>resolution=continue</code> , followed by an appropriate <code>isNone()</code> check

Compilers and development tools may provide a non-blocking recommendation when `continue` supplies none to a result that is subsequently used without an `isNone()` check. Such a recommendation is advisory and must not reject the program. Declaring a variant does not cause `continue` to manufacture one of its states; an appropriate variant state must be returned explicitly by the API implementation.

Constants and Immutability

FoLang separates two properties that are often confused. One is about *when* a value is known. The other is about *whether* it can change.

```
// RuntimePolicy.fol
// A co.object is one named singleton object. It owns independently
// constant, immutable, global, or shared state; loose package globals and
// class fields do not.
_ co.object->(for=[Producer, Consumer]) = {
  // compile-time constant – value known while compiling, substitutable
  @co.dap.const SIZE co.int = 1024;

  // immutable binding – cannot be reassigned, value need not be known
  early
  @co.dap.final startedAt co.int = co.sys.now();
}
```

Annotation	Guarantees	Value known at compile time	Usable as an index
<code>@co.dap.const</code>	value is fixed and known while compiling	✓	✓
<code>@co.dap.final</code>	binding cannot be reassigned	✗ not required	✗

Every `@co.dap.const` is also immutable. The reverse does not hold: a `@co.dap.final` binding may be initialised from a function call, a file, or user input, so the compiler cannot substitute a literal for it.

`@co.dap.final` is the declaration-site form of the immutable object kind described in the object mutation policy. `makeImmutable` applies the same property to a value at run time.

`@co.dap.const`, `@co.dap.final`, independently shared storage, locks, and other global storage are forbidden as fields of `co.class`. They are also not declared as loose package globals. When such state is associated with one or more classes, it is owned by a named `co.object`; see [Associated Objects](#).

Outside a declaration whose signature binds a dependent index parameter, only `@co.dap.const` may supply a named array size or dependent type index. A signature-bound index name is permitted symbolically even when its concrete value is supplied by the caller. See [Dependent Type Index Rules](#).

Single Source Application File

A **single-source application** is an application whose entry file contains the complete program and does not depend on user package source files.

A single-source application file and an application entry file are the same fixed structural source, `src/app1.fo1`, and use the same entry-file grammar, context, and restrictions. A project is a single-source application when `src/app1.fo1` contains the complete program and there are no application package directories below `src/`. The rules below define the constructs permitted in that source context.

For the single-source application layout, see [Project Layout](#).

Allowed Constructs

The application file may contain:

- built-in directives that are valid for an entry file; directives occupy the entry-file metadata preamble and cannot appear inside any declaration or block
- built-in `@co.pdap.*` pragmas; `src/app1.fo1` is the only FoLang source location in which pragmas are permitted
- package and library imports
- import aliases declared with `as=`
- file-local aliases for `co.*` paths declared with `@co.ddap.alias`
- named non-UDT type definitions, including aliases needed locally for arrays, collections, ranges, refinements, and concrete generic specializations
- uses of named types imported from units, classes, modules, packages, libraries, and the standard library
- entry-local `let` function-pattern groups, which may capture zero or more surrounding runtime bindings
- variable declarations, initialization, assignment, and mutation
- calls to built-in methods and functions
- calls to imported package and library APIs
- ordinary expressions
- comprehensions
- ternary expressions
- conditions and loops
- containment operations such as `contains`
- iterators such as `each`
- pattern matching and destructuring

```
a co.int = 10;  
b co.int = 20;  
  
co.out.println( a + b);
```

`co` is a reserved word in FoLang; see [Reserved Words](#).

`co` is the built-in root package in FoLang; see [Builtin Packages](#).

`a + b` is an expression in FoLang.

For the expression rules, see [Expressions](#).

Built-in and Imported Names

The projected `co` root is always in scope without an import. A particular `co.*` declaration is available only when it is exported by the installed `co.folenc` artifact and permitted by the active capability rules.

A built-in declaration may be referenced through its complete `co.*` path:

```
co.out.println("Hello");
co.List.of(1, 2, 3);
```

A file-local alias may be declared for a `co.*` path:

```
@co.ddap.alias(co.out, as="out")
@co.ddap.alias(co.List, as="list")

out.println("Hello");
values := list.of(1, 2, 3);
```

Creating an alias does not hide the complete `co.*` name; both forms remain valid in that file.

Third-party packages, user packages, and libraries are not automatically available. They must be imported using `@co.ddap.import`. When `as=` is present, the imported API is accessed through that alias. When `as=` is omitted, the complete imported package or library path must be used.

```
@co.ddap.import(package="hr.employee", as="emp")
first := emp.EmployeeService.find(1001);

@co.ddap.import(package="finance.payroll")
second := finance.payroll.calculate(request);
```

A non-standard package must be imported before its declarations are used.

`@co.ddap.import` and `@co.ddap.alias` are built-in directives; see [Built-in Directives](#).

Import semantics are defined in [Import Details](#).

`println` is a built-in method on the `co.out` object.

Variable Kinds

Variable-kind availability is context-dependent; not every variable kind is valid in every source location. For the contexts in which each kind is supported, see [Variable Kind Support](#).

Simple Variable Declaration

```
someVar co.int;  
someString co.string;
```

With Initialization

```
someBool co.bool = co.const.true;  
someInt co.int = 42;
```

Static Type Inference and Dynamic Declaration

```
someVal := "Hello, World!"; // new statically typed binding; type inferred  
from initializer  
someNum := 3.14;           // new statically typed binding; redeclaration  
is an error  
  
someDynamic ::= 10;        // new dynamically typed binding  
someDynamic = "Kamesh";    // valid: concrete runtime type may change  
  
someR ?= "Kamesh";         // if not defined, define and infer; otherwise  
reassign compatibly
```

`:=` is the canonical inferred declaration form; inferred declaration does not require a separate source type.

Alpha Character and String Literals

The alpha release accepts only the basic literal subset:

```
message := "hello";
letter  := 'A';
```

A string is unprefixed, remains on one source line, and cannot contain a backslash or an embedded double quote. A character literal contains exactly one non-backslash character.

Encoding prefixes, escaped characters, raw strings, and universal character names are reserved for a later release. The scanner recognizes the complete spelling and reports an unsupported-feature error instead of tokenizing it as several unrelated expressions. For example, all of these are rejected:

```
x := "quoted: \"text\"";
x := '\n';
x := u8"hello";
x := R"(raw text)";
x := '\N{LATIN CAPITAL LETTER A}';
```

Pointer Declaration

```
IntPtr      co.type = co.int->>(*);
IntPtrPtr   co.type = co.int->(**);
IntPtrDeepPtr co.type = co.int->*****);

somePtr      IntPtr;
someDb1Ptr   IntPtr;
someDeepPtr  IntPtr;
```

The number of consecutive `*` characters is the pointer degree. Any positive degree is permitted; `*` and `**` above are common examples, not a maximum.

Array Declaration

```
FiveInts      co.type = co.int->([5]);    // single dimension
IntMatrix2x3   co.type = co.int->([2,3]);  // multi dimension
```

```

JaggedInts2x3  co.type = co.int->([2][3]); // jagged
VariableInts   co.type = co.int->([...]);  // variable length
ZeroInts       co.type = co.int->([0]);    // zero length array
ZeroDimInts    co.type = co.int->([.]);    // zero-dimensional array

someArray      FiveInts;
someDblArray   IntMatrix2x3;
someJaggedArray JaggedInts2x3;
someVLArray    VariableInts;
someZeroLA     ZeroInts;
someZeroDimA   ZeroDimInts;

```

Array Declaration with Initialization

```

ThreeInts      co.type = co.int->([3]);
InferredInts   co.type = co.int->([]);
InferredGrid   co.type = co.int->([,]);

someInitializedArray  ThreeInts      = ThreeInts{1, 2, 3};
someInitializedArray1 InferredInts   = InferredInts{1, 2, 3};
someInitializedDblArray InferredGrid = InferredGrid{1, 2, 3, 4};

```

Reference Declaration

```

IntRef          co.type = co.int->(&);    // reference
IntLValueRef    co.type = co.int->(&&);  // LValue reference
IntHeapRef      co.type = co.int->(~);    // heap allocated reference
IntAddress      co.type = co.int->(@);    // address
IntThunk        co.type = co.int->(^);    // thunk
IntSlice        co.type = co.int->[:]); // slice

someRef         IntRef;
someLValueRef   IntLValueRef;
someHpRef       IntHeapRef;
someAddr        IntAddress;
someThunk       IntThunk;
someSlice       IntSlice;

```

Range Declaration

```

// A derived range type is named before it is used by a variable.
IntRange co.type = co.int->(..);
someRange IntRange;

```



```
// Inferred range declarations
rangeI := 1 .. 10;      // [1, 10]   ExcludeStart=false, ExcludeEnd=false
rangeS := 0 <.. 5;      // (0, 5]    ExcludeStart=true,  ExcludeEnd=false
rangeL := 0 ..< 100;    // [0, 100)  ExcludeStart=false, ExcludeEnd=true
rangeB := 0 <..

```

Dynamic Variable Declaration

Dynamic typing is a property of a binding, not a `co.*` type. FoLang uses `::=` to declare and initialize a dynamically typed binding:

```
value ::= 10;
value = "Hello";
value = Employee{id: 1, name: "Rao"};
```

`name ::= expression;` introduces a new binding and therefore fails if `name` is already declared in the applicable scope. An initializer is mandatory because `::=` both declares and initializes the dynamic binding. The value stored in the binding retains its concrete runtime type, but the binding itself has no single source-level static value type. A later ordinary assignment may replace the current value with a value of another concrete runtime type when that value is otherwise legal in the current capability and execution context. Operations whose validity depends on the current concrete value are resolved and validated at run time.

`::=` is a declaration form, not a type expression. It cannot appear in a parameter type, result type, type alias, generic argument, or other position that requires a named FoLang type. Where a statically typed API must represent one of several possible concrete result types, use an explicit tagged/variant representation or another declared stable outer type.

The declaration operators therefore have distinct contracts:

```
name := expression;
    declare + infer one static type from the initializer
    later assignments must satisfy that binding's static type

name ::= expression;
```

```
declare + initialize a dynamic binding
later assignments may change the concrete runtime type
```

Lazy

```
@co.dap.lazy
x = add(1, 2); // evaluating/using x invokes add(1, 2); until then the
right-hand function is not invoked
```

Bind Variables

```
$[1-9][0-9]*
```

Bind variables are available in function-chaining expressions. At each chained call, `$1`, `$2`, `$3`, ... denote the return components of the **immediately preceding function invocation**, in declared return order. If the preceding function returns one value, only `$1` is available for that step; if it returns multiple values, `$1` through `$N` correspond to those return values. `$0` is not a bind variable.

```
dosomething(a co.int, b co.int)->(co.int)
=>> somePack.someMethod(a)
=>> someOthPack.someOtherMeth($1, b);

// If previous() returns (A, B, C), the next chained call may consume
// those return components as $1, $2, and $3 respectively.
previous()
=>> consume($1, $2, $3);
```

Discard / Wildcard Variable

```
_
```

`_` is contextual rather than an ordinary identifier. In discard/wildcard positions it means “ignore this binding or match position.” It is also used as the filename-derived primary-declaration placeholder and as an unnamed type-parameter slot in parameterized-type forms such as `F(_)`. In the predicate of a `co.refinementType` declaration, `_` instead denotes the candidate value of the base type being tested. See [Refinement Types](#).

Comma and Grouping

```
// Comma
x co.int = 10, y co.string = "Hello", z co.bool = co.const.true;

// Grouping
(x co.int = 10, y co.string = "Hello", z co.bool = co.const.true);

// Parentheses group expressions; they do not construct tuple values.
IntStringPair co.type = co.Tuple(co.int, co.string);
pair := IntStringPair{x, y};
```

A comma inside a parenthesized expression or grouped declaration must introduce another expression or declarator. Parentheses do not construct tuple values; `(x, y)` is therefore not a tuple literal. Therefore `(x,)`, `(x, y,)`, and `(x co.int = 10,)` are invalid. Typed composite initializers have their own separator and trailing-comma rules. Record patterns also require another field after every comma; `Employee{id: value,}` is invalid.

Fat Pointers

```
DefaultFatIntPtr co.type = co.int->>(*, kind="", meta={});
x DefaultFatIntPtr;

MetadataIntPtr co.type = co.int->>(*, meta={});

VTableIntPtr co.type = co.int->>(*, meta={len=co.usize, vtab=somepkg.VTable->(*)});
y VTableIntPtr;

RegionIntPtr co.type = co.int->>(*, kind=region, meta={});
z RegionIntPtr;
```

Pointer

```
├─ base_type: T
├─ kind: <FatKind>
│   ├── thin
│   ├── slice
│   ├── relative
│   ├── trait
│   └── buffer
```

```

|   |— view
|   |— opaque
|   |— custom
|   |— mem
|   |— nullptr
|   |— sptr
|   |— uptr
|   |— ptrdiff
|   |— usize
|   |— ssize
|   |— (region) ← optional syntactic sugar
|— meta:
   |— region: heap | stack | global | numa(N) | mmio | constant | ...
   |— len, cap, vtab, bits, endian, ...

```

Pointers for address manipulation

```

IntPtrWord  co.type = co.word->(repr=intptr);
UIntPtrWord co.type = co.word->(sign=unsigned, repr=uintptr);
PtrDiffWord co.type = co.word->(repr=ptrdiff);
USizeWord   co.type = co.word->(sign=unsigned, repr=usize);
ISizeWord   co.type = co.word->(repr=isize);
NullPtrWord co.type = co.void->(repr=nullptr);

y IntPtrWord;
z UIntPtrWord;
p PtrDiffWord;
n USizeWord;
m ISizeWord;
o NullPtrWord;

```

Relative Pointers

```

RelativeIntPtr co.type = co.int->(*, kind=relative, meta={});
z RelativeIntPtr;

```

Note A pointer, word, reference, array, range, or other derived type is defined in a `co.type` declaration and subsequently used by its alias.

Conditions, Loops and Iterators

Conditions

`then` is the one-shot conditional branch verb. Its argument may be a block or an ordinary value/expression. `otherwise(condition)` always introduces another Boolean condition; a conditionless `otherwise` form does not exist.

`default(result)` is the optional terminal fallback and may likewise receive a block or value.

```
(boolean truth).then({
}).otherwise(boolean truth).then({
}).default({
});

//conditionEg1.unit.fol

_ co.unit = {

  someFun()->()={

    x := co.const.true;

    x.then({

    }).default({

    });

  }

  someOtherFun()->()={

    (co.const.true).then({ //parenthesis mandatory

    }).default({

    });

  }

  someOtherFun1()->()={
    x co.bool =co.const.true;
    (x).then({ // parentheses optional

    }).default({
```

```

    });

}

someOtherDiffFun1()->()={

    x co.int=10;
    y := 30;

    (x > y).then({      // parentheses around (x > y) are mandatory

    }).otherwise(x < y).then({

    }).default({

    });

}
}

```

Loops

`loop` is the repeated-execution verb and always has exactly one loop condition and one loop body. Unlike `then`, a `loop(...)` form cannot participate in an `otherwise(condition)` chain, and `default(...)` cannot follow a loop. When the loop condition is false, whether on the first test or after one or more iterations, the loop simply terminates.

To choose between different looping behaviours, first use a `then` / `otherwise(condition)` / `default` selection chain and place the required ordinary loop inside the selected block. The selection chain chooses a behaviour; each nested `loop(...)` then performs repetition using only its own condition.

```

(boolean truth).loop({
});

//loopsEg1.unit.fol

_ co.unit = {

```

```

someFun()->()={

    x := co.const.true;

    x.loop({
    });
}

someOtherFun()->()={

    (co.const.true).loop({ //parenthesis mandatory
    });
}

someOtherFun1()->()={
    x co.bool =co.const.true;
    (x).loop({ // parentheses optional
    });

}

someOtherDiffFun1()->()={

    x co.int=10;
    y := 30;

    (x > y).loop({ // parentheses around (x > y) are mandatory
    });
}
}

```

```

//loopsEg2.unit.fol

_ co.unit = {

    someFun()->()={

        x := co.const.true;
        v := 0;
        x.loop({
            (v == 10).then({
                this ->|;
            });
            v += 1;
        });
    }
}

```

```

}

someOtherFun()->()={
    v := 20;
    (co.const.true).loop({ //parenthesis mandatory
        (v == 30).then({
            this ->>;
        });
        v += 5;
    });
}

}

```

Combining Conditions and Loops

A loop does not become a branch verb inside an `otherwise(condition)` chain and has no `default(...)` fallback of its own. When a program must choose between different looping behaviours, the selection is expressed separately with `then` / `otherwise(condition)` / `default`, and each selected block may contain its own ordinary single-condition loop.

```

(first condition).then({
    (first loop condition).loop({
        ...
    });
}).otherwise(second condition).then({
    (second loop condition).loop({
        ...
    });
}).default({
    ...
});

```

Ternary Operator

The ternary/value form uses the same `then` / `otherwise(condition)` / `default` selection vocabulary as block conditionals. Only the branch arguments differ: a value-producing chain supplies values or expressions instead of statement blocks.


```

s = (boolean truth).then(some var/value).default(some val/var);
s = (boolean truth).then(some var/val).otherwise(boolean truth).then(some
var/val).default(some var/val);

//TernaryExample.unit.fol

_ co.unit = {

    someFunction()->()={

        s := (co.const.true).then(20).default(10);
        y co.int;
        z co.int=20;
        y ?= (co.const.true).then(20).default(z);

    }

    someOtherFunction()->()={

        k co.int=10;
        p co.int =20;
        s ?= (k>10).then(30).otherwise(k<10).then(p).default(10);

    }

}

```

Parenthesizing the chain head

A conditional or ternary selection chain is a sequence of postfix method calls using `.then(...)`, `.otherwise(condition)`, and optional terminal `.default(...)`. A loop uses the same postfix-call style but is a single-condition repetition form:

`.loop(...)` cannot be followed by `.otherwise(condition)` or `.default(...)`.

These calls always carry their own call parentheses, whatever the argument is. Nothing about the argument changes that.

`otherwise` always has the form `.otherwise(condition)` and therefore never acts as a terminal else marker. The terminal fallback is `.default(...)`; its argument may be a block or a value/expression according to the applicable form.

The only place a choice exists is the **head**: the subject before the first `.then(...)` or `.loop(...)` call. The head must already be a complete postfix expression that cannot absorb the following control verb.

Chain head	Parentheses	Why
<code>x</code>	optional	an identifier is already a complete postfix expression
<code>arr[0]</code> , <code>f()</code>	optional	index and call suffixes are postfix too
<code>co.const.true</code> , <code>myPkg.flag</code>	required	a qualified name would absorb <code>.then</code> as a further segment, giving <code>co.const.true.then</code>
<code>x > y</code>	required	<code>.then</code> binds tighter than <code>></code> , so <code>x > y.then({...})</code> groups as <code>x > (y.then({...}))</code>

A qualified name needs parentheses whether it names a literal or an identifier, so `myPkg.flag` is the same case as `co.const.true`.

```
x.then({ ... });           // head is an
identifier
(co.const.true).then({ ... }); // head is a
qualified name
(x > y).then({ ... }).otherwise(x < y).then({ ... }); // head is an
expression
(k > 10).then(30).otherwise(k < 10).then(p).default(10);
```

In the last two lines the parentheses after `.otherwise` are its call parentheses, not an application of this rule.

Iterating Arrays / Lists / Maps / Ranges

`each` is itself the element-iteration operation. It does not produce a value that must subsequently be passed to `loop`, and `.loop(...)` must not follow an `each(...)` call.

The explicit-binding form takes the index/key binding, the value binding, and the per-element action directly. The action may be a block or any ordinary expression valid in that position. A callable may also be supplied directly as the sole callback argument; the callable receives the receiver's defined iteration tuple. A lambda is a callable callback and is permitted directly in this collection-operation context.

```
FiveInts co.type = co.int->([5]);
arr FiveInts = FiveInts{6,7,8,9,10};

// block action
arr.each(idx, val, {
  co.out.print(idx);
  co.out.print(" :: ");
  co.out.println(val);
});

arr.each(_, val, {
  co.out.println(val);
});

// expression action – evaluated once for each element
arr.each(_, val, co.out.println(val));

// direct callable callback – receives the receiver's iteration tuple
arr.each(printItem);

// direct lambda callback
arr.each(|idx, val| => co.out.println(val));
```

The explicit-binding form establishes its bindings separately for every iteration and evaluates its action exactly once for that element. The action is therefore the body of `each`; `each` never participates in a `then`, `otherwise`, `default`, or `loop` chain. In the single-argument callback form, the argument must resolve to a callable compatible with the receiver's iteration tuple.

Array / List / Map / Range — Contains Element

```
FiveInts co.type = co.int->([5]);
arr FiveInts = FiveInts{35,57,96,81,31};
k co.int = 31;
arr.contains(k).then({
  co.out.println(k);
});
```

```
}).default({
  co.out.println("Not Found");
});
```

Comprehensions

```
IntList      co.type = co.List(co.int);
IntSet       co.type = co.Set(co.int);
StringIntMap co.type =
  co.Map(co.string, co.int);

k := (1 .. 10).filter(|x| => x % 2 == 0).map(|x| => x * x);

result := for (x <- IntList{1,2,3}).yield(x * 2);      // IntList{2, 4, 6}
result := for (x <- IntSet{1,2,3}).yield(x * 2);      // IntSet{2, 4, 6}
result := for (x <- Some(5)).yield(x * 2);            // Some(10)
result := for (x <- fetchData()).yield(x.process()); // Future

ages := StringIntMap{"A":30,"B":40,"c":66,"e":88};
upper := for ((name, age) <- ages).yield(name.toUpperCase, age);
```

Pattern Matching

```
x co.int = 10;

x.match().case(n: n > 10 => { n = n+100; "GT" }).case( n: n < 10 =>
"LT").default("EQ");
x.match().case(n: n > 10 => { n = n+100; "GT" }).case( n: n < 10 =>
"LT").case(_=>"EQ");
x.match(co.pattern.Type).case(co.int => ...).case(co.float => ...);
x.match(co.pattern.Value).case(0 => ...).case(1 => ...);
x.match(co.pattern.Instance).case(xx.CAT => ...).case(xx.DOG =>
...).default("Animal");
x.match(co.pattern.Object).case(xx.Ball => "Ball").case(xx.CAT =>
"CAT").default("Unknown");
x.match(co.pattern.Shape).case(Point{x, y} => ...).default(...);

x.match(co.pattern.Any).case(co.int => ...).case(co.float => ...).case(0 =>
...).default( ...);

x.match(PositiveEvenMatcher).case(0 => "Neither even nor odd").case(2 =>
"First Even Prime").default(...);
```

A match chain contains one or more `.case(...)` arms followed by at most one terminal `.default(...)` arm. `.otherwise` is not a match arm; it always introduces another Boolean condition in conditional `then` chains. Loops do not participate in `otherwise(condition)` chains. `default` is the common terminal fallback vocabulary for Boolean selection and match/case selection. A case or default result may itself be a ternary then expression, and that nested expression may consequently contain `.default(...)`.

For value dispatch, `match().case(...).default(...)` is also FoLang's generalized analogue of a traditional `switch / case / default` construct; match cases additionally support the pattern, type, object, and custom-matcher forms defined by this specification.

Matcher Selection

FoLang distinguishes automatic matcher selection from explicit matcher selection by whether `.match(...)` receives an argument:

```
value.match().case(...).default(...);    // automatic/default matcher
selection
value.match(co.pattern.Type).case(...);    // explicit matcher
value.match(PositiveEvenMatcher).case(...).default(...); // explicit custom
matcher
```

`.match()` is the only no-argument matcher-selection form. A bare `.match` without call parentheses is invalid. When `.match(matcher)` supplies an expression, that expression explicitly identifies the matcher. A user-defined matcher follows the ordinary matcher declaration, import, and name-resolution rules.

`_` is a special discard/wildcard variable. In a call it is permitted only as the first, index/key binding of the explicit receiver-qualified `.each` form, as in `items.each(_, value, { ... })`. Transparent grouping around the member callee, for example `(items.each)(_, value, { ... })`, does not change this rule. The value binding and the iteration-action argument of `each` cannot be discarded, and `contains(_)` and `containsVal(_)` are invalid because containment must compare a real value. Patterns and the filename-derived declaration-name form, parameterized-type placeholder forms such as `F(_)`,

and refinement-type predicates give `_` their own explicitly described meanings. In a refinement predicate, `_` denotes the candidate value being tested; it is not a general expression identifier.

`PositiveEvenMatcher` is a custom matcher; its declaration rules are defined in [Custom Matcher](#).

Type Declarations

Named Type Use Outside Type-Producing Declarations

FoLang separates **defining a type expression** from **using the resulting type**. A complete type expression is written on the right-hand side of an explicit type-producing declaration and receives a name there. Ordinary fields, variables, parameters, receivers, and function results use that name; they do not repeat an anonymous derived type expression inline.

This restriction does **not** change ordinary named function or method declarations. Their own parameter list and result clause remain direct parts of the function declaration:

```
calculate(a co.int, b co.int)->(co.int) = {  
    this => a + b;  
}
```

Here `(a co.int, b co.int)->(co.int)` is the declaration's signature, not an anonymous function type used in a type position. A named function-type alias is required only when a function type is itself the type of a field, variable, parameter, receiver, or result.

`co.type` creates a transparent alias. It does not create a new type identity:

<code>Binary</code>	<code>co.type = (co.int, co.int)->(co.int);</code>
<code>PolyBinary</code>	<code>co.type = forall(T).(T, T)->(T);</code>
<code>IntPtr</code>	<code>co.type = co.int->(*);</code>
<code>TenInts</code>	<code>co.type = co.int->([10]);</code>
<code>IntRange</code>	<code>co.type = co.int->([1...100]);</code>
<code>IntContainer</code>	<code>co.type = Container(co.int);</code>

The aliases are then used in ordinary declarations:

```
operation Binary;
polymorphicOperation PolyBinary;
pointer IntPtr;
values TenInts;
bounded IntRange;
Container IntContainer;

apply(operation Binary)->(co.int) = {
  this => operation(10, 20);
}
```

By contrast, `co.newtype` creates a distinct nominal type:

```
EmployeeId co.newtype = co.int;
```

Consequently, naming `co.int->(*)` with `co.type` does not make the pointer nominally different from that pointer type, while naming `co.int` with `co.newtype` creates a distinct `EmployeeId` type. The alias/newtype distinction is independent of whether the right-hand side is simple or derived.

The following direct uses are invalid:

```
operation (co.int, co.int)->(co.int); // invalid: inline function type
pointer co.int->(*);                  // invalid: inline pointer
type
values co.int->([10]);                 // invalid: inline array
type
bounded co.int->([1...100]);           // invalid: inline range
type
consume(value forall(T).(T)->(T))->(); // invalid: inline
polymorphic type
box Box(co.int);                      // invalid: unnamed
parameterized-type application
```

The same restriction applies to annotation values. An annotation may name a built-in type or a declared alias, but it cannot contain an inline or parameterized type expression:

```
@config(type=co.int)                // valid
@config(type=IntPtr)                 // valid named alias
@config(type=Vector(co.int))         // invalid
```

```
@config(type=co.int->(*))           // invalid
@config(callback=(co.int)->(co.bool)) // invalid
```

An overloaded declaration reference is the deliberate signature-shaped exception. Its parameter and result entries are built-in or named types, never inline type expressions:

```
@handler(target=find(co.int)->(Employee)) // valid overload reference
@handler(target=find(IntPtr)->(Employee))   // valid alias in
signature
```

The full type-expression grammar is available on the right-hand side of `co.type`, `co.newtype`, and the other explicitly defined type-producing declaration forms. It is also available as the produced expression of a type-valued function. In every ordinary type-use position, the grammar accepts a built-in type name, a declared type name, a named alias, or a generic parameter already established by the enclosing built-in metadata. A concrete instantiation of an annotation-based generic declaration, such as `Container(co.int)`, is a complete type expression and must first be named on a `co.type` right-hand side:

```
IntBox co.type = Container(co.int); // complete expression: legal here
value IntContainer;                // ordinary type use
```

Direct `Container(...)` use in a field, variable, parameter, receiver, function result, annotation value, or other ordinary type position is a compiler error. Built-in collection specializations such as `co.List(co.int)` follow the same rule: name the complete type expression with `co.type`, then construct and use values through that alias. A parameterized/dependent type family's own application syntax, such as `Vector(n)`, remains a separate dependent-value grammar category.

This alias-first rule also applies to typeclass contracts and instance specialization signatures. A typeclass names abstract applications such as `F(A)` and `G(B)` in its built-in `aliases=[...]` metadata. An implementing instance uses those same alias names; it does not repeat either the abstract application or its concrete specialized spelling inline. The compiler obtains the concrete representation by specializing the contract aliases from the instance's ordered `type=` or `types=[...]` bindings.

This rule applies equally to function types, polymorphic types, pointers, references, arrays, slices, ranges, unions, dependent types, and other type derivations. Higher-rank parameters and results therefore use named polymorphic `co.type` aliases rather than direct `forall(...)` expressions. It does not require an ordinary function declaration to replace its own parameter and result clauses with one function-type alias.

```
// Alias
x co.type = co.int;

// New
x co.newtype = co.int;

// Opaque
EmpIdType co.opaquetype = co.int;
DeptIdType co.opaquetype = co.int;

empId EmpIdType = 10;           // valid: base representation is accepted
when constructing EmpIdType
deptId DeptIdType = 20;        // valid

empId = deptId;                 // invalid: distinct opaque types are not
assignment-compatible
empId2 EmpIdType = empId;       // valid: same opaque type

x co.int = empId;               // invalid: an opaque value does not implicitly
become its base type

// ADT (tagged union)
y co.type = co.int | co.char;

// Proper subtypes of Employee; Employee itself is excluded.
// Employee is a class.
empSubType co.subtype = somePackage.Employee;

// PermanentEmployee inherits Employee.
permanentEmp empSubType = PermanentEmployee{}; // valid: proper
subtype

// ContractualEmployee inherits Employee.
contractualEmp empSubType = ContractualEmployee{}; // valid: proper
subtype
```

```

dancerEmp empSubType = DashingDancer{};           // compiler error: not a
subtype of Employee
baseEmp    empSubType = Employee{};                // compiler error:
Employee itself is excluded

// To accept Employee itself together with all proper subtypes:
empPlusType co.type = empSubType | Employee;

// Proper supertypes of Toyota; Toyota itself is excluded.
superToyota co.supertype = somePackage.Toyota;

// somePackage.Toyota extends somePackage.Car extends
somePackage.FourWheeler extends somePackage.Vehicle

toyota    superToyota = somePackage.Toyota{};       // compiler error:
Toyota itself is excluded
car        superToyota = somePackage.Car{};         // valid
four       superToyota = somePackage.FourWheeler{}; // valid
vehicle    superToyota = somePackage.Vehicle{};     // valid
truck      superToyota = somePackage.Truck{};       // compiler error: not a
supertype of Toyota

// To accept Toyota itself together with all proper supertypes:
superToyotaPlus co.type = superToyota | somePackage.Toyota;

// Refinement type
positiveInt co.refinementType = (co.int).where(_ > 0);

percentage co.refinementType =
  (co.int).where(_ >= 0 && _ <= 100);

evenInt co.refinementType =
  (co.int).where(_ % 2 == 0);

nonEmptyString co.refinementType =
  (co.string).where(_.length > 0);

// Inside a refinement predicate, _ denotes the candidate value of the base
type.

```

For the normative refinement-type rules, see [Refinement Types](#).

For `co.opaquetype`, the declared base type supplies the representation accepted where opaque-type construction permits it, but the resulting opaque type has distinct type identity. Distinct opaque types are not assignment-compatible merely because they share the same base type, and an opaque value is not implicitly assignable back to its base type. Thus `x co.int = empId;` is invalid when `empId` has type `EmpIdType`.

`co.subtype` and `co.supertype`

`co.subtype` and `co.supertype` define type sets for these two declaration kinds only. Their semantics do not define or replace inheritance, interface, generic, object-model, variance, or other assignability rules elsewhere in FoLang; those facilities retain their own independently defined semantics.

A declaration of the form:

```
TSub co.subtype = BaseType;
```

defines a type whose admissible values have concrete types that are **proper transitive subtypes** of `BaseType`. `BaseType` itself is excluded. Direct and indirect subtypes are included, while unrelated types are excluded.

To admit `BaseType` itself together with those proper subtypes, form an explicit union:

```
TSubPlusBase co.type = TSub | BaseType;
```

A declaration of the form:

```
TSuper co.supertype = BaseType;
```

defines a type whose admissible values have concrete types that are **proper transitive supertypes** of `BaseType`. `BaseType` itself is excluded. Direct and indirect supertypes are included, while unrelated types are excluded.

To admit `BaseType` itself together with those proper supertypes, form an explicit union:

```
TSuperPlusBase co.type = TSuper | BaseType;
```

Canonical Composite Construction

A list, array, set, map, tuple, struct, class, or other composite runtime value is constructed with an explicit concrete type followed immediately by a braced initializer. Braces are the single construction delimiter for every composite category. The rule applies wherever the construction expression occurs; it is not limited to assignment right-hand sides. Scalar literals such as `10`, `3.14`, `"text"`, and `'A'` remain ordinary bare literals and do not require a type prefix:

```
b := B{age: 25.0};
emp := Employee{name: "Rao", id: 1};
point := Point{x: 10.0, y: 20.0};
```

Object field initializers use `:` between the field name and value and `,` between fields. Map entries likewise use `:` between key and value. Lists, arrays, and sets contain comma-separated element expressions. The resolved type determines which entry form is valid; punctuation does not classify the collection. `=` is not an object-field or map-entry binder.

There is no untyped composite runtime literal. Thus `Employee{name: "Rao"}` is valid, while `{name: "Rao"}`, `[]`, `(...)` used as a composite value, and an unqualified collection body are not construction expressions.

Built-in generic collection types are named before use, exactly like other complete type expressions. A construction expression uses that named alias and does not repeat the arrow tail:

```
StringList  co.type = co.List(co.string);
IntSet      co.type = co.Set(co.int);
StringIntMap co.type =
    co.Map(co.string, co.int);

x StringList = StringList{"A","B","C"};
y IntSet = IntSet{1,2,3};
map StringIntMap = StringIntMap{"A":1,"B":2};

// Type-deduced variables still construct through the named type.
x2 := StringList{"A","B","C"};
y2 := IntSet{1,2,3};
map2 := StringIntMap{"A":1,"B":2};
```

Bare `{...}`, `[...]`, and `(...)` forms do not construct composite runtime values, even when the destination variable has an explicit declared type. Both inferred and explicitly typed declarations repeat the construction type:

```
children := SyntaxNodeList{};
children SyntaxNodeList = SyntaxNodeList{};

children := {};                // invalid: no construction type
children SyntaxNodeList = {};  // invalid: the expression still has no
                                type prefix
```

`Alias{...}` is interpreted from the declaration to which `Alias` resolves.

`Alias[...]` remains indexing and `Alias(...)` remains invocation. Construction never performs generic instantiation itself.

Only a concrete named specialization may construct a generic collection value; an unspecialized built-in generic collection name cannot prefix a runtime construction.

Let Bindings

```
y co.int = let({x = 10}).in({x + 1});
y co.int = let({$ = 10}).in({$ + 1}); // $ refers to the value being
defined

offset := 100;

let adjust(0) = offset;
let adjust(n) = n + offset;
```

`$` is a special identifier usable in ordinary `let` binding expressions for recursive expressions that refer to their current binding. Ordinary value binding uses only the `let(...).in(...)` form; `.where(...)` is not a value-binding spelling and retains only its separately defined type-predicate/refinement roles.

Ordinary `let` value-binding expressions remain available in language contexts that permit them, but they are forbidden directly in the application entry file. In the entry file, `let` is reserved for named `let` function-pattern groups. A `let` function-pattern group may capture zero or more surrounding runtime bindings;

zero capture is valid and does not create a separate non-capturing function-pattern form. It cannot introduce an anonymous function, a general closure value, or a curried function.

Let Function Patterns

A `let` function-pattern group is the single FoLang form for defining a named callable by pattern-dispatched clauses. FoLang has no separate bare `f(pattern) => ...` function-pattern declaration. The `let` form covers both non-capturing and capturing groups.

```
Option(T) co.type = co.variants(Some(T), None);

// Zero captures.
let f(Some(x)) = x + 1;
let f(None)    = 0;

// One capture.
offset := 100;
let adjust(0) = offset;
let adjust(n) = n + offset;
```

A clause may use either a concise expression body or a direct block body. An expression-bodied clause returns the value of its right-hand expression and is terminated by `;`. A block-bodied clause uses the ordinary callable-result control form `this =>` and terminates at its closing `}` without a trailing semicolon:

```
let g(Some(x)) = {
    trace(x);
    this => x + 1;
}

let g(None) = {
    this => 0;
}
```

All clauses with the same function-pattern name form one pattern-dispatch family. Pattern bindings introduced by a clause are local to that clause. A reference to an already initialized runtime binding in an enclosing lexical scope is a capture; using an identifier never implicitly creates a local binding. If a clause pattern introduces a

binding with the same spelling as an enclosing binding, the clause-local pattern binding is the binding referenced within that clause. The function-pattern family's own name is visible to its clauses for recursion and is not an enclosing capture.

A group may therefore capture zero, one, or many enclosing runtime bindings. Capture count does not change the source construct or its pattern-dispatch semantics. A conforming implementation may represent a zero-capture group as an ordinary callable and a capturing group with an environment or another equivalent mechanism, but that representation is not observable language semantics.

In the application entry file, `let` function-pattern groups are restricted entry-local dispatch helpers even though ordinary function declarations are forbidden there. They cannot be imported, exported, returned, stored as ordinary function values, passed as ordinary callbacks, curried, partially applied, or used as general closure values.

The former bare spelling `f(pattern) => ...` is not FoLang function-pattern syntax. `=>` retains only its other explicitly defined meanings, such as lambda/match-case syntax and the `this =>` callable-result control form; `=>>` remains function delegation. FoLang has no separate expression-form closure/curry operator.

In FoLang, file-backed primary declarations use their own `<Name>.fo1` files. Package functions and non-UDT type declarations are grouped in any number of `*.unit.fo1` files, while struct-associated behavior is placed in `<StructName>.comp.unit.fo1`. These are all [package source files](#).

Project Layout

This section is the **single canonical definition of FoLang project filesystem structure**. It defines the project-root domains, their allowed direct contents, structural source files, component slots, package roots, and artifact locations. Later sections describe application, library, component, packaged/export, operator, import, and compilation semantics without redefining these filesystem rules; they link back here when structural context is needed.

Compilation Invocation

The compiler is invoked with the project root:

```
folangcc <project-root>
```

The basename of the canonical project-root directory is the logical project name and the default output basename.

Root Domains

A FoLang project uses four standardized root domains:

```
<project-root>/
├─ src/          <- mandatory primary project source
├─ components/   <- optional project-owned isolated/specialized source
├─ lib/          <- optional developer-managed compiled dependencies
└─ build/        <- compiler-managed generated output
```

These names are compiler-defined filesystem domains. They are not packages and never contribute package-name components.

Root domain	Presence	Allowed direct content	Not allowed
src/	mandatory, non-empty	exactly one primary surface (<code>appl.fol</code> or <code>component.fol</code>) plus package directories	both primary surfaces, no primary surface, any other direct file
components/	executable application: optional; projected application library: optional only as <code>components/operators/</code> ; every other standalone library: forbidden	application component kinds listed below, or the single <code>operators/</code> exception for a projected application library	unknown immediate child, empty component root, or any component tree forbidden by the project kind
lib/	Optional contains 3rd party libraries	zero or more direct <code><name>.folenc</code> compiled dependency artifacts	FoLang source files, or non- <code>.folenc</code> project content
build/	compiler-managed	generated frontend/backend	source/package discovery input

If `components/` is unused, it is omitted rather than created empty. `lib/` is unused omitted, when the project has no other compiled dependency. `build/` may be absent before compilation; the compiler creates/manages it as needed.

`src/` — Primary Project Source

`src/` contains exactly one primary project surface:

```
src/app1.fol
```

or:

```
src/component.fol
```

The two are mutually exclusive:

- `src/app1.fol` classifies the project as an executable application;
- `src/component.fol` classifies the project as a standalone distributable component project;
- both present -> compile-time project-layout error;
- neither present -> compile-time project-layout error.

No other file may occur directly in `src/`. Every other direct entry under `src/` must be a non-empty package directory containing valid FoLang package source. Ordinary project package dot paths begin below `src/`.

Every `src/component.fol` contains exactly one `_ co.component = { ... }` structural declaration. The source then selects one of two mutually exclusive standalone exposure models:

1. **projected library** — `@co.dap.library` annotates the component declaration. Omitted `type` means `application`; explicit projected kinds are `application`, `dynamicvmrt`, and `native`;
2. **packaged library** — no `@co.dap.library` is present, and the component body contains the applicable `@co.dap.export(packages={...})` selector. Selected `src/` package contexts are the distributable open-package surface.

`native` and `dynamicvmrt` standalone libraries are always projected and therefore require `@co.dap.library(type=...)`; they cannot use the packaged exposure model. Standalone libraries do not use project-local components as a library-composition mechanism: reusable dependencies belong in `lib/` and are consumed as standalone libraries. The sole structural exception is a projected **application** library, which may contain exactly one optional project-local component kind, `components/operators/`, because custom operator syntax must be registered while compiling that library's own source. A projected application library may not contain `components/application/`, `components/native/`, `components/dynamicvmrt/`, or `components/packaged/`. Standalone packaged, `native`, and `dynamicvmrt` libraries must not contain a `components/` tree at all. Restricted library forms may still overload FoLang-owned built-in or pre-declared operators, but they cannot create new operator spellings. Detailed standalone semantics are defined in [Libraries](#) and [Operators](#).

`components/` — Project-Owned Components

For an **executable application**, `components/` contains project-owned implementation domains that are parsed and compiled as part of that application. A project-local component is **not a library** and never produces an independent `.folenc` artifact. Its immediate folder determines its component kind before `component.fol` is parsed, so no file below `components/` uses `@co.dap.library` to identify itself.

The full component-kind set below is valid only for an executable application. A standalone projected application library has only the separate `components/operators/` exception described above; every other standalone library has no `components/` tree.

For an executable application, only these immediate child directories are valid:

```
components/  
├─ application/  
├─ native/  
├─ dynamicvmrt/  
├─ packaged/  
└─ operators/
```

Every component-kind directory contains exactly one direct structural source file named `component.fo1`, and every such file contains exactly one `_ co.component = { ... }` declaration. No alternative direct component-surface filename is valid.

Component path	Kind supplied by folder	<code>component.fo1</code> role	Descendant package directories
<code>components/application/</code>	<code>application</code>	projected ordinary-application API surface	allowed; private to component
<code>components/native/</code>	<code>native</code>	projected native/foreign-interop API surface	allowed; private to component
<code>components/dynamicvmrt/</code>	<code>dynamicvmrt</code>	projected dynamic-runtime API surface	allowed; private to component
<code>components/packaged/</code>	<code>packaged</code>	selective application-package export surface	allowed; all descendant packages are private by default; explicitly selected contexts are exposed only to the executable application's primary <code>src/</code> graph and never to peer components
<code>components/operators/</code>	<code>operators</code>	project-local custom-operator declaration surface	not allowed

Every descendant package below a component kind that permits package directories is **private to that component by default**. The `components/` tree is not an ordinary package-import root, and physical presence below it never makes a package directly importable or otherwise accessible from the owning application/project.

For projected component kinds (`application`, `native`, and `dynamicvmrt`), descendant packages remain private implementation source. Only the owning primary `src/` domain may consume APIs published by that component's `component.fo1` surface; a peer project-local component may not import or

reference that surface. For `components/packaged/`, descendant packages are likewise private by default; `@co.dap.export(...)` in `components/packaged/component.fo1` is the only mechanism that promotes selected package contexts into the executable application's primary `src/` open package graph. Those selected contexts are **application-facing only**: no other project-local component (`application`, `native`, `dynamicvmrt`, `operators`, or another component context) may import, qualify, reference, or otherwise consume them. Every unselected packaged-component package remains private and cannot be referenced from outside that component. The operator component permits no descendant package directories and contributes syntax metadata only to the permitted owning primary `src/` domain; no source under `components/<kind>/` receives that custom operator table.

A component can affect its owner only through its `component.fo1` surface:

```
projected component API
  -> exposed through the component surface; implementation packages
  remain private

packaged component export selector
  -> only explicitly selected descendant packages leave component privacy
  and join the executable application's primary src/ open package
  graph
  -> never enters any peer component's package/import graph

operator component
  -> declared custom operator spellings enter the permitted owning
  operator table
```

Executable-application project-local components are **peer-isolated**. Dependency flow is one-way from the executable application's primary `src/` domain toward component surfaces:

```
executable application primary src/
  -> components/application surface
  -> components/native surface
  -> components/dynamicvmrt surface
  -> selected components/packaged package contexts (application src only)

components/<kind>/
  -X-> components/<other-kind>/
```

No project-local component may import, qualify, reference, or transitively acquire another project-local component surface. A component may consume a standalone **projected** library through its published `library=` surface when the ordinary capability/type-boundary rules permit it, but it may not consume a standalone packaged library because packaged artifacts expose no safe projected API. This component restriction does not create a pairwise dependency-direction matrix between independently built standalone **projected** libraries.

The exact projected-component semantics are defined in [Components](#), packaged selection in [Packaged Component](#), and operator declaration rules in [Operators](#). The operator component is additionally isolated from every executable-application project-local component: its custom spellings are available only to the executable application's ordinary primary `src/` domain, or to the projected application library's ordinary primary `src/` domain under that library's sole `components/operators/` exception; they are never available to source under another `components/<kind>/` domain.

Installed Standard-Package Artifact

The compiler distribution installs the language-provided exported standard package at the fixed toolchain-relative location:

```
<install-root>/
├── bin/
│   └── folcc
└── stdlib/
    └── co.folenc
```

Private Standard Namespace and Public Projection

The standard-package source is compiled under the compiler-owned private root `f0λ.*`. That root is a canonical implementation namespace, not a developer API. Ordinary source cannot import, qualify, alias, shadow, or otherwise name `f0λ.*`. The installed artifact exposes only its projected public `co.*` names.

The standard bootstrap may project one or more internal package subtrees into one public package by using the `as=` form of `@co.dap.export`:

```
_ co.component = {
  // flatten the private language, network, and core domains directly
```

```

into co
  @co.dap.export(
    packages={f0λ.lang, f0λ.net, f0λ.core},
    as=co
  )

  // preserve a public subpackage where that organization is useful
  @co.dap.export(
    packages={f0λ.out},
    as=co.out
  )
}

```

The privileged standard-package component may contain multiple projection-form `@co.dap.export` entries. Each entry is independent. For this form, `packages` is an unordered set of canonical source package roots and `as` is one destination package root. Projection is a prefix substitution over each selected source subtree: the selected internal package prefix is removed and the destination prefix is inserted. Thus, for the example above:

```

f0λ.lang.string  -> co.string
f0λ.lang.int     -> co.int
f0λ.lang.type    -> co.type
f0λ.net.http     -> co.http
f0λ.net.tcp      -> co.tcp
f0λ.core.List    -> co.List
f0λ.core.Map     -> co.Map
f0λ.core.Set     -> co.Set
f0λ.out.println  -> co.out.println

```

Projection changes only the source-visible qualified name. It does **not** clone a declaration, create a type alias, create a second type identity, or change declaration ownership. `co.string` and the private canonical declaration `f0λ.lang.string`, for example, denote one declaration identity; ordinary source can name only `co.string`. Diagnostics and serialized public API names use the projected `co.*` spelling rather than leaking `f0λ.*`.

Multiple source package roots may project into the same destination, and separate projection entries may target different public destinations. After all prefix substitutions are applied, two distinct canonical declarations or package contexts must not produce the same public qualified name; such a collision is a standard-

package compile-time error. Source-package order and projection-entry order have no semantic effect. The `as=` projection form is privileged standard-package bootstrap metadata; ordinary packaged libraries/components continue to use the package-selection form defined in [Packaged Component](#) and [Libraries](#). An implementation may reorganize the private `f0λ.*` hierarchy without changing source compatibility so long as the exported `co.*` projection remains unchanged.

The command environment, normally `PATH`, selects which `folcc` executable runs. The running compiler then obtains its own executable path through the host operating system, resolves symbolic links to the real executable, derives `<install-root>` as the parent of its `bin/` directory, and loads `<install-root>/stdlib/co.folenc`. It must not derive this location from the current working directory, `argv[0]` text alone, a project file, a manifest, or a separately required environment variable.

The frontend must locate, validate, and deserialize this installed `co.folenc` before parsing any project source under `src/` or `components/`. This early load establishes the reserved **public** `co` package root, its projected standard type/member signatures, and runtime-operation metadata needed by later parsing and semantic analysis. Private canonical `f0λ.*` package names remain bootstrap implementation identities and are not inserted into ordinary source name resolution. Failure to locate, read, validate, or deserialize the installed artifact is a compiler-installation error and stops compilation.

The standard artifact is loaded directly from the installation and is never copied into the project. Any project dependency claiming the reserved standard-package identity, is a compile-time error; project content cannot shadow or replace the installed `co.*` packages.

`lib/` — Compiled Dependencies

`lib/` contains only developer-managed compiled FoLang dependency artifacts. It is omitted when the project has no third-party compiled dependency:

```
lib/  
├─ first.folenc  
├─ second.folenc  
└─ ...
```

A `.fo1enc` is not FoLang source and is never passed through the source parser. After the installed standard package has been loaded, the frontend validates and deserializes each project dependency's stored library/package metadata, canonical symbol tables and contexts, type hierarchy, applicable overload-family information, applicable runtime-operation markers, and applicable AST/HIR information before dependent project-source parsing. Under the standalone-library topology rules, one artifact has one primary exposure model: a projected library surface **or** packaged/open package contexts.

Developers manage only these third-party `.fo1enc` dependencies. No project artifact may claim the reserved standard-package identity.

A projected `.fo1enc` preserves its internal library boundary and may be consumed by another library/application through its published surface. A packaged `.fo1enc` instead contributes explicitly exported open package contexts and may be consumed only by an executable application's primary open graph; libraries and project-local components cannot import those packaged contexts.

`build/` — Generated Output

`build/` is compiler-managed generated output. It is never a package, component, dependency, or source-discovery root. The compiler may create it when absent and may replace or update generated contents according to the selected backend interchange contract.

Canonical Full Layout

The following full tree is the **executable-application** layout. Optional component kinds are simply absent when unused. Standalone libraries use the restricted layouts stated immediately after the tree.

```
<project-root>/
├── src/
│   ├── app1.fo1                <- executable application project
│   │   OR
│   └── component.fo1          <- standalone distributable
├── component project
│   └── <package directories>/
├── components/
│   └── application/
```



```

├── component.fol
├── <private package directories>/
├── native/
│   ├── component.fol
│   └── <private package directories>/
├── dynamicvmrt/
│   ├── component.fol
│   └── <private package directories>/
├── packaged/
│   ├── component.fol
│   └── <export package directories>/
├── operators/
│   └── component.fol
├── lib/                                <- optional third-party compiled
dependencies
│   ├── <name>.folenc
│   └── ...
└── build/
    └── <compiler-generated artifacts>

```

The tree above is illustrative for an executable application: it uses `src/app1.fol`, not `src/component.fol`, and optional directories must not be created empty.

Standalone-library layouts are stricter:

```

projected application library
├── src/component.fol
├── src/<package directories>/
├── components/operators/              optional; the only permitted component
├── lib/                              optional projected-library dependencies
└── build/

packaged | native | dynamicvmrt standalone library
├── src/component.fol
├── src/<package directories>/
├── lib/                              optional projected-library dependencies
└── build/

```

For the second group, `components/` is forbidden. A packaged standalone library also cannot consume another packaged standalone library; its reusable dependencies must be projected libraries available through `lib/`.

Project Identity and Output

The project-root basename supplies project/artifact identity; structural filenames do not supply the project name.

```
/projects/payroll/  
└─ src/appl.fol
```

```
project kind      = application  
project name      = payroll  
output basename   = payroll
```

```
/libraries/hrlib/  
└─ src/component.fol
```

```
project kind          = standalone component/library producer  
project name          = hrlib  
compiled artifact     = hrlib.folenc
```

For an application, the backend/platform determines the executable convention. A standalone `src/component.fol` project produces `<project-name>.folenc`.

Structural Source Filenames

Filename	Canonical structural location	Meaning	Package file?
<code>appl.fol</code>	direct child of application <code>src/</code>	application entry surface; the only source location permitted to contain <code>@co.pdap.*</code> pragmas	no
<code>component.fol</code>	direct child of standalone producer <code>src/</code>	standalone projected-library or packaged-library surface	no
<code>component.fol</code>	direct child of standardized <code>components/<kind>/</code>	project-owned component surface; kind comes from folder	no
<code><Name>.fol</code>	package directory	file-backed primary declaration	yes

<code><Fragment>.unit.fol</code>	package directory	ordinary package unit	yes
<code><StructName>.comp.unit.fol</code>	package directory	struct companion unit	yes

Detailed package-file grammar belongs to [Package Source Files](#); this table defines only structural placement.

Common Source Parsing Rule

FoLang source under `src/` and `components/` uses the same lexical and syntactic parser. Filesystem placement supplies the semantic source mode: ordinary/project source under `src/` and component source under `components/`. `lib/*.folenc` loading is artifact deserialization, not source parsing.

Packages

Package Identity

For canonical filesystem ownership and reserved root-domain rules, see [Project Layout](#).

Ordinary project packages are directories below `src/`; their dot paths are relative to `src/`:

The compiler-owned standard-package bootstrap is the sole exception to the ordinary project root rule: its internal package contexts are canonically rooted at reserved `f0λ.*` and are projected to public `co.*` names when `co.folenc` is built. This privileged root is never available to ordinary source.

```
/appl/src/hr/           -> package "hr"
/appl/src/hr/employee/ -> package "hr.employee"
/appl/src/auth/         -> package "auth"
```

The project root and `src/` are not packages. Structural surfaces such as `src/appl.fol`, `src/component.fol`, and `components/<kind>/component.fol` are not package source files.

All descendant packages below `components/application/`, `components/native/`, `components/dynamicvmrt/`, and `components/packaged/` are component-private by default. Projected-component packages remain inaccessible outside their component and are represented externally only through APIs published by `component.fol`. Packaged-component packages remain equally private unless their package contexts are explicitly selected by `components/packaged/component.fol`; only those selected contexts enter the executable application's primary `src/` open package graph. They are not inserted into any peer component's package graph. The operator component creates no package namespace.

Multi-File Packages

Multiple `.fol` files in the same package directory automatically belong to that package:

```
src/hr/employee/  
├─ Employee.fol      -> hr.employee  
├─ EmpService.fol    -> hr.employee  
└─ EmpValidator.fol  -> hr.employee
```

Package Names and Import Aliases

For ordinary project, component, and third-party library source, FoLang package identity is derived directly from the physical directory path below the applicable package root. There is no source-level package declaration, package-metadata file, or logical package-name override. The compiler-owned standard-package bootstrap is deliberately separate: it assigns the reserved canonical root `f0λ.*` to its private package contexts and then publishes selected subtrees through `@co.dap.export(packages={...}, as=...)`.

For example:

```
/appl/src/hr/empl/Employee.fol  
-> package hr.empl  
-> declaration hr.empl.Employee
```

The canonical import therefore uses the folder-derived package path:

```
@co.ddap.import(package="hr.empl", as="emp")
```

`as=` is optional and creates only a file-local import alias. It does not rename the package, change its canonical identity, or alter package resolution. With the alias above, `Employee` may be referenced as `emp.Employee`; without an alias, the complete imported package path is used according to the ordinary import rules.

If the canonical package name must change from `hr.empl` to `hr.emp`, the directory itself must be renamed from `src/hr/empl/` to `src/hr/emp/`. The filesystem and package namespace therefore cannot diverge.

FoLang defines no `co.package` declaration kind and no reserved `package.fo1` metadata form. A file named `package.fo1`, if otherwise legal, has no structural package meaning and is classified by the ordinary `<Name>.fo1` filename rule.

UDT (User defined Data types)

FoLang provides the following user-defined data declaration kinds:

1. cstructs
2. structs
3. unions
4. enums
5. classes
6. modules
7. interfaces
8. signatures

Each ordinary file-backed primary declaration is placed in its own `<Name>.fo1` file. The declaration name is never repeated in source; `_` occupies the declaration-name position and the compiler derives the name from the filename.

For canonical declaration-usage rules for structs, cstructs, unions, enums, classes, interfaces, and related primary declarations, see [Unused Symbols, Liveness, and Reachability](#).

Built-in UDT kinds are defined in [Built In Kinds](#).

Struct Declaration

```
// Employee.fol
_ co.struct = {
  id    co.int;
  name  co.string;
}
```

More about structs: [Structs in detail](#).

C-Struct Declaration

`co.cstruct` is a C-like value type: it is passed by value, has a simple memory layout, and is safe to cross supported ABI boundaries.

```
// Point.fol
_ co.cstruct = {
  x co.int;
  y co.int;
}
```

```
// Rect.fol
_ co.cstruct = {
  origin Point;
  width  co.int;
  height co.int;
}
```

Enum Declaration

`co.enum` declares a closed tagged algebraic data type whose body contains only **enum states**. A state may declare zero or more typed parameters. Every parameter of a parameterized enum state is intrinsically **named**: each parameter declaration must contain both its name and its type. Enum states do not support positional or unnamed payload declarations.

```
// Status.fol
_ co.enum = {
```

```
Active,  
Inactive,  
Failed(code co.int, message co.string)  
}
```

A state with no parameters denotes an enum value directly and is written without call parentheses. A state with parameters behaves as a compiler-defined **state function**. Every state-function argument must be supplied by its declared parameter name using `name=value`; positional arguments are invalid. Named arguments may be written in any order, but every required state parameter must be supplied exactly once. Every resulting value has the enclosing enum as its static type; the state name is not an independent type.

```
current Status = Status.Active;  
failed Status = Status.Failed(code=500, message="Internal Error");  
alsoFailed Status = Status.Failed(message="Internal Error", code=500); //  
same named payload  
  
// invalid: enum state functions never accept positional payload arguments  
// failed Status = Status.Failed(500, "Internal Error");
```

The `~` marker used by ordinary named function parameters is unnecessary for enum state parameters: enum state-function parameters are named by definition. This enum-specific rule is stricter than ordinary function calling and does not permit a positional fallback.

FoLang does not introduce a constructor declaration category for enum states. In type-theory literature, a parameterized enum state may be described as a data constructor, but the normative FoLang terms are **state** and **state function**.

Union Declaration

```
// NumberOrText.fol  
_ co.union = {  
    intValue co.int;  
    strValue co.string;  
}
```

Class Declaration

```
// Employee.fol
_ co.class = {
  getEmployeeDetails()->(Employee) = empmodule.getEmployeeDetails;

  getEmployeeInfo()->(Employee) =>> empmodule.getEmployeeDetails();
  // delegating – internally redirecting the call to module function
}

// $1, $2, $3 ... are return components of the immediately previous chained
function
//Emp.fol
_ co.class = {
  dosomething(a co.int, b co.int)->
  (co.int)=>>somePack.someMethod(a)=>>someOthPack.someOtherMeth($1, b);
}
```

More about classes: [Classes in detail](#).

For the canonical class usage rule, including the distinction between developer-authored methods and interface-required methods, see [Unused Symbols, Liveness, and Reachability](#).

Interface vs Signature

```
// EmployeeContract.fol
_ co.signature = {
  ...
}
```

More about signatures: [signatures in detail](#).

```
// EmployeeApi.fol
_ co.interface = {
  ...
}
```

More about interfaces: [interfaces in detail](#).

Module Declaration

```
// Employee.fol
_ co.struct = {
    ...
}
```

```
// EmployeeModule.fol
_ co.signature = {
    ...
}
```

```
// EmployeeModImpl.fol
_ co.module->(matches=EmployeeModule) = {
    ...
}
```

More about modules: [Modules in detail](#).

Extension Declarations

A `co.extension` is a reusable collection of fully implemented methods that adds behavior to one explicitly selected class **without creating a subclass or changing the target class's nominal type identity or inheritance hierarchy**. The extension chooses its target through the mandatory `fortype` argument; the target class does not adopt the extension through `@co.dap.oops`.

```
// EmployeeExtension.fol

_ co.extension->(fortype=somePkg.Employee) = {

    someFun()->>() = {
        co.out.println(this.someName);
    }

    @co.dap.class
    someOtherFun()->>() = {
        co.out.println(this.clsVariable);
    }
}
```

```
}  
}
```

`fortype=somePkg.Employee` fixes `somePkg.Employee` as the extension target while the extension is compiled. Therefore receiver-dependent references are resolved against that target rather than deferred to a later class-adoption step:

```
unannotated method  
  this -> instance of fortype  
  
@co.dap.class method  
  this -> class/type context of fortype
```

An extension contributes callable behavior only. It does **not**:

- create a new nominal type;
- establish an `is-a` or subtype relationship;
- alter the target's inheritance hierarchy;
- make the target inherit extension state; or
- change subtype, substitution, overload, or virtual-dispatch relationships merely because the extension exists.

The distinction is semantic: extension membership does not carry the nominal subtype and substitution semantics of subclassing. For example, contributing `toCsv()` to `Employee` does not create a separate `CsvEmployee` subtype; `Employee` remains the same nominal type.

An extension may target a class owned by another package/library and may be maintained outside the target class source. Extension declarations are not required merely for source organization; behavior declared directly in an owning class remains ordinary class behavior.

Traits

// EmployeeTrait.fol

```
_ co.trait={  
  
  @co.dap.abstract
```

```

someFunction()->();

somerealFun()->()={
    co.out.println("Doing real work" );
}

}

```

A trait is interface-like but may provide default function implementations. A trait carries no instance state.

A class participating in the `@co.dap.oops` model includes a trait by listing it in the annotation's `traits=[...]` field.

A consuming class must implement every abstract function that remains unsatisfied.

Mixins

//EmployeeMixin.fol

```

_ co.mixin={

    someNum co.int;

    someFun1()->()={
        co.out.println(someNum);
    }

    @co.dap.abstract
    someotherFun()->();

    @co.dap.virtual
    someVirtFun()->()={
        ...
    }
}

```

```
}
```

A mixin is the dedicated abstract-class-like composition form; it avoids declaring an ordinary class merely to mark it abstract.

A mixin may contain state, abstract methods, fully implemented methods, and virtual methods where the mixin rules permit them.

A class incorporates a mixin by listing it in the `mixins=[...]` field of `@co.dap.oops`, implements its abstract methods, and may override its virtual methods.

FoLang has no abstract-class declaration. A mixin supplies the corresponding stateful abstract-composition capability through fields, abstract methods, concrete methods, and virtual methods. A trait supplies stateless behavioral composition and may provide default method implementations that interfaces do not provide.

Every declaration whose kind is `co.class` has class construction semantics. FoLang has no annotation or class subtype that turns a declared class into an abstract or unconstructable class. Ordinary lifecycle, constructor-signature, and accessibility rules still determine which construction expression is valid at a particular source location.

`@co.dap.sealed` changes inheritance or overriding availability, not constructability. On a class it prevents that class from being selected as a parent by another class. On a method it prevents overriding that method. A sealed class remains a constructable class, and a sealed method remains an ordinary callable method.

A `co.mixin` is not an unconstructable class. It is a different declaration kind whose semantics are composition into a consuming class; it has no independent class-instance construction operation.

`@co.dap.abstract` and `@co.dap.virtual` are permitted only on methods declared by traits or mixins. `@co.dap.override` is permitted only on methods declared by classes, traits, or mixins, and only when the method resolves an applicable inherited

or composed method with a compatible normalized signature.

Folang Traits and mixins provide abstract composition

Units

A unit is a stateless source container. A package may contain any number of ordinary unit files, and all their members are consolidated directly into the package namespace.

```
arithmetic.unit.fol  
conversion.unit.fol  
optional.unit.fol
```

Each ordinary unit file contains:

```
_ co.unit = {  
    ...  
}
```

The filename is organizational only. It does not create a unit symbol or another qualification level. For package `math`, a function `abs` declared in any ordinary unit is accessed as `math.abs(...)`, not `math.Arithmetic.abs(...)`.

A companion unit uses the reserved filename form `<StructName>.comp.unit.fol`. Its members attach to the matching same-package struct:

```
Employee.fol  
Employee.comp.unit.fol
```

Only `co.struct` supports a companion unit. See [Units in detail](#) and [Struct Companion Units](#).

Matchers

Custom Matcher

```
// PositiveEvenMatcher.fol
@co.dap.matcher
_ co.matcher->(type=co.int) = {
  matchCase(
    value    co.int,
    pattern  co.untyped
  )->(co.int, co.MatchBindings) = {
    // user logic
    // 0 = no match, >0 = match
  }
}
```

A matcher declaration supports exactly one matched-subject type, specified by `type=`. It must declare exactly one protocol function named `matchCase`; overloading `matchCase` inside a matcher is not permitted. A matcher for another subject type must be declared as a separate `<Name>.fol` primary declaration.

At compile time, the compiler resolves the type named by `type=` and the type of the first `matchCase` parameter. The two resolved types must be equivalent. Type aliases are compared after alias resolution. The second parameter must have type `co.untyped`, and the result must be exactly `(co.int, co.MatchBindings)`.

```
// InvalidMatcher.fol
_ co.matcher->(type=co.int) = {
  matchCase(
    value    co.string,
    pattern  co.untyped
  )->(co.int, co.MatchBindings) = {
    ...
  }
  // compiler error: declared matcher type and first parameter type
  differ
}
```

The parameter names themselves are not significant; their order and resolved types define the protocol shape.

Matcher liveness is defined in [Unused Symbols, Liveness, and Reachability](#).

Comprehensions

//comprehensionseg1.unit.fol

```
_ co.unit = {
  IntList      co.type = co.List(co.int);
  IntSet       co.type = co.Set(co.int);
  StringIntMap co.type =
    co.Map(co.string, co.int);

  someFun()->() = {
    k := (1 .. 10).filter(|x| => x % 2 == 0).map(|x| => x * x);

    result := for (x <- IntList{1,2,3}).yield(x * 2); // IntList{2, 4,
6}
    result := for (x <- IntSet{1,2,3}).yield(x * 2);  // IntSet{2, 4,
6}
    result := for (x <- Some(5)).yield(x * 2);        // Some(10)
    result := for (x <- fetchData()).yield(x.process()); // Future

    ages := StringIntMap{"A":30,"B":40,"c":66,"e":88};
    upper := for ((name, age) <- ages).yield(name.toUpperCase, age);
  }
}
```

Comprehension Semantics

A FoLang comprehension is a source-driven transformation. Its canonical form is:

```
result := for (pattern <- source).yield(resultExpression);
```

The core comprehension syntax defines the binding and transformation structure; the source type defines the source-specific comprehension behaviour. The syntax does not implicitly convert every source to a `List`, and an arbitrary value does not become a valid comprehension source merely because it appears to the right of `<-`. A source is valid only when it is a FoLang iterable, `Some(T)`, or `Future(T)`. No other non-iterable source category can acquire comprehension capability through extensions, ordinary package APIs, operators, or similarly named methods.

Permitted Comprehension Sources

FoLang comprehensions intentionally accept only the following source categories:

1. **Any FoLang iterable**, including standard iterable forms such as arrays, lists, sets, maps/dictionaries, and ranges.
2. `Some(T)`.
3. `Future(T)`.

This set is closed. Comprehension support is **not** an open extension mechanism. No other non-iterable source category participates in `for ... yield`.

For iterable sources, the comprehension consumes the values exposed by the source's ordinary iteration semantics. The precise traversal order, entry shape, duplicate behaviour, and result-container behaviour remain those defined for that iterable type.

`Some(T)` and `Future(T)` are the only permitted non-iterable comprehension sources. They do not become iterable; instead, `for ... yield` applies their language-defined transformation semantics.

For example:

```
for (x <- IntList{1,2,3}).yield(x * 2); // valid: iterable
for (x <- 1 .. 10).yield(x * 2);        // valid: iterable range
for ((k, v) <- valuesMap).yield(k, v);  // valid: iterable map/dictionary
for (x <- Some(5)).yield(x * 2);        // valid: permitted non-iterable
source
for (x <- someFuture).yield(f(x));      // valid: permitted non-iterable
source
```

A struct, class, module, or other UDT that is **not iterable** is not a valid comprehension source. Defining `map`, `each`, extensions, operators, or similarly named functions does not make such a type comprehension-capable.

```
emp Employee = ...;

for (x <- emp).yield(x.salary + 1000); // compiler error if Employee is not
iterable
```


A user-defined type may therefore appear as a comprehension source only when it participates in FoLang's ordinary iterable model. It cannot define a new non-iterable comprehension meaning comparable to `Some(T)` or `Future(T)`.

The parts of the form have the following meanings:

- `source` is the value being traversed or transformed;
- `pattern` introduces the comprehension-local binding or destructuring pattern for each value produced by the source;
- `<-` associates that binding pattern with the source;
- `.yield(...)` supplies the result expression evaluated for each participating source value according to that source type's comprehension semantics.

Bindings introduced by `pattern` are local to that comprehension. They are visible in the corresponding `yield` expression and do not introduce names into the enclosing scope. A destructuring pattern, such as `(name, age)`, introduces each successful component binding in the same comprehension-local scope.

The result shape is source-defined rather than selected by a universal core-language conversion rule. The examples above establish the following current forms:

```
F(A) --yield B--> F(B)  // abstract collection-family relationship
Some(A)  --yield B--> Some(B)
Future(A) --yield B--> Future(B)
```

For example:

```
result := for (x <- IntList{1,2,3}).yield(x * 2);
// IntList{2, 4, 6}

result := for (x <- IntSet{1,2,3}).yield(x * 2);
// IntSet{2, 4, 6}

result := for (x <- Some(5)).yield(x * 2);
// Some(10)

result := for (x <- fetchData()).yield(x.process());
// Future
```

The `Map` form demonstrates source destructuring and pair production:

```
ages := StringIntMap{"A":30,"B":40,"c":66,"e":88};
upper := for ((name, age) <- ages).yield(name.toUpperCase, age);
```

Here `(name, age)` deconstructs the source entry for the current comprehension step. The result-container rules, duplicate-key behaviour, ordering, and other map-specific properties are those defined by the applicable `Map` API rather than by the core `for ... yield` grammar.

Source-specific semantics also govern cardinality, emptiness, deferred execution, failure propagation, ordering, duplicate handling, and similar behaviour. Thus the `Some` and `Future` forms preserve the source abstraction shown by their examples, while the precise empty/failure/scheduling behaviour belongs to the corresponding type's defined API semantics. The comprehension syntax itself does not introduce implicit blocking, concurrency, retries, or error suppression.

The current core `for (pattern <- source).yield(...)` form does not define an inline filter clause. Filtering is expressed separately through the source's supported operations, for example:

```
k := (1 .. 10)
    .filter(|x| => x % 2 == 0)
    .map(|x| => x * x);
```

If a later source-specific comprehension form defines filtering, its filter conditions are evaluated before its result expression as required by the general evaluation-order rules. No filter is implied by the core `for ... yield` syntax shown above.

A comprehension does not imply parallel or concurrent traversal. Execution is sequential unless the selected source semantics or an explicitly requested FoLang execution model states otherwise.

Extension Methods

Extension functions may be declared in an ordinary package unit:

```
// string_extension.unit.fol
_ co.unit = {

    @co.dap.extension(fortype=co.string, what=extends)
```

```

    upperCase()->(co.string) = {
        this => this.upper();
    }

    @co.dap.extension(fortype=[co.string], what=overrides)
    equals(str co.string)->(co.bool) = {
        this => this == str;
    }
}

```

Because ordinary unit filenames create no symbol, extension activation identifies the contributing package, not the unit file.

For the current package, omit `from`:

```

@co.ddap.use(methods=[equals, upperCase])
k.upperCase();

```

For another package, use its alias or complete package path:

```

@co.ddap.import(package="text.util", as="tu")

@co.ddap.use(from="tu", methods=[upperCase])
@co.ddap.use(from="text.util", methods=[upperCase])

```

See [Activating Instance Methods](#) for activation of typeclass instances.

`@co.dap.extension(fortype=..., what=...)` on a unit function is distinct from a `co.extension` declaration. The function-level form attaches an individual function to a supported existing target type and therefore identifies its receiver owner in the annotation. The declaration-level `co.extension->(fortype=Class)` form groups reusable implemented methods for one explicit class target.

`@co.dap.extension` may be combined with `@co.dap.operator` when an operator implementation is contributed to an existing type. In that combination, `@co.dap.operator` classifies the callable as an operator overload and `@co.dap.extension` supplies the existing target/owner. The operator declaration itself is never generic: attaching `@co.dap.generic` to the same declaration is a compile-time error.

When the extension target is a generic type declaration, operator ownership is associated with the target's canonical declaration identity, such as `List` or `Set`, rather than with a new operator-level generic parameter. For an operator extension, `fortype=` therefore identifies that canonical target declaration; it does not declare or infer the target's element/type arguments for the operator. Different instantiations of the same generic owner remain instances of that same operator-owning type.

Neither extension mechanism creates a subtype merely to obtain additional methods or operators. In particular, class extension behavior does not require subclassing and therefore does not introduce an accidental `is-a` relationship, substitution contract, or inheritance-hierarchy change.

Modules and other declaration kinds do not acquire class-extension semantics merely by using these forms. An overriding extension method cannot override a method sealed with `@co.dap.sealed`.

Reflections

The `co.meta` reflection form shown below is a dynamic-runtime facility and is valid only inside a `dynamicvmrt` capability domain; it does not grant `co.meta` access to ordinary application or packaged code.

```
@co.dap.reflection(enable=true, package="co.meta")

x co.int = 10;
x.reflect().getType();    //co.int
x.reflect().getValue();   //10;
x.reflect().getKind();    // value
```

Type Classes

Monads, Applicatives, Functors, Monoids and Transformers

`@co.dap.typeclass(kind=..., shape=(...), aliases=[...])` is the single annotation for all typeclass definitions. `kind` specifies the algebraic structure — `Functor`, `Applicative`, `Monad`, `Monoid`, `Transformer`, or any user-defined kind. `shape` declares the ordinary or higher-kinded type parameters. Optional `aliases` name complex type representations used by the contract's function specifications. Instances of any typeclass always use `co.instance`.

Typeclass and instance liveness is defined in [Unused Symbols, Liveness, and Reachability](#).

In a file-backed typeclass declaration, `_` is only the filename-derived declaration-name placeholder. Typeclass parameters occur exclusively in the built-in annotation's `shape=(...)` option; declaration-head spellings such as `_ (F(_)) co.typeclass` are invalid. A parameter such as `T` denotes an ordinary type, while `F(_)` denotes a unary parameterized type and `G(_, _)` denotes a binary parameterized type. Otherwise-unbound type variables introduced in an operation signature, such as `A` and `B`, are implicitly universally quantified within that operation. Built-in metadata may classify syntax in this way; custom annotations cannot introduce a new declaration shape.

The optional `aliases=[{name=..., type=...}]` field follows the same alias rule as `@co.dap.generic`: an entry only gives a shorter name to its type expression and does not introduce a type, generic marker, nominal identity, or specialization rule. A typeclass alias is visible throughout that typeclass contract. Free type variables in an alias, such as `A` and `B` in `(A)->(B)`, are bound by the operation specification that uses the alias, just as they would be when the same type expression was written inline. This permits typeclass operations to follow the alias-first rule used by generic and ordinary function signatures.

An instance automatically receives the alias vocabulary of the typeclass named by its `for=` option. The instance repeats the alias name in the corresponding method signature; it does not redeclare the alias and must not replace it with an inline type expression. During typeclass-instance checking, the compiler specializes the alias by

the instance's `type=` or `types=[...]` binding. For example, `FlatMapFunction = (A)->(F(B))` becomes `(A)->(Option(B))` in an `Option` instance. The serialized typeclass annotation, its owned alias symbols, and the instance's `for=` identity preserve this relationship for semantic resolution; it is not textual substitution performed by the syntax parser.

Typeclass-instance conformance is checked after that specialization. The number of `type=` / `types=[...]` bindings must equal the number of parameters in `shape=(...)`. Every required operation and overload must be implemented with the contract's parameter-type and result-type alias names, and every such alias must specialize successfully. A missing operation, an extra operation, a mismatched parameter or result signature, an unknown typeclass, a shape-binding arity mismatch, or a cyclic/unresolved contract alias is a compile-time `SignatureConformanceFailure`.

Typeclass contracts use parameterized-type application notation such as `F(A)` and `G(B)`. `F(_)` and `G(_)` declare the required parameterized-type shapes; `F(A)` and `G(B)` apply those abstract parameters to ordinary type arguments. When an instance binds such a parameter to a concrete constructor, the abstract application is lowered to the concrete constructor's instantiation syntax. For example, `type=co.List` binds `F=co.List`, so `F(A)` specializes to `co.List(A)` and `F(B)` specializes to `co.List(B)`. Parentheses are the uniform type-application notation for abstract parameters, typeclasses, parameterized types, and annotation-declared or built-in generic types.

Functor

```
//Functor.fol
@co.dap.typeclass(
  kind=Functor,
  shape=(F(_)),
  aliases=[
    {name=MapFunction,    type=(A)->(B)},
    {name=InputContainer, type=F(A)},
    {name=ResultContainer,type=F(B)}
  ]
)
_ co.typeclass = {
  map(value InputContainer, f MapFunction) -> (ResultContainer);
}
```

```
// ListFunctor.fol
_ co.instance->(for=Functor, type=co.List) = {
  map(value InputContainer, f MapFunction)->(ResultContainer) = {
    result := ResultContainer{};
    value.each(_, item, { result.append(f(item)) });
    this => result;
  }
}
```

For this instance, the binding is explicit: `F = co.List`. Therefore the abstract contract occurrences `F(A)` and `F(B)` specialize to `co.List(A)` and `co.List(B)` in the instance signature. `A` and `B` remain the element/result type variables supplied by the operation signature; they are not part of the `type=` binding.

Applicative

```
//Applicative.fol
@co.dap.typeclass(
  kind=Applicative,
  shape=(F(_)),
  aliases=[
    {name=MapFunction,      type=(A)->(B)},
    {name=FunctionContainer,type=F(MapFunction)},
    {name=InputContainer,   type=F(A)},
    {name=ResultContainer,  type=F(B)}
  ]
)
_ co.typeclass = {
  pure(x A) -> (InputContainer);
  apply(fab FunctionContainer, fa InputContainer) -> (ResultContainer);
}

// OptionApplicative.fol
_ co.instance->(for=Applicative, type=Option) = {
  pure(x A)->(InputContainer) = { this => Some(x); }
  apply(fab FunctionContainer, fa InputContainer)->(ResultContainer) = {
    this => (fab, fa)
      .match()
      .case((Some(f), Some(x)) => Some(f(x)))
      .default(None);
  }
}
```

Monad

```
//someunit.unit.fol

_ co.unit = {

    Option(T) co.type = co.variants(Some(T), None);

}

//Monad.fol
@co.dap.typeclass(
    kind=Monad,
    shape=(F(_)),
    aliases=[
        {name=InputContainer, type=F(A)},
        {name=ResultContainer, type=F(B)},
        {name=FlatMapFunction, type=(A)->(ResultContainer)}
    ]
)
_ co.typeclass = {
    pure(x A) -> (InputContainer);
    flatMap(fa InputContainer, f FlatMapFunction) -> (ResultContainer);
}

// OptionMonad.fol
_ co.instance->(for=Monad, type=Option) = {
    pure(x A)->(InputContainer) = { this => Some(x); }
    flatMap(fa InputContainer, f FlatMapFunction)->(ResultContainer) = {
        this => fa.match().case(Some(x) => f(x)).default(None);
    }
}
```

Monoid

`Monoid` needs no `aliases=` entry because its contract contains no derived or inline function type: `T` is already the shape parameter used directly by its operations. Adding a second name for `T` would provide no simplification.

```
//Monoid.fol
@co.dap.typeclass(kind=Monoid, shape=(T))
_ co.typeclass = {
    empty() -> (T);
    combine(a T, b T) -> (T);
}
```



```

}

// IntMonoid.fol
_ co.instance->(for=Monoid, type=co.int) = {
  empty()->(co.int) = { this => 0; }
  combine(a co.int, b co.int)->(co.int) = { this => a + b; }
}

```

Transformer

```

//Transformer.fol
@co.dap.typeclass(
  kind=Transformer,
  shape=(F(_), G(_)),
  aliases=[
    {name=MapFunction,    type=(A)->(B)},
    {name=InputContainer, type=F(A)},
    {name=ResultContainer,type=G(B)}
  ]
)
_ co.typeclass = {
  map(value InputContainer, f MapFunction) -> (ResultContainer);
}

// ListToSetTransformer.fol
_ co.instance->(for=Transformer, types=[co.List, co.Set]) = {
  map(value InputContainer, f MapFunction)->(ResultContainer) = {
    result := ResultContainer{};
    value.each(_, item, { result.insert(f(item)) });
    this => result;
  }
}

```

Using an Instance

An instance is selected **by name**. There is no implicit search.

```

@co.ddap.import(package="abc.tc", as="tc")

IntList co.type = co.List(co.int);
xs IntList = IntList{1, 2, 3};
double(x co.int)->(co.int) = { this => x * 2; }

```

```
ys := tc.ListFunctor.map(xs, double);
```

`map` takes the container as its first argument, exactly as the typeclass declares it. The call names `ListFunctor`, so the compiler resolves it the same way it resolves any other imported declaration.

FoLang does **not** search visible packages for an instance that happens to match a type. A typeclass is a contract, an instance is a named implementation of that contract, and the caller names the one it means. Nothing is inferred, so an unresolved call reports a name the developer actually wrote rather than a failed search they never saw.

Method syntax such as `xs.map(f)` is unrelated to typeclasses. It calls an associated function declared in a companion unit for the type. Companion units live in the type's own package, so method calls are unambiguous by construction. A companion function and a typeclass instance may both be named `map`; they are different declarations and are reached differently.

Activating Instance Methods

An instance may also be **activated**, which makes its functions callable as methods on the receiver. Activation uses the same directive that activates extensions.

```
@co.ddap.import(package="abc.tc", as="tc")
@co.ddap.use(from="tc.ListFunctor", methods=[map, reduce])

ys := xs.map(double);           // resolves to tc.ListFunctor.map(xs, double)
```

Activation is explicit and file-scoped. `@co.ddap.use` is a directive and therefore appears only in the source-file metadata preamble. Importing a package does **not** activate anything in it, so adding an import can never change how an existing call resolves.

`methods` and `from`

`methods` selects the functions to activate.

For extension functions contributed by ordinary package units:

- omit `from` to select the current package
- use an imported package alias to select another package
- use the complete package path when no alias exists

```
@co.ddap.use(methods=[upperCase])           // current package
@co.ddap.use(from="tu", methods=[upperCase]) // imported package alias
@co.ddap.use(from="text.util", methods=[upperCase]) // complete package path
```

Ordinary unit filenames are not accepted by `from`, because they do not create symbols.

For a typeclass instance, `from` continues to name the instance declaration:

```
@co.ddap.use(from="tc.ListFunctor", methods=[map, reduce])
```

Listing names is optional. Omit `methods` to activate every eligible method from the selected package or instance; provide it to activate a subset. Conflict detection remains receiver-aware and file-scoped.

How a method call resolves

For `xs.map(f)`, where `xs` has ordinary concrete type `co.List(A)`:

1. a class method or companion-unit function on `co.List`
2. an activated extension for `co.List`
3. an activated instance function whose typeclass declares `map` with the receiver as its first parameter
4. otherwise, an error

The first match wins. A type's own declarations therefore always take precedence over anything activated into scope, and no activation can silently replace behaviour the type already defines.

Within one source file a given method name may be activated at most once for a given receiver type. Activating `map` for `co.List` from two sources is an error at the second `@co.ddap.use`, which names both. The conflict is reported where the activation is written, never at a distant call site.

The parser's built-in-method classification is only a lookup candidate. If the frontend recognizes a control-chain shape before receiver-aware lookup, it must retain the complete original member/call chain. A receiver-owned, companion, extension, or activated-instance declaration that wins the order above restores/remains the ordinary method call represented by that chain. An early dedicated control-flow node is therefore only a lowering candidate and becomes final only when the built-in meaning wins.

Activation never affects generic code. A function polymorphic over a typeclass takes the instance as an ordinary parameter and calls it by name.

A Typeclass Is a Type

A typeclass may be used wherever a type is expected. Its values are its instances. This is the same relationship a signature has to a module:

```
mm EmployeeModule = EmployeeModImpl;      // signature as type, module as
value

ListFunctorType co.type = Functor(co.List);
f ListFunctorType = tc.ListFunctor;      // named typeclass type, instance
value
```

`Functor(co.List)` is the complete typeclass application and is therefore legal on the `co.type` right-hand side. `ListFunctorType` is the ordinary type used by variables, fields, parameters, receivers, and function results. Writing `f Functor(co.List) = tc.ListFunctor;` directly is a compiler error. The same alias-first rule applies to every concrete typeclass application, including `Applicative(...)`, `Monad(...)`, `Monoid(...)`, `Transformer(...)`, and user-defined typeclasses. Abstract contract applications such as `F(A)` are unchanged: `F` is a parameterized-type placeholder declared by the typeclass contract, not a concrete typeclass application.

An instance is therefore an ordinary first-class value. It can be held in a variable, stored in a collection, returned from a function, and chosen at run time.

```
inst := (useCache).then(cachedFunctor).default(plainFunctor);
result := inst.map(xs, transform);
```

Nothing about an instance is special to the compiler, which is why FoLang needs no instance resolution algorithm and no coherence rules.

Writing Code Generic Over a Typeclass

Activation makes `xs.map(f)` work for a **known** container. Code that must work for **any** container cannot activate anything, because the container type is not known until the caller supplies it. Such code takes the instance as an ordinary parameter.

```
@co.dap.generic(  
  types=[{name=F}, {name=A}, {name=B}],  
  aliases=[  
    {name=FunctorOf,      type=Functor(F)},  
    {name=MapFunction,    type=(A)->(B)},  
    {name=InputContainer, type=F(A)},  
    {name=ResultContainer, type=F(B)}  
  ]  
)  
mapAll(  
  inst FunctorOf,  
  value InputContainer,  
  fn MapFunction  
)->(ResultContainer) = {  
  this => inst.map(value, fn);  
}
```

Parameter	What it is
<code>F</code>	the parameterized type — <code>co.List</code> , <code>Option</code> , <code>Tree</code>
<code>inst</code>	the instance, an implementation of <code>Functor</code> for <code>F</code>
<code>value</code>	the container itself, an ordinary value

The caller supplies the instance:

```
ys  := mapAll(tc.ListFunctor,  xs,  double);  
opt2 := mapAll(tc.OptionFunctor, opt, double);
```

The function never learns what `F` is. It knows only that `inst` provides `map`, which is the whole contract. One definition therefore serves every container that has a `Functor` instance.

A type is never “a Functor” in FoLang. `co.List` does not become a Functor; an instance implements Functor operations *for* `co.List`, and the list stays a plain list. The Functor-ness lives entirely in `inst`.

When a wrapper is worth writing

`mapAll` above forwards directly to `inst.map`, so it adds nothing a caller could not write themselves. A wrapper earns its place when it does more than forward — when it combines several typeclass operations, adds logic, or fixes some parameters and leaves others open.

```
@co.dap.generic(
  types=[{name=F}],
  aliases=[
    {name=FunctorOf,    type=Functor(F)},
    {name=IntContainer, type=F(co.int)},
    {name=IntTransform, type=(co.int)->(co.int)}
  ]
)
doubleAll(inst FunctorOf, value IntContainer)->(IntContainer) = {
  transform IntTransform = (x co.int)->(co.int) {
    this => x * 2;
  };
  this => inst.map(value, transform);
}
```

This one fixes the element type and the operation, so it says something a bare `inst.map` call does not.

Where an Instance Is Declared

An instance is declared in **the package that defines the typeclass**, or in **the package that defines the type**. That exact package, not a sub-package.

```
abc.tc.ListFunctor    for=Functor, type=co.List    // OK  typeclass's
package
myapp.ab.TreeFunctor  for=Functor, type=myapp.ab.Tree  // OK  type's
package
other.util.ListFunctor for=Functor, type=co.List    // ERR neither is
theirs
```

A typeclass is an ordinary declaration and may live in any package; `abc.tc` above is a user package, not a built-in one. Sub-packages are distinct packages, so an instance for `myapp.ab.Tree` belongs in `myapp.ab`, not in `myapp` or `myapp.ab.instances`. This matches the rule for companion units, which also sit in their type's own package. Because a package spans every `.fo1` file in its folder, each instance still gets its own `<InstanceName>.fo1` file and uses `_ co.instance` in source.

The rule is permissive on both sides on purpose. Requiring the typeclass's package alone would make a typeclass usable only by its own author, since nobody could implement it for their own types. Requiring the type's package alone would stop a library from shipping instances for common built-in types.

This placement rule is semantic, not syntactic. A misplaced instance parses correctly and is reported during name resolution, so the diagnostic can name the typeclass, the type, and the two packages in which the instance would have been legal.

Typeclass Member Access Rules

The typeclass declaration itself is a package-owned declaration and follows [Package Declaration Access Rules](#). After the typeclass has been resolved and found accessible, access to one of its members is checked independently:

Annotation	Inside the Typeclass Ownership Boundary	Outside the Typeclass Ownership Boundary
<code>@co.dap.public</code>	✓	✓
<code>@co.dap.private</code>	✓	✗

A public typeclass member does not widen access to an inaccessible typeclass. Private member access is confined to the typeclass's own semantic ownership boundary. These access rules do not alter the separately defined instance placement, coherence, conformance, or orphan rules.

Labels and Named Blocks

FoLang labels are **lexical structured-control targets**. They are not ordinary variables, values, package symbols, named blocks, or raw instruction addresses, and they do not introduce a `goto` facility.

A label uses an apostrophe-prefixed identifier. The same lexical form is used both when declaring the label and when referring to it:

```
'outer:    label declaration
'outer     label reference
```

The `:` belongs to the declaration syntax; it is not part of the label name.

Labeled Block

```
'outer: {
    // statements

    (someCondition).then({
        this ->| 'outer;
    });
}
```

`this ->| 'outer;` exits the nearest enclosing structured control region whose label is `'outer`. A labeled block is therefore a structured outward-exit target, not a program-counter destination.

An unlabeled:

```
this ->|;
```

retains its ordinary nearest applicable structured-exit meaning.

Labeled Loop

A loop statement may also be labeled:

```
'outer: (condition).loop({
    ...
});
```

A labeled structured exit may exit that loop:

```
'outer: (condition).loop({
    (done).then({
        this ->| 'outer;
```



```
});  
});
```

A labeled iteration advance may target an enclosing **labeled loop**:

```
'outer: (condition).loop({  
  ...  
  (retry).then({  
    this ->> 'outer;  
  });  
  ...  
});
```

`this ->> 'label;` is invalid when the resolved label denotes a plain labeled block rather than a loop, because a block has no next iteration.

Unlabeled:

```
this ->>;
```

retains its ordinary nearest-loop iteration-advance meaning.

Label Resolution

Labels use a dedicated lexical/control namespace.

```
'label
```

is not looked up as an ordinary variable, function, type, package, or named block. A label reference resolves only to an enclosing active labeled structured control region. It cannot refer forward to a later unrelated statement, escape its labeled region as a first-class value, or target an arbitrary source location.

When nested active regions reuse the same label spelling, the innermost enclosing matching label is selected. Code outside the labeled region cannot refer back into that region.

Labels therefore provide structured control only:

```
allowed:  
  this ->| 'label;           // structured exit  
  this ->> 'label;          // iteration advance; only when label denotes a
```

```
loop
```

```
not introduced:
```

```
goto 'label;
```

```
arbitrary jump into a block
```

```
arbitrary jump across initialization/scope boundaries
```

Label Lexing and Character Literals

The apostrophe form is lexically distinct from FoLang character literals:

```
'c'          // character literal  
'outer      // label identifier/reference  
'outer:     // label declaration prefix
```

A character literal has both opening and closing apostrophes. A label identifier has one leading apostrophe followed by an ordinary identifier and no closing apostrophe. The lexer recognizes the complete form deterministically; the apostrophe used by a label is not an operator.

Anonymous Blocks

An ordinary anonymous block remains unchanged:

```
{  
    // statements  
}
```

An anonymous block is an executable block value, not a function declaration or function expression. It introduces a block scope, but it does **not** introduce a callable, receiver, parameter list, result list, or ordinary return target. When such a block is supplied as a call argument, `this ^=>` explicitly exits the block and produces the result of the nearest lexically enclosing function or method invocation:

```
absolute(value co.int)->(co.int) = {  
    (value < 0).then({  
        this ^=> 0 - value; // terminates absolute, not then  
    });
```

```
this => value;
}
```

Passing the block to `then`, `default`, `loop`, `each`, or another operation does not turn the block into a function. The `^` in `this ^=>` visibly records the one permitted outward transfer from the anonymous argument block to its enclosing callable.

Ordinary `this =>` cannot cross that argument-block boundary. Likewise, `this ->>` and `this ->|` retain their defined nearest applicable iteration or structured-control targets.

A function-shaped expression is different. It establishes a new callable and therefore a new `this =>` target:

```
callback := ()->(co.int) {
  this => 0; // terminates callback
};
```

Thus `{ ... }` alone introduces no callable target, whereas `()->(Result) { ... }` creates one. `this ^=>` is admitted only in the former when it is a call argument; it is invalid directly in a callable body, in a bare or named block, and inside a function-shaped expression. The language defines no `^^=>` or other multi-level spelling.

Named Blocks

A named `co.block` is distinct from a control label. It is an ordinary named declaration/value according to the block rules:

```
labelBlock co.block = {
}

labelBlock.expand();
```

The two forms therefore have different purposes:

<code>'outer: { ... }</code>	structured control label
<code>labelBlock co.block = {...}</code>	named block declaration/value

Blocks have their own variable scope and context. A variable declared outside a block remains accessible inside the block unless it is shadowed. A block may declare a variable with the same name and either the same or a different type; that declaration shadows the outer binding and is scoped only to the block.

Blocks cannot appear outside functions or methods. Inner executable blocks are prohibited directly inside classes, structs, typeclasses, modules, and other non-function/non-method declarations; such usage is a compile-time error.

```
somefun(a co.int, b co.int)->(co.int) = {  
  
    some_other co.float = 20.1f;  
  
    {  
        some_other co.char = 'c';  
        co.out.println(some_other);    // prints c  
    }  
  
    co.out.println(some_other);        // prints 20.1  
  
    {  
        some_other co.float = 11.1f;  
        co.out.println(some_other);    // prints 11.1f  
    }  
  
    co.out.println(some_other);        // still prints 20.1f  
  
    {  
        some_other = some_other + 1.1f;  
        co.out.println(some_other);    // prints 21.2  
    }  
  
    co.out.println(some_other);        // prints 21.2  
}
```

imports

FoLang supports package imports, standalone projected-library imports, and executable-application project-local component imports. `component=` is deliberately **executable-application-primary-`src/` only** and never appears in standalone-library source or source under `components/<kind>/`. Standalone packaged-library exports are likewise executable-application-open-graph inputs rather than general library dependencies. Import targets must be available in the prepared compilation environment before semantic resolution consumes them.

An import declaration makes the target context available for name resolution; import liveness and zero-use validation are defined in [Unused Symbols, Liveness, and Reachability](#).

1. Package Import

Use `package=` for ordinary/open package contexts. A package may come from:

```
the current project's src/ package tree
selected package source under components/packaged/ (executable application
primary `src/` only)
a packaged/exported package context reconstructed from a loaded
`lib/*.folenc` artifact (executable application primary `src/` only)
the language-provided co.* package artifact
```

Example:

```
@co.ddap.import(package="hr.employee", as="emp")

e := emp.getEmployee(1);
co.out.println(e.name);
```

Conceptual resolution:

```
package="hr.employee"
  -> exactly one matching open package context from the prepared package
index
```

The package path must resolve uniquely. Multiple providers of the same package path are an ambiguity error. A `.folenc` filename never becomes an implicit package-name prefix.

Project-local descendant source under `components/application/`, `components/native/`, or `components/dynamicvmrt/` is not an open package root and cannot be imported with `package=`. Packages explicitly selected by `components/packaged/component.fol` enter **only the executable application's primary `src/` package index**. They are deliberately absent from the package indexes used while parsing or resolving every project-local component, so no peer component can import them with `package=`, reach them by qualification, or acquire them indirectly through another component surface.

2. Component Import

Use `component=` only from an **executable application's primary** `src/` domain to consume a same-owner projected component surface:

```
@co.ddap.import(component="native", as="native")
```

Resolution:

```
component="native"  
  -> <project-root>/components/native/component.fol  
  -> projected component surface context
```

The permitted `component=` values are:

```
application  
native  
dynamicvmrt
```

Those values are fixed component identities. Their matching `components/<kind>/` folder determines capability context. The component is not a standalone library and does not produce an independent artifact.

`component=` is an **executable-application-root-to-component composition mechanism**, not a library dependency mechanism and not a component-to-component dependency mechanism. It is invalid in every `src/component.fol` standalone library project and anywhere under `components/application/`, `components/native/`, `components/dynamicvmrt/`, `components/packaged/`, or `components/operators/`. Full qualification, aliases, re-export tricks, or transitive resolution cannot bypass this rule.

For projected components:

- the resolved surface is the fixed `components/<kind>/component.fol`;
- the surface uses `_ co.component` and no `@co.dap.library`;
- only declarations exposed through that surface are visible to the owning open source graph;
- implementation packages below the component root remain private to that component compilation domain.

`components/packaged/` is consumed through ordinary `package=` imports **only by executable application code in the primary `src/` domain** after selection because it contributes application-open package contexts rather than a projected API namespace. No project-local component may consume those selected package contexts. `components/operators/` is compilation infrastructure and is never imported.

These restrictions are enforced by **separate component/package-resolution environments**. Preparing an executable application's primary `src/` domain makes same-owner projected component surfaces addressable through `component=`; preparing any standalone library or any `components/<kind>/` domain makes **no project-local projected component surface addressable through `component=`**. The executable application's primary `src/` package index additionally receives package contexts explicitly selected by `components/packaged/`; component and library package indexes do not. A component/library therefore cannot obtain a forbidden component or packaged context by aliasing, full qualification, transitive lookup, or re-export.

This peer-component prohibition does not impose a pairwise architectural matrix on standalone **projected** libraries. Code inside an executable-application component may import an independently built projected library with `library=` when normal visibility, capability, boundary-type, liveness, and cycle rules are satisfied. The compiler does not reject such a projected-library dependency merely because of the architectural direction between projected libraries. A standalone packaged library is different: it has no safe projected surface, so its exported package contexts are not importable by another library or by a project-local component; they may join only an executable application's primary open graph.

3. Standalone Projected Library Import

Use `library=` for the projected library surface of a prebuilt `.folenc` artifact in `lib/`:

```
@co.ddap.import(library="hrlib", as="hr")
```

Resolution:

```
library="hrlib"  
-> <project-root>/lib/hrlib.folenc
```

-> reconstructed projected component/library surface context

The source used to build the `.folenc` is not reparsed. The artifact loader reconstructs canonical symbol/context metadata and applicable AST information before source parsing begins.

A standalone packaged `.folenc` contains the package contexts selected by its producer rather than a projected `library=` surface. Those selected contexts are consumed with ordinary `package=` imports **only by an executable application's primary `src/` open graph**. A standalone library or project-local component attempting to import a packaged-library context is a compiler error.

Import Directive Fields

Field	Required	Meaning
<code>package</code>	exactly one of <code>package</code> , <code>library</code> , or <code>component</code>	logical open-package dot path resolved from the prepared package index
<code>library</code>	exactly one of <code>package</code> , <code>library</code> , or <code>component</code>	logical standalone projected-library identity resolved from <code>lib/<name>.folenc</code>
<code>component</code>	exactly one of <code>package</code> , <code>library</code> , or <code>component</code>	standardized same-owner projected component identity: <code>application</code> , <code>native</code> , or <code>dynamicvmrt</code>
<code>as</code>	optional	local alias; valid FoLang identifier; when omitted, the complete imported package/library/component identity is used

Notes:

- `package`, `library`, and `component` are mutually exclusive; exactly one must be supplied;
- `as` is optional;
- dots are not allowed in `as`;

- component identities and canonical paths are defined in [Project Layout](#);
- packaged-component and packaged-library contexts are addressed through `package=`;
- the operator component is not imported;
- a packaged `.fo1enc` provides open package contexts only; it has no projected `library=` surface and is consumable only by an executable application's primary open graph.

Examples:

```
@co.ddap.import(package="hr.employee", as="emp")
em emp.Employee;

@co.ddap.import(package="hr.employee")
em hr.employee.Employee;

@co.ddap.import(component="native", as="native")
native.someSurfaceFunction();

@co.ddap.import(library="hrlib", as="hr")
hr.someSurfaceFunction();
```

Valid `as` Values

<code>as="hr"</code>	✓
<code>as="v1_hr"</code>	✓
<code>as="v1.hr"</code>	✗
<code>as="123hr"</code>	✗

Cycles

Compiler error if any dependency cycle exists through effective package, component-surface, or standalone-library dependencies. A written but unused import is not an effective edge and is handled by the unused-import rules.

Symbol Resolution

Resolution Order

For symbols without a prefix:

1. current package symbol table
2. parent package chain upward
3. error if not found

For symbols with an alias prefix:

1. current package imported-context map
2. parent package chain imported-context maps
3. error if not found

For imports declared **without** `as`:

1. use the full imported package path directly
2. resolve it through imported-context lookup using that full path
3. error if not found

Example Context Graph

```
pp1 (grandparent package)
  ├── SymbolTable
  ├── ImportedContexts
  └── ChildCtxs

p1 (parent package)
  ├── Parent -> &pp1
  ├── SymbolTable
  ├── ImportedContexts
  └── ChildCtxs

c1 (current package)
  ├── Parent -> &p1
  ├── SymbolTable
  ├── ImportedContexts
  └── ChildCtxs
```

Resolution Examples

```
lookup "OwnType"
  -> current symbol table

lookup "ParentType"
  -> parent chain
```

```
lookup "hr.Employee"
  -> imported-context lookup for alias "hr"

lookup "hr.employee.Employee"
  -> imported-context lookup for full imported package path when no alias exists

lookup "unknown.Type"
  -> imported-context lookup fails -> compiler error
```

Package in detail

Package identity and multi-file membership are defined in [Packages](#). Canonical project, component, library, and artifact placement is defined only in [Project Layout](#). This section continues with package visibility and access semantics.

Package Declaration Access Rules

Four access levels control the visibility of package-owned declarations across package boundaries. These rules apply to package-owned structs, classes, functions, modules, objects, typeclasses, macros, and every other declaration kind for which the applicable declaration rule permits an ordinary access classifier.

Annotation	Same Package	Parent Package	Descendant Package	Unrelated Package	Entry File / Ordinary Consumer
@co.dap.public	✓	✓	✓	✓	✓
@co.dap.package	✓	✓	✓	✗	✗
@co.dap.protected	✓	✗	✓	✗	✗
@co.dap.private	✓	✗	✗	✗	✗

Meaning:

- @co.dap.public -> visible everywhere

- `@co.dap.package` -> visible across the package family
- `@co.dap.protected` -> visible downward into subpackages only
- `@co.dap.private` -> visible only inside the declaring package

Declaration Access Precedes Member Access

Access is checked hierarchically:

1. resolve the owning package declaration and check its package-level access;
2. only when that declaration is accessible, resolve the requested member and apply the member-access rules of the owning construct.

If the owning declaration is inaccessible, it is not available to the requesting source context and member lookup does not proceed. A public field, method, associated function, or other member never widens the accessibility of its owner. When the owner is accessible but the selected member is not, the compiler reports

`InaccessibleSymbol` for the member access.

Package accessibility is also independent of export and projection. In particular, `@co.dap.public` does not by itself import, export, publish, or place a declaration on a projected component or library surface. Projected surfaces continue to accept only the declaration kinds and boundary forms permitted by [Projected Library and Component Surface Rules](#).

Example:

```
// hr/employee-access.unit.fol – package "hr"

_ co.unit = {

    @co.dap.public
    getEmployee(id co.int)->(Employee) = { ... }    // anyone can call

    @co.dap.package
    validateId(id co.int)->(co.bool) = { ... }    // hr package family

    @co.dap.protected
    baseQuery()->(co.string) = { ... }            // visible to
subpackages

    @co.dap.private
```

```
normalizeId(id co.int)->(co.int) = { ... } // private declaration
}
```

`co.*` Paths and Aliases

`co.*` Is Always Available

The FoLang distribution provides the standard `co.*` package tree through `<install-root>/stdlib/co.folenc`. The command environment selects `fo1cc`; the running compiler resolves its real executable location, derives the installation root, and loads this artifact directly before parsing project source. The frontend makes the `co` root implicitly available, so source code never imports standard `co.*` packages through `@co.ddap.import`. The artifact is neither copied into the project nor managed as a third-party dependency.

Implicit availability changes package discovery only. Once loaded, declarations supplied by `co.*` packages use ordinary package, symbol, type, member, visibility, extension, and overload resolution in the same way as declarations reconstructed from exported package contexts in another `.folenc` artifact.

```
co.out.println("hello");
co.in.readLine();
x co.int = 42;
```

Being **in scope** does not automatically mean **permitted in every context**.

A package rule, capability domain, projected boundary, or other compiler restriction may still forbid use of a particular `co.*` facility. In such cases the name is still known to the compiler, but its use is rejected at compile time.

`@co.ddap.alias`

Use aliases only to shorten `co.*` paths.

```
@co.ddap.alias(co.out, as="out")
@co.ddap.alias(co.List, as="list")
@co.ddap.alias(co.encoding, as="enc")

out.println("hello");
list.of(1, 2, 3);
```

```
enc.json.serialize(data);

// full form still works alongside
co.out.println("world");
```

Rules:

- target must be a `co.*` path
- alias must be a valid identifier
- alias scope is file-local
- aliasing user packages is not allowed; use `as=` in `@co.ddap.import`
- duplicate aliases in the same file are compiler errors
- when no alias is declared, the complete `co.*` path is used
- declaring an alias does not disable or hide the complete `co.*` path ***

Package Source Files

A package folder may contain three ordinary source-file categories. The fixed project surfaces `src/app1.fo1` and `src/component.fo1` are not ordinary package-source files and are invalid inside a package. Likewise, every direct `components/<kind>/component.fo1` surface is a structural component file and is invalid inside an ordinary package. The compiler classifies ordinary package files from filenames before parsing, using the longest recognized suffix first:

```
<Name>.comp.unit.fo1  -> companion unit
<Fragment>.unit.fo1   -> ordinary package unit
<Name>.fo1            -> file-backed primary declaration
```

A filename ending in `.comp.unit.fo1` is recognized before the ordinary `.unit.fo1` suffix and is never classified as an ordinary unit file. Structural project/component filenames are validated by [Project Layout](#) before ordinary package-file classification.

File-Backed Primary Declarations

A primary declaration file has the form `<Name>.fo1` and contains exactly one primary top-level declaration. The source must use `_` in the declaration-name position:

```
// Employee.fo1
_ co.struct = {
  id    co.int;
  name  co.string;
}
```

The compiler derives the declaration name `Employee` from the filename. An explicit primary name is invalid:

```
// Employee.fo1
Employee co.struct = { ... }
// compiler error: file-backed primary declarations must use `_`
```

This rule applies to ordinary file-backed declarations that use a `<name> co.<kind>` primary form, including classes, structs, cstructs, enums, unions, interfaces, signatures, modules, instances, matchers, objects, and other ordinary package-owned declaration kinds. `co.component` is not an ordinary `<Name>.fo1` primary. It is valid only as the single structural declaration in `src/component.fo1` or `components/<kind>/component.fo1`, as defined in [Project Layout](#), [Libraries](#), and [Components](#).

The following declaration forms are stated exceptions and keep an explicit name in the head, because filename derivation cannot express what they need:

Form	Why
surface <code>struct</code> / <code>cstruct</code> declarations inside a projected <code>_ co.component</code>	one projected component surface may carry several boundary declarations
<code>co.data</code> algebraic data type	the head names the variants
parameterized <code>co.type</code>	a filename cannot carry <code>(T)</code>

File-level directives, imports, and aliases occupy the source file's top-level metadata region and do not count as additional primary declarations. A directive is never part of the following declaration merely because it appears immediately before that declaration. Annotations and decorators are different: they may attach to declarations at the locations permitted by their own rules.

FoLang permits the following top-level declaration kinds across ordinary package source files and their reserved special source forms:

1. struct
2. cstruct
3. class
4. trait
5. mixin
6. extension
7. module
8. signature
9. interface
10. enum
11. union
12. typeclass
13. instance
14. matcher
15. annotation or object declaration
16. component declaration in its reserved structural surface
17. unit declaration

Ordinary package-source placement is:

- filename-backed primary declarations use `<Name>.fol`;
- ordinary units use `<Fragment>.unit.fol`;
- companion units use `<StructName>.comp.unit.fol`.

`src/app1.fol`, `src/component.fol`, and `components/<kind>/component.fol` are structural project/component surfaces rather than ordinary package-source forms. Their canonical locations and allowed structural forms are defined once in [Project Layout](#).

A package source file is invalid when it contains multiple unrelated primary declarations or places a unit declaration outside a recognized unit filename. Structural-surface placement errors are validated according to [Project Layout](#).

Package-level functions and non-UDT type declarations belong inside ordinary unit files.

Filename Canonicalization

Filename-derived declaration identity is independent of filesystem case sensitivity. The compiler normalizes and case-folds the filename stem to construct the duplicate-detection and lookup key:

```
canonical file key = caseFold(normalize(filename stem))
```

`normalize` is the same derivation that produces the declaration name, so the key is the case fold of that name. Two stems therefore share a key exactly when they are case variants of one derived declaration, which is the property a duplicate-detection key has to have.

The language-level declaration is then represented using FoLang's canonical declaration spelling. Therefore:

```
employee.fol  
Employee.fol  
EMPLOYEE.fol
```

all denote the canonical declaration name `Employee`. They conflict rather than declaring separate types, even on a case-sensitive filesystem.

The same rule applies to companion owners:

```
employee.comp.unit.fol -> companion of Employee
```

Underscores are word boundaries in the stem rather than part of its identity, so they do not survive normalization:

```
employee_service.fol    -> EmployeeService  
EmployeeService.fol     -> EmployeeService  
employeeService.fol     -> EmployeeService  
employeeservice.fol     -> Employeeservice
```

All four share one key and therefore conflict. The last derives a different declaration name, because a single-case segment carries no word boundary to recover, but it is a case variant of `EmployeeService.fol`, which this section already requires to conflict; conflict is transitive across the whole set.

This rule is specific to filename-derived primary declarations and companion-owner identity; it does not make every FoLang identifier case-insensitive. The compiler must not rely on operating-system filename comparison.

The filename stem must form a valid FoLang declaration identifier after normalization. Case variants and canonically equivalent spellings produce the same package-index key.

Ordinary Package Units

An ordinary package-unit file has the form `<Fragment>.unit.fol`:

```
// arithmetic.unit.fol
_ co.unit = {
  abs(value co.int)->(co.int) = {
    ...
  }
}
```

The fragment name is organizational only. It does not create a public `Arithmetic` symbol. All ordinary unit members in the package are merged into one package namespace:

```
math/
├─ arithmetic.unit.fol
├─ comparison.unit.fol
└─ optional.unit.fol
```

```
math.abs(-10);
math.max(10, 20);
value math.Option(co.int);
```

The following qualification is invalid:

```
math.Arithmetic.abs(-10); // compiler error
```

Any number of ordinary unit files may exist in one package. During package indexing, the compiler shallow-parses their declarations, merges their members with filename-derived primary declarations, and reports duplicate names or duplicate function signatures according to FoLang overload rules.

Companion Unit Files

A companion unit has the form `<StructName>.comp.unit.fol` and must contain `co.unit`. The canonical owner name comes from the filename, not from the source body.

```
Employee.fol
Employee.comp.unit.fol
```

The compiler can detect the following before parsing either body:

- duplicate primary declarations after filename canonicalization
- duplicate companion files for the same canonical owner
- an orphan companion file for which no same-package primary declaration exists

After the owner header is known, the compiler also verifies that the owner is a `co.struct`. Other primary declaration kinds cannot own companion units.

Package Indexing Levels

Level 1: filename and package indexing

- classify primary, ordinary-unit, and companion-unit files
- canonicalize primary names and companion owners
- detect duplicate primary files
- detect duplicate or orphan companion files
- shallow-parse ordinary units
- merge ordinary-unit members into the package namespace
- report package-namespace conflicts

Level 2: companion validation

- parse companion declarations
- validate explicit receivers against the filename-derived owner
- merge companion members into the owner's companion namespace
- report duplicate companion members and receiver mismatches

Level 3: full semantic resolution

- resolve declaration bodies, expressions, calls, patterns, and types

This organization lets an imported source package build most of its symbol index cheaply before full parsing. Compiled packages may store the same index as metadata so imports need no source parsing.

Application Entry File

For the canonical application project structure and root-domain rules, see [Project Layout](#). The application-specific structural slice is simply:

```
src/  
├─ appl.fol  
└─ <package directories>/
```

The application entry file is the fixed **special executable source form**

`src/appl.fol`. It is not a package, does not create an importable namespace, and is not subject to the ordinary package-file rule requiring exactly one primary declaration.

The compiler creates a dedicated **entry-file context** for it:

```
ApplicationEntryContext  
├─ file directives, imports, and aliases  
├─ `let` function-pattern groups (zero or more lexical captures)  
└─ executable statements and expressions
```

Everything declared directly in this context is private to the entry file. Entry-local symbols cannot be imported, exported, or referenced by package or library source files.

Allowed Entry-File Constructs

The application entry file uses exactly the same grammar and allowed-construct rules described in [Single Source Application File](#). This section additionally defines the entry file's formal context, privacy, and dependency direction.

Entry-Local Let Function Patterns

`let` function-pattern groups are allowed as a special entry-file construct even though ordinary function declarations are forbidden. FoLang provides one function-pattern form: `let name(pattern) = body`. The group may capture zero or more already initialized runtime bindings from the entry file's lexical context. Zero capture is an ordinary case of the same construct, not a separate bare function-pattern form. Clause-local pattern bindings and lexical captures follow the rules in [Let Function Patterns](#).

Entry-File Dependency Direction

The entry file may depend on packages and libraries, but packages and libraries may never depend on the entry context:

```
application entry file
  ↓ uses
packages and libraries
```

This allows the entry file to coordinate application startup while preserving package and library independence.

Packaged Component

For canonical component placement, allowed descendant directories, and the `component.fo1` structural form, see [Project Layout](#).

Purpose

The packaged component is a component-private package domain with **selective application-package publication**. Every descendant package begins private to `components/packaged/`. Its `component.fo1` may explicitly select package contexts that are permitted to leave that private domain and participate directly in the executable application's primary `src/` open package/type graph. This publication is one-way toward the application: selected contexts are never made visible to any peer project-local component. It introduces no projected API adapter, snapshot boundary, or separate type identity for packages that are exported.

Its structural position is:

```
components/packaged/
├─ component.fo1
└─ <package directories>/
```

The filesystem constraints are defined only in [Project Layout](#). This section defines selection and open-package semantics.

Packaged Selector Surface

`components/packaged/component.fo1` contains exactly one `_ co.component` structural declaration. It does **not** use `@co.dap.library` because a project-local component is not a library. Inside this special component body, `@co.dap.export(...)` is a component-surface metadata entry applying to the containing `_ co.component`; it does not annotate a following declaration:

```
// components/packaged/component.fo1
_ co.component = {
  @co.dap.export(
    packages={
      hr.employee={recurse=true},
      text.format={recurse=true}
    }
  )
}
```

The `packages` map names package contexts relative to the `components/packaged/` component root. Selection changes **package-context accessibility**, not the visibility of declarations inside the selected package:

- every descendant package is private to the packaged component before selection;
- `recurse=false` exports exactly the named package context;
- `recurse=true` exports the named package context and every descendant package context;
- every selected package path must exist;
- duplicate or overlapping selections normalize to one selected package context;
- selected package names are preserved exactly;
- an unselected package remains component-private even when another package in the same subtree is exported, unless that package is covered by an applicable recursive selection;
- an unselected package cannot be imported, reached through a fully qualified package name, or otherwise referenced from outside `components/packaged/`;
- a selected package is visible only from the executable application's primary `src/` domain; every other project-local component is treated as outside the

publication target and cannot import, qualify, reference, or otherwise consume it.

Exporting a package context does not widen declaration visibility. A private declaration remains private, a protected declaration remains protected, and ordinary subtype, extension, typeclass, ownership, overload, and visibility rules continue to be determined by the declarations themselves.

Open Package Semantics

Only a package context explicitly selected by the packaged surface participates as an ordinary open package context **for the executable application's primary `src/` domain**. Unselected packaged-component packages remain outside this graph, and peer components never receive selected packaged contexts:

```
src package
selected components/packaged package
packaged package context reconstructed from lib/*.folenc
language-provided co.* package context
    |
    v
ordinary package / symbol / member / overload resolution
```

Executable application source under the primary `src/` domain imports a selected package through the ordinary package form:

```
@co.ddap.import(package="hr.employee", as="emp")
```

It never imports the `packaged` component itself. Source under any `components/<kind>/` domain cannot perform this import; its package-resolution environment does not contain the selected packaged-component contexts.

Packaging into `.folenc`

A project-local packaged component does not produce an independent artifact.

For an executable application project, only package contexts explicitly selected by `components/packaged/component.fol` become project-owned open source contexts in the application's **primary `src/` compilation graph**. The frontend does not add those contexts to any project-local component's package index. Unselected descendant packages remain component-private implementation source.

A standalone projected **application** library cannot contain `components/packaged/` or any other project-local component except its optional `components/operators/`. It therefore produces a purely projected `.fo1enc` surface. Reusable implementation dependencies for that library come from `lib/`, not from project-local components.

Standalone `native` and `dynamicvmrt` libraries likewise have no `components/` tree and remain isolated projected domains.

A standalone packaged library selects `src/` package contexts from its own `src/component.fo1`; see [Libraries](#). It has no `components/` tree at all, cannot create project-local custom operator spellings, and may only overload built-in or language-predeclared operators according to the ordinary operator rules.

Application-Wide Semantic Participation

Packaged/open source is intentionally integrated rather than isolated **only after explicit export selection and only into the executable application's primary `src/` graph**. Once a selected package context enters that application graph, its accessible types, inheritance relations, overload families, and applicable AST participate in application-wide semantics. The same context is not injected into any peer component's semantic graph. Unselected packaged-component packages remain private and do not enter the application-wide dispatch/type universe. Therefore application-wide semantic directives that operate on the open application graph, including `@co.ddap.dynamicdispatch(true)`, apply to participating declarations and overload call sites from exported packaged contexts. See [Dynamic Multi Dispatch](#).

Interaction with Operators

Packaging does not export or consume a project-local parser operator table. A packaged library or packaged component may overload built-in or language-predeclared operators according to the ordinary operator ownership rules, but it cannot create a new operator spelling. In an executable application project, packages selected from `components/packaged/` **also cannot use custom operator spellings declared by the owning application's `components/operators/`**, even after those packages are exported into the application's open package graph. Custom operator syntax is available only to the owning application's ordinary `src/` source domain.

Custom operator creation and application-only consumption are governed exclusively by [Operators](#).

Libraries

A standalone distributable producer uses the canonical structure defined in [Project Layout](#). Its fixed primary surface is `src/component.fol`.

Standalone Structure

```
<project-root>/
├── src/
│   ├── component.fol
│   └── <package directories>/
```

`components/`, `lib/`, and `build/`, when applicable, follow [Project Layout](#).

Common Structural Container

Every standalone `src/component.fol` contains exactly one:

```
_ co.component = {
  ...
}
```

`co.component` is the structural surface/container. It does not by itself say whether the produced artifact exposes a projected library API or open package contexts. That exposure model is determined as follows.

Projected Library Form

A projected standalone library annotates the component declaration with `@co.dap.library`:

```
@co.dap.library
_ co.component = {
  // projected application APIs
}
```

Omitting `type` means `application`. The explicit equivalent is:

```
@co.dap.library(type=application)
_ co.component = {
    // projected application APIs
}
```

Other legal projected kinds are:

```
@co.dap.library(type=dynamicvmrt)
@co.dap.library(type=native)
```

Each annotation applies to the single `_ co.component` declaration in `src/component.fo1`. The resulting `<project-name>.fo1enc` exposes that projected surface through `@co.ddap.import(library="...")`. `@co.dap.library` is valid for this standalone `src/component.fo1` projected form only; it is invalid on ordinary package-owned declarations and on every project-local `components/<kind>/component.fo1` surface.

`native` and `dynamicvmrt` are always projected. Their internal packages cannot be exposed directly as packaged/open packages; consumers reach them only through the declared projected surface. A projected `application` library may contain **only** the optional `components/operators/` exception; it may not contain any other project-local component kind. Standalone `native` and `dynamicvmrt` libraries have no `components/` tree. They cannot create new operator spellings, though they may provide legal overloads for FoLang-owned built-in or pre-declared operators.

Packaged Library Form

A packaged standalone library has no `@co.dap.library` annotation. Its component body contains the packaged export selector. In this structural context `@co.dap.export(...)` applies to the containing `_ co.component` and is not waiting for a following declaration: The package-selector form of `@co.dap.export(...)` is valid only in packaged `src/component.fo1` and `components/packaged/component.fo1` structural contexts.

```
// src/component.fo1
_ co.component = {
    @co.dap.export(
        packages={
            hr.employee={recurse=true},
            text.format={recurse=true}
```

```
    }  
  )  
}
```

All `src/` packages of a packaged standalone library are producer-private until selected by the `@co.dap.export(...)` entry in `src/component.fo1`. Selected package paths are relative to `src/`; only those selected package contexts become intentional distributable roots of the resulting `<project-name>.fo1enc`. Because this form has no projected/safe API surface, those contexts are consumable through `@co.ddap.import(package="...")` **only by an executable application's primary open graph**. Standalone libraries and project-local components may not import them. Every unselected `src/` package remains producer-private and is retained only as internal implementation when required by reachable exported code.

A packaged library uses ordinary application-language capabilities. If it needs native/foreign-interop or dynamic-runtime functionality, it must consume those capabilities through projected native or dynamic-runtime library/component surfaces rather than acquiring those capabilities itself.

A packaged standalone library has no `components/` tree. Consequently it cannot declare new project-local operator spellings. It may still provide overload implementations for built-in and language-predeclared operators whose syntax is already owned by FoLang, subject to the normal operator ownership/signature rules.

Form Exclusivity

For `src/component.fo1`:

```
@co.dap.library present  
  -> projected standalone library  
  
@co.dap.library absent  
+ @co.dap.export(...) in the component body  
  -> packaged standalone library
```

A standalone `src/component.fo1` that establishes neither form is invalid.

`@co.dap.library` and the standalone packaged-library selector are mutually exclusive for the primary exposure model.

Packaged/Open Integration

A packaged library is intentionally open to an **executable application's** type and overload graph only through package contexts explicitly selected by its export selector. It is not a general library-to-library dependency form because it has no projected/safe surface API. Metadata required by those exported contexts—including hierarchy information, overload families, and applicable AST/HIR—is retained in `.fo1enc` so that the consuming application can merge and validate the exported open graph under application-wide semantic directives. Unselected producer packages remain private implementation contexts and are not published into the consumer's package graph. This includes dynamic multiple dispatch when enabled by the application.

A projected library is different: its implementation remains behind its declared surface and the consumer does not reinterpret internal projected-library calls merely because the consumer enables an application-wide directive.

Components

For the authoritative list of component kinds, filesystem locations, direct `component.fo1` form, and descendant-package constraints, see [Project Layout](#).

Every project-local component surface has the structural form:

```
// components/<kind>/component.fo1
_ co.component = {
  ...
}
```

The ordinary project-local component model is executable-application-owned source compiled as part of that application. A component is **not** a standalone library, uses no `@co.dap.library`, and produces no independent `.fo1enc` artifact. The immediate `components/<kind>/` folder supplies the kind and capability context. The sole standalone-library exception is `components/operators/` in a projected application library; that exception contributes parser metadata only and is not an importable component API.

All descendant packages of every component kind that permits package directories are **private to that component by default**. `components/` is not an ordinary package-import root. The only outward gateway is the owning `component.fo1`

surface:

projected component

- > publishes only its declared surface APIs
- > descendant packages remain private

packaged component

- > publishes only package contexts selected by `@co.dap.export(...)`
- > publication target is the executable application's primary `src/` graph only
- > no peer component can consume selected package contexts
- > unselected descendant packages remain private

operator component

- > publishes operator syntax metadata to its permitted owning domain
- > has no descendant packages

Projected Components

The projected component kinds are `application`, `native`, and `dynamicvmrt`.

```
// components/native/component.fol
_ co.component = {
  // projected APIs exposed to the owning project
}
```

The owning primary `src/` domain sees only declarations published through a projected component surface. Every descendant implementation package remains private to that component compilation context and cannot be imported, qualified, or referenced directly through `package=` or any physical component path. **Peer components do not see the surface at all.** A project-local component is therefore a leaf dependency domain with respect to other project-local components.

Projected components are imported by component identity only from the executable application's primary `src/` domain:

```
@co.ddap.import(component="native", as="native")
```

The same directive inside any `components/<kind>/` source is a compiler error; peer component surfaces are not importable from a component.

The packaged component exposes only package contexts explicitly selected by its `@co.dap.export(...)` surface entry rather than a projected component namespace; those selected contexts are application-facing only and are not visible from any other project-local component. All unselected descendant packages remain private. See [Packaged Component](#). The operator component contributes project-local parser/operator metadata rather than an importable API; see [Operators](#).

Component Boundary Model

For projected component and projected standalone-library surfaces, the kind controls boundary representation and transfer semantics:

Kind	Boundary data	Transfer semantics
<code>application</code>	<code>co.struct</code>	automatic deep snapshot
<code>dynamicvmrt</code>	<code>co.struct</code>	automatic deep snapshot
<code>native</code>	<code>co.cstruct</code>	declared native/foreign ABI value

`co.struct` is a FoLang semantic data contract; `co.cstruct` is a physical ABI-compatible value contract. Application and dynamic-runtime projected surfaces therefore use managed `struct` contracts transferred as deep snapshots, while native projected surfaces use restricted `cstruct` contracts for explicitly declared native/foreign ABI transfer.

Projected Library and Component Surface Rules

The following rules apply to projected `_ co.component` surfaces: a standalone `src/component.fol` annotated with `@co.dap.library`, or a projected project-local component under `components/application/`, `components/native/`, or `components/dynamicvmrt/`. They do not apply to packaged/export selector surfaces or to the operator component. Structural placement is defined in [Project Layout](#).

Allowed Projected Surface Declarations

This subsection applies to every projected `_ co.component` surface covered by this section. The canonical source locations for those surfaces are defined in [Project Layout](#).

A projected component/library surface may contain only:

- file- or library-level imports needed by its adapter implementations
- `co.struct` boundary declarations for `application` and `dynamicvmrt`
- `co.cstruct` boundary declarations for `native`
- public free-function API declarations with boundary-adapter definitions

The following declaration kinds are forbidden directly in every projected `co.component` surface:

- classes
- modules
- interfaces
- signatures
- units and companion units
- associated functions
- operator functions
- enums, unions, and other ADTs
- type aliases, newtypes, and opaque types
- objects, instances, type classes, and dependent types
- generic declarations, generic type parameters, `@co.dap.generic`, and generic public surface signatures
- macros, templates, annotations, and decorators
- global variables, pointers, references, addresses, or mutable surface state

Surface `struct` and `cstruct` declarations are data contracts only. They cannot have companion units, associated functions, operators, methods, or other behavior on the surface.

Declarations directly inside a projected `co.component` body are exported by default. Imports and implementation details are never exported.

Public Signature Type Closure

Every public surface function signature must be closed over the projected surface's exported boundary type set. Generic parameters are not boundary types: projected surface functions are concrete API contracts and cannot declare or expose generics. Packaged/exported source is different because selected package declarations join the open graph rather than crossing a projected surface.

For `application` and `dynamicvmrt` surfaces, parameters and results may use only:

- approved built-in types
- `co.struct` types declared in the same projected surface

For `native` surfaces, parameters and results may use only:

- ABI-safe built-in types
- `co.cstruct` types declared in the same projected surface

The same closure rule applies recursively to fields of surface boundary types:

- a surface `struct` field may use an approved built-in type or another surface-declared `struct`
- a surface `cstruct` field may use an ABI-safe built-in type or another surface-declared `cstruct`
- an internal package type may never appear in a public function signature or surface boundary-type field
- pointers, references, and addresses may never cross any projected public surface

A built-in type is not automatically surface-safe merely because it belongs to `co`. An approved surface built-in must be concrete, fully resolved, and transferable under the applicable projected kind's boundary semantics.

The following categories are forbidden in public surface fields and signatures:

- inference-only types such as `co.infer`
- dynamically declared `::=` bindings and unconstrained carriers such as `co.any`, `co.typed`, and `co.untyped`
- function, closure, delegate, loader, AST, reflection, or runtime implementation values
- pointer, reference, address, thunk, and implementation-handle types
- any type whose reachable representation contains a forbidden type

For application-family surfaces, managed built-ins such as `co.string` are permitted when the compiler defines deep-snapshot reconstruction for them. For native surfaces, only built-ins with a defined ABI representation are permitted; for example, `co.string` is not directly `cstruct`-compatible.

Valid:

```
// Employee.fol
_ co.struct = {
    id      co.int;
    name    co.string;
    address Address;
}

// Address.fol
_ co.struct = {
    city co.string;
}
```

Invalid:

```
getEmployee(id co.int)->(emp.internal.Employee);
// compiler error: an internal type escapes through the public signature
```

Boundary-Adapter Functions

A public surface function may contain a definition, but its definition is restricted to boundary adaptation.

A boundary-adapter function may:

- read its input parameters and their fields
- declare temporary local values used only for conversion
- construct internal input values
- perform deterministic field-to-field or representation conversion
- invoke exactly one internal implementation operation
- receive that operation's result
- construct and return an allowed surface `struct`, surface `cstruct`, or built-in value
- use a direct delegation expression when no conversion is required

A boundary-adapter function may not:

- implement business rules or domain calculations
- perform business validation or authorization decisions

- orchestrate multiple implementation operations
- call another surface function
- perform persistence, networking, file I/O, retries, caching, or transactions
- start concurrency, scheduling, asynchronous work, processes, or continuations
- contain loops, recursion, workflow branching, or externally observable state mutation
- expose an internal value directly without converting it to an allowed boundary type

The compiler enforces the restricted adapter statement set. The restriction is structural: arbitrary executable logic is not accepted merely because it is placed in a surface file.

A direct delegate is the simplest adapter form:

```
health()->(co.bool)
=>> health.internal.Service.health();
```

A converting adapter may map between the public contract and an internal model.

Standalone Projected Application-Library Surface Example

```
// src/component.fol – standalone application library
@co.dap.library
_ co.component = {

  @co.ddap.import(package="emp", as="emp")

  Employee co.struct = {
    name co.string;
    id   co.int;
  }

  getEmployee(empId co.int)->(Employee) = {
    internalEmployee := emp.EmployeeService.getEmployee(empId);

    this => Employee{
      name: internalEmployee.name,
      id: internalEmployee.id
    };
  };
}
```

```
}  
}
```

The surface owns conversion from the internal `emp.Employee` representation to the public `hrlib.Employee` contract. The `emp` package owns validation, business rules, persistence, and workflow.

The consumer sees the equivalent API contract:

```
// Employee.fol  
_ co.struct = {  
  name co.string;  
  id    co.int;  
}  
  
getEmployee(empId co.int)->(Employee);
```

The consumer does not see the body of `getEmployee` or the `emp` package.

Standalone Projected Native-Library Surface Example

The following example is a separately authored library project. Project-local `components/native/component.fol` does not repeat the kind annotation because its fixed path already establishes `native`.

```
// src/component.fol – standalone native library  
@co.dap.library(type=native)  
_ co.component = {  
  
  @co.ddap.import(package="driver.internal", as="impl")  
  
  Point co.cstruct = {  
    x co.int;  
    y co.int;  
  }  
  
  getOrigin()->(Point) = {  
    internalPoint := impl.DriverService.getOrigin();  
  
    this => Point{  
      x: internalPoint.x,  
      y: internalPoint.y  
    };  
  }  
}
```

```
}  
}
```

Pointers, addresses, native bindings, and hardware state belong in `driver.internal`. They are forbidden in the public surface and cannot appear in the exported signature.

Boundary Transfer Semantics

For `application` and `dynamicvmrt` projected surfaces:

```
consumer boundary value  
  ↓ automatic deep snapshot of the complete reachable value graph  
library-local built-in or surface struct  
  ↓ surface conversion  
internal implementation value  
  ↓ surface conversion  
library-local built-in or surface struct  
  ↓ automatic deep snapshot  
consumer-local boundary value
```

The snapshot rule applies to both surface structs and approved built-in arguments/results. The library never receives the consumer's live object identity, and the consumer never receives a live internal-library object.

For projected `native` surfaces, `cstruct` values cross according to the explicitly selected native/foreign ABI contract. That contract may describe a platform ABI, C ABI, calling convention, layout, alignment, linkage, or other supported native representation metadata. FFI and low-level native operations may exchange the same ABI values internally without crossing another projected surface.

Surface-to-Internal Dependency Direction

The source-level dependency is one-way:

```
projected surface  
  ↓ imports and invokes  
internal packages
```

Internal packages:

- do not import the projected surface

- do not use surface-declared `struct` or `cstruct` types
- define their own implementation and domain types
- return internal values to the surface adapter
- contain all business logic, validation, workflow, I/O, and state management

This prevents a surface/internal compilation cycle and keeps the public contract independent from the internal domain model.

An internal implementation symbol may be visible to its own projected surface without becoming part of the consumer API. Library encapsulation takes precedence over ordinary package visibility: consumers cannot resolve internal package symbols even when those symbols are callable from the surface during library compilation.

Consumer API Projection

The compiler does not expose the complete surface compilation context to a consumer. It creates a projected imported symbol table.

The projected API contains:

- the projected library/component identity and kind
- complete public surface `struct` or `cstruct` definitions, including field names and permitted field types
- public function names
- public function parameter names and types
- public function result types
- calling-convention, effect, error, and linkage metadata where applicable

The projected API does not contain:

- function bodies or implementation AST/HIR
- imports used by the surface
- local conversion variables
- delegate or implementation targets
- internal package paths or symbols
- private compiler-generated helpers
- business classes, modules, units, or internal data types

Therefore, a function definition may exist in the surface source and compiled artifact while the consumer's symbol table contains only its signature.

The backend may retain implementation bodies for code generation, linking, or whole-program optimization. Backend possession of implementation code does not make that code semantically visible to the consumer.

Project-local projected components follow the same boundary rule. Availability of their source code inside the owning project does not make their private implementation packages semantically visible outside the component surface.

Library Compilation Order

A library uses the same import-usage and symbol-usage validator as an application. It is processed in these logical stages:

1. parse the surface header, boundary data declarations, and public function signatures;
2. parse/prepare internal packages without depending on surface types;
3. register internal imports as provisional context references rather than copying symbol tables;
4. type-check and link surface boundary-adaptor bodies against internal package symbols, activating an import only when a symbol from it is actually used;
5. complete semantic resolution and mark exact usage-checkable symbols live;
6. prune and report zero-use imports;
7. reject unused usage-checkable symbols and unreachable internal source/packages;
8. emit the compiled implementation and projected consumer API.

The declarations intentionally exported by a projected surface (`src/component.fol` or a projected `components/<kind>/component.fol`) are semantic roots of the applicable library/component internal compilation graph. This order permits the surface to call internal packages while preventing internal packages from depending on the surface. The different producer, project-local component, and `.folenc` consumer usage rules are defined in [Unused Symbols, Liveness, and Reachability](#).

Projected Capability Kinds

Filesystem placement and component slots are defined in [Project Layout](#). This section defines capability semantics only.

FoLang has three projected capability kinds:

```
application
dynamicvmrt
native
```

Macros, templates, concurrency, asynchronous execution, continuations, scheduling, transformation facilities, and other ordinary non-privileged language features belong to the application capability domain unless another section explicitly restricts them.

A packaged library/component is not a fifth privileged capability kind. It uses ordinary application capabilities and differs by **exposure model**: only packages explicitly selected by `@co.dap.export(...)` join the open package/type graph; every unselected package remains private to the packaged producer/component.

application

`application` is the ordinary FoLang programming capability domain. It permits normal language features including classes, modules, interfaces, signatures, units, structs, generics, macros, templates, concurrency, async/parallel abstractions, continuations, scheduling, transformations, and other standard application-level facilities.

It does not directly acquire `native` or `dynamicvmrt` privileged capabilities. Application code may consume those domains only through their projected surfaces.

Public projected boundary:

- surface `struct` contracts;
- approved built-in types;
- automatic deep-snapshot transfer.

`dynamicvmrt`

`dynamicvmrt` includes application capabilities plus the dynamic-runtime/metaprogramming capabilities defined for `co.meta`, including runtime reflection, instrumentation, dynamic loading, patching, eval-based facilities, and related dynamic-runtime operations.

It does not acquire `native` capability. Native facilities remain accessible only through projected surfaces.

Public projected boundary:

- surface `struct` contracts;
- approved built-in types;
- automatic deep-snapshot transfer.

`native`

`native` is the single isolated low-level capability domain. It combines the facilities formerly separated as `system` and `ffi` because native implementation and foreign interoperability share ABI values, pointers, memory representations, linkage information, and platform operations and may need to exchange those values internally.

The native domain includes raw pointers, references and addresses; pointer arithmetic; word-level and ABI-oriented types; `@co.dap.native`; `co.native` assembly/machine-level facilities; platform/runtime implementation; MMIO and low-level process/thread facilities; extern declarations; foreign symbol bindings; supported C/native ABIs; calling conventions; linkage metadata; C-compatible/native `cstruct` representations; and related marshalling or invocation logic.

FFI remains a **feature area inside** `native`, not a separate capability domain. Native code may therefore receive a pointer or ABI value from a foreign call and pass it directly to raw-memory, platform, assembly, or other native logic without leaving the native isolation boundary or round-tripping through another projected surface.

`native` does not inherit application or dynamic-runtime capabilities merely because those features exist elsewhere in FoLang. Cross-domain use occurs only through projected surfaces.

Public projected boundary:

- surface `cstruct` contracts;
- ABI-safe built-in types;
- explicitly declared native/foreign ABI value transfer.

packaged

A packaged library or packaged component uses application capabilities but has no projected API boundary for the packages it explicitly exports. For a project-local `components/packaged/`, only explicitly selected package contexts participate directly in the executable application's primary `src/` open package/type/overload graph; they are not visible to peer components. A standalone packaged library likewise contributes its explicitly exported package contexts **only to an executable consuming application's primary open graph** through the loaded artifact. It cannot be imported into another standalone library or project-local component. Unselected packages remain private implementation contexts.

A packaged domain cannot directly acquire native or dynamic-runtime capabilities. Packaged library code and packaged-component code may call independently built **projected libraries** through their published `library=` surfaces when the applicable boundary rules permit it; they may not reach peer project-local components. The executable application remains the composition root for its own project-local component surfaces.

Dependency Direction

FoLang capability domains are isolated rather than arranged in the former linear capability ladder.

```
application
  -> ordinary language capabilities

packaged
  -> ordinary application capabilities + open package exposure

dynamicvmrt
  -> application capabilities + dynamic-runtime capabilities
```

native

-> isolated native / foreign-ABI / platform capability domain

The absence of direct capability inheritance does **not** prevent independently built **standalone projected libraries** from calling other standalone projected libraries through their published surface APIs. Calling a projected library API does not grant the caller the callee's internal capability set or expose the callee's private types.

FoLang deliberately does **not** encode a pairwise architectural dependency matrix for standalone projected `application`, `native`, or `dynamicvmrt` libraries. The compiler instead enforces the actual language boundary: import/exposure legality, public API types, capability use, visibility, liveness, cycles, and other applicable semantic constraints. Standalone packaged libraries are excluded from this rule: their exported contexts may be consumed only by an executable application and never by another library or component.

Project-local components are different. They are not independently consumable libraries; they are implementation components owned by one project. Their dependency direction is fixed:

owning primary src/ -> project-local component	allowed
project-local component -> peer component	compiler error

No `components/application/`, `components/native/`, `components/dynamicvmrt/`, `components/packaged/`, or `components/operators/` source may consume another project-local component. `components/packaged/` publishes selected packages only to the executable application's primary `src/` graph, and `components/operators/` publishes custom syntax only to the executable application's ordinary primary `src/` domain. A projected application library may also own `components/operators/` as its sole component exception, with syntax visible only to that library's ordinary primary `src/` source.

Examples:

application src -> components/native surface	allowed
components/native -> components/application	compiler error
components/application -> components/dynamicvmrt	compiler error
components/packaged -> components/application	compiler error
standalone native library -> another projected library API allowed if	

```
API/boundary rules pass
standalone projected library -> native library API allowed if API/boundary
rules pass
standalone library/component -> standalone packaged-library packages
compiler error
caller -> use callee-private privileged capability      forbidden

native remains an isolated implementation domain
application or packaged code reaches native facilities only through a
projected native surface
```

Dependency cycles through effective imports remain compile-time errors.

Capability Isolation

`native` and `dynamicvmrt` are distinct privileged capability domains. `dynamicvmrt` additionally includes ordinary application capabilities; `native` does not. No source file gains another domain's privileged capability merely by importing its projected surface.

Types crossing a projected boundary must satisfy the boundary rules for that domain. Internal implementation types remain private and cannot be imported directly.

Packaged/Open Code

Packaged/open package contexts are application-domain code from the **executable application consumer's** perspective. Their actual declarations and applicable AST are merged into that application's open graph rather than hidden behind a projected boundary. They are not legal imports into standalone libraries or project-local components. Consequently, once admitted to the executable application graph, they inherit application-wide semantic rules, including dynamic multiple dispatch when enabled.

Units in detail

A `co.unit` is a stateless file-level declaration container. It is not instantiable and does not create an object, runtime scope, or public unit namespace.

FoLang has two unit-file forms:

1. **ordinary package unit** — `<Fragment>.unit.fol`

2. struct companion unit — `<StructName>.comp.unit.fol`

Both forms use `_ co.unit`; explicit unit names are invalid.

A unit is an organizational container rather than a public symbol namespace.

Visibility annotations apply to the declarations contributed by the unit; they do not apply to `_ co.unit` itself.

Ordinary Package Units

Example package layout:

```
math/  
├─ arithmetic.unit.fol  
├─ comparison.unit.fol  
└─ optional.unit.fol
```

```
// arithmetic.unit.fol  
_ co.unit = {  
  abs(value co.int)->(co.int) = { ... }  
  
  max(  
    a co.int,  
    b co.int  
  )->(co.int) = { ... }  
}
```

```
// optional.unit.fol  
_ co.unit = {  
  Option(T) co.type =  
    co.variants(Some(T), None);  
  isSome(value Option(co.int))->(co.bool) = {  
    ...  
  }  
}
```

All declarations are contributed directly to the `math` package namespace:

```
math.abs(-10);  
math.max(10, 20);  
value math.Option(co.int);
```

The unit filenames never appear in qualified names:

```
math.Arithmetic.abs(-10); // compiler error
math.Optional.Option(co.int); // compiler error
```

Within the same package, the functions and types may be referenced without the package prefix according to ordinary package name-resolution rules.

An ordinary unit may contain:

- receiverless functions
- `co.type` aliases and parameterized/variant-based type declarations
- newtype and opaque-type declarations
- subtype and supertype declarations
- macros and template declarations
- other non-instantiable type declarations explicitly permitted by their sections
- public, package, protected, or private members

An ordinary unit may not contain:

- fields or mutable unit state
- executable loose statements
- classes, structs, cstructs, enums, unions, modules, interfaces, or signatures
- lifecycle declarations for the unit
- a declaration that creates an independent nested primary namespace

A unit file may group closely related declarations or merely organize a convenient set of package-level functions and types. Semantic cohesion is recommended but not enforced.

For canonical unit and per-symbol usage rules, see [Unused Symbols, Liveness, and Reachability](#).

Package-Namespace Consolidation

All ordinary unit files in one package are shallow-parsed and consolidated into one package symbol table. Their order and filenames have no semantic effect.

```
package members =
  filename-derived primary declarations
+ declarations from every ordinary unit file
```

A duplicate is reported as soon as the package index is built. Conflicts include:

- the same type or value name contributed by two ordinary units
- an ordinary-unit member colliding with a filename-derived primary declaration
- duplicate function signatures not permitted by overload rules
- declarations that normalize to the same canonical package symbol

Diagnostics should identify both source files.

Companion Units

A companion unit is governed by the filename and owner-validation rules in [Struct Companion Units](#). Its members do not merge directly into the package namespace; they attach to the owner struct's companion namespace.

Companion-unit functions follow the canonical strict per-symbol usage rule in [Unused Symbols, Liveness, and Reachability](#).

Units have no fields, identity, instances, inheritance, polymorphic dispatch, or independent construction.

CStructs

`co.cstruct` is a C-like value type — passed by value, simple memory layout, safe to cross zone boundaries. Unlike `co.struct` which is passed by reference, `co.cstruct` is always copied on pass.

```
// Point.fol
_ co.cstruct = {
  x co.int;
  y co.int;
}

// Rect.fol
_ co.cstruct = {
  origin Point;
  width  co.int;
  height co.int;
}
```

cstruct Rules

always passed by value – never by reference
simple memory layout – no metadata
can contain only simple types and other cstructs
cannot contain co.struct ❌ has metadata
cannot contain co.string ❌ heap allocated
cannot contain dynamically typed storage (``:=``) ❌ no fixed ABI type / runtime type info
cannot contain classes ❌ vtable, metadata
cannot contain modules ❌
cannot contain any heap allocated type ❌
cannot have methods
cannot have associated functions
cannot embed co.struct
safe to cross direct ABI and zone boundaries

allowed field types:

co.int, co.uint, co.float	✓	primitives
co.bool, co.char, co.byte	✓	primitives
FixedArrayN	✓	named co.type defined as
co.int->([N])		
co.cstruct	✓	other cstructs

Packed cstruct — no padding, exact memory layout

Used for hardware registers, binary protocols, exact memory mapped formats:

```
// Register.fol
@co.dap.packed
_ co.cstruct = {
  flags  co.uint8;
  status co.uint8;
  data   co.uint16;
}
```

SIMD cstruct — aligned for vector operations

Used for math, graphics, signal processing:

```
// Vec4.fol
@co.dap.simd(align=16)
_ co.cstruct = {
  x co.float;
```

```
y co.float;
z co.float;
w co.float;
}
```

Both together

```
// AVXVec.fol
@co.dap.packed
@co.dap.simd(align=32)
_ co.cstruct = {
    data co.float;
}
```

`@co.dap.packed` and `@co.dap.simd` are specialisations of `co.cstruct` — same rules, same zone boundary safety. They are not separate types.

Structs

```
// myStruct.fol
_ co.struct={
    field1 co.int;
    field2 co.string;
    field3 co.bool;
}
```

Struct Rules

- structs cannot extend other structs
- structs cannot contain methods directly
- structs can compose other structs
- structs cannot have default values to fields/members
- structs can embed other structs (Go lang like)
- structs can have a same-package companion file named ``<StructName>.comp.unit.fol``

The struct declaration remains pure data. Associated behaviour is declared separately in `<StructName>.comp.unit.fol`.

Struct Embedding

Embedding promotes fields of an embedded struct directly into the outer struct — they act as the outer struct's own fields at construction and access sites. This is distinct from composition where the embedded struct is a named field.

```
// E.fol
_ co.struct = {
    id    co.int;
    name co.string;
}

//  No conflict – id and name promoted as B's own fields
// B.fol
_ co.struct = {
    age co.float;
    E;           // embedded – id and name promoted
}

b := B{ age: 25.0, id: 1, name: "Rao" }; // all fields at same level
b.id    // direct – no b.E.id needed
b.name  // direct – no b.E.name needed
b.age   // direct
```

```
//  Compiler error – name conflict between B.name and E.name
// B.fol
_ co.struct = {
    name co.string; // conflicts with E.name
    E;
    age  co.float;
}

// Fix 1 – rename B's conflicting field
// Fix 2 – use explicit composition instead: e E;
```

```
// Explicit composition – no promotion, always qualified access
// B.fol
_ co.struct = {
    name co.string;
    e    E;           // named field – no conflict, no promotion
    age  co.float;
}

b.name ; // B's own name
```

```
b.e.id ;    // E's id – always explicit
b.e.name;  // E's name – always explicit
```

Embedding Rules

Situation	Behavior
Embedded field, no conflict	Promoted — acts as the outer struct's own field
Embedded field, name conflict with outer	✗ Compiler error — rename or use composition
Multiple embeds, no conflict between them	All fields promoted
Multiple embeds, conflict between embedded structs	✗ Compiler error
Explicit composition (<code>e E</code>)	No promotion — always accessed via <code>b.e.field</code>

FoLang does **not** silently shadow conflicting fields. Any such name conflict is a compile-time error; the conflicting declaration must be renamed or the relationship expressed through an allowed explicit-composition mechanism.

For member-style embedding, an accessible public instance-associated function with an explicit value receiver follows the same promotion and conflict rules as a field. Receiverless companion functions, type-associated functions, operator functions, and companion-associated type declarations remain in their defined companion or operator lookup domains and are not promoted as instance members merely because their owner struct is embedded.

Embedding never widens member access. A promoted field or instance-associated function retains its original declaration owner, symbol identity, and access classifier. A private member remains accessible only inside the embedded struct's ownership boundary, including that struct's companion unit; it is not made accessible to the embedding struct. Conflict checking considers the accessible members eligible for promotion and reports a compiler error when no unique promoted member can be selected.

Struct Declaration Relationships

A struct cannot physically declare another struct, enum, class, module, function, signature, interface, or other named declaration inside its body. A struct body remains a pure data declaration containing fields and permitted embeddings only.

Use one of the following instead:

- **composition** through a named field
- **embedding** where the declaration kind's embedding rules permit it
- a separately declared target-local declaration restricted with `@co.dap.local`

```
// EmployeeAddress.fol
@co.dap.local(for=hr.employee.Employee)
_ co.struct = {
  street co.string;
  city   co.string;
}
```

```
// Employee.fol
_ co.struct = {
  id      co.int;
  name    co.string;
  address EmployeeAddress; // composition
}
```

The following physical nesting is invalid:

```
// Employee.fol
_ co.struct = {
  Address co.struct = { // ❌ nested declaration
    city co.string;
  }

  address Address;
}
```

structs cannot declare inner structs	❌	compiler error – only through <code>@co.dap.local</code>
structs can declare inner enums/ADTs	❌	compiler error – only through <code>@co.dap.local</code> or <code>@co.dap.nested</code>
structs cannot declare inner classes	❌	compiler error – struct is pure data

structs cannot declare inner modules
pure data



compiler error – struct is

`@co.dap.local` controls declaration visibility only. It does not automatically compose or embed the annotated declaration into the target struct.

Struct Companion Units

A struct companion unit is declared in a file named `<StructName>.comp.unit.fol`. The matching struct remains in `<StructName>.fol`, and both files must belong to the same package.

```
// Vector.fol
_ co.struct = {
  x co.float;
  y co.float;
}
```

```
// Vector.comp.unit.fol
_ co.unit = {
  distance(
    left Vector,
    right Vector
  )->(co.float) = {
    ...
  }

  zero()->(Vector) = {
    this => Vector{x: 0.0, y: 0.0};
  }

  (value Vector) magnitude()->(co.float) = {
    this => co.math.sqrt(
      value.x * value.x + value.y * value.y
    );
  }

  (Vector) create(
    x co.float,
    y co.float
  )->(Vector) = {
    this => Vector{x: x, y: y};
  }
}
```

```
}  
}
```

The filename establishes ownership. No companion member needs an ordinary parameter merely to prove association.

Call forms:

```
d      := Vector.distance(first, second);  
origin := Vector.zero();  
length := value.magnitude();  
point  := Vector.create(10.0, 20.0);
```

A companion unit may declare:

- receiverless companion functions, including zero-parameter functions and factories
- instance-associated functions with an explicit value receiver
- type-associated functions with an explicit type receiver
- operator functions permitted for the owner struct
- non-UDT type declarations associated with the owner when their own sections permit them

```
receiverless non-operator companion function  
  -> has no explicit receiver  
  -> ordinary parameters may have any legal types  
  -> no owner-typed parameter is required merely for association
```

```
instance-associated function  
  -> explicit receiver is a value of the matching struct
```

```
type-associated function  
  -> explicit receiver is the matching struct type itself
```

These are the general companion rules. Operator functions have the additional operand requirement defined in the Operator Functions subsection.

Companion declarations do not make the struct a class and do not add object identity, inheritance, virtual dispatch, lifecycle methods, or unit-level state.

Filename and Owner Validation

The compiler classifies companion files before parsing:

```
Vector.comp.unit.fol -> canonical owner Vector
```

At package-indexing level, the compiler verifies:

- a canonical primary declaration named `Vector` exists in the same package
- no second companion file resolves to the same canonical owner
- the companion is not orphaned

After the owner declaration header is available, the compiler verifies that the owner is a `co.struct`. A class, cstruct, enum, union, module, interface, signature, or imported declaration with the same short name cannot become the owner.

Receiver Validation

The companion filename determines the required receiver root. Any explicit receiver must match that owner.

```
// Employee.comp.unit.fol
_ co.unit = {
  create(id co.int)->(Employee) = { ... } // valid: receiverless

  (emp Employee) isValid()->(co.bool) = { ... } // valid

  (Employee) empty()->(Employee) = { ... } // valid

  (dept Department) isValid()->(co.bool) = { ... }
  // compiler error: receiver Department does not match companion owner
  Employee

  (Department) create()->(Department) = { ... }
  // compiler error: type receiver Department does not match companion
  owner Employee
}
```

Ordinary parameters may have any legal types. They are not used to establish companion ownership.

For a generic struct owner, receiver validation resolves the receiver's named alias to its canonical root declaration and generic arity. For example, `BoxGOfT co.type = BoxG(T)` may be declared in `BoxG.comp.unit.fol`, and a receiver of type `BoxGOfT` belongs to that companion because the alias resolves to `BoxG`. An alias resolving to `BoxG(T, E)` with the wrong arity, or to an unrelated root, is rejected.

Companion Namespace and Conflicts

Companion members are resolved through the owner:

```
employee := Employee.create(1);
valid     := employee.isValid();
```

The complete companion namespace is formed from all legal owner-associated declarations and the single companion file. Duplicate names and normalized duplicate signatures are compile-time errors.

The following are invalid:

```
Employee.comp.unit.fol without Employee.fol
Employee.comp.unit.fol and employee.comp.unit.fol in the same package
Employee.comp.unit.fol when Employee.fol declares a class or cstruct
an explicit receiver whose canonical root is not Employee
```

Operator Functions

Operator functions associated with a struct belong in its companion unit. Ownership comes from the companion filename; receiver validation follows the same rule as other companion functions.

```
// Employee.comp.unit.fol
_ co.unit = {
  @co.dap.operator(symbol="+")
  (emp Employee) add(other Employee)->(Employee) = {
    ...
  }

  @co.dap.operator(symbol=="")
  equals(
    left  Employee,
    right Employee
  )->(co.bool) = {
```

```

    ...
}

@co.dap.operator(symbol=">")
(Employee) greater(
    left Employee,
    right Employee
)->(co.bool) = {
    ...
}
}

```

Companion ownership always comes from `<StructName>.comp.unit.foo1`. An operator function has an additional operand rule: a value receiver already supplies the owner instance, but a receiverless operator function or a type-receiver operator function must declare the companion owner type as its first ordinary parameter. This allows operator lowering to pass the actual owner instance to the function. The compiler compares the resolved first-parameter type with the companion owner type at compile time. This requirement establishes the operator operand, not companion ownership. An operator declaration that does not operate on the owner struct is invalid.

Struct Member Access Rules

The struct declaration itself follows [Package Declaration Access Rules](#). A struct's companion unit belongs to the struct's semantic ownership boundary; it is not a separate access owner. After the struct is accessible, its field and eligible associated-function access is checked as follows:

Annotation	Struct and Its Companion Unit	Outside the Struct Ownership Boundary
<code>@co.dap.public</code>	✓	✓
<code>@co.dap.private</code>	✓	✗

A public struct member does not widen access to an inaccessible struct. Private fields and private associated functions may be used by the owning struct's companion unit, but they are not exposed through embedding, composition, an import, or another public member.

Unions

`co.union` declares an untagged ADT. Its body lists the alternative members defined by the union.

```
// myUnion.fol
_ co.union={
    intValue co.int;
    strValue co.string;
}
```

Enums

`co.enum` is FoLang's closed tagged algebraic-data-type declaration. The body of an enum contains only **state declarations**. States are owned by the enclosing enum and form its complete set of alternatives.

```
// Shape.fol
_ co.enum = {
    Circle(radius co.float),
    Square(side co.float),
    Rectangle(width co.float, height co.float),
    Point
}
```

Enum States and State Functions

Every enum member is a **state**. A state is not an independent type and does not introduce a struct, class, object, or constructor declaration.

A state with no parameters is a complete value of the enclosing enum type. It is referenced directly, without `()`:

```
origin Shape = Shape.Point;
```

`Shape.Point()` is invalid because `Point` declares no parameters and is not a zero-argument ordinary function call.

A state with one or more parameters defines a compiler-provided **state function**. Every state-function parameter is intrinsically named. The declaration therefore gives each payload component both a name and a type:

```
// valid named payload declarations
_ co.enum = {
  Move(x co.int, y co.int),
  Text(body co.string),
  Quit
}

// invalid model: enum payload components cannot be positional/type-only
// Move(co.int, co.int)
```

Calling a state function produces a value of the enclosing enum type. A call must bind every supplied payload value explicitly by parameter name using `name=value`; a positional state-function call is invalid:

```
circle Shape = Shape.Circle(radius=5.0);
square Shape = Shape.Square(side=4.0);
rect  Shape = Shape.Rectangle(width=4.0, height=6.0);
rect2  Shape = Shape.Rectangle(height=6.0, width=4.0); // order is
irrelevant

// invalid: positional payloads are never accepted for enum state functions
// Shape.Circle(5.0);
// Shape.Rectangle(4.0, 6.0);
```

Each required state parameter must be supplied exactly once. Unknown, duplicate, missing, or positional state-function arguments are invalid. The ordinary-function `~` named-parameter marker is not written on enum state parameters because enum state parameters are named by definition. Unlike ordinary named functions, enum state functions have no positional-call fallback.

Conceptually, the state functions above have these value-level mappings:

```
Shape.Circle      : (radius: co.float) -> Shape
Shape.Square      : (side: co.float) -> Shape
Shape.Rectangle   : (width: co.float, height: co.float) -> Shape
Shape.Point       : Shape
```

These mappings are explanatory signatures, not separate source declarations. A state function has no developer-defined body and no separately declared result type: its result is always the enclosing enum type. The state function's payload is a

named set of fields defined by its parameter declarations; parameter declaration order does not give the payload a positional identity.

The state name identifies the selected alternative and the supplied named arguments are that value's payload. Therefore these are distinct enum values even though they share the same static type:

```
a Shape = Shape.Circle(radius=5.0);  
b Shape = Shape.Circle(radius=10.0);  
c Shape = Shape.Square(side=5.0);
```

Conceptually:

value payload	static type	state	named
Shape.Circle(radius=5.0)	Shape	Circle	{radius=5.0}
Shape.Circle(radius=10.0) {radius=10.0}	Shape	Circle	
Shape.Square(side=5.0)	Shape	Square	{side=5.0}
Shape.Point	Shape	Point	{}

A backend may represent the state identity and payload using a tag, discriminant, tagged union, compact integer encoding, or another representation. The representation is not observable language semantics; the observable requirement is that the state identity and every named payload component remain available to operations such as pattern matching.

Pattern Matching

Pattern matching distinguishes an enum value first by state and then binds or tests that state's named payload. Parameterized enum-state patterns are named just like state-function calls: each payload pattern identifies the declared state parameter with `name=pattern`. Positional enum-state patterns are invalid. Zero-parameter states remain bare state names.

```
shape.match()  
  .case(Shape.Circle(radius=r) => r)  
  .case(Shape.Square(side=s) => s)  
  .case(Shape.Rectangle(width=w, height=h) => w * h)  
  .case(Shape.Point => 0.0);
```

Because matching is name-based, pattern order is not semantically significant; for example `Shape.Rectangle(height=h, width=w)` denotes the same payload fields. A payload binding such as `r` is local to the matching case according to the ordinary pattern-binding rules. `Shape.Circle(radius=5.0)` and `Shape.Circle(radius=10.0)` have the same enum type and state but different payload values; `Shape.Circle(radius=5.0)` and `Shape.Square(side=5.0)` have numerically equal payload values under different payload names and different states.

State Identity Is Not Type Identity

An enum state is not a subtype or an independent nominal type. For the declaration above, `Shape` is the type; `Circle`, `Square`, `Rectangle`, and `Point` are states owned by `Shape`.

```
Shape.Circle(radius=5.0) : Shape
Shape.Square(side=4.0)   : Shape
Shape.Point              : Shape
```

A declaration such as `x Circle;` does not follow from the enum declaration because `Circle` is not a type. Code that requires an independently reusable `Circle` type must declare that type separately, for example as a `co.struct`, and may then use that type as payload of an enum state if required.

This differs from a union of existing types such as `ShapeType co.type = Circle | Square`: that form composes already existing types, whereas `co.enum` declares a closed family of states whose state functions directly produce values of the one enclosing enum type.

Relationship to `co.variants(...)`

A closed variant-based `co.type` created with `co.variants(...)` uses the same state/state-function terminology. A parameterized variant entry defines a state function; a bare entry defines a zero-parameter state value. The **named-only payload declaration, call, and pattern rule defined above is normative for `co.enum` state functions**; `co.variants(...)` follows the separately specified variant syntax. `co.variants(...)` remains useful where the enclosing declaration itself is a parameterized `co.type`, while `co.enum` remains the file-backed enum UDT declaration.

An enum value's constant expression may use a registered custom operator at any declared precedence. Runtime assignment is forbidden everywhere in that constant-expression subtree, including inside grouping, arguments, and collection or object elements. Whether the resulting expression can actually be evaluated at compile time—including whether a custom operator is foldable—is checked after parsing.

classes

```
// Employee.fol
_ co.class = {
  getEmployeeDetails()->(Employee) = empmodule.getEmployeeDetails;
  // assigning module function to class's method

  getEmployeeInfo()->(Employee) =>> empmodule.getEmployeeDetails();
  // delegating – internally redirecting the call to module function
}

// $1, $2, $3 ... are return components of the immediately previous chained
function
// Emp.fol
_ co.class = {
  dosomething(a co.int, b co.int)->
  (co.int)=>>somePack.someMethod(a)=>>someOthPack.someOtherMeth($1, b);
}
```

Class Storage and OOP Model

In storage terms, a `co.class` is deliberately as simple as a struct: its declared fields are ordinary mutable per-instance fields. A class does not contain field-level constants, field-level immutable values, static/global fields, independently shared fields, field locks, or field-level CopyOnWrite state. The complete class instance may later receive an Immutable, Shared, or CopyOnWrite object-graph policy through `co.utils`, but an individual class field cannot receive one of those policies independently.

The difference between a struct and a class is therefore not hidden static storage. A class adds the object-oriented facilities defined by FoLang:

- inheritance;
- abstraction;

- polymorphism;
- encapsulation and access control;
- interface implementation;
- trait and mixin composition;
- extension and explicit behaviour modification; and
- dynamic dispatch.

The following class fields are invalid:

```
_ co.class = {
  @co.dap.const limit co.int = 100;           // compiler error
  @co.dap.final identifier co.int;           // compiler error
}

customer Customer = getCustomer();
co.utils.makeShared(customer.queue);         // compiler error: field
root
co.utils.copyOnWrite(customer.cache);         // compiler error: field
root
```

If one instance must become immutable, shared, or CopyOnWrite, the policy is applied to that complete instance graph:

```
customer Customer = getCustomer();
co.utils.makeImmutable(customer); // complete customer graph, never one
field
```

Constants, immutable bindings, locks, global variables, and shared storage that must exist independently of a class instance belong to a named `co.object`.

The same prohibition applies to a field introduced into a class instance by a parent class or mixin. Composition cannot be used to bypass the class-storage rule.

Associated Objects

`co.object` declares one named singleton object. It is already the object kind used to implement an annotation:

```
// AuditAnnotation.fol
_ co.object->(for=someAnnotation) = {
```

```
...  
}
```

An associated object follows the ordinary primary-declaration filename rule:

`<ObjectName>.fo1`. The filename may use any legal object name. There is no `<ClassName>.comp...` filename, no companion-owner name, and no requirement that the object be named after one target class. For example, `ProducerConsumerShared.fo1` declares the `ProducerConsumerShared` object used below.

The same declaration kind may be associated with one class or with an explicit list of classes:

```
// ProducerConsumerShared.fo1  
_ co.object->(  
  for=[Producer, Consumer]  
) = {  
  @co.dap.const capacity co.int = 100;  
  @co.dap.final configuration QueueConfiguration = loadConfiguration();  
  
  queue Queue;  
  queueLock co.lock;  
  
  enqueue(value Item)->>() = {  
    lock(queueLock) {  
      queue.add(value);  
    }  
  }  
  
  dequeue()->(Item) = {  
    lock(queueLock) {  
      this => queue.remove();  
    }  
  }  
}
```

`for=Target` is the scalar spelling for one target; `for=[Target1, Target2]` associates the object with an explicit target set. The target declaration kind determines the association role: an annotation target denotes an annotation implementation, while one or more class targets denote a shared support object for

those classes. A class-target list contains class declaration references, not class instances or runtime expressions. A target list cannot mix annotation and class declaration kinds.

An associated object has an ordinary independent name and is accessed through that name. Its members are not copied or promoted into any target class:

```
_ co.class = {  
  produce(value Item)->>() = {  
    ProducerConsumerShared.enqueue(value);  
  }  
}  
  
_ co.class = {  
  consume()->(Item) =>>  
    ProducerConsumerShared.dequeue();  
}
```

The association is explicit, non-owning, non-structural, and non-transitive:

- the associated object is not embedded in a target class;
- its fields do not become class fields;
- its methods are not inherited, promoted, overridden, or dynamically dispatched as class methods;
- its storage is not part of any class instance or reachable instance graph merely because of `for=`;
- one associated object may name multiple classes, and multiple independently named objects may name the same class;
- a subclass does not inherit an association from its parent; it must be named explicitly when association-specific access requires it;
- ordinary public access remains governed by normal name resolution and access rules rather than by inheritance;
- constructing, copying, freezing, sharing, locking, or destroying a class instance does not construct, copy, freeze, share, lock, or destroy an associated object; and
- locking or changing the associated object does not implicitly lock or change any associated class instance.

For example:


```

producer Producer = getProducer();
co.utils.makeImmutable(producer);

producer.currentItem = nextItem;           // compiler/runtime error
ProducerConsumerShared.enqueue(nextItem);  // still permitted

```

Even a deep instance policy does not traverse through a `for=` association, because the association is metadata rather than a managed-reference edge in the instance graph. The associated object has its own singleton lifetime and its own explicitly declared constant, immutable, shared, and locking rules.

Classes with Operator methods

```

// Employee.fol
_ co.class = {
  @co.dap.operator(symbol="+")
  add(other Employee)->(Employee) = {
    // implicit instance method: operands are this and other
  }

  @co.dap.operator(symbol=="")
  @co.dap.static
  equals(
    left Employee,
    right Employee
  )->(co.bool) = {
    // static operator implementation
  }

  @co.dap.operator(symbol">")
  @co.dap.class
  greater(
    left Employee,
    right Employee
  )->(co.bool) = {
    // class-associated operator implementation
  }
}

```

An unannotated operator function in a class is an instance method; the hidden `this` value contributes the first operand. FoLang has no explicit annotation for the default instance-method category. `@co.dap.static` and `@co.dap.class` do not contribute an implicit operand, so their declared parameters are the complete operator operand list. Operator signature normalization includes these method categories so equivalent declarations are diagnosed as duplicates.

Class Declaration Relationships

A class cannot physically contain named class, struct, enum, module, function, signature, interface, or other declaration definitions as nested declarations. Class bodies contain fields and methods permitted by the class model. Every class has the compiler-owned lifecycle members `@@new` and `@@init` as inherited lifecycle machinery; source-defined lifecycle overrides or overloads are additionally permitted only for generic classes whose `@co.dap.generic(types=[...], lifecycle=true)` metadata enables developer lifecycle customization.

Types and helper declarations used only by one class are declared in their ordinary legal source locations and restricted to that class with `@co.dap.local`:

```
// EmployeeAddress.fol
@co.dap.local(for=hr.employee.Employee)
_ co.struct = {
    street co.string;
    city   co.string;
}
```

```
// EmployeeStatus.fol
@co.dap.local(for=hr.employee.Employee)
_ co.enum = {
    Active,
    Inactive,
    Pending
}
```

```
// Employee.fol
_ co.class = {
    address EmployeeAddress;
    status  EmployeeStatus;

    getAddress()->(EmployeeAddress) = {
```

```

        this => this.address;
    }
}

```

The local declarations are visible while compiling `hr.employee.Employee`, but they do not become nested names such as `Employee.EmployeeAddress` or `Employee.EmployeeStatus`.

The following is invalid:

```

// Employee.fol
_ co.class = {
    Address co.struct = { // ❌ physical nested declaration
        city co.string;
    }
}

```

Ordinary visibility annotations do not widen a target-local declaration beyond the declaration or closed target set named by `@co.dap.local` or `@co.dap.nested`.

Method Types

```

// Employee.fol
_ co.class = {

    @co.dap.static
    getEmployee()->(Employee) ={}

    // unannotated: instance method
    getEmployee()->(Employee)={}

    @co.dap.class
    getEmployee()->(Employee) ={}

    @co.dap.object
    getEmployee()->(Employee)={}
}

@co.dap.oops(
    interfaces=[someInterface1, someInterface2, someInterface3],
    classes=[someClass1, someClass2],
    mixins=[someMixin1, someMixin2, someMixin3],
    traits=[someTrait1, someTrait2, someTrait3],
)

```

```
// test.fol
_ co.class = {
  getTest(id co.int)->(test) ={}
}
```

@co.dap.oops Relationship Fields

The four relationship fields have fixed target kinds and cardinalities:

Field	Required declaration kind	Direct entries
<code>classes</code>	<code>co.class</code>	zero, one, or two
<code>interfaces</code>	<code>co.interface</code>	zero or more
<code>mixins</code>	<code>co.mixin</code>	zero or more
<code>traits</code>	<code>co.trait</code>	zero or more

Every entry must resolve to the declaration kind required by its field. An entry repeated in the same field, an entry appearing in incompatible fields, a relationship cycle, an unresolved entry, or an entry of the wrong kind is a compile-time error. Each list is finite and preserves source order.

The `classes=[...]` list contains the direct concrete class parents. Its first entry is the **primary parent** and its optional second entry is the **secondary parent**. Class inheritance uses virtual-base identity semantics: when multiple inheritance paths reach the same canonical ancestor class, the resulting class contains one shared inherited ancestor portion rather than one copy for each path.

Named Relationship and Direct Parent Selection

`classes`, `mixins`, `traits`, and `interfaces` are contextual compile-time selectors available through `this` in an applicable class context. `this` denotes the current class/type receiver in a class method and the current instance receiver in an instance method. The selectors organize the four direct `@co.dap.oops(...)` relationship lists without combining their different semantics:

The `->` spelling in these forms is a compiler-owned relationship selector. It is recognized only with the hard-reserved `this` token on its left and one of the language-defined relationship names on its right. It is not general member access and cannot be used as `value->member`. Ordinary user-defined members, including

members named `parent`, `parents`, `super`, `classes`, `mixins`, `traits`, or `interfaces`, continue to use `.`; consequently `this->parent` and `this->parent` are distinct and cannot collide.

Selector category	Source relationship list	Selection meaning
<code>this->classes[Type]</code>	<code>classes = [...]</code>	exact direct concrete class-parent branch
<code>this->mixins[Type]</code>	<code>mixins = [...]</code>	exact directly composed mixin branch
<code>this->traits[Type]</code>	<code>traits = [...]</code>	exact directly composed trait branch
<code>this->interfaces[Type]</code>	<code>interfaces = [...]</code>	exact directly implemented interface view

These selectors describe direct relationships only; inherited or transitively composed declarations are not appended to the direct lists. The selector categories are not runtime objects or collections and cannot be used as standalone values.

Every relationship selection uses a compile-time type name:

```
this->classes[someClass1]    // selected direct class-parent receiver
this->classes[someClass2]    // selected second class-parent receiver
this->mixins[someMixin1]     // selected composed mixin receiver
this->traits[someTrait1]     // selected composed trait receiver
this->interfaces[someInterface1] // selected implemented-interface view
```

In a class method these expressions select type contexts. In an instance method they select the corresponding views or implementation branches of the current instance. Within these relationship-selector expressions, `this` is the ordinary receiver expression; the callable category determines whether that receiver is a type or an instance. This receiver meaning is distinct from the `this` token used by the symbolic executable-control productions.

The bracket operand is a declaration reference, not a numeric index, string, runtime expression, or `co.type` value. It may use the complete imported type name or a file-local import alias:

```
@co.ddap.import(package="company.behavior", as="behavior")

this->classes[behavior.BaseEmployee]
```

```
this->mixins[behavior.Auditable]
this->traits[behavior.Serializable]
this->interfaces[behavior.EmployeeContract]
```

The frontend resolves the written name or alias and records the canonical fully qualified declaration identity. Reordering a relationship list therefore cannot change selector meaning. An unresolved alias, wrong declaration kind, type absent from the corresponding direct relationship list, ambiguous type reference, numeric key, computed key, or missing key is a compile-time error.

`parent`, its `super` alias, and type-named `parents[Type]` are convenience selectors for direct class parents. None exposes a runtime collection:

```
this->parent           // primary direct class-parent receiver
this->super            // alias of this->parent
this->parents[someClass1] // same as this->classes[someClass1]
this->parents[someClass2] // same as this->classes[someClass2]
```

The singular `this->parent` and `this->super` forms always select the first entry of `classes = [...]`. Only plural `this->parents[Type]` is keyed. Consequently, `this->parent[Type]`, an unkeyed `this->parents`, and every numeric parent selector are invalid. When `classes` is empty, `this->parent`, `this->super`, `this->parents[Type]`, and `this->classes[Type]` are all invalid.

Only a `this->classes[Type]` or equivalent `this->parents[Type]` selection denotes a class-parent branch and can participate in parent lifecycle lookup. A `this->mixins[Type]` or `this->traits[Type]` selection may explicitly qualify a composed implementation when conflict resolution requires it. A `this->interfaces[Type]` selection produces the corresponding interface view; calls through that view use ordinary interface dispatch.

Lifecycle lookup follows the named parent:

```
this->parent::new();           // primary parent type in @@new
this->classes[someClass1]::new(); // explicitly named parent type
this->classes[someClass2]::new(); // explicitly named second parent type

this->parent::init();          // primary parent instance in @@init
this->classes[someClass1]::init(); // explicitly named parent instance
this->classes[someClass2]::init(); // explicitly named second parent
instance
```

If two parent branches contribute the same normalized method signature and neither implementation uniquely overrides the other, unqualified inherited lookup is ambiguous and is a compile-time error. The class may resolve the conflict by declaring a compatible override. A method body may explicitly select a source through `this->classes[Type]` or `this->parents[Type]` as appropriate. Composed mixin or trait conflicts are resolved through the corresponding `this->mixins[Type]` or `this->traits[Type]` selector.

Lifecycle Members: `@@new`, `@@init`, and `::` Invocation

`@@new` and `@@init` are compiler-owned **class lifecycle members**, not ordinary methods. Every `co.class` receives the compiler-provided lifecycle implementations as part of the language-defined class base behavior. Semantically, those inherited implementations behave like protected lifecycle members: they are available to the compiler/runtime and to source contexts that are permitted to reach protected parent lifecycle behavior, but they are not automatically exposed as ordinary source-callable lifecycle APIs.

The `@@` prefix is declaration syntax for customization of this compiler-owned lifecycle family. A source declaration named `@@new` or `@@init` never creates a new lifecycle name.

Normal class construction does not require developer lifecycle customization. The compiler may use the inherited lifecycle machinery internally while processing ordinary class construction according to the composite-construction rules.

Developer lifecycle customization is configured through the existing generic annotation. No separate lifecycle annotation exists:

```
@co.dap.generic(  
  types=[{name=T}, {name=R}],  
  lifecycle=true  
)  
_ co.class = {  
  ...  
}
```

The `lifecycle` field is an optional field of `@co.dap.generic(...)`.

For a **generic class**, the compiler interprets it as follows:

```
lifecycle field absent    -> inherited compiler lifecycle remains;  
                           developer override/overload is forbidden  
  
lifecycle=false          -> inherited compiler lifecycle remains;  
                           developer override/overload is forbidden  
  
lifecycle=true           -> inherited compiler lifecycle remains;  
                           developer may override or overload @@new /  
@@init
```

For other declaration kinds that legally use `@co.dap.generic(...)`, such as generic structs, generic functions, and generic methods, the `lifecycle` field is **not considered**. Its presence does not by itself produce a diagnostic and has no lifecycle effect on those declarations.

The lifecycle customization rules are:

1. source declarations named `@@new` or `@@init` are valid only as members of `co.class`;
2. the enclosing class must be generic through a valid `@co.dap.generic(types=[...])`;
3. that same generic annotation must have `lifecycle=true`;
4. `lifecycle=true` grants permission to **override or overload** the existing compiler-owned lifecycle family; it does not create the lifecycle family and does not by itself expose any lifecycle call;
5. a normal/non-generic class cannot source-declare a lifecycle override or overload;
6. a generic class with `lifecycle` absent or `lifecycle=false` cannot source-declare a lifecycle override or overload;
7. each developer-defined lifecycle override/overload has ordinary FoLang accessibility. A public lifecycle implementation is externally accessible; an implementation carrying any other valid accessibility classifier follows the normal rules of that classifier;
8. ordinary `Type::new(...)` / `object::init(...)` lookup considers the developer-defined lifecycle override/overload candidates for the resolved class. The

inherited compiler-provided lifecycle implementation is not automatically exposed as an ordinary source-callable candidate;

9. therefore `::new(...)` or `::init(...)` is valid for an ordinary caller only when a matching developer-defined lifecycle implementation exists and is accessible to that caller;
10. inside a valid lifecycle customization, access to an inherited parent lifecycle implementation is permitted when the ordinary protected/accessibility rules allow it; `this->parent::new(...)` in `@@new` and `this->parent::init(...)` in `@@init` select the primary parent, while `this->classes[Type]` or its `this->parents[Type]` alias explicitly selects a direct class parent by its resolved type identity; mixin, trait, and interface relationship categories do not participate in lifecycle lookup;
11. lifecycle customization eligibility is independent of project/package/component placement and follows the ordinary placement rules of the enclosing generic class.

Lifecycle invocation uses the dedicated `::` form for source-visible developer lifecycle implementations:

```
Employee::new(...)
employee::init(...)
```

`::` is **not** a second general-purpose method-call operator. It performs lifecycle-member lookup and may name only a lifecycle invocation name defined by the language. In the current language profile those names are `new` and `init`.

Ordinary member lookup and lifecycle lookup therefore remain separate:

```
Employee.new(...)      // ordinary class/static method named new
Employee::new(...)     // developer-defined lifecycle @@new candidate

employee.init(...)     // ordinary instance method named init
employee::init(...)    // developer-defined lifecycle @@init candidate
```

Consequently, FoLang does not reserve or block ordinary methods named `new` or `init`. Ordinary `.` lookup and lifecycle `::` lookup use different semantic channels.

The declaration/invocation mapping is fixed:

Developer lifecycle declaration	Lifecycle invocation	Meaning
<code>@@new(...)</code> override/overload	<code>Type::new(...)</code>	customized allocation / uninitialized-instance lifecycle operation
<code>@@init(...)</code> override/overload	<code>object::init(...)</code>	customized instance-initialization lifecycle operation

A lifecycle call always includes its call parentheses. Bare `Type::new` or `object::init` is not a first-class member reference.

This is a category-wide rule: any future compiler-owned lifecycle declaration added with the declaration spelling `@@name` must use the corresponding invocation spelling `receiver::name(...)` when a developer-defined accessible customization is invoked. Lifecycle declarations are never invoked through ordinary `.` member syntax.

```
// Employee.fol
@co.dap.generic(
  types=[{name=T}, {name=R}],
  lifecycle=true
)
_ co.class = {

  id T;
  name R;

  // Public lifecycle override: externally accessible through
  Employee::new(...).
  @co.dap.class
  @co.dap.public
  @co.dap.override
  @@new(a co.hokrlt, b co.hokrlt)->(co.uninit) = {
    T co.type = a;
    R co.type = b;

    // Valid protected parent-lifecycle access from a lifecycle
    customization.
    this->parent::new();

    this => co.uninit.instance(Employee, this);
```

```

}

// Private lifecycle overload/override: accessibility remains private.
@co.dap.override
@co.dap.constructor(access=private)
@@init() = {}

// Public lifecycle override/overload: externally accessible through
::init(...).
@co.dap.override
@co.dap.constructor(access=public)
@@init(id T, name R) = {
    // Valid protected parent-lifecycle access from a lifecycle
    customization.
    this->parent::init();

    this.id    = id;
    this.name  = name;
}

getEmployee(id T)->(Employee) = {}
}

a := Employee::new(co.int, co.string); // valid: public developer @@new
b := a::init(1, "Rao");                // valid: public developer
@@init

```

`lifecycle=true` alone does not expose the inherited compiler lifecycle implementations. If the generic class declares no developer lifecycle override/overload, it continues to use ordinary construction rules and has no newly exposed lifecycle API merely because the field is `true`.

A generic class without lifecycle customization permission remains an ordinary generic class:

```

@co.dap.generic(types=[{name=T}])
_ co.class = {
    value T;
}

```

```

// plain_generic_types.unit.fol
_ co.unit = {
    IntPlainGeneric co.type = PlainGeneric(co.int);
}

```

```

    someFun()->()={
        x IntPlainGeneric;
    }
}

```

The same metadata field is harmless on other generic declaration kinds:

```

@co.dap.generic(
    types=[{name=T}],
    lifecycle=true
)
_ co.struct = {
    value T;
}

```

The struct remains an ordinary generic struct; the `lifecycle` field is ignored for lifecycle semantics. The same target-insensitive metadata rule applies to generic functions and generic methods: `lifecycle=true` does not grant lifecycle customization to them and does not cause an error merely because the field is present.

```

_ co.unit = {
    @co.dap.generic(
        types=[{name=T}],
        lifecycle=true
    )
    identity(value T)->(T) = {
        this => value;
    }
}

```

`identity` remains an ordinary generic function. Its `lifecycle` field is not considered. If source attempts to place `@@new` or `@@init` in a struct, function, method, unit, or any other non-class context, the lifecycle declaration itself is rejected because lifecycle customization is class-only.

Anonymous Classes/Types

Anonymous/inline ordinary classes use the compiler-provided class lifecycle machinery internally but cannot opt into developer lifecycle customization because they do not satisfy the generic-class `lifecycle=true` contract. They therefore expose no developer-defined lifecycle API. `// somunit.unit.fol`

```

_ co.unit = {

    someFun ()->()={
        emp := co.class{};

        empObj := emp;

        empobj1 := co.class{
            name co.string;

            @co.dap.public
            doSomething(s co.string)->(co.int)={
                co.out.println(s);
                this.name=s;
                this => 33;
            }
        }.init(); //parameter less init only as co.class doesn't have any
other init

        empObj1.doSomething("abc"); //Compiler error as co.class doesn't
have this method
    }
}
// someInterface.fol
_ co.interface={

    doSomething(s co.string)->(co.int);
}

//someUnit.unit.fol

_ co.unit={

    someFun()->()={

        empObj1 someInterface = co.class{
            name co.string;

            @co.dap.public
            doSomething(s co.string)->(co.int)={
                co.out.println(s);
                this.name=s;
                this => 33;
            }
        }
    }
}

```

```
    }.init(); //when assigned to interface only default init no
parameters
```

```
    empObj1.doSomething("abc"); // successful
    // internally the anonymous class implements the interface
someInterface as empObj1 is now the type someInterface
    }
}
```

```
//SomeClass.fol
```

```
_ co.class = {

    name co.string;

    @co.dap.public
    doSomething(s co.string)->(co.int)={
        co.out.println(s);
        this.name=s;
        this => 33;
    }

}
```

```
//someUnit.unit.fol
```

```
_ co.unit={

    someFun()->()={

        empObj1 SomeClass = co.class{
            name co.string;

            @co.dap.public
            doSomething(s co.string)->(co.int)={
                co.out.println(s);
                this.name=s;
                this => 33;
            }
        }.init(); //default init always valid

        empObj2 SomeClass = co.class{
            name co.string;
```

```

        @co.dap.public
        doSomething(s co.string)->(co.int)={
            co.out.println(s);
            this.name=s;
            this => 33;
        }
    }.init(name="somename"); //valid as by default class will have
init with all field member implemented as optional named parameters

    empObj1.doSomething("abc"); // successful
    // internally the anonymous class inherits the class SomeClass as
empObj1 is now the type SomeClass
    }
}

```

`foLang` internally creates anonymous class and object using init method

Their ordinary construction/use continues to follow the anonymous-class rules independently of the lifecycle facility.

Conflicts and Resolutions

Inherited state and inherited behavior use different conflict rules. Field declarations describe storage and may be structurally merged when their state contracts are compatible. Concrete methods contain executable behavior and use method-source resolution instead. The first-position method rule described below never resolves a field conflict.

Inherited State Merging

Class and mixin fields accessible to the child are inherited automatically. Accessible fields with the same identifier are grouped before the child layout is produced.

Two or more declarations in one identifier group become one child instance field only when their canonical field types match. Class fields cannot carry field-level immutability, constant, Shared, locking, or CopyOnWrite status, so none of those policies participates in inherited-field compatibility.

The canonical type comparison includes any refinement or other constraint represented by the type. FoLang does not compare a separate field-constraint property.

When the compatible declarations have different access specifiers, the child receives the broader accessible level:

```
public > protected > internal
```

Private fields are not visible to the inheriting child and do not participate in its accessible-field merge. Fields are always instance state. Class-associated fields, static fields, inline class-field initializers, field ownership modifiers, and separate field atomicity modifiers are not part of this merge model.

Inherited field situation	Child result
Identifier occurs in only one accessible source	Inherit that field
Same identifier and canonical type	Merge into one child storage slot
Same identifier but different canonical types	Compile-time error
Compatible fields with different accessible access specifiers	Merge and assign the broader access specifier
Private source field	Excluded from the child's accessible-field merge

For example, these declarations produce one public child field:

```
// someClass1.fol
_ co.class = {
  @co.dap.public
  value co.int;
}

// someClass2.fol
_ co.class = {
  @co.dap.protected
  value co.int;
}

// child.fol
@co.dap.oops(
  classes = [someClass1, someClass2]
```



```
)  
_ co.class = {  
}
```

The child contains one `value co.int` slot with public accessibility. Methods reached through either parent operate on that same child storage slot.

The following declarations conflict because their canonical types differ:

```
// someClass1  
@co.dap.public  
value co.int;  
  
// someClass2  
@co.dap.protected  
value co.string;
```

`@co.dap.final`, `@co.dap.const`, `Shared`, field locking, and `CopyOnWrite` are rejected on every class or mixin field that would contribute to class-instance storage before inherited-state merging begins. `Shared` or independently fixed state used by multiple classes belongs to an associated `co.object`, not to an inherited class-storage slot.

Method Resolution

Methods are matched by their complete normalized signatures. The compiler does not merge or compare distinct concrete method bodies merely because their signatures match.

Inherited method situation	Resolution
Same canonical method declaration reached through multiple paths	Deduplicate as one inherited method
Distinct matching concrete methods within <code>classes</code> , within <code>mixins</code> , or within <code>traits</code>	Select the method from the first source in that relationship list
Distinct matching concrete methods across relationship categories	Compile-time error until the child explicitly resolves the conflict
Matching abstract obligations from interfaces, traits, or mixins	One common implementation may satisfy all exact matching slots

Source-specific abstract behavior required	Use type-qualified <code>@co.dap.implement</code> declarations
Matching mandatory virtual methods	One common <code>@co.dap.override</code> may override all exact matching slots
Child supplies a compatible override	The child method becomes the effective implementation

The first-source rule applies separately within one relationship category. It does not choose between a class and a mixin, a class and a trait, or a trait and a mixin. Those cross-category conflicts require an explicit child method.

The child may override the automatically selected same-category method when it wants different behavior or explicit delegation. Type-named selectors identify the exact source implementation:

```
@co.dap.override
run() -> () = {
  this->classes[someClass2].run();
}
```

Every implementation or override must match the applicable normalized method signature. Accessibility rules continue to apply; conflict resolution cannot select an inaccessible source method.

```
// someInterface1.fol

_ co.interface = {
  m1( a co.int)->(co.int);
}

// SomeInterface2.fol

_ co.interface = {
  m1( a co.int)->(co.int);
}

// SomeInterface3.fol

_ co.interface = {
  m1( a co.int)->(co.int);
```

```

}

// someMixin1.fol

_ co.mixin= {

    m8 (a co.int)->()={}

    m9 (b co.string)->()={}

    @co.dap.virtual
    m10 ()->()={

    }

    @co.dap.abstract
    m11()->();

}

//someTrait1.fol

_ co.trait = {

    m14 (a co.int)->()={}

    m15 (b co.string)->()={}

    @co.dap.virtual
    m10 ()->()={

    }

    @co.dap.abstract
    m11()->();

}

@co.dap.oops(
    interfaces = [
        someInterface1,
        someInterface2,
        someInterface3

```

```

],

classes = [
    someClass1,
    someClass2
],

mixins = [
    someMixin1
],

traits = [
    someTrait1
]
)

_ co.class = {
    // Interface-specific implementation.
    // The target method is inferred from the attached declaration.
    @co.dap.implement(type = someInterface1)
    m1(a co.int) -> (co.int) = {
        this => a;
    };

    // A second interface-specific slot with the same source
name/signature.
    @co.dap.implement(type = someInterface2)
    m1(a co.int) -> (co.int) = {
        this => a + 1;
    };

    // Explicit mapping from someInterface3.m1 to child method m1_3.
    @co.dap.implement(
        type = someInterface3,
        method = m1(co.int)->(co.int)
    )
    m1_3(a co.int) -> (co.int) = {
        this => a + 2;
    };

    // Overrides both matching mandatory virtual slots contributed
    // by someMixin1 and someTrait1.
    @co.dap.override
    m10() -> () = {
    };

```

```

// Separate source-qualified abstract implementations.
@co.dap.implement(type = someMixin1)
m11() -> () = {
};

@co.dap.implement(type = someTrait1)
m11() -> () = {
};

// Alias/wrapper around automatically composed concrete methods.
m15t(a co.string) -> () =>>
    this->traits[someTrait1].m15(a);

m9M(a co.string) -> () = {
    this->mixins[someMixin1].m9(a);
};
}

```

folang provides type specific override/implementation apart from common one

Class Declaration and Member Access Rules

The class declaration itself is package-owned and follows [Package Declaration Access Rules](#). `@co.dap.internal` is a class-member access classifier; it is not valid on the class declaration itself. Applying it to a class declaration produces an `InvalidMetadataTarget` diagnostic.

After the class declaration is accessible, its fields, methods, and developer-defined lifecycle implementations follow this class-member table:

Annotation	Same Package	Parent Package	Subclass in Another Package	Descendant Package	Unrelated
<code>@co.dap.public</code>	✓	✓	✓	✓	✓
<code>@co.dap.package</code>	✓	✓	✗	✓	✗
<code>@co.dap.protected</code>	✓	✗	✓	✗	✗

@co.dap.private	✓	✗	✗	✗	✗
@co.dap.internal	✓	✗	✗	✓	✗

Class-member access never widens access to the class declaration. Inheritance, override, implementation, and inherited-field merging first operate on members that are accessible under this table and then apply their separately defined relationship and conflict rules.

Associated Object Member Access Rules

The associated object declaration itself follows [Package Declaration Access Rules](#). After the object is accessible, its fields and functions follow this table:

Annotation	Inside the Object Ownership Boundary	Outside the Object Ownership Boundary
@co.dap.public	✓	✓
@co.dap.private	✓	✗

A public object member does not widen access to an inaccessible associated object.

Interfaces

```
// IEmployee.fol
_ co.interface = {
  storeEmployee(emp Employee)->(Employee);
}
```

Signatures

```
// Employee is an ordinary package-level declaration.
// MEmployee.fol
_ co.signature = {
  storeEmployee(emp Employee)->(Employee);
}
```

Structurally they look similar — both are lists of contracts. The difference is **who implements them and how**.

	<code>co.signature</code>	<code>co.interface</code>
Implemented by	module via <code>matches=</code>	class via <code>implements=[]</code>
Number of implementations	Any number of distinct modules may match one signature	Any number of classes may implement one interface
Runtime cardinality of one implementation declaration	One module object per loaded module identity	Any number of independently constructed class objects
State model	One shared state for all references to the same module	Separate per-instance state for each class object
May specify required values	✓	✗ — methods only
May reference package-level types	✓	✓
May declare associated or fixed/manifest type components	✓	✗
May require parameterized types	✓	✗
May declare physical nested/local types	✗	✗
May use <code>@co.dap.local</code>	✗	✗
Instantiation involved	Module is declared once, not constructed	Class objects are constructed
Reference use	Compatible module references may be used through the signature type without creating another module	Interface references may refer to any implementing object instance
OOP dispatch	✗	✓ virtual/dynamic
Contract style	module values, functions, and associated/fixed type components	behavioral methods on object instances

- A `signature` is a **module contract** over values, functions, existing package-level types, associated-type requirements, and fixed/manifest type components. A type-component specification is a contract slot, not a physical nested type definition. Multiple modules may match one signature, but each module declaration denotes one module object with shared module state.
- An `interface` is a **behavioral contract** tied to class dispatch and polymorphism. It cannot declare associated or fixed module type components or own nested type definitions. A class implementing an interface may create any number of independent runtime objects.

Modules

A module is a language-level implementation component governed by an optional signature. A module declares conformance only through `matches=Signature` on its `co.module` declaration; this is the single source-level module-to-signature relationship. A module may use package-level types, satisfy associated-type requirements declared by its matched signature, and use fixed/manifest type components established by that signature. It does not physically own or nest arbitrary type declarations.

```
// Employee.fol – ordinary package-level type
_ co.struct = {
  Id    co.int;
  Name  co.string;
}

// EmployeeModule.fol
_ co.signature = {
  getEmployee(id co.int)->(Employee);
}

// EmployeeModImpl.fol
_ co.module->(matches=EmployeeModule) = {

  getEmployee(id co.int)->(Employee) = {
    this => Employee{
      Id: 10,
      Name: "Rao"
    };
  };
}
```



```
}  
  
mm EmployeeModule = EmployeeModImpl;  
v Employee = mm.getEmployee(10);
```

Usage and Liveness

Module, signature, interface, typeclass, instance, matcher, data-shape, and annotation/object liveness is defined centrally in [Unused Symbols, Liveness, and Reachability](#).

Module Cardinality and Identity

A module declaration defines exactly one named module object for that loaded module identity. It is not an instantiable blueprint and does not create a new module object each time its name is referenced.

```
first EmployeeModule = EmployeeModImpl;  
second EmployeeModule = EmployeeModImpl;
```

`first`, `second`, and `EmployeeModImpl` refer to the same module object. These bindings copy or retain the module reference; they do not clone or instantiate the module. Consequently, values declared directly by the module represent one shared module state for all references to that module.

A signature does not itself create a module object. It is a contract, and any number of separately declared modules may conform to the same signature:

```
DatabaseBackend signature  
├─ PostgreSQLBackend module -> one named module object  
├─ MySQLBackend module     -> one named module object  
└─ TestDatabaseBackend module -> one named module object
```

Each conforming module declaration contributes its own single module object and its own module state. The signature does not restrict the number of distinct conforming module declarations.

Modules are also broader than ordinary interface implementations. A matching module may provide module values and functions, bind associated types required by its signature, and define required type-alias components from those resolved associated types. Fixed/manifest type components are established by the signature

itself and are used by the module without being rebound. An interface constrains object behaviour; it does not provide the same module type-component abstraction.

By contrast, a class declaration defines an instantiable type. Every class construction creates a distinct object with independent identity, state, and lifetime:

```
PostgreSQLBackend module
```

```
└─ one shared module object and module state
```

```
PostgreSQLConnection class
```

```
└─ connection1 -> independent object and state
```

```
└─ connection2 -> independent object and state
```

```
└─ connection3 -> independent object and state
```

Module instantiation A FoLang class or struct declaration introduces an instantiable type but does not create an instance. A FoLang module declaration introduces one named module component directly into its package. The module name acts as the binding for that component, so no separate construction expression is required. The module's runtime state is initialized once according to the language's module-initialization rules.

A module declaration is a singleton component declaration and binding, rather than merely an object definition.

Module Signature Contents

A `co.signature` is a declarative contract for a module. It may specify required module values and functions, associated-type requirements, and fixed/manifest type components. A signature does not allocate storage, initialize variables, execute statements, or provide function bodies.

A signature may contain:

- value specifications
- function signatures
- references to already existing accessible types
- associated-type specifications

- required type-alias component specifications
- fixed or manifest type-component specifications

A signature may not contain:

- value initializers
- executable statements
- function bodies
- concrete class, struct, enum, module, interface, or signature definitions
- arbitrary nested or target-local declarations

Associated and fixed/manifest type-component specifications are part of module conformance. They are contract slots, not Java-, C++-, or C#-style inner types, and they do not participate in `@co.dap.local`.

Value Specifications

A declaration such as:

```
// Counter.fol
_ co.signature = {
  count co.int;
  increment(amount co.int)->();
}
```

requires a matching module to provide a value named `count` of type `co.int` and a compatible `increment` function. The signature does not initialize `count` and does not define `co.int`; the built-in type already exists.

```
// CounterImpl.fol
_ co.module->(matches=Counter) = {
  count co.int = 0;

  increment(amount co.int)->() = {
    count.value = count + amount;
  }
}
```

The same rule applies when a value or function specification uses an existing accessible package type:

```
// EmployeeRepository.fol
_ co.signature = {
  current hr.employee.Employee;
  find(id co.int)->(hr.employee.Employee);
}
```

The matching module must provide `current` and `find`. It does not redefine `hr.employee.Employee`.

Abstract Type Components

A bodyless `co.type` declaration is an ordinary unresolved type component in a signature. Every matching module must supply its concrete type alias:

```
// Repository.fol
_ co.signature = {
  Entity co.type;

  current Entity;
  find(id co.int)->(Entity);
}
```

`Entity co.type;` leaves the identity of `Entity` abstract in the signature. It is not generic or parameterized merely because different matching modules can supply different concrete aliases:

```
// EmployeeRepositoryImpl.fol
_ co.module->(matches=Repository) = {
  Entity co.type = hr.employee.Employee;

  current Entity = ...;
  find(id co.int)->(Entity) = { ... }
}
```

Within a matching module, `Entity co.type = ...` fulfills the signature's ordinary abstract-type requirement. Its name must correspond to the bodyless `co.type` component declared by the matched signature. A module cannot use this form to introduce unrelated module-local types.

If the signature itself requires `hr.employee.Employee`, it uses that concrete type directly. The `Entity` slot is useful only when different matching modules may choose different concrete entity types.

An abstract type-component requirement differs from `forward` and `extern` declarations:

```
abstract type-component requirement
    -> each matching module supplies its own compatible type binding

forward declaration
    -> one specific declaration is completed later

extern declaration
    -> one specific declaration is defined in another linkage or
    compilation unit
```

Fixed or Manifest Type Components

A signature may fix a type component to an already known type:

```
// IntegerRepository.fol
_ co.signature = {
    Id co.type = co.int;

    find(id Id)->(co.bool);
}
```

Here `Id` is predetermined as `co.int`. This is a fixed/manifest type component rather than an abstract type requirement: the signature itself establishes the type equality, so a matching module uses `Id` but does not bind or redefine it.

```
Entity co.type;                -> abstract type slot; matching module
supplies `Entity co.type = ...`
Id co.type = co.int;           -> fixed/manifest; signature supplies the type
equality
```

Associated Types and Required Type-Alias Components

A signature may require an associated element type and a separate type-alias component without defining either representation:

```
// StackSignature.fol
_ co.signature = {
    T      co.associatedType;
    Stack co.type;
```

```

empty(value T)->(Stack);
push(value T, stack Stack)->(Stack);
pop(stack Stack)->(T, Stack);
}

```

`T co.associatedType;` declares the unknown type name associated with the generic-container representation of `Stack`. `Stack co.type;` states that the matching module must define the `Stack` alias by applying that `T` to a generic container type. The signature's functions can use both names before their concrete bindings are known.

Unlike the ordinary abstract component `Entity co.type;`, `T` is an `associatedType` because its purpose is to parameterize and determine another type component. Here `Stack` depends on `T`.

An associated type is therefore the module/signature mechanism for passing a type name into a type alias whose right-hand side instantiates a generic container type. It is not a general-purpose abstract module type. If no module type alias instantiates a generic container, no `co.associatedType` component is needed. This mechanism does not use `@co.dap.generic`: the signature declares the associated container argument, and each matching module supplies its concrete type during module conformance. `@co.dap.generic` remains the mechanism for generic classes, structs, and functions.

A matching module supplies both bindings:

```

// ListStackModule.fol
_ co.module->(matches=StackSignature) = {
  T co.associatedType = co.int;
  Stack co.type = co.List(T);

  empty(value T)->(Stack) = { ... }
  push(value T, stack Stack)->(Stack) = { ... }
  pop(stack Stack)->(T, Stack) = { ... }
}

```

The module first resolves `T` to `co.int`; the `Stack` alias therefore resolves to `co.List(co.int)`. Full generic application belongs on the right-hand side of the alias. Parameters and results use the alias name `Stack`, not repeated `Stack(T)` applications.

Another matching module may choose another element and representation:

```
// ArrayStackModule.fol
_ co.module->(matches=StackSignature) = {
  T co.associatedType = Employee;
  Stack co.type = collections.ArrayStack(T);
  ...
}
```

Therefore:

```
StackSignature
  -> requires associated type T and type alias Stack

ListStackModule
  -> binds T to co.int
  -> defines Stack as co.List(T)

ArrayStackModule
  -> binds T to Employee
  -> defines Stack as collections.ArrayStack(T)
```

This form deliberately replaces a parameterized associated-type spelling such as `Stack(T) co.associatedType`. `Stack` is a type selected or derived by the implementation after `T` is known; it is not an implicitly applied generic constructor.

An associated-type binding or required alias definition does not permit physical type nesting. When an implementation needs a new concrete struct, class, or enum representation, that declaration remains an ordinary package-owned declaration and may be restricted to the implementing module with `@co.dap.local`; the module then refers to that declaration from the associated-type or alias binding.

Signature Conformance Rules for Type Components

For every type component in a matched signature:

- each `co.associatedType` component must receive exactly one compatible `co.associatedType` binding from the matching module
- every `co.associatedType` component must be referenced by at least one required `co.type` component whose matching-module definition instantiates a generic container type; an unreferenced associated type is a compiler error

- a matching module that binds an associated type must contain the corresponding required `co.type` alias definition that uses that binding
- each bodyless `co.type` component must receive exactly one alias definition from the matching module
- a fixed/manifest component must retain the type equality declared by the signature and must not be rebound by the module
- component names must be unique within the signature
- associated-type bindings and required alias definitions cannot contain executable code
- an associated-type binding or required alias definition that does not correspond to a component declared by the matched signature is a compiler error
- required aliases are resolved after their referenced associated types; cycles among type components are a compiler error
- types referenced by value and function specifications must resolve after applying the module's associated-type bindings, required alias definitions, and fixed/manifest type equalities
- function conformance is checked after this substitution

Module Declaration Relationships

A module cannot physically declare nested structs, enums, classes, modules, signatures, interfaces, or other arbitrary named declarations. It references ordinary package-level declarations through its functions and signature. The only type-like declarations permitted directly in a matching module are `co.associatedType` bindings and `co.type` definitions that satisfy corresponding components declared by its matched signature; such bindings do not create independent nested declarations.

A declaration intended only for one module may be restricted with `@co.dap.local`:

```
// EmployeeModuleConfig.fol
@co.dap.local(for=hr.employee.EmployeeModImpl)
_ co.struct = {
    timeout co.int;
    retries co.int;
}
```



```
// EmployeeModImpl.fol
_ co.module = {
  connect(cfg EmployeeModuleConfig)->(co.bool) = {
    ...
  }
}
```

The following remains invalid:

```
// EmployeeModImpl.fol
_ co.module = {
  Config co.struct = { // ❌ physical nested declaration
    timeout co.int;
  }
}
```

A target-local declaration does not automatically become a module member name and is not projected through the module's signature. It becomes part of the signature view only when an associated-type component is explicitly bound to it or when a signature value/function specification references it through an allowed type component.

Module Member Access Rules

The module declaration itself follows [Package Declaration Access Rules](#). After the module is accessible, its values and functions follow this table:

Annotation	Inside the Module Ownership Boundary	Outside the Module Ownership Boundary
@co.dap.public	✓	✓
@co.dap.private	✓	❌

A public module member does not widen access to an inaccessible module. A compatible signature view may expose only members permitted by both the module's member access and the signature-conformance rules.

Structs vs Classes vs Objects vs Modules vs Units vs Packages

	Struct	CStruct	Class	Object	Module	Unit	Package
--	--------	---------	-------	--------	--------	------	---------

Purpose	Pure data shape	C-like value type	Per-instance data plus OOP behaviour	Named singleton for annotation implementation or explicitly associated shared support	Signature-backed ML-style abstraction	Stateless package-fragment or struct-companion container	Folder-based grouping
Fields	✓	✓ simple only	✓ ordinary mutable fields per instance; no field-level policies	✓ singleton-owned constant, immutable, global, shared, and locked state	✗	✗	✗
Module-level values	✗	✗	✗	✗ — object-owned state is distinct	✓ when declared directly or required by a signature	✗	✗

Functions / methods	Companion functions through <code><StructName>.companionUnit.foo</code> ; explicit receivers must match the struct	✗	✓ instance/OOP methods	✓ singleton-object functions	✓ module functions	✓ package functions in ordinary units; companion functions in companion units	✗
Lifecycle (<code>@@new</code> / <code>@@init</code>)	✗	✗	✓ compiler-owned lifecycle exists for every class; source override/overload requires generic class + <code>lifecycle=true</code>	✗ — one object per declaration	✗	✗	✗
<code>this</code> receiver	✗	✗	✓ instance or class/type receiver according to method category	singleton-object receiver in object functions	✗	✗	✗

Value/literal construction	✓	✓	✗	✗	✗	✗	✗
Explicit lifecycle invocation (<code>::new</code> / <code>::init</code>)	✗	✗	✓ only matching developer-defined lifecycle override/overload candidates, subject to their accessibility	✗	✗ — one module object per declaration	✗	✗
Runtime state cardinality	Per bound struct object	Per value	Per class object	One singleton state per object declaration	One shared state for the module declaration	—	—
First class	✓	✓	✓	✓ singleton-object reference	✓ module reference; referencing does not instantiate	✗	✗
Pass by	Reference	Value	Reference	Reference to the same singleton object	Reference to the same module object	—	—
Contract / association	—	—	interface via interfaces=[]	explicit non-owning for=Target	signature via matches =	none	—

				<code>get</code> or <code>for=</code> <code>[...]</code> association			
OOP / inheritance	✗	✗	✓	✗	✗	✗	✗
Physically nested independent named type/container declarations	✗	✗	✗	✗	✗	✗	N/A — packages contain separate source declarations
May be an <code>@co.dap.local</code> target	✓	✗	✓	—	✓	✗	✗
Pattern matching	✓	✓	✓	✗	✗	✗	✗
Direct ABI / zone boundary safe	✗ — library boundaries require snapshots	✓	✗	✗	✗	✗	✗
Associated functions	✓ through <code><Struct Name>.comp.unit.fol</code>	✗	—	owns its named singleton functions	—	✓ only in a struct companion unit	✗
Embedding	✓	✗	—	✗	—	✗	✗

Declared with	<code>co.struct</code>	<code>co.cstruct</code>	<code>co.classes</code>	<code>co.objects</code>	<code>co.module</code>	<code>co.union</code>	folder path
---------------	------------------------	-------------------------	-------------------------	-------------------------	------------------------	-----------------------	-------------

Declaration scoping rule: FoLang does not permit physical nesting of independent file-backed primary declarations. Classes, structs, cstructs, enums, unions, modules, interfaces, signatures, instances, matchers, and other package-owned primary declarations remain in their own `<Name>.fo1` files. Ordinary and companion unit files are explicit package containers: they may contain functions and the non-UDT type declarations permitted by the unit rules, but they may not contain independent primary declarations such as classes, structs, enums, modules, interfaces, or signatures. Ordinary local functions and anonymous expressions remain the other explicit nesting exceptions. Supported package-owned declarations may restrict visibility to exact same-package targets with `@co.dap.local`; the annotation changes visibility, not physical ownership. Signature type components and matching-module `co.associatedType` or `co.type` bindings that fulfill those components are contract slots rather than arbitrary nested package declarations. ***

Local and/or Nested types and functions

FoLang does not provide Java-, C++-, or C#-style physical nesting of independent named type and container declarations. Such declarations remain in their ordinary legal source locations. Ordinary local functions and anonymous expressions are explicit exceptions governed by the rules below.

Physical Nesting Rules

Prohibited Independent Named Declarations

Independent file-backed primary declarations cannot be physically declared inside another class, struct, cstruct, enum, union, module, unit, interface, signature, function, or executable block. This includes:

- classes, structs, cstructs, enums, and unions;
- modules, interfaces, signatures, and additional units;
- instances, matchers and other file-backed primary declarations.

Named non-UDT type declarations are lexical declarations and may be introduced wherever this specification admits declarations: directly in `src/app1.fo1`, in every non-operator component surface, in units, classes, interfaces, modules, mixins and signatures, and inside functions or nested executable blocks. This family includes aliases, parameterized and variant-based `co.type` declarations, polymorphic type definitions, newtypes, opaque types, dependent types, refinement and predicate types, subtypes, supertypes, and every other named type-producing declaration. Their names belong to the lexical scope in which they are declared. A function-local alias therefore exists only in that function/block and is not contributed to the package namespace.

This rule does not turn independent file-backed UDTs into nested types: classes, structs, modules, and the other primary declaration kinds still use their own `<Name>.fo1` files. A signature may contain both named type definitions and abstract `co.type` requirements. A signature `co.associatedType` member remains an abstract contract requirement whose matching binding is supplied by a module; associated types retain those narrower signature/module rules and are not general class, mixin, or unit type declarations. Macros, templates, and decorators follow their own declaration rules.

File-backed primary declarations retain package-owned identity and follow their normal `<Name>.fo1` placement rules. An association or visibility annotation such as `@co.dap.local`, `@co.dap.nested`, or `@co.dap.inner` does not physically move a separately declared declaration inside its target.

`co.associatedType` is the exception to general lexical type placement. It may be declared only as an abstract requirement in a signature or as the corresponding concrete binding in a matching module.

Local-Function Exception

A function body may contain an ordinary named local or inner function where the grammar permits a local-function declaration:

```
outer()->() = {  
    value co.int = 10;  
  
    inner()->() = {  
        co.out.println(value);  
    }  
}
```

```
    inner();  
}
```

The local function has block-local declaration identity. Its free runtime names are resolved from its lexical declaration context, and its lifetime and escape behavior follow the ordinary inner-function rules. It is not a package member and cannot be independently imported or exported.

An inner/local function cannot be generic and must not carry `@co.dap.generic`. It may use a generic type marker already introduced by an enclosing generic class or function, but it cannot introduce a new generic parameter list or its own generic specialization boundary. A generic function must be declared directly in a unit, class, module, mixin, or signature.

This exception permits local functions only. It does not permit a named class, struct, enum, module, unit, interface, signature, or another named type/container declaration inside a function body.

Anonymous-Expression Exception

The physical-nesting restriction does not apply to anonymous constructs that are expressions or type expressions rather than independent named declarations. They may be nested wherever their specific grammar category is permitted.

These include:

- anonymous function expressions;
- lambdas and callback blocks;
- anonymous class or anonymous type expressions;
- references to named polymorphic types declared through `co.type`;
- ordinary nested block, composite construction, and other value-producing expressions.

```
process()->() = {  
    operation := (value co.int)->(co.int) {  
        this => value * 2;  
    };  
  
    worker := co.class {  
        run(value co.int)->(co.int)={
```



```

        this => operation(value);
    }
};
}

transformer co.type = forall(T).(T)->(T);

```

An anonymous construct has no independently addressable package-owned declaration identity. Its scope, capture, lifetime, type, and escape behavior are determined by the rules for that specific construct. Syntactic containment of an anonymous expression does not create a Java-, C++-, or C#-style named nested declaration and does not violate the one-primary-declaration-per-package-file rule.

Relationship to Association Annotations

The exceptions above are distinct from FoLang's association annotations:

- an ordinary local function is physically declared inside an enclosing function and is lexically scoped;
- an anonymous construct is an expression without independent declaration identity;
- `@co.dap.local`, `@co.dap.nested`, and `@co.dap.inner` apply to separately declared declarations that retain their normal source identity.

The remainder of this section defines those separately declared association forms.

```
@co.dap.local(for=<declaration-reference>)
```

or:

```
@co.dap.local(
    for=[
        <declaration-reference>,
        <declaration-reference>
    ]
)
```

The annotated declaration is called a **target-local declaration**. The declarations named by `for` form its **local target set**.

Supported Declaration and Target Kinds

`@co.dap.local` may be applied only to these declaration kinds:

- `co.class`
- `co.struct`
- `co.enum`
- `co.module`
- a named function declared in a context that normally permits functions

Every entry in the local target set must resolve to exactly one:

- class
- struct
- enum
- module
- function overload

A target list may contain any combination of supported target kinds.

Signatures and interfaces do not support target-local or physically nested declarations. A `co.signature` or `co.interface` cannot be annotated with `@co.dap.local` and cannot appear in a local target set. A signature may declare abstract, fixed, or generic **type-component specifications** as part of its module contract; these are contract requirements rather than local or nested type definitions. Interfaces do not declare signature type components.

Other declaration kinds are not target-local unless a later section explicitly permits them.

Target-Set Syntax

The `for` argument accepts either:

1. one declaration reference; or
2. a non-empty list of declaration references.

The scalar form is equivalent to a singleton target list:

```
@co.dap.local(for=hr.employee.Employee)
```

```
@co.dap.local(for=[hr.employee.Employee])
```

The scalar form is canonical when only one target is required. Use the list form when two or more declarations require access:

```
@co.dap.local(  
    for=[  
        hr.employee.Employee,  
        hr.employee.EmployeeService,  
        hr.employee.EmployeeValidator  
    ]  
)  
// EmployeeState.fol  
_ co.enum = {  
    Active,  
    Inactive  
}
```

Rules:

- the target list must contain at least one declaration reference
- each reference must resolve independently and unambiguously
- duplicate references to the same resolved declaration are a compiler error
- list order does not affect visibility or declaration identity
- the target list is a closed set; access is not granted to declarations omitted from it
- repeated `@co.dap.local` annotations are not used to accumulate targets; use one annotation with one complete target list

Invalid:

```
// State.fol  
@co.dap.local(for=[])  
_ co.struct = { ... }  
// ❌ empty target list
```

```
@co.dap.local(  
    for=[  
        hr.employee.Employee,  
        hr.employee.Employee  
    ]  
)
```

```
)  
// State.fol  
_ co.struct = { ... }  
// ❌ duplicate resolved target
```

Declaration-Reference Syntax

Each `for` entry is a compiler-resolved declaration reference, not a string, runtime expression, function call, or `co.meta.symbol` value.

For a non-function target, use only its complete qualified declaration name:

```
@co.dap.local(for=hr.employee.Employee)  
@co.dap.local(for=hr.employee.EmployeeStatus)  
@co.dap.local(for=hr.employee.EmployeeRules)
```

For a function target, use its complete qualified callable reference because FoLang permits overloads. The callable identity and canonical parameter signature select the overload; the written return tail records and validates the selected declaration's return contract but does not participate in overload selection:

```
@co.dap.local(  
  for=hr.employee.Employee.calculate(co.float)->()  
)
```

Function references in a target list follow the same rule:

```
@co.dap.local(  
  for=[  
    hr.employee.Employee.calculate(co.float)->(),  
    hr.employee.Employee.calculate(co.int)->(),  
    hr.employee.Employee.validate()->(co.bool)  
  ]  
)  
// CalculationState.fol  
_ co.struct = { ... }
```

Parameter names are not part of the reference:

```
// ✅ canonical  
hr.employee.Employee.calculate(co.float)->()
```

```
// ✗ parameter names are not declaration identity  
hr.employee.Employee.calculate(amount co.float)->()
```

The qualified callable identity and parameter types must select exactly one overload. The return tail must then match that already-selected overload's declared return signature; it is validation metadata, not an overload discriminator. An abbreviated function name, unresolved target, parameter-ambiguous overload, or mismatching return tail is a compiler error.

```
@co.dap.local(for=hr.employee.Employee.calculate)  
// ✗ ambiguous when calculate is overloaded
```

Each listed overload is an independent target. Listing one overload does not grant access to sibling overloads with the same function name.

Same-Package Requirement

The target-local declaration and **every declaration in its local target set** must belong to the same exact package. The compiler compares their complete folder-derived package identities; matching only a parent package, package family, import alias, library, or source root is not sufficient.

For a function target, the target package is the package that owns the function's enclosing class, struct companion unit, module, unit, or other legal function container. A target-local function is checked in the same way: the package containing its legal function-owning declaration must match the package of every listed target.

The invariant is:

```
package(target-local declaration)  
  == package(target 1)  
  == package(target 2)  
  == ...
```

```
// hr/employee/Employee.fol  
_ co.class = { ... }  
  
// hr/employee/EmployeeService.fol  
_ co.class = { ... }
```

```
// hr/employee/EmployeeState.fol
@co.dap.local(
  for=[
    hr.employee.Employee,
    hr.employee.EmployeeService
  ]
)
// EmployeeState.fol
_ co.enum = { Active, Inactive }
// ✅ all declarations belong to package hr.employee
```

A declaration in another package cannot participate in the local target set, even when ordinary package visibility, imports, aliases, parent/subpackage relationships, or library membership would otherwise make the declaration resolvable.

```
// EmployeeState is declared in package hr.internal
@co.dap.local(
  for=[
    hr.employee.Employee,
    hr.internal.EmployeeService
  ]
)
// EmployeeState.fol
_ co.enum = { Active, Inactive }
// ❌ local declaration and every target must have the same package
```

```
// CalculationState is declared in package hr.employee.internal
@co.dap.local(
  for=hr.employee.Employee.calculate(co.float)->()
)
// CalculationState.fol
_ co.struct = { ... }
// ❌ subpackages are distinct packages; both sides must be exactly
hr.employee
```

The same-package rule prevents `@co.dap.local` from becoming a cross-package friendship or visibility-bypass mechanism. A local declaration is a selectively shared implementation detail inside one package, not an imported helper attached from elsewhere.

Visibility Domain

A target-local declaration may be resolved only from:

1. each exact declaration in its local target set; and
2. lexical scopes contained within each listed target's implementation.

For targets `A`, `B`, and `C`, the visibility domain is the union of their implementation scopes:

```
visibility(local declaration)
  = scope(A) ∪ scope(B) ∪ scope(C)
```

It is not an intersection, and code does not need to be simultaneously inside every target.

Consequences:

- a declaration local to a class is available to that class body and its methods
- a declaration local to a module is available to that module body and its functions
- a declaration local to a struct is available while resolving that struct declaration
- a declaration local to an enum is available while resolving that enum declaration
- a declaration local to a function is available only in the body of the exact targeted overload and its nested statement scopes
- when several targets are listed, each listed target receives access independently
- sibling declarations and sibling overloads not listed cannot resolve it
- subclasses, companion units, extensions, callees, callers, and related declarations do not gain access automatically
- visibility is not inherited, transitive, or propagated through calls, composition, embedding, imports, or other local declarations
- it cannot be imported, exported, or projected through a library surface

```
@co.dap.local(
  for=[
    hr.employee.Employee,
    hr.employee.EmployeeService
  ]
)
// EmployeeState.fol
_ co.enum = {
  Active,
  Inactive
}
```

```

}

// Employee.fol
_ co.class = {
  state EmployeeState; // ✅ listed target
}

// EmployeeService.fol
_ co.class = {
  state EmployeeState; // ✅ listed target
}

// Payroll.fol
_ co.class = {
  state EmployeeState; // ❌ not listed
}

```

Interaction with Ordinary Visibility

`@co.dap.local` establishes the final visibility boundary. Ordinary visibility annotations such as `@co.dap.public`, `@co.dap.package`, `@co.dap.protected`, or `@co.dap.private` cannot widen the declaration beyond its closed local target set.

```

@co.dap.public
@co.dap.local(
  for=[
    hr.employee.Employee,
    hr.employee.EmployeeService
  ]
)
// EmployeeState.fol
_ co.enum = { Active, Inactive }

```

`EmployeeState` is still visible only to the listed targets. The ordinary visibility annotation is redundant for external resolution and may produce a compiler warning, but it never overrides `@co.dap.local`.

When most or all declarations in a package require access, ordinary package visibility should be used instead of maintaining a large target list.

Source Placement and Identity

A target-local declaration remains in the source position normally required for its declaration kind:

- a class, struct, enum, or module remains a normal primary declaration and follows the one-primary-declaration-per-package-file rule
- a function remains inside a unit, class, module, or another declaration context that normally permits functions
- `@co.dap.local` does not permit a free function to appear loose at package-file scope

The declaration retains a stable package-owned compiler identity. Its package identity must be exactly the same as the package identity of every target. It does not become physically nested and is not addressed as `Target.LocalName`.

```
hr.employee.EmployeeState
  local-for [
    hr.employee.Employee,
    hr.employee.EmployeeService
  ]
```

The normalized local target set is declaration metadata. Target-list order does not create a different declaration identity.

No Implicit Relationship or Privilege

`@co.dap.local` changes visibility only. It does not imply:

- composition
- embedding
- inheritance
- lifecycle ownership
- memory ownership
- friendship or privileged private-member access
- automatic membership in any listed target
- shared runtime state among the targets

Composition must still be written explicitly through a field or another supported relation. Embedding remains a separate facility: only struct and enum declarations are eligible for embedding, and each use must follow the embedding rules defined for that declaration kind.

Escape Restrictions

A target-local type must not leak outside its complete visibility domain.

It cannot appear in:

- a public, package, or protected API visible outside the local target set
- a library surface
- the parameter or return types of a function to which it is local
- a field or method signature that exposes it beyond the local target set
- an exported generic specialization or type alias

A value whose static type is target-local must be converted to an externally visible type before leaving the union of the listed targets' visibility domains.

Usage

```
@co.dap.public
@co.dap.local(
  for=[
    hr.employee.Employee,
    hr.employee.EmployeeService
  ]
)
// EmployeeState.fol
_ co.enum = { Active, Inactive }

// Employee.fol
_ co.struct={

  state EmployeeState;
}
```

Invalid Physical Nesting

```
// Employee.fol
_ co.class = {
```

```

    Address co.struct = { ... } // ✗
}

// EmployeeModule.fol
_ co.module = {
    Config co.struct = { ... } // ✗
}

process()->() = {
    State co.enum = { Ready, Done } // ✗ named type declaration
}

// EmployeeContract.fol
_ co.signature = {
    Employee co.struct; // ✗
}

// EmployeeApi.fol
_ co.interface = {
    Result co.struct; // ✗
}

```

The `process` example rejects the named enum declaration; it does not reject ordinary local functions or anonymous expressions. Those are permitted only under the explicit exceptions in [Physical Nesting Rules](#).

Use separately declared package-owned declarations, composition, embedding where allowed, and `@co.dap.local` when a closed set of declarations requires selective access.

There is another annotation `@co.dap.nested` which is similar to `local` but captures target state.

`@co.dap.nested(target=hr.emp.Employee)`

Instead of `for` we use `target` and `target` is always a single fully qualified type/function.

`@co.dap.nested` provides nested/inner-style target-state capture while the declaration itself remains physically separate, as defined above.

Comparison Table

Annotation	attribute	multiple targets	capture the target state
@co.dap.local	for	✓ as list for single target can mention without list syntax	✗
@co.dap.nested	target	✗	✓

@co.dap.inner Declarations

`@co.dap.inner` is an association annotation. It does **not** create a physically nested type, method, or function. The annotated declaration remains in its ordinary legal source location, retains its package-owned identity, and must satisfy the same exact-package rule that applies to `@co.dap.local` and `@co.dap.nested`.

Unlike `@co.dap.local` and `@co.dap.nested`, `@co.dap.inner` does not contain a `for` or `target` attribute. Its target relationship is established at the use site by explicitly embedding or attaching the declaration to a legal target. An `@co.dap.inner` declaration cannot be used as a standalone declaration.

Usage

```
@co.dap.public
// EmployeeState.fol
@co.dap.inner
_ co.enum = {
    Active,
    Inactive
}

// Employee.fol
_ co.struct = {
    EmployeeState;
    state EmployeeState;
}
```

The use of `EmployeeState;` establishes the inner association for this target. The annotation itself does not name the target.

Scope of `@co.dap.inner` Executable Declarations

For an `@co.dap.inner` function or method, parameters and local declarations resolve normally. A free runtime name is resolved from the **lexical context of the active call or attachment site**, not from the declaration site of the separately declared `@co.dap.inner` function.

The lookup order is:

1. parameters and local declarations of the `@co.dap.inner` function or method
2. the innermost lexical scope at the active call or attachment site
3. enclosing lexical scopes of that call or attachment site
4. statically resolved types, annotations, imports, and built-in `co.*` names
5. compiler error

This model is called **call-site lexical-context resolution**.

It is distinct from an ordinary inner function. An ordinary inner function is physically declared inside another function and resolves free runtime names from its enclosing lexical **declaration** context. An `@co.dap.inner` function is declared separately and therefore obtains its runtime context from the lexical scope in which it is actively attached or called.

It is also distinct from `@co.dap.dynamicscope`. Call-site lexical-context resolution does not search arbitrary runtime caller frames beyond the lexical scope chain of the active call site, and it does not use mixed-scope fallback. At every statically known use or call site, the compiler validates that each required runtime binding exists, is definitely initialized, has a compatible type, and permits the requested access or mutation.

For `@co.dap.inner` types and other non-executable declarations, ordinary static name and type resolution continues to apply. Call-site lexical-context resolution concerns only free runtime names used by executable function or method bodies.

An implementation may lower an `@co.dap.inner` declaration by copying, inlining, specialization, or another equivalent mechanism. Such lowering is not observable language semantics; the association and scope rules above are normative.

Not Allowed

`@co.dap.inner`, `@co.dap.local`, and `@co.dap.nested` cannot be used with `co.cstruct`.

Statements

A statement is a complete executable or declarative instruction. It may contain one or more expressions and may change program state, control execution, introduce declarations, or produce observable effects.

Common statement categories in `FoLang`

1. Declaration Statement
2. Initialization Statement
3. Expression Statement
4. Conditional Statement
5. Loop Statement etc.,

Effects and Handling Effects

FoLang handles recoverable execution failures as typed **effects** without `try`, `catch`, `throw`, or `finally` statements. Effect declaration and effect handling have deliberately separate owners:

Construct	Owner and placement	Purpose
<code>@co.dap.effects</code>	Callable declaration or definition	Declare effects the callable may emit
<code>@co.dap.onEffect</code>	Immediately before a call expression	Handle effects from that invocation
<code>@co.dap.defer</code>	Inside a callable	Register unconditional completion work

The defining library knows which effects an exported callable may emit, but it does not know the caller's logging, retry, cleanup, resource, or return policy. The definer therefore publishes only `@co.dap.effects`; the caller chooses `@co.dap.onEffect` handlers and resolutions independently at each ordinary call site. An execution-

model invocation is the deliberate exception: the caller may register handlers, but its resolution is fixed implicitly to `return_with_error` because effects cannot propagate across that execution boundary.

A non-handler library implementation may still use `@co.dap.onEffect` for calls made inside its own implementation. At those sites the library is itself the caller. It may not attach `@co.dap.onEffect` to an exported callable definition to impose handling on external callers. Effect-handler modules are the explicit exception: `@co.dap.onEffect` is forbidden throughout them.

Effect Channel and Error Result Channel

An emitted effect and a returned `co.error` value use different channels:

Channel	Meaning
Effect channel	Non-local typed failure that must be handled, converted, or propagated
Ordinary result channel	A normal returned value, including a <code>co.error</code> value

Returning a non-empty `co.error` does not emit it as an effect. `return_with_error` explicitly consumes an effect at a call site, converts it to a `co.error` object, and returns it from the enclosing caller through the ordinary result channel.

An error object may carry its category, origin, message, source location, cause, and backend/runtime diagnostics. Native or backend exceptions must be translated into typed FoLang recoverable effects before FoLang resolution begins; the translated type need not have been advertised by `@co.dap.effects`.

Recoverable Error Contract, Mixin, and Effect Emission

Every recoverable effect value is an instance of a class satisfying the standard `co.error` interface. Error classes may add category-specific fields and helper methods. The standard `co.AbstractError` mixin supplies common state and concrete behavior for that interface. Custom error classes normally compose the mixin, declare `co.error` as an interface, and add their own information:

```
@co.dap.oops(  
  interfaces=[co.error],  
  mixins=[co.AbstractError]
```

```

)
_ co.class = {
    // category-specific error fields and methods
}

```

`co.AbstractError` is declared as `_ co.mixin`, not as a class and not as a distinct abstract-class kind. As a mixin it participates only in composition and has no independent class-instance construction semantics. Its exact matching concrete methods may satisfy `co.error` interface slots when the consuming error class composes the mixin. If an explicit source mapping or remapping is required, the consuming class uses `@co.dap.implement(type=co.error)` according to the ordinary method- resolution rules. Interface method declarations themselves do not carry that annotation.

`@co.dap.implement` and `@co.dap.implementation` remain distinct:

Form	Purpose
<code>@co.dap.implement</code>	Satisfy an interface, trait, or mixin method contract
<code>@co.dap.implementation</code>	Bind an authorized bodyless standard declaration to a backend-neutral runtime operation

FoLang source and HIR use one backend-neutral effect-emission operation. Its standard declaration is conceptually:

```

@co.dap.implementation(
    kind=co.dap.implementationKind.runtime,
    operation=co.runtime.operation.effect.emit
)
emit(error co.error)->();

```

The operation accepts only a `co.error` value. Each backend may implement it using native exceptions, tagged control results, runtime unwinding, task completion, or another mechanism, but it must preserve FoLang matching, handler, resolution, and defer semantics. A backend or native-library failure that is recoverable in FoLang must first be translated into a class instance satisfying `co.error`; a raw backend exception must not enter FoLang handling directly.

Fatal runtime failures are outside this hierarchy. They do not satisfy `co.error`, cannot appear in `@co.dap.effects` or `@co.dap.onEffect`, and cannot be continued, retried, returned as an error, propagated as a FoLang effect, or delivered to an effect handler. They terminate the affected application or runtime execution according to runtime policy.

`@co.dap.effects`: Definer Contract

`@co.dap.effects` is informative public symbol metadata, not a closed or enforced catch-or-declare contract. Its `emits=[...]` list advertises effect types currently known to possibly leave the callable:

```
@co.dap.effects(  
  emits=[  
    co.DatabaseError,  
    co.NetworkError  
  ]  
)  
loadCustomer(id co.int)->(Customer) = {  
  ...  
}
```

It is valid on ordinary callable definitions, bodyless declarations, interface methods, native declarations, and other ordinary callable contracts. It is invalid on an execution-model declaration because that declaration converts all recoverable effects at its public boundary. It is also invalid on a call, expression, statement, field, or non-callable declaration.

Every `emits` entry must resolve to an accessible class type satisfying `co.error`. Duplicate canonical types are invalid, and a broader declared error class covers its subtypes. The list is an open set of known possible effects, not an exhaustive closed effect row and not a claim that every listed effect occurs on every invocation.

Absence of `@co.dap.effects`, or an empty `emits` list, means that the source explicitly advertises no effects. It does not mean that the callable is effect-free. An undeclared effect may still arise and propagate like an unchecked exception.

The compiler computes the callable's known outgoing-effect metadata from its explicitly declared effects plus statically known unhandled callee effects, minus effects consumed at call sites. When a statically selected concrete handler has a

known outgoing effect, the compiler adds the possibility of `co.GenericError`, because that is the value exposed beyond the failed handler pipeline. It serializes this known set in exported `.foIenc` symbols. Unknown runtime effects remain possible even when absent from that metadata.

Because the contract is open, an implementation or override may advertise additional effects not named by its interface or overridden declaration. Effect metadata informs callers and tooling but does not distinguish overloads or make undeclared propagation invalid.

`co.EffectHandler`

The standard library defines the handler-module contract:

```
// co.EffectHandler
_ co.signature = {
  handle(error co.error)->();
}
```

Every `handlers=[...]` entry must be an accessible singleton module matching this signature. It is not a constructed class instance or arbitrary function reference. The signature limits only the module's public contract; a handler may use private functions, associated singleton objects, configuration, registries, locks, thread-local state, and other accessible facilities.

`co.EffectHandler` constrains only the structural member signature. The signature itself does not declare handler effects. Each concrete handler implementation may independently advertise effects on its actual `handle` implementation:

```
@co.dap.effects(
  emits=[co.LogWriteError]
)
handle(error co.error)->() = {
  writeLog(error);
}
```

Omitting `@co.dap.effects` from that implementation does not prohibit an unexpected effect. It means only that the implementation advertises no effect explicitly. Since `handlers=[...]` contains statically resolved concrete module

references, the compiler can use their declared and inferred known effects to include the possibility of `co.GenericError` when analysing the caller.

An effect-handler module must not use `@co.dap.onEffect`, either on its public `handle` implementation or at a call site inside one of its private functions. Handlers are the terminal declarative stage of the current handling pipeline; allowing them to start another local handler pipeline would permit recursive handling chains with no clear semantic boundary. A handler should therefore use ordinary values, variants, explicit checks, or non-effectful operations for problems it can recover from locally. If it nevertheless emits a recoverable effect, the handler-failure rules below apply.

`@co.dap.effects` remains valid on the concrete `handle` implementation because it advertises such possible outgoing handler effects without handling them locally.

`@co.dap.onEffect`: Caller Policy

`@co.dap.onEffect` prefixes exactly one call expression. It defines how that specific invocation handles matching effects from its callee:

```
showCustomer(id co.int)->(co.error) = {
  customer Customer =
    @co.dap.onEffect(
      co.DatabaseError={
        handlers=[
          DBConnectionCloseHandler,
          LogErrorHandler
        ],
        resolution=return_with_error
      },
      co.NetworkError={
        resolution=propagate
      }
    )
  loadCustomer(id);
}
```

It is invalid on a callable definition or declaration, and it does not create a statement-oriented `try/catch` region. The prefix belongs to the immediately following call AST node. For nested calls, it handles only that decorated call:

```
outer(
  @co.dap.onEffect(
    co.NetworkError={resolution=propagate}
```

```
)  
  inner()  
);
```

The policy above applies to `inner()`, not to `outer(...)`. A separately placed `@co.dap.onEffect` is required to handle effects emitted by `outer`.

Each top-level field name must resolve to a valid effect type. It need not appear in the callee's advertised `emits` list because that list is open and an unexpected effect may still occur. A call-site entry for a non-advertised type is valid proactive handling, although tooling may identify it as not currently advertised by the callee. Keys normalize to canonical type identity; duplicates are invalid. When multiple entries match, the most specific type wins.

For an ordinary invocation, each effect record contains exactly one singular `resolution=` field. A record may additionally contain an optional non-empty ordered `handlers=[...]` list. When handlers are present, all of them execute before the resolution. The absence of `handlers` does not imply propagation: a matching ordinary-call record without handlers still applies its declared resolution.

For an invocation of a callable declared with `@co.dap.executionmodel`, an effect record instead contains a required non-empty `handlers=[...]` list and must omit `resolution`. Its resolution is fixed by the language to `return_with_error`.

Supplying any explicit resolution, including `return_with_error`, is a compile-time error. The call-site annotation remains useful because it registers application handlers even though the developer cannot change the execution-boundary resolution.

For an ordinary invocation, an effect with no matching `@co.dap.onEffect` entry propagates from the enclosing caller by default. A statically known unmatched effect is included in the caller's computed outgoing-effect metadata, and an unknown runtime effect propagates even when it appears in no `@co.dap.effects` list. For an execution-model invocation, an unmatched effect cannot propagate across the execution boundary; it is converted to the callable's ordinary `co.error`-compatible result without running call-site handlers.

There is no `effects={...}` wrapper, no `resolutions=[...]` list, and no `invoke` resolution.

Execution-Model Call-Site Policy

An execution-model declaration must expose exactly one result position compatible with `co.error`. The first unhandled recoverable effect that reaches its execution boundary is retained as the primary error object and is returned through that ordinary result position. Other ordinary result positions that the failed execution did not produce receive `co.const.none` only when those positions admit none. A refinement/dependent result position makes this failure-conversion shape invalid and must be redesigned with an explicit initialized alternative, commonly a variant.

The caller may attach ordered handlers to a particular submitted invocation. For execution governed by `co.cPCA`, the prefix is placed on the canonical submission call and its policy is bound to the statically selected execution-model target:

```
// appl.fol
SubmitParams  co.type = co.List(co.any);
SubmitResults co.type = co.List(co.any);

@co.dap.onEffect(
  co.DatabaseError={
    handlers=[
      DBConnectionCloseHandler,
      LogErrorHandler
    ]
  }
)
co.cPCA.submit(
  loadCustomerAsync,
  params=SubmitParams{id},
  results=SubmitResults{customer, error}
);
```

The omitted resolution is not default propagation. In this context it denotes the mandatory implicit `return_with_error` policy. These records are invalid:

```
co.DatabaseError={resolution=continue}
co.DatabaseError={resolution=propagate}
co.DatabaseError={resolution=return_with_error}
co.DatabaseError={} // no handler and no caller policy to add
```

The handlers execute synchronously and sequentially in the execution context that receives the effect, before completion is published to the submitting caller. The annotation registers statically resolved singleton modules with the invocation; it does not require the submitting caller's stack to remain active. Unmatched effects receive the same implicit conversion without handlers.

For concurrent or parallel execution, "first error" means the first effect observed by the applicable execution-model runtime. Unless a particular model defines deterministic observation priority, scheduling may affect which of simultaneously occurring effects becomes primary.

Resolution Records

The available resolutions are:

- `continue`
- `retry`
- `return`
- `return_with_error`
- `propagate`

Examples:

```
co.ValidationError={
  resolution=continue
}

co.NetworkError={
  handlers=[LogRetryHandler],
  resolution=retry,
  retry={
    max_attempts=3,
    on_exhausted=propagate
  }
}

co.DatabaseError={
  handlers=[DBConnectionCloseHandler, LogErrorHandler],
  resolution=return_with_error
}
```

Invalid records include:

```
co.DatabaseError={} // missing resolution

co.DatabaseError={
  resolution=[continue, return] // resolution is singular
}

co.DatabaseError={
  resolution=continue,
  resolution=return // duplicate resolution
}

co.DatabaseError={
  handlers=[],
  resolution=propagate // an explicitly supplied handlers list is empty
}

co.DatabaseError={
  resolution=retry // missing retry configuration
}
```

Ordered Handler Execution

Handlers execute synchronously and sequentially in declared order. Every handler receives the same original `co.error`, and each `handle` call must complete before the next begins. The implementation must not implicitly reorder, parallelize, schedule, or detach the list.

For an ordinary invocation, handlers run on the caller thread and execution context receiving the effect. For an execution-model invocation, they run in the execution context that observes the effect before completion is published to the submitting caller. This permits handler modules to use state accessible in the context where handling actually runs. An ordinary function, class object, incompatible module, inaccessible module, duplicate module, or empty list is invalid.

If a handler emits a recoverable effect, the remaining handlers for the original effect are skipped and the original effect's resolution is not applied. Because `@co.dap.onEffect` is forbidden throughout an effect-handler module, the handler cannot start a nested local handler pipeline for the new effect. A known handler

effect makes `co.GenericError` part of the caller's computed outgoing-effect metadata; an undeclared handler effect remains valid and follows the same runtime rule without necessarily appearing in advance metadata.

Handler Failure and `co.GenericError`

Let `E0` be the original effect being handled, `H` the handler currently executing, and `E1` a recoverable effect emitted while `H` handles `E0`. The standard library creates one `co.GenericError` class instance that satisfies `co.error` and records:

- the original handled error `E0`;
- the handler failure `E1`;
- the canonical identity of `H`; and
- available causal, source, and runtime diagnostic context.

Creation of this wrapper terminates the current handler pipeline immediately. The remaining handlers are skipped, the resolution selected for `E0` is cancelled, and neither `E0` nor the new `GenericError` is submitted to the same handler list again.

The boundary then determines what happens to the wrapper:

Context	Result of handler failure
Ordinary call	<code>co.GenericError(E0, E1, H)</code> propagates from the enclosing caller to its caller
Execution-model invocation	<code>co.GenericError(E0, E1, H)</code> is returned through the execution model's ordinary error-result position; otherwise unproduced none-admitting results receive <code>co.const.none</code> , while refinement/dependent result positions make that substitution invalid

`co.GenericError` is used only for a recoverable effect emitted by a handler while processing another recoverable effect. It is not created for an ordinary `propagate`, an ordinary `return_with_error`, or the normal implicit `return_with_error` conversion at an execution-model boundary. When handlers complete successfully, those paths preserve the original error object `E0`. A higher ordinary caller may handle the propagated `GenericError` at its own call site according to ordinary rules.

A fatal failure inside a handler is not `E1`, is never wrapped in `co.GenericError`, and terminates the application immediately like every other fatal failure.

Resolution Semantics

Resolution	Consumes effect	Behavior at the decorated call
<code>continue</code>	Yes	Skip the failed call result and continue after the call
<code>retry</code>	Conditionally	Invoke the same callee again until success or exhaustion
<code>return</code>	Yes	Exit the enclosing caller normally
<code>return_with_error</code>	Yes	Convert to <code>co.error</code> and return it from the enclosing caller
<code>propagate</code>	No	Emit the effect from the enclosing caller

`continue` does not mean retry. It resumes after the decorated invocation. If the failed call owes one or more values to its surrounding expression, every missing result position receives the universal `co.const.none` value. An assignment therefore completes with none in the affected destination rather than becoming a compile-time error.

`return` exits the enclosing caller, not the failed callee. It is directly valid for a unit-returning caller. In a non-unit caller, every ordinary result position not otherwise produced by the resolution receives `co.const.none` only if that position admits none. A missing refinement/dependent result makes the resolution invalid.

`return_with_error` exits the enclosing caller. That caller's return signature must contain exactly one position compatible with `co.error`. Additional result positions not otherwise produced receive `co.const.none` only if they admit none. An unproduced refinement/dependent result position, or a missing or ambiguous error-compatible result, is a compile-time error.

`propagate` emits the effect from the enclosing caller. Explicit `propagate` is useful when handlers must run before propagation; without a matching record, propagation occurs automatically without handlers. A known propagated effect is advertised through computed outgoing-effect metadata even if the caller did not spell it in a source annotation.

Retry

`retry` repeats only the decorated invocation, never the complete enclosing caller. It requires exactly one `retry={...}` record:

```
@co.dap.onEffect(  
  co.NetworkError={  
    handlers=[LogRetryHandler],  
    resolution=retry,  
    retry={  
      max_attempts=3,  
      on_exhausted=return_with_error  
    }  
  }  
)  
sendRequest();
```

`max_attempts` is a positive compile-time integer and includes the initial invocation. `max_attempts=3` therefore permits at most three total invocations, not one initial invocation plus three retries.

`on_exhausted` must be exactly one of `continue`, `return`, `return_with_error`, or `propagate`; it cannot be `retry`. It is applied to the final error after the final failed attempt. All ordinary validity rules for the selected exhaustion action still apply.

The caller evaluates the decorated call's receiver and argument expressions once before the first attempt and captures their resulting values. Retries use those same values according to ordinary value/reference parameter semantics; FoLang does not deep-copy mutable arguments. A caller that needs fresh argument evaluation must move that computation into the retried callee.

For each failed attempt, handlers run in declared order before the next attempt or the exhaustion action. A handler failure aborts the retry sequence. Each callee invocation is a distinct invocation: defers registered inside the callee execute when

that attempt exits before another attempt begins. Defers registered by the enclosing caller do not execute merely because one call attempt failed or was retried.

Retry repeats observable callee behavior. The developer must therefore ensure that the callable is idempotent, transactional, compensated, or otherwise safe to invoke repeatedly.

Caller-Owned Database Handling

An exported database library declares effects but does not register application handlers:

```
@co.dap.effects(  
    emits=[co.DatabaseError]  
)  
executeDatabaseOperation(command DBCommand)->() = {  
    ...  
}
```

The application defines its own `DBConnectionCloseHandler` and `LogErrorHandler` modules conforming to `co.EffectHandler`. It may keep a connection pool and current connection in an associated singleton object or another caller-owned facility. At the call site it chooses both handlers and their order:

```
runDatabaseWork(command DBCommand)->(co.error) = {  
    @co.dap.onEffect(  
        co.DatabaseError={  
            handlers=[  
                DBConnectionCloseHandler,  
                LogErrorHandler  
            ],  
            resolution=return_with_error  
        }  
    )  
    executeDatabaseOperation(command);  
}
```

When the call emits `co.DatabaseError`, FoLang closes the caller-owned connection, logs the same original error, converts it to `co.error`, and returns it from `runDatabaseWork`. Another caller may choose retry or direct propagation for the

same library operation without changing the library.

Static Effect Information and Default Propagation

At each call the compiler combines:

```
callee declared and inferred known effects
+ matching call-site @co.dap.onEffect entries
+ co.GenericError when concrete handlers have known outgoing effects
+ enclosing caller return signature
```

Effects consumed by `continue` or `return`, converted by `return_with_error`, or successfully eliminated by `retry` do not escape. An exhausted retry ending in `propagate`, an explicit `propagate`, and every unmatched callee effect do escape. Statically known escaping effects are included in the caller's computed outgoing metadata. Undeclared effects also escape but cannot be promised to callers in advance. When the enclosing declaration is an execution-model declaration, this analysis describes its internal implementation only: the public execution boundary converts the first escaping recoverable effect to its ordinary error result, so no effect is advertised outward.

At the outermost application/runtime boundary there is no caller to receive an unhandled effect. The runtime reports the effect and terminates the affected execution flow according to runtime policy.

`@co.dap.defer`

`@co.dap.defer` registers a callable for execution when the enclosing function or method actually exits. It performs unconditional completion work whether the exit is successful, ordinary, or effect-driven. It is not an effect handler, does not inspect the completion error, and does not choose among effect resolutions.

`@co.dap.defer` does not implicitly capture arguments through `this->args`. Values required by the deferred callable must be supplied/passed explicitly at registration.

```
someErrorFun(a co.int)->>() = {
    resource Resource = acquireResource();

    @co.dap.defer
    cleanup(resource Resource, originalArgument co.int)->>() = {
        releaseResource(resource);
    }
}
```

```
    logArgument(originalArgument);  
  }(resource, a);  
  
  performOperation(resource);  
}
```

The explicit arguments are evaluated and captured when execution reaches the defer registration. The deferred callable itself executes later. FoLang never injects a `co.error` object or any other completion-status binding into that callable. Values needed by deferred work must be declared as ordinary parameters and supplied explicitly at registration.

Defer Registration and Ordering

A defer is registered only when execution reaches it. A defer located after a failing operation is not registered when that failure prevents execution from reaching the defer. Completion work intended to cover the complete function must therefore be registered before operations that may terminate the function.

Every registered defer:

- executes exactly once when the enclosing function actually exits;
- executes after normal completion, ordinary return, `return`, `return_with_error`, or `propagate`;
- does not execute merely because an effect was handled with `continue` while the function remains active;
- participates in last-in, first-out ordering with other registered defers; and
- is not guaranteed after forced process termination, unrecoverable runtime shutdown, or a hardware failure that prevents further execution.

The completion order is:

1. execute the function body
2. resolve locally handled effects
3. determine that the function will exit
4. execute registered deferred callables in LIFO order
5. incorporate an error produced by a deferred callable
6. complete the ordinary return, `return_with_error`, or propagation

If an error already caused the function to exit, that first error remains the primary error. A later deferred-call error does not replace it and may be attached only as causal/diagnostic information supported by the primary error representation. If no primary error exists and a deferred callable emits an effect, that effect becomes the primary completion effect and propagates from the enclosing function unless it was handled at a call site inside the deferred callable. A statically known deferred-call effect is included in the enclosing function's computed outgoing-effect metadata; an unknown effect still propagates. Remaining deferred callables continue to execute.

Effect-Handling Summary

Construct	Purpose
<code>@co.dap.effects</code>	Definer-owned open metadata advertising known effects an ordinary callable may emit
<code>@co.dap.onEffect</code>	Caller-owned policy attached to one invocation; execution-model calls permit handlers but use implicit <code>return_with_error</code>
<code>co.EffectHandler</code>	Standard signature requiring <code>handle(error co.error) ->()</code>
<code>handlers=[...]</code>	Ordered list of developer-defined handler-module references
<code>co.GenericError</code>	Standard wrapper created only when a handler emits a recoverable effect while processing another recoverable effect
<code>continue</code>	Consume the effect, substitute <code>co.const.none</code> for missing results, and continue
<code>retry</code>	Repeat only the decorated invocation under a bounded retry policy
<code>return</code>	Consume the effect and terminate through the ordinary return channel
<code>return_with_error</code>	Consume and return the current error as an ordinary result
<code>propagate</code>	Forward the effect from the enclosing caller; known effects remain advertised upstream

```
@co.dap.defer
```

Execute error-independent completion work
in LIFO order on every function exit

FoLang effect handling is therefore explicit, typed, lexically scoped, and declarative. It provides local recovery, early return, propagation, callbacks, and guaranteed completion work without statement-oriented exception syntax.

Expressions

An expression is a construct that evaluates to a value, produces an effect, or both, and may be contained within a larger expression or statement.

Expression Evaluation Order

FoLang defines a deterministic evaluation order for expressions.

Expression structure is determined first by:

1. explicit grouping with parentheses;
2. operator precedence;
3. operator associativity.

After the expression structure has been determined, operands and subexpressions are evaluated from left to right in their source-code order, except where a construct explicitly defines conditional, lazy, asynchronous, concurrent, or otherwise specialized evaluation behavior.

Ordinary Expressions

For an ordinary expression:

```
result = first() + second();
```

the evaluation order is:

1. evaluate `first()`;
2. evaluate `second()`;
3. apply the `+` operator;
4. assign the resulting value to `result`.

Precedence and Evaluation Order

Left-to-right evaluation does not override operator precedence.

For example:

```
result = first() + second() * third();
```

operator precedence determines the expression structure as:

```
result = first() + (second() * third());
```

The function calls are evaluated in source order:

1. evaluate `first()`;
2. evaluate `second()`;
3. evaluate `third()`;
4. multiply the results of `second()` and `third()`;
5. add the multiplication result to the result of `first()`;
6. assign the final result to `result`.

Therefore, precedence determines how values are combined, while evaluation order determines when each operand or subexpression is evaluated.

Function Calls

For a function call, the callable expression is evaluated first, followed by the arguments from left to right.

```
result = calculate(first(), second(), third());
```

The evaluation order is:

1. evaluate the callable expression `calculate`;
2. evaluate `first()`;
3. evaluate `second()`;
4. evaluate `third()`;
5. invoke `calculate` with the resulting argument values;
6. assign the returned value to `result`.

Name resolution performed at compile time does not constitute runtime evaluation.

Method Calls

For a method call, the receiver expression is evaluated first, followed by the arguments from left to right.

```
result = getEmployee().calculate(first(), second());
```

The evaluation order is:

1. evaluate `getEmployee()`;
2. resolve the resulting object as the method receiver;
3. evaluate `first()`;
4. evaluate `second()`;
5. invoke `calculate`;
6. assign the returned value to `result`.

The receiver expression must be evaluated exactly once.

Effect-Handled Calls

`@co.dap.onEffect` is evaluated as metadata on the immediately following call AST node; the decorator itself has no runtime value. The callable/receiver and arguments are evaluated in the ordinary order before the first invocation.

When the selected resolution is `retry`, those evaluated receiver and argument values are retained and reused. They are not evaluated again for later attempts. On successful completion, evaluation continues with the returned value exactly as for an undecorated call. On `continue`, no value is produced, so the surrounding expression must not require one.

Member Access

For member access, the expression producing the containing object is evaluated before the member is accessed.

```
value = getEmployee().address.city;
```

The evaluation order is:

1. evaluate `getEmployee()`;
2. access `address` on the resulting object;
3. access `city` on the resulting address;
4. assign the resulting value to `value`.

Each intermediate receiver is evaluated only once.

Indexing

For an indexing expression, the indexed object is evaluated first, followed by index expressions from left to right.

```
value = getMatrix()[row()][column()];
```

The evaluation order is:

1. evaluate `getMatrix()`;
2. evaluate `row()`;
3. perform the first indexing operation;
4. evaluate `column()`;
5. perform the second indexing operation;
6. assign the resulting value to `value`.

Assignment

For assignment, FoLang evaluates the assignment target first, then evaluates the right-hand-side expression, and finally performs the assignment.

```
array[index()] = calculate();
```

The evaluation order is:

1. evaluate `array`;
2. evaluate `index()`;
3. determine the destination location;
4. evaluate `calculate()`;
5. assign the resulting value to the destination.

Determining the assignment target does not read the previous value stored at that location unless the assignment operation explicitly requires it.

Simple Variable Assignment

For assignment to a simple variable:

```
result = first() + second();
```

the evaluation order is:

1. determine the binding represented by `result`;
2. evaluate `first()`;
3. evaluate `second()`;
4. apply the `+` operator;
5. assign the resulting value to `result`.

Compound Assignment

A compound assignment evaluates its target exactly once.

```
array[index()] += calculate();
```

The evaluation order is:

1. evaluate `array`;
2. evaluate `index()`;
3. determine the destination location;
4. read the current value from that location;
5. evaluate `calculate()`;
6. apply the `+` operator;
7. assign the resulting value to the same destination.

The expression above must not behave as though it were expanded into a form that evaluates `array` or `index()` more than once.

Multiple Assignment

When an assignment contains multiple right-hand-side expressions, those expressions are evaluated completely from left to right before values are assigned to their targets.

```
a, b = first(), second();
```

The evaluation order is:

1. determine the target for `a`;
2. determine the target for `b`;
3. evaluate `first()`;
4. evaluate `second()`;
5. assign the first resulting value to `a`;
6. assign the second resulting value to `b`.

Right-hand-side evaluation must complete before any target receives its new value.

This permits value exchange without an explicit temporary variable:

```
a, b = b, a;
```

Operator Expressions

Built-in and user-defined operators follow the same operand-evaluation rules.

For a binary operator:

```
left() + right();
```

FoLang evaluates `left()` before `right()`.

For a prefix unary operator:

```
-operand();
```

FoLang evaluates `operand()` before applying the operator.

For a postfix unary operator:

```
operand()!;
```

FoLang evaluates `operand()` before applying the operator.

Operator overloading must not change the specified evaluation order of operands.

Short-Circuit Boolean Operations

Short-circuit Boolean operators evaluate the left operand first.

The right operand is evaluated only when it is required to determine the result.

For logical AND:

```
left() && right();
```

the evaluation order is:

1. evaluate `left()`;
2. when the result is false, return false without evaluating `right()`;
3. otherwise, evaluate `right()` and use its Boolean result.

For logical OR:

```
left() || right();
```

the evaluation order is:

1. evaluate `left()`;
2. when the result is true, return true without evaluating `right()`;
3. otherwise, evaluate `right()` and use its Boolean result.

A skipped operand produces no value, mutation, side effect, or error.

Conditional Expressions

A conditional expression evaluates its condition first and then evaluates exactly one selected result expression.

```
result = condition()  
    .then(whenTrue())  
    .default(whenFalse());
```

The evaluation order is:

1. evaluate `condition()`;
2. when the condition is true, evaluate `whenTrue()` only;
3. otherwise, evaluate `whenFalse()` only;
4. assign the selected result to `result`.

The unselected expression must not be evaluated.

Conditions and Branches

For a condition-and-branch construct, conditions are evaluated sequentially from left to right.

```
firstCondition().then({
    firstBranch();
}).otherwise(secondCondition()).then({
    secondBranch();
}).default({
    finalBranch();
});
```

The evaluation order is:

1. evaluate `firstCondition()`;
2. when true, execute `firstBranch()` and skip the remaining conditions and branches;
3. otherwise, evaluate `secondCondition()`;
4. when true, execute `secondBranch()` and skip the final branch;
5. otherwise, execute `finalBranch()`.

Only the selected branch is evaluated. `otherwise(condition)` is considered only after every earlier condition fails. If all conditional branches fail, `default(...)`, when present, supplies the terminal branch.

Pattern Matching

A pattern-matching subject is evaluated exactly once.

Cases are examined in source order unless a particular matcher explicitly defines another ordering rule.

```
getValue().match()  
  .case(firstPattern => firstResult())  
  .case(secondPattern => secondResult())  
  .default(defaultResult());
```

The evaluation order is:

1. evaluate `getValue()` exactly once;
2. test `firstPattern`;
3. when it matches, evaluate `firstResult()` and stop;
4. otherwise, test `secondPattern`;
5. when it matches, evaluate `secondResult()` and stop;
6. otherwise, evaluate `defaultResult()`.

Results belonging to unmatched cases are not evaluated.

Pattern guards are evaluated only after their corresponding structural pattern has matched.

Composite Construction Evaluation

Elements and entries of a composite construction are evaluated from left to right as they appear in the source. A runtime composite always carries its concrete type prefix; a bare bracketed or braced value is not an expression.

```
ValueList co.type = co.List(Value);  
values = ValueList{first(), second(), third()};
```

The evaluation order is:

1. evaluate `first()`;
2. evaluate `second()`;
3. evaluate `third()`;
4. construct the collection from the resulting values.

Range Expressions

For a bounded range, the lower-bound expression is evaluated before the upper-bound expression.

```
range = lower() .. upper();
```

The evaluation order is:

1. evaluate `lower()`;
2. evaluate `upper()`;
3. construct the range.

An omitted bound is not evaluated and does not produce an implicit function call or side effect.

Comprehensions and Iteration

Comprehension binding, result-shape, and source-specific behaviour are defined in [Comprehension Semantics](#). This section defines only their interaction with the general evaluation-order rules.

Within each iteration:

- source expressions are evaluated in the order required by the applicable comprehension or iteration form;
- when an explicitly defined filter is present, its condition is evaluated before the corresponding result expression;
- a result expression is evaluated only for a source value that participates according to the source's defined semantics and any explicit filter;
- individual operand evaluation remains left to right.

The core `for (pattern <- source).yield(...)` form does not itself introduce an inline filter. The language does not implicitly evaluate comprehension iterations concurrently unless concurrency is explicitly requested or the selected source semantics explicitly define another execution model.

Lazy Expressions

An expression declared lazy is not evaluated when the lazy binding is created.

Its evaluation occurs only when demanded according to the rules of the corresponding lazy construct.

Once evaluation begins, the expression itself follows the normal FoLang evaluation-order rules unless the lazy construct specifies otherwise.

The specification for each lazy construct must also state whether its result is:

- evaluated once and cached;
- evaluated again for every demand;
- safe for concurrent demand;
- permitted to propagate errors more than once.

Asynchronous and Concurrent Expressions

Left-to-right evaluation determines the order in which asynchronous or concurrent operations are created, invoked, or submitted.

It does not necessarily determine the order in which independently executing operations complete.

```
submit(first());  
submit(second());
```

FoLang evaluates and submits `first()` before `second()`. However, completion order depends on the execution-model rules unless explicit ordering is requested.

Concurrent execution never arises implicitly from an ordinary expression. It must be introduced by an explicit FoLang concurrency, parallelism, asynchronous-execution, scheduling, or execution-model construct.

Errors During Evaluation

When evaluation of a subexpression produces an error that prevents further evaluation, later subexpressions are not evaluated.

```
result = first() + failing() + third();
```

When `failing()` terminates evaluation with an error:

1. `first()` has already been evaluated;
2. `failing()` produces the error;
3. `third()` is not evaluated;

4. no final value is assigned to `result`.

The applicable rules in [Effects and Handling Effects](#) determine whether the error is continued from, returned, returned as an error, or propagated.

Side Effects

All observable side effects occur according to the defined evaluation order.

Observable effects include:

- object mutation;
- variable rebinding;
- input and output;
- file-system operations;
- network operations;
- synchronization;
- effect or error generation;
- interaction with foreign code;
- interaction with the runtime environment.

An implementation must not reorder operations when doing so could change any observable effect.

Compiler Optimizations

A compiler may optimize, combine, eliminate, or internally reorder operations only when the transformation preserves all behavior required by this specification.

In particular, an optimization must not change:

- the resulting value;
- the order or presence of observable side effects;
- the identity or aliasing behavior of objects;
- the timing or presence of specified errors;
- the selected conditional or pattern-matching branch;
- the number of times an effectful expression is evaluated;
- synchronization or concurrency guarantees.

Pure internal operations may be reordered when no conforming FoLang program can observe the difference.

Implementation Requirement

Every conforming FoLang implementation must preserve the evaluation order defined in this section.

An implementation may use any parser, intermediate representation, optimizer, runtime, or backend, but those implementation choices must not alter the externally observable behavior required by these rules.

Operators

FoLang distinguishes a **symbolic spelling** from an **operator**. A sequence of symbol characters is not automatically an operator merely because it resembles one. Its syntactic role is determined by the grammar context in which the whole spelling occurs.

For example:

```
IntDb1Ptr co.type = co.int->(**);  
a IntDb1Ptr;
```

In the alias declaration, `->` is the structural type-derivation marker and `**` is pointer-degree metadata inside the derivation. Neither occurrence is parsed as an expression operator. The ordinary declaration uses the resulting alias. The same `**` spelling in `left ** right` is parsed as the registered power operator because it occurs in an operator-expression position.

FoLang uses one uniform implementation model for every expression operator. The difference between built-in, pre-declared, and project-local custom operators is only who registers the symbol and its parse properties.

built-in operator

- symbol and parse properties registered by the language
- one or more implementations may already exist

pre-declared operator glyph

- symbol and parse properties registered by the language

no implementation is required to exist

project-local custom operator

symbol and parse properties registered once in
components/operators/component.fol
no implementation is contained in the operator declaration

all three categories

implementations are ordinary mode=overload operator functions
duplicate normalized operand signatures are errors

Symbolic Runs, Classification, and Boundaries

After comments, literals, and closed scanner-known composite spellings such as `@@new`, `@@init`, and the lifecycle invocation marker `::` are recognized, the lexer consumes each remaining complete contiguous run of symbol characters as one symbolic token candidate. It does not backtrack or split an unrecognized run into shorter valid tokens. Whitespace, comments, and delimiters such as `(`, `)`, `[`, `]`, `{`, and `}` end a symbolic run. Comment openers are recognized before ordinary symbolic-run scanning.

The complete run is then classified by its grammar context as one of the following:

1. a fixed structural or declaration spelling such as `->`, `=>`, `:=`, `::=`, or `?=`;
2. a contextual metadata spelling, such as a contiguous run of one or more `*` characters inside `->(…)` to express pointer degree;
3. a registered expression operator; or
4. an unrecognized symbolic token, which is a compilation error.

There is no fallback splitting. Therefore, when `++` is not registered:

```
++ a;      // error: unrecognized symbolic token `++`
a ++ b;    // error: unrecognized symbolic token `++`
a++b;      // error: the contiguous run is still the single candidate `++`

+ +a;      // valid: two `+` operators separated by whitespace
+(+a);     // valid: `(` creates the token and operand boundary
a + +b;    // valid: binary `+` followed by unary `+`
```

A registered expression operator whose spelling contains more than one symbol character must have an explicit boundary on every operand-facing side. A boundary may be whitespace, a comment, or an applicable delimiter. Boundary presence is checked from the original source before whitespace and comments are discarded. The rule is based on the operator's fixity:

- an infix multi-symbol operator requires a boundary before and after it;
- a prefix multi-symbol operator requires a boundary after it;
- a postfix multi-symbol operator requires a boundary before it.

Suppose `+ -` is registered as an infix operator:

```
a +- b;           // valid
(a)+-(b);         // valid: the parentheses provide both operand boundaries
a+-b;            // error: missing boundaries
a +-b;           // error: missing the right boundary
a+- b;           // error: missing the left boundary
a + -b;          // valid: separate `+` and unary `-` operators
a + (-b);        // valid: the parenthesis separates the operators
```

This boundary requirement applies uniformly to built-in, pre-declared, and custom multi-symbol expression operators. It does not apply merely because a structural token or metadata spelling contains multiple symbols. Thus `co.int->(**)` remains valid without spaces around `->` or inside the pointer metadata.

The statement-level definition spellings `:=`, `::=`, and `?=` are not expression operators, but they deliberately use the analogous two-sided boundary rule. Write `name := value`, `name ::= value`, and `name ?= value`; compact forms such as `name:=value`, `name::=value`, and `name?=value` are invalid.

Operator `mode=override` and `mode=extends` are unsupported. Ordinary class method overriding through `@co.dap.override` remains a separate class feature.

Built-In Operator Parse Table

Larger precedence numbers bind more tightly. Precedence and associativity determine the parse tree; FoLang's left-to-right operand evaluation rule independently determines evaluation order within that tree.

Precedence	Operator/form	Fixity	Associativity	Arity / parse role
------------	---------------	--------	---------------	--------------------

700	call <code>(...)</code> , index <code>[...]</code> , member <code>.</code> , lifecycle call <code>::name(...)</code> , postfix <code>!</code>	postfix	left	call/index/mem- ber/lifecycle- call syntax; postfix <code>!</code> unary
650	<code>**</code>	infix	right	binary
600	<code>+</code> , <code>-</code> , <code>!</code>	prefix	right	unary
550	<code>*</code> , <code>/</code> , <code>%</code>	infix	left	binary
500	<code>U</code> , <code>n</code>	infix	left	binary
450	<code>+</code> , <code>-</code>	infix	left	binary
400	<code>..</code> , <code><..</code> , <code>..</code> <code><</code> , <code><..<code><</code></code>	infix/range	none	range form; a bound may be omitted where the range grammar permits
350	<code><</code> , <code><=</code> , <code>></code> , <code>>=</code> , <code>:></code> , <code><:</code>	infix	none	binary
300	<code>==</code> , <code>!=</code>	infix	none	binary
250	<code>&</code>	infix	left	binary
200	<code>^</code>	infix	left	binary
150	<code> </code>	infix	left	binary
100	<code>&&</code>	infix	left	binary; short- circuit
50	<code> </code>	infix	left	binary; short- circuit
10	<code>=</code> , <code>+=</code> , <code>-=</code> , <code>*=</code> , <code>/=</code> , <code>%=</code> , <code>**=</code> , <code>&=</code> , <code>^=</code> , <code> =</code>	infix assignment	right	binary assignment

The definition spellings `:=`, `::=`, and `?=` are statement-level definition operators, not general expression operators, so they do not receive an expression-precedence level. Structural spellings such as `::`, `=>`, `=>>`, `->`, `<-`, `->>`, and `<->` are

likewise not ordinary expression operators merely because they contain symbol characters.

Operator Implementations

An operator implementation is a normal function with operator metadata. It cannot be declared loose at package scope. Every implementation must be in one of these legal function-owning locations:

Operand owner	Required implementation location
existing type extended outside its defining owner	an <code>@co.dap.extension</code> operator function inside a legal unit
<code>co.struct</code> defined by the current source owner	the struct's same-package companion unit
<code>co.class</code> defined by the current source owner	an operator method declared by the class
module, enum, union, interface, signature, <code>co.cstruct</code>	unsupported

An operator implementation is **never a generic declaration**. A function-shaped declaration carrying `@co.dap.operator` must not also carry `@co.dap.generic`; that combination is a compile-time error. This remains true when the owning class, struct, or existing extension target is itself generic. The genericity belongs to that type declaration, not to the operator declaration.

For a generic owner, operator ownership is determined from the owner's canonical declaration identity. For example, `List(co.int)` and `List(co.string)` are both owned by `List` for the purpose of locating operators defined for `List`. The owner's generic arguments do not become operator type parameters and do not cause generic-operator inference or instantiation. An operator declared inside a generic owner may still execute in that owner's ordinary type context; that does not make the operator declaration generic.

For example, a generic class may directly own a non-generic unary operator:

```
@co.dap.generic(types=[{name=T}])
_ co.class = {
  @co.dap.operator(symbol='!')
  isEmpty()->(co.bool) = {
```

```
    ...  
  }  
}
```

Here `T` belongs to the class declaration. The `!` implementation belongs to the class's canonical owner and does not declare, infer, or instantiate an operator-level `T`.

An implementation uses `mode=overload`, or omits `mode` because omission is shorthand for `mode=overload`. A one-character symbol may use a character literal; a multi-character symbol must use a string literal:

```
@co.dap.operator(symbol='+', mode=overload)  
add(left Employee, right Employee)->(Employee) = {  
    ...  
}  
  
@co.dap.operator(symbol="+-", mode=overload)  
combine(left Employee, right Employee)->(Employee) = {  
    ...  
}
```

FoLang does not attempt to prove that an overload follows the conventional meaning of its symbol. A developer may overload `+` with any implementation. Using one symbol consistently for one recognizable concept is recommended for readability, but it is not a compiler-enforced semantic restriction.

For a receiverless struct-companion operator function, the first declared operand must have the matching struct type. A matching struct type in a later operand is insufficient. For a unary operator, the sole operand must have the matching struct type.

A matching instance receiver contributes the first operand. A matching type receiver establishes ownership but contributes no operand; its ordinary parameter list is the complete operator signature. In a class, an ordinary operator method has an implicit `this` first operand, while `@co.dap.static` and `@co.dap.class` operator methods use only their declared parameters and require the first declared operand to have the enclosing class type.

After receiver normalization, the operand count must match the registered arity of the operator. Each distinct normalized operand signature is an overload. A second implementation with the same normalized operand signature is a conflict and is rejected even when its declared result type differs. This rule applies equally to built-in operators, pre-declared glyphs, and project-local custom operators.

The operator result type is part of the complete implementation signature and type contract, and **different normalized operand signatures may have different result types**. The result type does not participate in selecting between two operator implementations with the same normalized operand signature. Therefore:

```
// valid: distinct operand signatures may have distinct result types
+(co.int, co.int)      -> (co.int)
+(co.float, co.float)  -> (co.float)
+(Employee, Employee)  -> (Employee)

// invalid: the same normalized operands cannot be overloaded by result
type
+(co.int, co.int)      -> (co.int)
+(co.int, co.int)      -> (co.float) // compiler error
```

This operator rule is separate from the ordinary named-callable rule that sibling overloads share one declared return signature.

Operator Type Resolution, Conversion, and Overflow

Operator overload resolution requires an **exact normalized operand-type match** after operator ownership has been established. It does not perform nominal subtype-to-supertype widening, C-/C++-/Java-style numeric promotion, or implicit casting/conversion while selecting an operator implementation.

The owner-bearing operand is the applicable receiver when one contributes an operand; otherwise it is the first declared operand required by the operator ownership rules. When that owner is generic, operator-owner normalization uses the canonical owner declaration and does not treat the owner's generic arguments as operator-generic parameters or candidate-selection dimensions. Thus different instantiations of `List` still locate the operator family owned by `List`. This owner normalization does not make unrelated operand types interchangeable and does not alter ordinary type compatibility outside operator ownership.

A subtype does not inherit a supertype's operator implementation merely because the supertype signature would be applicable under ordinary function-overload rules. The subtype must have an explicit matching operator implementation when that operation is intended to be valid for the subtype.

This exactness is deliberate: an operator implementation for a supertype may rely on invariants or semantics that are not valid for every subtype. Operator overloading therefore remains separate from ordinary named-callable overload widening. An overflow condition likewise never causes the compiler or runtime to widen an operand or select another numeric operator overload.

An operator implementation has its own declared result type. After that operator has been selected, an enclosing declaration, assignment, argument, or return position may require the produced value to satisfy another target type. That is handled by FoLang's ordinary conversion model, not by operator promotion.

The language-provided simple types, such as `co.int` and `co.float`, provide standard `to` and `from` conversion methods through their ordinary package APIs. When the required target type has an applicable conversion and the value is representable by that target, FoLang uses that conversion to satisfy the target context. The operator overload that produced the value is not changed by that conversion.

User-defined types may participate in the same model by supplying supported `to` and/or `from` conversions through the extension mechanisms defined by FoLang. A conversion provided by an extension is an ordinary type conversion; it does not create automatic promotion between the operands of an operator expression.

Consequently:

```
exact normalized operand types
  -> exact operator-overload resolution
  -> operator result type
  -> optional target-context conversion through to/from
```

Overflow never implies promotion. If a declared result/target context provides an applicable conversion to a type capable of representing the produced value, that conversion handles the value at the target boundary. Otherwise the selected

operator and result type retain responsibility for their own overflow, division-by-zero, range, and other value-level arithmetic behavior.

For the standard language-provided simple types, arithmetic, ordering, equality, logical, and bitwise operators follow their conventional meanings. `!=` is the logical negation of `==`; `&&` and `||` are short-circuit Boolean operators; and `&`, `|`, and `^` are the ordinary bitwise operators for operand types that support those operations. Exact exceptional-value behavior belongs to the selected type/operator implementation rather than to parser precedence or operator-token classification.

Language-Owned Operators

Language-owned operators already have a registered symbol, fixity, precedence, associativity, and arity. They must not be redeclared in the project operator source. They receive additional implementations through `mode=overload`.

This category includes:

1. ordinary built-in operators such as `+`, `-`, `*`, and `==`;
2. pre-declared operator glyphs whose parse properties are fixed by the language but which may initially have no implementation.

Pre-Declared Operator Glyphs

The current alpha profile pre-declares exactly two mathematical operator glyphs: `u` and `n`. Their parse properties are fixed by the language and are listed in the [Built-In Operator Parse Table](#): both are binary infix operators with precedence `500` and left associativity. These glyphs are language-owned and therefore cannot be redeclared with `co.operator`.

Unlike an explicitly reserved future/unsupported operator spelling, `u` and `n` are enabled expression operators in the current alpha profile. The lexer recognizes them as registered operator tokens and the parser applies their language-defined fixity, precedence, associativity, and arity. Their concrete behavior is supplied through ordinary `mode=overload` operator implementations using the same ownership, signature-normalization, and overload-resolution rules as other language-owned operators.

A use of `u` or `n` therefore parses as an operator expression even when no applicable implementation exists. If overload resolution finds no matching implementation for the operand types, compilation fails during operator resolution rather than during lexing or parsing.

Hard-reserved spellings such as `->>`, `<->`, backtick, backslash, `#`, and comment openers are different: they are not overloadable or declarable unless a later language revision explicitly assigns them operator semantics.

Project-Local Custom Operator Source

A custom operator is a symbol that is neither language-owned nor hard-reserved. Its symbol and parse properties are registered only in the fixed operator component surface:

```
<project-root>/components/operators/component.fol
```

The operator component's canonical placement and no-subdirectory constraint are defined in [Project Layout](#). If the operator component is absent, the owning compilation introduces no custom operator spellings.

`components/operators/component.fol` uses the common structural component declaration and no library annotation:

```
// components/operators/component.fol
_ co.component = {

  ⊗ co.operator = {
    fixity = co.operator.fixity.infix,
    precedence = 60,
    associativity = co.operator.associativity.left,
    arity = co.operator.arity.binary,
    commutative = co.const.false,
    idempotent = co.const.false,
    identity = co.const.none,
    foldable = co.const.false,
    vectorizable = co.const.false,
    distributes_over = [],
    desugar = "intrinsic:tensor_product"
  };

  +- co.operator = {
```

```

    fixity = co.operator.fixity.infix,
    precedence = 60,
    associativity = co.operator.associativity.left,
    arity = co.operator.arity.binary
};
}

```

The body may contain only `co.operator` declarations. Imports, functions, ordinary types, variables, executable expressions, implementation packages, and nested component/library declarations are forbidden.

`co.operator` is accepted only while parsing `components/operators/component.fol` in the folder-derived operator component context. It cannot appear in package source, `src/app1.fol`, `src/component.fol`, or another component kind.

Where Custom Operator Creation Is Allowed

A project-local `components/operators/` domain is permitted only for:

1. an executable application project rooted at `src/app1.fol`; or
2. a standalone **projected application library** rooted at `src/component.fol` with `@co.dap.library` (or `@co.dap.library(type=application)`).

It is forbidden for:

- standalone packaged libraries — this prohibition is absolute even though packaged code may overload language-owned operators;
- standalone `native` or `dynamicvmrt` libraries;
- peer project-local components such as `components/native/`, `components/dynamicvmrt/`, `components/application/`, or `components/packaged/` as independent operator domains;
- loaded `lib/*.folenc` dependencies.

For an executable application, the resulting `ProjectOperatorTable` applies **only to the executable application's own `src/` source domain**. It is not made visible while parsing or resolving any project-local component, including `components/packaged/`. Exporting a packaged-component package into the application's open package graph does not retroactively grant that package access to application-defined custom operator spellings.

Consequently, no project-local component may consume, reference, implement, or use a custom operator spelling declared by `components/operators/`. This restriction applies uniformly to `components/application/`, `components/native/`, `components/dynamicvmrt/`, and `components/packaged/`. Those components may still use and legally overload FoLang-owned built-in or pre-declared operators because those spellings belong to the language rather than to the application's `ProjectOperatorTable`.

For a projected application library producer, the table likewise applies only while compiling that producer's own primary `src/` package domain. It is not visible to any project-local component owned by that producer. Custom operator syntax is fully resolved/lowered before `.fo1enc` emission and does not become importable syntax for consumers.

Operator declaration attributes

Every custom declaration must contain each required parse attribute exactly once. Optional metadata may appear at most once. Duplicate and unknown attributes are errors.

Every operator declaration attribute uses `=` between its name and value. This follows the language-wide declarative-field rule used by annotations, decorators, pragmas, and directives. `:` is not an operator-attribute binder and is a syntax error in a `co.operator` body.

Attribute	Required	Accepted value	Meaning
<code>fixity</code>	yes	<code>infix</code> , <code>prefix</code> , <code>postfix</code> , or a reserved future fixity name (<code>circumfix</code> , <code>postcircumfix</code> , <code>precircumfix</code> , etc)	parser placement
<code>precedence</code>	yes	decimal integer from <code>0</code> through <code>100</code>	binding strength
<code>associativity</code>	yes	<code>left</code> , <code>right</code> , or <code>none</code>	grouping for equal precedence

arity	yes	unary, binary, ternary, or a positive decimal integer	normalized operand count
commutative	no	co.const.true or co.const.false	semantic/optimization metadata
idempotent	no	co.const.true or co.const.false	semantic/optimization metadata
identity	no	a FoLang literal, including co.const.none	identity-value metadata
foldable	no	co.const.true or co.const.false	compile-time folding metadata
vectorizable	no	co.const.true or co.const.false	vectorization metadata
distributes_over	no	a list of quoted operator symbols	distributivity metadata
desugar	no	a string literal	intrinsic or lowering hook metadata

In the alpha profile, only `infix`, `prefix`, and `postfix` are implemented. Consequently, an alpha `prefix` or `postfix` declaration must be unary and an alpha `infix` declaration must be binary. Other fixity names remain reserved for future delimiter and slot grammars and are rejected by the alpha conformance profile.

The four required parse attributes construct the tokenizer and precedence table. Optional semantic/optimization attributes do not change tokenization or parsing.

Declaration and implementation are separate

The operator component registers only the symbol and parse/semantic metadata. It contains no implementation. Implementations use the same `mode=overload` form as built-in and pre-declared operators:

```
// vector/Vector.comp.unit.fol
_ co.unit = {
  @co.dap.operator(symbol='⊗', mode=overload)
  tensorProduct(left Vector, right Vector)->(Vector) = {
    ...
  }
}
```

A custom symbol has exactly one declaration in the operator component, but may have any number of distinct implementation overloads in legal owners. Duplicate normalized operand signatures are errors.

Built-In / Pre-Declared Overloading Does Not Require Operator Creation

`native`, `dynamicvmrt`, packaged code, and other domains that are forbidden from creating new operator spellings may still provide legal overload implementations for **language-owned** built-in or pre-declared operators such as `+`, `==`, `U`, or `n`, subject to the ordinary operator ownership and signature rules. They do not need `components/operators/` because the parser already knows those spellings.

Creating syntax and implementing already-known syntax are therefore separate capabilities:

```
new operator spelling / precedence / fixity
-> components/operators/component.fol
-> only application project or projected application-library producer

mode=overload for language-owned operator
-> legal owner declaration
-> no custom-operator component required
```

Symbol registration and exact recognition

When parsing the operator component surface, the scanner reads an operator declaration name as one maximal contiguous symbol run. After the permitted operator table is built, ordinary source scanning in that compilation domain uses the same whole-run rule. A registered custom symbol is recognized only when the complete run matches that symbol; an unknown run is rejected without being split into shorter operators.

Multi-symbol operator uses must also satisfy the operand-boundary rule defined in [Symbolic Runs, Classification, and Boundaries](#).

A custom symbol:

- must not be language-owned;
- must not be a hard-reserved spelling;
- must not contain `//` or `/*`;
- must have exactly one declaration in the permitted operator component.

Operators Do Not Cross Projected or Packaged Artifact Boundaries

A `.fo1enc` never contributes an importable parser operator table. Projected surfaces export named signatures and boundary contracts, not custom syntax. Packaged/open `.fo1enc` contexts may carry implementations of already language-known operators, but loading them cannot register a new operator spelling or parse properties in the consumer.

A consuming executable application that wants a custom spelling must register that spelling in its own permitted `components/operators/component.fo1` and provide a legal local implementation/delegation. A projected application library may likewise register custom syntax only for its own ordinary primary `src/` compilation through its sole permitted `components/operators/` exception; that parser table is never exported in `.fo1enc`.

Frontend Preparation, Parsing Order, Imports, and Reachability

FoLang uses one common source parser, but compilation preparation occurs in a defined order so that artifact metadata, component surfaces, application directives, and operator syntax are available before dependent semantic resolution.

Preparing or loading a component does **not** by itself make its symbols visible elsewhere. Except for the implicit `co` context and the operator bootstrap, visibility is established only by an import attached to the lexical context that declares it. A project-local component may import a component kind prepared earlier in the fixed dependency order; it cannot import a later kind. Packaged-component exports are published as package contexts rather than as a component surface. Standalone projected-library imports follow the independent library exposure rules, while standalone packaged-library contexts are application-open-graph-only.

Preparation and Parsing Order

1. project discovery and structural validation
↓
2. identify primary src surface
src/appl.fol OR src/component.fol
↓
3. load the installed standard-package artifact before project parsing
obtain and canonicalize the running folcc executable path
resolve symbolic links and derive install-root from bin/folcc
validate and deserialize install-root/stdlib/co.folenc
establish the reserved co package and standard symbol environment
↓
4. deserialize optional project-root lib/*.folenc dependencies
reject co.folenc or any artifact claiming the standard-package identity
reconstruct projected surfaces when present
reconstruct packaged/open package contexts when present
reconstruct type hierarchy + overload-family metadata
reconstruct runtime-operation identifiers and contracts
reconstruct applicable AST/backend information
↓
5. inspect and classify the fixed primary surface during project discovery
application: collect application-wide directives such as
@co.ddap.dynamicdispatch(...)
standalone src/component.fol:
classify projected @co.dap.library form
OR packaged @co.dap.export form
↓
6. when permitted by project kind, process
components/operators/component.fol
executable application OR projected application library only
common parser + componentKind=operators
build ProjectOperatorTable
↓
7. for executable applications only, parse remaining project-local components
native -> application -> dynamicvmrt -> packaged
common parser + folder-derived componentKind
do not expose the owning ProjectOperatorTable to component source
permit imports only from libraries and component kinds already prepared
build canonical contexts/symbol tables/ASTs
for standalone libraries, reject every other components/<kind>/ tree
↓

8. establish prepared dependency/component environment
 - projected component surfaces use component import identities
 - packaged exports use their exported package identities
 - standalone library contexts remain available according to their exposure rules
- ↓
9. parse the complete primary src domain
 - src packages first, then src/appl.fol
 - OR
 - src packages first, then src/component.fol
- ↓
10. resolve imports by canonical context reference at the declaring context
- ↓
11. for executable applications, merge selected packaged/open package contexts into the application open graph
- ↓
12. semantic/name/type/operator resolution to fixed point
 - including dynamic-multidispatch validation when enabled
- ↓
13. unused-symbol/liveness/reachability/capability/source-context validation
- ↓
14. merge applicable AST material -> Final AST
- ↓
15. serialize frontend artifact beneath build/
- ↓
16. validate required runtime-operation handlers and invoke backend

`.folenc` handling is pre-parse artifact loading; `.folenc` source is never reparsed.

Executable-application project-local components are source parsing. Every `component.fol` and applicable descendant source file uses the common FoLang grammar with its filesystem-derived source context. Component resolution is directional: `native` may use prepared libraries, `application` may additionally import `native`, `dynamicvmrt` may additionally import `native` and `application`, and `packaged` may import any earlier ordinary component kind. A reverse or cyclic component dependency is unavailable at the import site and is a semantic error. Standalone library projects reject project-local component kinds except for the projected application library's `components/operators/` exception. Standalone projected-library dependencies are resolved independently through `lib/`; packaged-library contexts are application-open-graph-only inputs.

The operator component is processed before the permitted ordinary primary `src/` source that may use custom operator spellings: executable application source, or projected application-library source under its sole operator-component exception. Its `ProjectOperatorTable` is withheld from every executable-application project-local component parse, including `components/packaged/`. The application directive preamble is established early enough that `@co.dap.dynamicdispatch(true)` can affect later overload-family validation and packaged/open AST integration.

For standalone `src/component.fol`:

```
@co.dap.library [type omitted/application/dynamicvmrt/native]
+ _ co.component
  -> projected standalone library

no @co.dap.library
+ _ co.component containing @co.dap.export(...)
  -> packaged standalone library
```

For project-local components, the folder supplies the kind and every `component.fol` uses `_ co.component`; no project-local component uses `@co.dap.library`.

One canonical symbol table; imports store references

The frontend keeps one canonical context/symbol-table representation for each prepared package, packaged package, projected component surface, or standalone library surface. An import does not copy or merge the target symbol table into the importer.

When a symbol is successfully resolved through a provisional imported context, the compiler marks the exact `SymbolId` used, marks the import used, activates the dependency edge, and makes the target context reachable.

Packaged/open contexts differ from projected surfaces: only explicitly exported packaged contexts admitted by an **executable application** participate directly in that application's open package/type/overload graph. Unselected packaged contexts remain private to their producer/component, and packaged contexts are not admitted into standalone-library or project-local-component graphs. Projected component/library implementation contexts remain isolated behind their surface.

Unused Symbols, Liveness, and Reachability

This section is the **single canonical definition** of FoLang unused-symbol, declaration liveness, import liveness, project reachability, project-owned component/packaged-component usage, library-project producer usage, and packaged `.fo1enc` consumer usage. Other sections define syntax and mechanics and link here rather than restating these rules.

Core Meaning

FoLang distinguishes availability from semantic participation:

```
parsed / loaded / deserialized
  != used
  != live
  != reachable
```

- **used** — semantic resolution actually consumes the declaration or operation, or a language-defined contract/protocol rule establishes its required liveness;
- **live** — the declaration or project entity satisfies the usage condition in the matrix below;
- **reachable** — the package, component domain, internal package, or packaged artifact is connected to the applicable project root through effective dependencies;
- **unused** — a usage-checkable project-owned declaration remains non-live after semantic resolution reaches a fixed point;
- **orphan** — physically participating project content remains unreachable after the effective dependency graph is constructed.

Parsing a declaration, importing a package/library, loading a `.fo1enc`, preparing a symbol table, naming a declaration without exercising the required semantic use, or merely making something visible does not by itself establish liveness.

Governing Principle

FoLang checks **behavior, not shape**.

A function or method has to be written, understood, tested, and maintained. An orphan one is cost with no return, so callables are usage-checked. A type or declaration is lighter, but someone must still know it exists, so it is checked as a

whole. A field, variant, or member is a slot in a data shape: it carries no logic to test or reason about, so it is not checked at all.

Everything below follows from that. It is also why a contract-required member is exempt: it is not orphan behavior, because the contract needs it and the developer could not remove it. And it is why a function-local variable is exempt: a local is not behavior anyone can call, test, or depend on — it is scratch inside behavior that is already checked.

FoLang applies three member-level rules:

```
developer-chosen behavior
  -> independently usage-checked

contract/protocol-required behavior
  -> live by conformance once the implementation itself is live

data/object shape
  -> declaration/type is usage-checked;
      fields, members, or enum/variant states are not checked
independently
```

A callable the developer could remove without breaking a declared contract is independently usage-checked. A callable that must exist because a contract or protocol requires it does not need an artificial separate call merely to avoid an unused-symbol error.

Complete Usage and Liveness Matrix

Entity / declaration	What makes it live / valid	What is independently checked	Unused result
class	class/type participates in the live semantic graph	every developer-authored method not required by an implemented interface	unused class or unused freely-authored method = compile-time error; fields are not checked
module	module participates as a live module value/declaration	every developer-authored function/method not	unused module or unused freely-authored member = compile-time error;

		required by its signature	state values are not checked
struct	struct type is semantically used	declaration/type only	unused struct = error; fields are not independently checked
cstruct	cstruct type is semantically used	declaration/type only	unused cstruct = error; fields are not independently checked
union	union type or member is semantically used	declaration/type only	unused union = error
enum	enum type or state is semantically used	declaration/type only	unused enum = error
user-defined object	declaration is semantically referenced	declaration as a whole	unused object = error
user-defined annotation object	annotation is actually applied	declaration as a whole	never-applied annotation object = error
matcher	matcher is selected in a live <code>.match(...)</code> chain	declaration as a whole	unused matcher = error
interface	at least one live class implements it	declaration/contract as a whole	interface with no live implementing class = error
signature	at least one live module conforms/matches it	declaration/contract as a whole	signature with no live conforming module = error
typeclass	at least one live instance implements it	declaration/contract as a whole	typeclass with no live instance = error
typeclass instance	at least one operation is exercised through that instance	instance as a whole	unused instance = error

ordinary unit	its contributed symbols are used	every usage-checkable contributed symbol	every unused contributed symbol = error
struct companion unit	its contributed companion functions are used	every user-declared companion function	unused companion function = error
ordinary import	at least one symbol is consumed through the imported context	import edge	zero-use import = error and no effective dependency edge
ordinary <code>src/package</code>	reachable from <code>src/app1.fol</code> or standalone <code>src/component.fol</code> roots through effective use edges	ordinary project-owned declarations	unreachable package/unused declaration = error
project-local projected component surface	exposed APIs are consumed by the owning graph	every exposed surface API	unconsumed projected API = error
project-local projected component internal package	reachable from its component surface implementation	ordinary declarations	unreachable/unused internal implementation = error
<code>components/package/component.fol</code>	selector is structurally valid	selector validity and package-publication boundary	invalid/missing selection = error; unselected packages remain component-private
selected packaged-component package in application	explicit selection permits it to enter the open graph; actual package/symbol use determines ordinary liveness	ordinary package/declaration rules	ordinary unused/reachability error
selected packaged-component package in projected application-library producer	selection makes it an intentional artifact package root	selected exports are producer roots; implementation dependencies remain strict	invalid selection/unreachable required implementation = error

<code>components/operators/component.folenc</code>	structurally valid and permitted for owning application or projected application-library producer	operator-specific declaration rules	handled by operator rules
projected standalone <code>src/component.folenc</code>	intentional projected API root	projected surface declarations are producer roots; internals remain strict	unused implementation source = error
packaged standalone <code>src/component.folenc</code>	valid export selector establishes intentional package roots	selected <code>src/</code> package contexts are producer roots	invalid selection/unreachable required implementation = error
installed <code>stdlib/co.folenc</code>	compiler-owned standard-package root loaded before project parsing; implicitly available	reachable standard declarations only	never treated as a project or unused third-party dependency
loaded developer-supplied <code>lib/<name>.folenc</code>	at least one projected or packaged/exported symbol is used	consumer artifact/import liveness	zero used exports = error; unused sibling exports valid
private projected implementation inside <code>.folenc</code>	producer already validated it	not consumer-revalidated	no consumer-side unused-symbol analysis
application project	graph rooted at <code>src/app1.folenc</code> reaches participating project-owned/open source	all applicable rows	orphan project-owned source/entity = error
projected standalone library project	graph rooted at projected <code>src/component.folenc</code> surface reaches producer implementation; a projected application library may	all applicable rows	orphan producer source/entity = error

	additionally own only its optional operator metadata component		
packaged standalone library project	graph rooted at package contexts selected by <code>src/component.fo</code> 1 export selector reaches required producer source	all applicable rows	orphan producer source/entity = error

Contract and Protocol Relationships

Interfaces, signatures, typeclasses, and matchers are contracts/protocols rather than collections of unrelated callable utilities:

```
interface -> live implementing class
signature -> live conforming/matching module
typeclass -> live implementing instance
matcher -> actual matcher selection in .match(...)
```

Their required members are not independently usage-checked.

For a class or module implementing a contract:

```
contract-required member
  -> live by conformance

additional developer-authored member
  -> independently usage-checked
```

Conformance therefore does not excuse helper/debug APIs that the developer freely added.

Typeclass instance

A typeclass instance becomes live only when at least one operation is actually exercised through that specific instance:

```
OptionMonad instance
├─ pure()      required by Monad
└─ flatMap()   required by Monad
```

```
no operation exercised through OptionMonad
-> OptionMonad UNUSED
```

```
OptionMonad.flatMap(...) resolves successfully
-> OptionMonad LIVE
-> pure() valid by conformance
-> flatMap() valid by conformance
```

An exercised operation includes:

- a qualified operation call through the instance;
- an activated method call that resolves to that instance through `@co.ddap.use`;
- live generic code that is supplied that instance and actually invokes a typeclass operation through it.

Merely importing, naming, binding, storing, passing around, or activating the instance without an operation being exercised does not by itself satisfy the instance's usage requirement.

Direct Usage Declarations

The following are live by their own semantic use rather than by implementation of a separate contract:

```
struct / cstruct / union / enum
-> type use

object
-> object declaration/reference use

annotation object
-> annotation application

matcher
-> matcher selection in .match(...)
```

For data-shape declarations, fields/members/states are not independently checked. For matchers, `matchCase` is protocol-required and therefore not independently checked.

Language-owned annotations, directives, and pragmas such as `@co.dap.*`, `@co.ddap.*`, and `@co.pdap.*` are compiler/language facilities rather than project-owned declaration symbols and are outside project unused-symbol validation.

Strict Per-Symbol Declarations

Ordinary units, struct companion units, freely-authored class methods, and freely-authored module functions/methods are strict because each symbol was independently chosen by the developer.

```
utility.unit.fol
├─ Id alias          USED
├─ Result type      USED
├─ parse()          USED
└─ format()         UNUSED -> compile-time error
```

```
Employee struct      USED
Employee.comp.unit.fol
├─ calculateTax()    USED
├─ serialize()       USED
└─ debugDump()       UNUSED -> compile-time error
```

Using one unit symbol does not use its siblings. Using a struct does not use its companion functions. Using one companion function does not use its siblings.

Imports, Components, and Packaged Artifacts

All dependency-bearing entities follow the same general rule:

```
written / discovered / loaded
    != live dependency

actual semantic contribution
    -> effective dependency edge
```

A zero-use import is pruned and reported as unused.

Project-local projected components are strict because the same project owns both their surface and implementation: every exposed API must be consumed, and every internal implementation package/declaration must be reachable from that surface.

Project-local packaged components contribute **only explicitly selected** package contexts, and those contexts are published only to the executable application's primary `src/` graph. Every unselected descendant package remains component-private and cannot be imported or referenced from outside the component. No peer project-local component receives selected packaged contexts in its package index. In an executable application, selected package-owned declarations follow ordinary application package reachability. A projected application-library producer has no packaged component; its only permitted project-local component is `components/operators/`. Packaged/open artifact roots therefore arise only from standalone packaged libraries or from `components/packaged/` inside executable applications.

A loaded `.folenc` is consumer-relaxed within its legal consumer domain: at least one projected surface symbol or, for an executable application consuming a packaged artifact, one packaged/exported symbol must be consumed for the physical dependency to be live. Unused sibling exported symbols do not become consumer-side errors. Packaged `.folenc` contexts are never legal consumer inputs for standalone libraries or project-local components.

Closed-Project Reachability

For an executable application:

```
root = src/appl.fol
open graph additionally includes:
  ordinary src/ packages
  selected components/packaged packages
  packaged/open contexts loaded from lib/*.folenc when imported
```

For a projected standalone library producer:

```
root = projected src/component.fol surface
+
permitted intentional packaged-component roots, if any
```

For a packaged standalone library producer:

```
roots = src/ package contexts selected by
        src/component.fol @co.dap.export(...)
```

After semantic/name/type/operator/dynamic-dispatch resolution reaches a fixed point, the compiler validates zero-use imports, declaration/member usage, package reachability, projected component implementation reachability, packaged selection validity, dependency liveness, and orphan project-owned entities.

Bootstrap Order

The normative high-level bootstrap sequence is the preparation order defined in [Frontend Preparation, Parsing Order, Imports, and Reachability](#). In particular:

```
project discovery
-> resolve real folcc path and load install-root/stdlib/co.folenc
-> optional project-root lib/*.folenc deserialization
-> early primary-surface/directive classification
-> permitted operator component
-> remaining components
-> complete src parsing
-> import/open-graph integration
-> semantic/type/dispatch fixed point
-> liveness/capability validation
-> Final AST
-> build artifact
-> backend
```

All FoLang source uses the common lexical and syntactic parser.

`src/component.fol` and `components/<kind>/component.fol` use the same `_co.component` structural declaration but receive different semantics from their source context and metadata. `.folenc` loading remains artifact preparation rather than source parsing.

Runtime-Operation Declarations

Backend-neutral runtime-operation marker

The exported standard `co.*` package may declare a callable, property, constructor, type/layout declaration, or another explicitly supported standard declaration without a FoLang body when `@co.dap.implementation` supplies the implementation classification required for that declaration kind. A runtime classification identifies a compiler-owned runtime operation. The marker defines the operation's backend-

independent meaning; it does not name a C++ header, C++ function, JVM member, WASM import, linker symbol, or any other backend-specific implementation.

```
// standard-package private source: f0λ.out.Console
_ co.unit = {

    @co.dap.implementation(
        kind = co.dap.implementationKind.runtime,
        operation = co.runtime.operation.out.println
    )
    println(value co.string) -> ();
}
```

The declaration above contributes two linked identities:

```
public FoLang symbol    = co.out.println
runtime operation       = co.runtime.operation.out.println
signature               = (co.string) -> co.unit
```

The currently defined standard implementation classification is:

kind	Meaning
co.dap.implementationKind.runtime	Lower through a registered backend/runtime handler identified by <code>operation</code> .

The `operation` field is required and must resolve to an authorized compiler-owned operation symbol whose contract is compatible with the annotated declaration's complete signature. Additional implementation classifications require an explicit specification revision.

The runtime-operation marker satisfies the implementation requirement for that bodyless standard declaration. A bodyless ordinary declaration with no valid forward, abstract, intrinsic, or runtime-operation classification is a compile-time error.

Strictly, this is divided between phases: the parser recognizes the bodyless declaration and the built-in annotation shape, while semantic analysis verifies that the marker is authorized, complete, and signature-compatible. A consumer compiling ordinary program source does not reparse the standard-package source;

before parsing that program source, the frontend deserializes `<install-root>/stdlib/co.folenc` and establishes the standard symbol environment used to validate `co.*` references.

When the standard package is compiled, `co.folenc` preserves the exported symbol, complete signature, and runtime-operation identifier. A consuming frontend deserializes that information, resolves a call to the public symbol, validates its arguments/result, and records the resolved operation identifier in backend-neutral HIR. The selected backend maps the operation identifier to its own internal handler and implementation.

```
co.out.println("Hello")
  -> resolved symbol co.out.println
  -> runtime operation co.runtime.operation.out.println
  -> HIR RuntimeCall(operation-id, arguments, result-type)
  -> selected backend handler
```

For example, the reference C++ HIR backend may internally map this operation to a call into a separately compiled C++ runtime function. That mapping, required header, generated C++ name, and runtime source are backend implementation details and are not serialized as FoLang source semantics in `co.folenc`. Another conforming backend may use a JVM call, WASI import, native runtime entry, or another mechanism while preserving the same observable operation contract.

Each backend advertises or registers the runtime-operation identifiers it implements. Before final code generation, the frontend/backend contract must reject every required operation for which the selected backend has no compatible handler. Standard `co.runtime.operation.*` identifiers are compiler-owned and may be claimed only by the authorized standard package. A third-party package cannot reuse those identifiers; it must provide an ordinary FoLang implementation, a `.folenc` implementation, or its own explicitly supported extension mechanism.

Runtime-operation identifiers are resolved symbols internally, not raw source fragments. An implementation may intern them as stable numeric/enum IDs in HIR and dispatch through a backend operation table. Directly appending arbitrary target-language source strings from FoLang annotations is not part of this contract.

Standard-package bootstrap boundary

Building `co.folenc` is a privileged compiler-bootstrap operation. The frontend already knows the core FoLang grammar, hard-reserved words, built-in `@co.*` metadata registry, primitive semantic identities required by the language, and the compiler-owned runtime-operation registry. It indexes and resolves the standard-package source as one closed compilation without first loading an installed standard-package artifact. During this bootstrap only, the reserved `f0λ.*` root is enabled as the private canonical package namespace; `@co.dap.export(packages={...}, as=...)` then projects the selected internal package subtrees into their public `co.*` names before serialization.

This bootstrap knowledge does not include ordinary standard-library API symbols such as `co.out.println`. Those symbols, their signatures, their owning package contexts, and their implementation markers come from the standard-package source and are serialized into the resulting `co.folenc`. The compiler distribution installs that result as `<install-root>/stdlib/co.folenc`; ordinary project compilations load it directly from that location before parsing project source. The selected backend separately supplies the handlers for the operation IDs that reachable code requires.

Forward / Extern Declarations

Variables extern declaration

```
// someUnit.unit.fol
```

```
_ co.unit = {  
  @co.dap.declare(type=extern)  
  someBool co.bool;  
}
```

Functions forward declaration

```
// someOtherUnit.unit.fol
```

```
_ co.unit = {  
  
  @co.dap.declare(type=forward, namespace="hr.Employee")
```

```
    getEmployee(id co.int)->(somepack.Employee); // @co.dap.declare is must  
    it will inform parser from where getEmployee is referred  
  
}
```

`namespace` attribute doesn't introduce any new kind it just tells parser the property or method's complete qualified name.

`hr.Employee` in `namespace` is only a qualified symbol string. With an import such as `@co.ddap.import(package="hr", as="hr")`, code may resolve `hr.Employee.getEmployee` against the declared external symbol.

A bodyless `@co.dap.generic` function or method declaration is an ordinary forward declaration under the rules above. A bodyless generic declaration **must not carry `mapping=`**; doing so is a compiler error. Generic mapping augmentation is metadata that extends the effective mapping set of an inherited generic method as defined in [Generic Mapping, Result Resolution, and Class-Inheritance Augmentation](#); it is not represented by a bodyless callable declaration.

Types external declaration

```
// Employee.fo1  
_ co.class = {  
  
    @co.dap.declare(type=extern)  
    Dept co.struct;  
  
}
```

For functions and types `@co.dap.declare` is optional. For variables it is required.

Functions

FoLang does not allow free-flowing package functions. Package functions must be declared inside an ordinary `<Fragment>.unit.fo1` file. Their public identity is the package member name, not the unit filename.

Function-Shaped Declaration Classification

FoLang deliberately reuses ordinary function-shaped surface syntax for several declarations that have distinct semantics. Any declaration with a callable shape such as `name(parameters)->(returns) = { ... }` or `name(parameters) = { ... }` is classified by the following metadata **when that metadata is attached to the function-shaped declaration**:

```
@co.dap.generic          -> GenericFunctionDecl
@co.dap.decorator        -> DecoratorDecl
@co.dap.extension        -> ExtensionMethodDecl
@co.dap.macro            -> MacroDecl
@co.dap.template         -> TemplateDecl
@co.dap.native           -> NativeFunctionDecl
@co.dap.executionmodel   -> ExecutionModelFunctionDecl
@co.dap.operator         -> OperatorOverloadDecl
@co.dap.indexer          -> IndexerDecl
```

These declarations may share the ordinary callable grammar and parsing machinery, but they are **different declaration kinds**. Each function-shaped declaration has exactly one AST declaration kind. A specialized declaration is not an ordinary `FunctionDecl` carrying flags that make it generic, decorator, extension, macro, template, native, execution-model, operator, or indexer behavior; its specialized AST node owns those semantics directly.

The function-shape-classifying metadata forms listed above are **mutually exclusive** on one function-shaped declaration except for the deliberate `@co.dap.operator` + `@co.dap.extension` composition. In that combination, the declaration remains an `OperatorOverloadDecl`; `@co.dap.extension` supplies the existing target/owner and does not create a second AST declaration kind.

`@co.dap.operator` and `@co.dap.generic` are explicitly incompatible on the same declaration. An operator declaration never introduces generic parameters, so `@co.dap.operator` + `@co.dap.generic`, with or without `@co.dap.extension`, is a compile-time error. A generic enclosing class or struct does not change this rule: the enclosing type may be generic while the operator declaration itself remains non-generic.

An operator overload contributed to an existing type through the function-level extension mechanism uses `@co.dap.extension(fortype=...)`. An operator owned directly by a user-defined struct companion or class does not need that extension annotation. Every other combination of two function-shape-classifying forms is a compiler error because one declaration cannot simultaneously have two declaration kinds.

A function-shaped declaration not classified by one of the metadata forms above is an ordinary `FunctionDecl`, irrespective of other non-classifying metadata that is valid at that declaration's source location. Such metadata may affect visibility, validation, optimization, or other behavior without changing the declaration's AST kind. This rule does not relax metadata-placement restrictions; in particular, `@co.pdap.*` pragmas are valid only in an executable application's `src/app1.fol` and cannot be attached to package-, component-, or library-owned function declarations.

The classification is local to function-shaped declarations. For example, `@co.dap.generic` attached to a `co.struct` or `co.class` does not create a `GenericFunctionDecl`; the explicit struct/class declaration kind remains authoritative. Likewise, explicitly distinguishable declarations such as classes, structs, type classes, extensions, modules, variables, and type constructs are outside this function-shape disambiguation rule.

Normal

```
// general.unit.fol
_ co.unit = {

    fun1(k co.int, b co.char)->(co.int, co.char) = {
        // function body
    }
}
```

A FoLang function may return multiple values.

Default Parameters

```
// somefununit.unit.fol
```

```

_ co.unit = {
  fun1(k co.int, b co.char = 'A')->(co.int, co.char)={
  }
}

usage:
fun1(10, 'B'); // k = 10 and b = 'B'
fun1(10);      // k = 10 and b = 'A', the declared default value

```

When an argument for a default parameter is not supplied, the parameter assumes its declared default value.

Variadic Functions

Curried functions are not allowed to be variadic, and vice versa.

```
//someCurried.unit.fol
```

```

_ co.unit = {
  fun1 (k co.int, ...b co.char)->(co.int, co.char)={
  }
}

```

Optional Parameters

```
//someOptional.unit.fol
```

```

_ co.unit = {
  fun1(k? co.int)->()={
    k.omitted.then({

    }).default({

    });
  }
}

```

When an optional argument is not supplied, the parameter is in the language-defined omitted/unprovided state and its `omitted` flag is `true`. This state does **not** assign `co.const.none` (or any other sentinel value) to the parameter.

`k.omitted` tests whether a value was supplied; developers must not assume that reading or printing an omitted `k` produces `co.const.none`.

Named Parameters

```
// someNamedParam.unit.fol
```

```
_ co.unit = {  
  fun1(~k co.int, ~v co.int)->()={  
  
  }  
}
```

Usage:

```
fun1(v=10,k=20); // valid  
fun1(10,20); //valid here k =10 and v=20
```

Named-parameter declaration is all-or-none within a function: a function cannot mix `~`-named parameters with ordinary parameters. When a call uses argument names, their order does not matter. A function whose parameters are all declared with the named form may still be called positionally where otherwise valid, as shown above.

Function Results

A function result clause contains zero or more **result types** and never declares result names. For a declaration `->(R1, R2, ..., Rn)`, result position `i` has type `Ri`. Result positions are ordered and positional; they do not introduce identifiers, bindings, or callee-local variables.

A `this =>` callable-result statement that produces values must produce the required number of values, and each returned value must satisfy the corresponding declared result type. The caller may bind returned values to its own names, but

those caller-local names are not part of the callable signature. Invoking a function cannot implicitly create or reuse caller-local variables.

```
_ co.unit = {
  doSomething(input co.int)->(co.int, co.bool) = {
    this => 20, co.const.true;
  }

  printValue(value co.int)->() = {
    (value < 0).then({ this ^=>; });
    co.out.println(value);
  }
}
```

A declaration-shaped result entry is invalid. For example, `->(result co.int)` is invalid; write `->(co.int)` and bind the returned value explicitly at the call site. The frontend must not treat a function result position as a named declaration.

Function Delegates

// someFunDelegate.unit.fol

```
_ co.unit = {
  myFunc(s co.int, t co.int)->(co.int, co.int) = {
    this => 10, 10;
  }

  mySecondFun(s co.int, t co.int)->(co.int, co.int) = {
    this => 20, 20;
  }
}
```

Usage inside a legal executable function or method body:

```
@co.dap.delegate someDelegate co.delegate =
  (a co.int, b co.int)->(co.int, co.int);

someDelegate = myFunc;
someDelegate(10, 20); // invokes the currently registered function

someDelegate = myFunc;
```

```
someDelegate += mySecondFun;  
someDelegate(10, 20); // invokes the registered delegate functions
```

A delegate may register multiple compatible functions. Function chaining is the distinct mechanism for direct redirection or sequential call composition.

Multicast Delegate Invocation

When a delegate has multiple registered compatible functions, invocation proceeds sequentially through the registered functions in delegate invocation order. A function added with `+=` is invoked after the functions already registered before it.

The value returned by the delegate invocation is always the result of the **last invoked function**. Results produced by earlier registered functions do not become the result of the delegate invocation. For a multi-return delegate, the complete set of return components produced by the last invoked function is returned.

For example, if `myFunc` returns `(10, 10)` and `mySecondFun` returns `(20, 20)`, then a call made after:

```
someDelegate = myFunc;  
someDelegate += mySecondFun;
```

invokes `myFunc` and then `mySecondFun`, and the delegate call returns `(20, 20)`.

Function Chaining

```
// somefunctionChaining.unit.fol
```

```
_ co.unit = {  
    fetchEmployee(empId co.string)->(Employee)=>>empMod.getEmployee(empId);  
  
    dosomething(a co.int, b co.int)->(co.int)  
        =>> somePack.someMethod(a)  
        =>> someOthPack.someOtherMeth($1, b);  
}
```

For a chained invocation, bind variables refer only to the return values of the immediately preceding function invocation. Return components are numbered from one in declared return order: `$1` is the first return value, `$2` the second, and

so on. Therefore a preceding multi-return function exposes all of its return components to the next chaining step as `$1 ... $N`. A single-return function exposes only `$1`.

Anonymous Functions

```
// someAnonymousFun.unit.fol
```

```
_ co.unit = {  
  add co.function = (a co.int, b co.int) -> (co.int){  
    this => a + b;  
  };  
  
  res co.int = (a co.int, b co.int) -> (co.int){  
    this => a * b;  
  }(10, 20);  
}
```

Why is there no equals sign after the function signature? It is deliberately omitted because the function signature acts as the type and the body acts as the literal value. The declaration is therefore a function-object initialization, analogous to initializing any other UDT from an object literal.

Binding requirement: An anonymous-function literal may occur only as the root initializer of a variable, field, `let`, or `co.function` binding. The binding may store the function object, or it may immediately invoke the literal and store the invocation result. A literal cannot stand alone and cannot be written directly as an argument or return value. Bind it first, then pass or return the named function object. This restriction gives the literal a useful owner and lets the parser select it from the binding context without searching or backtracking. Detailed function semantics are defined in [Functions in Detail](#).

Functions in detail

Inline

```
// math_functions.unit.fol
_ co.unit = {
  @co.dap.inline
  add(a co.int, b co.int)->(co.int) ={
    this => a + b;
  }
}
```

Anonymous Functions

```
// someOtherAnonymousfun.unit.fol
```

```
_ co.unit = {
  add co.function = (a co.int, b co.int) -> (co.int) {
    this => a + b;
  };

  res co.function = (a co.int, b co.int) -> (co.int) {
    this => a * b;
  }(10, 20);
}
```

In Folang Anonymous is applicable to classes, functions and blocks

Lambda

Only allowed as an inline callback argument to receiver-qualified collection operations (e.g. `each`, `map`, `filter`, `reduce`, `forEach`, `sortBy`, `groupBy`).

Transparent grouping around the member callee is permitted, so `(nums.map)(|x| => x*x)` is equivalent to the ordinary call spelling. A bare function call such as `map(|x| => x)` is not a collection-method context. Using `|...|` anywhere else is a syntax/lint error.

```
// somelanbda.unit.fol
```

```
// Callback literal syntax, shown only in its valid collection-call context
_ co.unit = {
```

```

    funs ()->()={
      nums.map(|x| => x*x);
      words.filter(|s| => s.len() > 3);
      pairs.reduce(|acc, e| => acc + e, 0);
      dict.map(|k, v| => v * 10);
      list.sortBy(|a, b| => a.score - b.score);
    }
  }
}

```

The lambda must be a direct argument of the allowed collection call. That call may itself be nested, for example `consume(nums.map(|x| => x*x))`; the enclosing call does not make the lambda an argument of `consume`.

Inner Function

```
//someInnerFun.unit.fol
```

```

_ co.unit = {
  myfun(a co.int, b co.int)->(co.int)={
    p co.int = 10;
    someother()->()={
      co.out.println(p);
    }
    someother();
    p = 20;
    someother();
  }
}

```

The function `someother` is an ordinary inner function because it is physically declared inside `myfun`. Its free runtime names are resolved from the lexical declaration context of `myfun`; therefore `p` denotes the binding declared in `myfun` regardless of where `someother` is called within its permitted lifetime.

`@co.dap.local`, `@co.dap.nested`, and `@co.dap.inner` apply to separately declared functions and do not change the lexical-scope rule for an ordinary inner function. They provide visibility, target-state capture, or call-site association without requiring the function body to be physically placed inside the target declaration.

The permitted local-function form is the named local-function exception described in [Physical Nesting Rules](#).

Curried

A function with one parameter group may use an empty group and is then an ordinary zero-argument function. A curried function has two or more consecutive parameter groups, and every group must contain at least one explicitly typed parameter. Empty stages consume no argument and carry no type information, so forms such as `f()(x T)`, `f(x T)()`, and `f()()()` are compile-time syntax errors. This rule applies equally to ordinary functions, methods, and named local functions.

```
// someOtherCurried.unit.fol
```

```
_ co.unit = {  
  add(first co.int)(second co.int)->(co.int)={  
    this => first + second;  
  }  
}
```

Closure

```
//someClosure.unit.fol
```

```
_ co.unit = {  
  IntAdder co.type = (co.int)->(co.int);  
  
  adder()->(IntAdder) = {  
    sum co.int = 0;  
    this => (x co.int)->(co.int){  
      sum += x;  
      this => sum;  
    };  
  }  
}
```

Functions Taking and Returning Functions

Syntax 1 — Inline signature (not permitted)

```
//someInlineSignature.unit.fol
```

```
_ co.unit = {  
  // Invalid: ordinary parameter and result positions require named
```

```
types.  
  someFunction (r (co.int, co.int)->(co.int))->((co.int)->(co.int))={}  
}
```

An inline function type is a type expression, not a type-use name. Declare the argument and result types with `co.type` and use Syntax 2 instead.

Syntax 2 — Named type alias

//sommeNamedTypeAliases.unit.fol

```
_ co.unit = {  
  someFArg co.type = (co.int, co.int)->(co.int);  
  someFRet co.type = (co.int)->(co.int);  
  
  someFunction (r someFArg)->(someFRet)={}  
}
```

Syntax 3 — Function objects

//someFunObject.unit.fol

```
_ co.unit = {  
  someFArg co.function = (a co.int, b co.int) -> (co.int){  
    this => a + b;  
  };  
  
  someFRet co.function = (a co.int) -> (co.int){  
    this => a * 2;  
  };  
}
```

Function Objects and Function Types

//someAdditionalleg.unit.fol

```
_ co.unit = {  
  myobj co.function = (a co.int, b co.int)->(co.int){  
    this => a + b;  
  };  
  
  add (a co.int, b co.int)->(co.int)={ this => a + b; }  
  someFun ()->()={  
    oObj co.function = add;  
  }
```

```
}  
funtype co.type = (a co.int, b co.int)->(co.int);  
}
```

FoLang has no separate expression-form closure or curry declaration. A closure is an ordinary anonymous/local function that captures lexical bindings, and currying is expressed by an ordinary named or local function with two or more consecutive parameter groups, as defined above. ***

Associated Functions

For a user-defined struct, associated functions must be declared inside the same-package companion unit whose name matches the struct; see [Associated Functions in a Companion Unit](#).

Some Restrictions on Special Functions

Non-overloadable Function Forms

The following function forms cannot be overloaded:

1. functions with named parameters;
2. functions with optional parameters;
3. functions with default parameters;
4. variadic functions;
5. curried functions;
6. functions that use inline function-signature syntax directly in a parameter or return position;
7. functions that use a function type as a parameter or return type;
8. multi-return functions;
9. functions having a pointer, address, reference, thunk, or slice in any parameter or return position.

These categories are signature-level restrictions. A declaration that falls into more than one category remains non-overloadable for the same purpose; the categories do not create separate overload families.

Callable Identity and Static Overload Resolution

FoLang first resolves the **canonical callable identity** before comparing overload parameter signatures. For an ordinary named callable, that identity includes:

1. the canonical owning context;
2. the callable name; and
3. the applicable callable/receiver category.

For class methods, categories such as `instance`, `class`, `static`, and `object` are therefore distinct identity dimensions before parameter overload resolution. For struct companion units, receiverless, instance-associated value-receiver, and type-associated type-receiver functions likewise retain their defined receiver category as part of callable identity. A declaration in a different owner or callable/receiver category is not a sibling overload merely because its visible name and ordinary parameter list are the same.

Within one canonical callable identity, ordinary **static overload resolution** uses the compile-time argument-type tuple. Generic candidates first perform only the parameter-position/explicit generic deduction needed to establish their candidate parameter signatures; return context and generic `mapping=` rows are not candidate-selection inputs. For static tuple `(S1, S2, ..., Sn)`, a candidate with parameter tuple `(P1, P2, ..., Pn)` is applicable when every `Si` is the same as or a nominal subtype of the corresponding `Pi`. Exact matches are therefore maximally specific; when an exact signature is absent, one or more positions may widen through the nominal supertype hierarchy. Arbitrary `to`/`from` conversions and the expected return type are not used to manufacture an overload candidate.

Candidate signature `A` is more specific than candidate signature `B` when every parameter type in `A` is the same as or a subtype of the corresponding parameter type in `B`, and at least one parameter type in `A` is a proper subtype of the corresponding parameter type in `B`. Static overload resolution requires one **unique most-specific applicable** candidate. No applicable candidate is a compiler error; multiple incomparable most-specific candidates are an ambiguity error. Declaration order and left-parameter priority do not resolve ambiguity.

```
static argument-type tuple
  -> applicable parameter signatures
  -> exact match when present
```

- > otherwise nominal parent/supertype widening
- > unique most-specific applicable overload

Application dynamic multiple dispatch uses the same applicability and specificity relation; its only fundamental selection difference is that the tuple comes from runtime argument types rather than compile-time argument types.

Overload-Family Parameter and Return Rules

Within one canonical ordinary named-callable identity, overload selection is determined by the applicable **parameter signature**. A return type or return signature never participates in overload selection and cannot distinguish two sibling declarations. Therefore declarations that differ only in return type are invalid rather than separate overloads.

```
select(x Animal)->(Animal) = { ... }  
select(x Animal)->(Dog)    = { ... } // ❌ invalid: differs only by return  
type
```

Every sibling declaration in the same ordinary overload family must declare an **identical return signature**. Parameter signatures may vary according to the ordinary overload rules, but the declared result contract of the family is invariant.

```
collide(a Animal, b Animal)->(co.bool) = { ... }  
collide(a Dog,    b Cat)->(co.bool)    = { ... } // ✅ same return  
signature  
  
transform(a Animal)->(Animal) = { ... }  
transform(a Dog)->(Dog)       = { ... } // ❌ same family, different  
return signature
```

This rule applies equally to ordinary static overload resolution, static multi-parameter overload resolution, and application dynamic multiple dispatch. FoLang does not implicitly widen a selected overload's return type, infer a common result type across sibling overloads, or use the expected destination type to choose an overload. The caller therefore has one stable result contract after overload selection.

Operator implementations follow their separate normalized-operand rule: distinct operand signatures may have different result types, while identical operand signatures cannot be distinguished by result type. Operator candidate selection

itself is exact by normalized operand types, subject to the operator-owner normalization defined in the operator section: nominal subtype widening, numeric promotion, and `to` / `from` conversion do not make an operator candidate applicable. Operator declarations themselves are never generic.

For an **ordinary generic named-callable declaration**, the return-family rule is not violated merely because a declared result generic resolves to different concrete types for different valid generic instantiations. In that case the declaration still has one return-signature structure (for example `->(T)`), and `T` is resolved only after parameter/generic resolution according to the generic rules, including `mapping=` where present.

Existing Callback and Execution-Model Restrictions

Curried functions, functions with named, optional, variadic, or default parameters, functions that take or return functions or function types, and dynamically scoped or mixed-scoped functions retain the existing restrictions that they cannot be used as ordinary callbacks and cannot participate in [Execution Models and Control Abstractions](#).

Scoping Rules for Functions

All functions in FoLang have a defined scope — the set of variables a function can access.

Default — Lexical / Static Scope

All of the following are **always** lexically scoped — this cannot be changed:

```
methods
inner methods
free functions
inner functions
target-local named functions
closures
lambdas
Generic Functions
Anonymous functions
Curried functions
```

Lexical scope means a function resolves names from its **declaration site**, not from the scope of its caller. Unit-level variables are forbidden, so a unit-scoped free function receives runtime values through parameters or introduces them locally. An ordinary inner function captures from the enclosing lexical declaration context. Calling that inner function does not replace its captured context with the caller's runtime scope.

```
// scope_example.unit.fol
_ co.unit = {

  foo()->() = {
    x co.int = 10;

    printX()->() = {
      co.out.println(x); // ✅ x from the enclosing lexical scope
    }

    printX();
  }
}
```

`@co.dap.inner` — Call-Site Lexical Context

`@co.dap.inner` is not the syntax for an ordinary inner function. It annotates a separately declared function or method that becomes associated with a target at an explicit use or attachment site.

Its free runtime names use **call-site lexical-context resolution**:

1. `@co.dap.inner` parameters and local declarations
2. the innermost lexical scope at the active call or attachment site
3. enclosing lexical scopes of that call or attachment site
4. statically resolved types, annotations, imports, and `co.*` names
5. compiler error

The compiler validates these requirements at every statically known use or call site. This is a fixed property of `@co.dap.inner`; it is not selected through `@co.dap.dynamicscope` or `@co.dap.mixedscope`. The ordinary unannotated associated-function scope remains declaration-site lexical scope.

This differs from ordinary lexical inner functions, whose free runtime names are fixed by their declaration site, and from dynamically scoped associated functions, which may search outward through the runtime caller activation chain.

Associated Functions — Additional Scope Options

Only associated functions support non-lexical scoping via annotations:

The examples below are members of the same-package `Employee` companion unit.

Default lexical scope — unannotated

```
// Employee.comp.unit.fol
_ co.unit = {
  (emp Employee) process()->() = {
    co.out.println(emp.name);  // ✅ declaration scope
  }
}
```

Declaration-site lexical scope is the default and has no explicit annotation.

`@co.dap.dynamicscope` — accesses caller's scope

```
// Employee.comp.unit.fol
_ co.unit = {
  @co.dap.dynamicscope
  (emp Employee) process()->() = {
    co.out.println(name);  // name comes from caller's scope
  }
}
```

`@co.dap.mixedscope` — accesses both scopes

```
// Employee.comp.unit.fol
_ co.unit = {
  // caller scope takes priority – shadows declaration scope on conflict
  @co.dap.mixedscope
  (emp Employee) process()->() = {
    co.out.println(name);  // caller scope takes priority
    co.out.println(emp.id); // falls back to declaration scope
  }
}
```

Callback Scope Inside Dynamically Scoped Associated Functions

A callback block or lambda does not independently select lexical, dynamic, or mixed scope. The associated function that executes the callback determines how the callback's free runtime names are resolved.

Callback parameters and declarations made inside the callback always belong to the callback's local scope. Names that are not callback parameters or callback-local declarations follow the executing associated function's scope policy.

//someOperation.unit.fol

```
_ co.unit = {  
  someFun ()-> ()={  
    nums.reduce(|acc, e| => {  
      total.value += e;  
      acc + e;  
    }, 0);  
  }  
}
```

For this example:

acc, e	-> callback parameters
temporary locals	-> callback-local context
total	-> reduce's dynamic caller context
co.* and imports	-> normal built-in/import resolution

The callback syntax remains uniform. `reduce` is dynamically scoped, so unresolved runtime names in the callback are resolved through the active caller-context chain. A lexically scoped associated function would instead resolve free runtime names through its declaration context.

Why Dynamic Scope Exists — `.then`, `.loop`, `.each`, and Collections

FoLang's control-flow model is built on dynamically scoped associated functions. The executing function supplies the scope policy, so blocks do not require separate capturing and non-capturing forms.

//someScopeEg1.unit.fol

```

_ co.unit = {
    FiveInts co.type = co.int->([5]);

    someFun()->()={
        x      co.int = 10;
        total co.int = 0;
        arr    FiveInts = FiveInts{1, 2, 3, 4, 5};

        // .then reads and modifies the caller's x
        (x > 5).then({
            x.value = 20;
            co.out.println(x);
        });

        // .loop modifies the caller's x
        (x > 0).loop({
            x.value -= 1;
        });

        // .each(..., action) performs the iteration and modifies the
caller's total
        arr.each(_, val, {
            total.value += val;
        });

        // .filter, .map, and .reduce use the same dynamic caller context
        nums.filter(|x| => x > 10);
        nums.reduce(|acc, e| => {
            total.value += e;
            acc + e
        }, 0);
    }
}

```

The compiler does not create a separate capture description for each control block. It resolves names according to the scope mode of the associated function executing that block.

FoLang Control Flow Uses Dynamic Scope

```

no if/else keywords    -> .then / .otherwise / .default  – dynamic scope
no for/while keywords  -> .loop                        – dynamic scope
no foreach keywords    -> .each(..., action) – dynamic scope

```

```
no in keyword          -> .contains          – dynamic scope
no map/filter keywords -> .map / .filter      – dynamic scope
```

all control flow is implemented as associated functions
control associated functions are dynamically scoped
caller variables are accessible and mutable under the dynamic lookup rules
block syntax requires no separate capture mode

Dynamic and Mixed Lookup Order

For `@co.dap.dynamicscope`, runtime variable lookup follows this order:

1. function/callback parameters and local declarations
2. immediate caller activation context
3. caller activation contexts outward through the dynamic call chain
4. built-in ``co.*`` names and imported declarations visible to the source file
5. compiler error if no compatible declaration is available

For `@co.dap.mixedscope`, lookup follows this order:

1. function/callback parameters and local declarations
2. immediate caller activation context
3. caller activation contexts outward through the dynamic call chain
4. lexical declaration context
5. built-in ``co.*`` names and imported declarations visible to the source file
6. compiler error if no compatible declaration is available

Local declarations shadow caller declarations. Caller declarations shadow declarations from the lexical declaration context in mixed scope.

Types, annotations, imports, and other compile-time declarations continue to use ordinary static resolution. Dynamic lookup applies to runtime bindings, not to the identity of types or imported APIs.

Call-Site Validation of Dynamic Requirements

A dynamically or mixed-scoped associated function may reference a caller-provided runtime name that is not declared in its lexical context. Because it has an explicit receiver, it is declared directly in the receiver type's companion unit:

```
//Employee.comp.unit.fol
```

```

_ co.unit = {
    @co.dap.dynamicscope
    (Employee) addToTotal(value co.int)->() = {
        total.value += value;
    }
}

```

At every statically known call site, the compiler validates that the active caller-context chain provides a definitely initialized binding named `total` with a compatible type and permitted mutability. A call is rejected when the required binding cannot be proven to exist.

Because non-lexically scoped functions are non-first-class and non-escaping, their call sites remain statically identifiable. This avoids runtime string-based name lookup and allows the compiler to represent dynamic requirements using context and symbol-table references.

Scope Rules Summary

Function Type	Lexical	Dynamic	Mixed
methods	✓ declaration-site only	✗	✗
ordinary inner methods	✓ declaration-site only	✗	✗
free functions	✓ declaration-site only	✗	✗
ordinary inner functions	✓ enclosing declaration context only	✗	✗
target-local named functions	✓ declaration-site only	✗	✗
<code>@co.dap.inner</code> functions/methods	call-site lexical context	✗ no unrestricted caller-chain lookup	✗
anonymous functions	✓ declaration-site only	✗	✗
closures	✓ declaration-site capture	✗	✗

lambdas/callback blocks	determined by executing associated function	determined by executing associated function	determined by executing associated function
associated functions	✅ default	✅ opt-in	✅ opt-in
<code>.then</code> / <code>.loop</code> / <code>.each</code> / <code>.contains</code>	❌	✅ built-in	❌
<code>.map</code> / <code>.filter</code> / <code>.reduce</code>	❌	✅ built-in	❌

Additional Restrictions

- dynamically or mixed-scoped functions cannot be returned
- dynamically or mixed-scoped functions cannot be stored as values
- dynamically or mixed-scoped functions cannot be passed as ordinary callbacks
- dynamic or mixed caller contexts cannot cross thread or process boundaries
- dynamic or mixed execution cannot continue after the caller frame ends
- dynamic or mixed-scoped functions cannot participate in [Execution Models and Control Abstractions](#)
- dynamically or mixed-scoped functions cannot be curried
- named, optional, variadic, and default-parameter forms follow the same non-escaping and call-site-validation rules

Types

Named non-UDT type definitions—`co.type` aliases and polymorphic or parameterized definitions, newtypes, opaque types, dependent types, refinement and predicate types, subtypes, supertypes, and related type-producing declarations—are lexical declarations. They may be declared in `src/app1.fo1`, non-operator component surfaces, units, classes, interfaces, modules, mixins, signatures, functions, and nested executable blocks. File-backed primary UDT declarations and associated-type requirements follow their own placement rules.

This permission is necessary for alias-first construction in a local scope:

```
build()->() = {
  IntList co.type = co.List(co.int);
  FiveInts co.type = co.int->([5]);
```



```

values IntList = IntList{1, 2, 3};
fixed FiveInts = FiveInts{1, 2, 3, 4, 5};
}

```

The permission to declare `co.type` does not grant native memory access. An alias whose resolved type expression contains a pointer, reference, address, borrow, or other native indirection derivation is valid only in a native library or native component context. A non-native library may still declare aliases for arrays, slices, ranges, function types, refinement types, parameterized types, and generic specializations; it is the resolved native indirection derivation, not the alias declaration itself, that is restricted.

Examples in this section that show only a type declaration are fragments from inside a legal unit or other legal enclosing declaration.

The three axes — each adds one new power:

Axis 1: Polymorphism (terms depend on types)

```

// "Give me a type, I'll give you a value"

// Without: write separate functions
//sometypes1.unit.fol
_ co.unit = {

    identityInt(x int) → int={}
    identityStr(x string) → string={}

    // With: one function works for all types
    identity(T)(x T) → T={}

}
// This is generics / parametric polymorphism
// System F, Java generics, your @co.dap.generic

```

Axis 2: Type operators (types depend on types)

```

// "Give me a type, I'll give you a type"
//sometypes2.unit.fol
_ co.unit = {
    List(Int)      → List ={}           // List of ints type → type
    Map(String, Int) → Map  ={}         // type → type → type
    Option(T)      → variants(Some(T), None) // parameterized type
}

```

```

}
// This is kinds / higher-kinded types
// Your FoLang: Option(T) co.type = co.variants(Some(T), None)

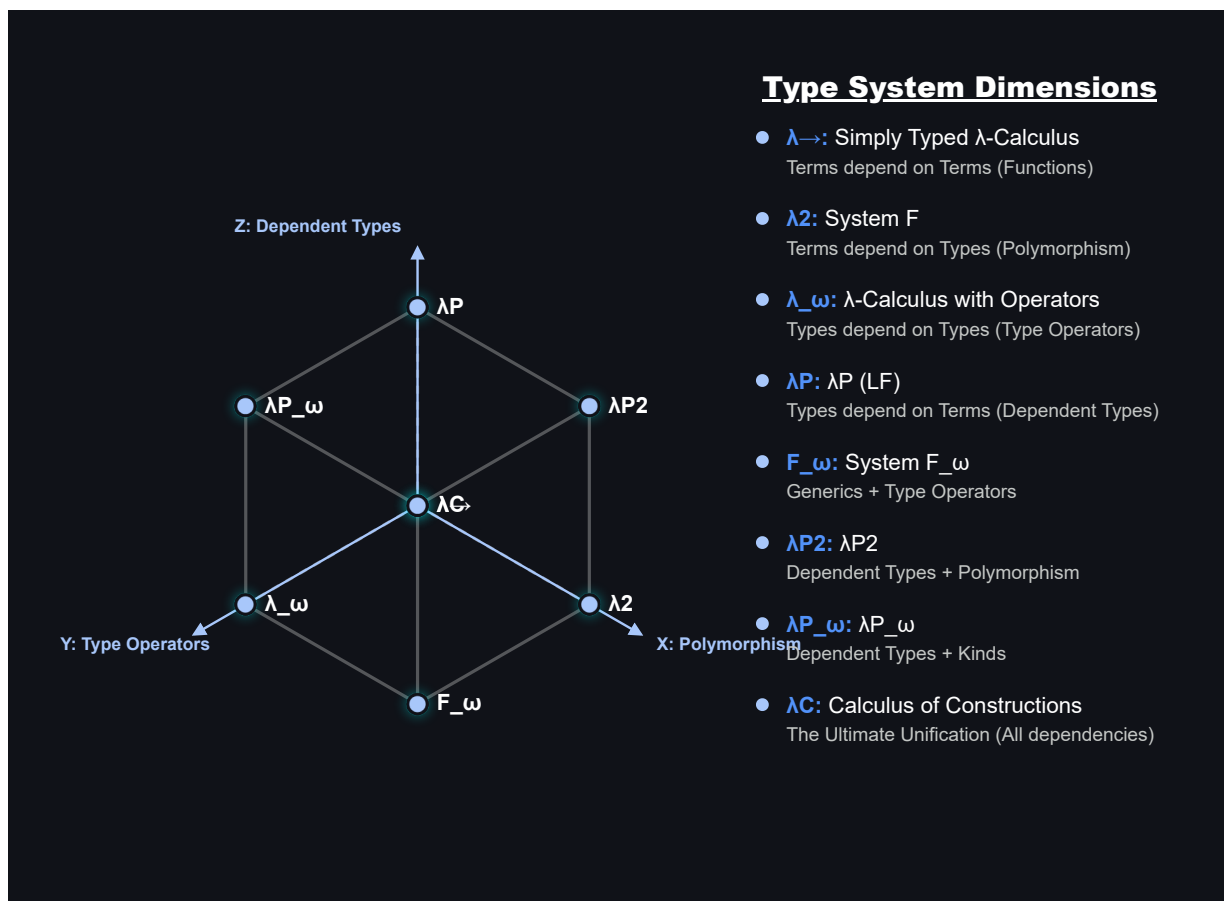
```

Axis 3: Dependent types (types depend on values)

```

// "Give me a value, I'll give you a type"
// sometypes3.unit.fol
_ co.unit = {
  Vector(3)      → array of exactly 3 elements
  Matrix(2, 3)   → 2x3 matrix
}
// The type is indexed by a value. The index may be a compile-time constant
// or a
// symbolic value parameter bound by the enclosing declaration signature.
// Value predicates such as "x > 0" belong to refinement types, not
// dependent indexing.

```



Lambda Cube

Refinement Types

A refinement type restricts the admissible values of an existing base type by a Boolean predicate. It does not introduce a separate binder name merely so the predicate can refer to the value being tested. The contextual token `_` denotes that **candidate value** inside the refinement predicate.

The canonical declaration form is:

```
// types.unit.fol
_ co.unit = {
  positiveInt co.refinementType =
    (co.int).where(_ > 0);
}
```

The declaration above means that `positiveInt` admits values of `co.int` that satisfy `_ > 0`. The `_` placeholder has the base type (`co.int` in this example), and every occurrence of `_` in the same refinement predicate refers to the same candidate value.

```
// types.unit.fol
_ co.unit = {
  percentage co.refinementType =
    (co.int).where(_ >= 0 && _ <= 100);

  evenInt co.refinementType =
    (co.int).where(_ % 2 == 0);

  nonEmptyString co.refinementType =
    (co.string).where(_.length > 0);
}
```

The predicate supplied to `.where(...)` must resolve to `co.bool`. The candidate placeholder is available only while resolving that refinement predicate. It does not introduce a variable into the enclosing unit, does not escape the declaration, and cannot be rebound or assigned. Member access and ordinary expressions may use it according to the operations supported by the base type.

`_` remains contextual. In pattern and discard positions it retains its wildcard/discard meaning, and in filename-derived primary declarations it retains its declaration-name-placeholder meaning. Only the predicate belonging to a

`co.refinementType` declaration gives `_` the candidate-value meaning.

For a statically known candidate, the compiler can evaluate the refinement predicate directly. A statically known value that makes the predicate false cannot inhabit the refinement type:

```
// types.unit.fol
_ co.unit = {
  positiveInt co.refinementType =
    (co.int).where(_ > 0);

  good positiveInt = 10;    // valid: 10 > 0
  bad  positiveInt = -5;    // compile-time error: -5 does not satisfy the
  predicate
}
```

Refinement validation is normatively a **run-time operation**. Whenever a value enters a `co.refinementType` through assignment, argument passing, return, conversion, initialization, or another admissible value-transfer operation, the refinement predicate is validated at run time. If the predicate evaluates to `co.const.true`, the value is admitted; if it evaluates to `co.const.false`, refinement validation fails.

The compiler may additionally evaluate the predicate when the candidate value is statically known. This permits an invalid statically known candidate to be rejected earlier, as in the `bad` declaration above. Such compile-time analysis is an early diagnostic and does not change the language's validation model: refinement validity is defined by the run-time predicate check rather than by a requirement that the predicate be statically provable.

`co.const.none` is outside every refinement type's admissible domain and is rejected before its predicate is evaluated; a refinement predicate never receives none as its candidate. A refinement-typed declaration may omit its initializer, but any access before a definitely established valid assignment is the compile-time diagnostic `Uninitialized`, and `.isNone()` is invalid for the refinement-typed value. See [Non-None Refinement and Dependent Values](#).

Refinement, Dependent, and Associated Types

These three mechanisms solve different problems:

Form	What determines or constrains the type	FoLang role
<code>T co.refinementType = (Base).where(predicate)</code>	a predicate restricts which values of <code>Base</code> are admitted	value-set restriction
<code>Vector(n)</code> / a function returning <code>co.dependentType</code>	a value participates in the resulting type identity or shape	value-indexed type
<code>Entity co.type;</code>	a signature requires a matching module to supply an ordinary concrete type alias	abstract module type component
<code>T co.associatedType;</code>	a signature declares an unknown type parameter associated with another generic or parameterized type component	associated generic type component

A refinement predicate therefore does not make its candidate value an index of the type in the dependent-type sense. Likewise, an associated type is not a predicate-restricted value set; it is a type component selected by a matching module.

A `co.refinementType` declaration is also distinct from `co.subtype` and `co.supertype`. Refinement adds a value predicate to a base type. It does not by itself define the inheritance, variance, or assignability rules of the separate subtype/supertype declaration kinds.

Predicate Types

Folang Predicate type has different meaning than defined universally. A predicate type is type which holds type as a value based on conditions/constraints/refinements/capabilites applied

```
// somePredicateType.unit.fol
```

```
_ co.unit={
  someType co.predicateType =
    co.type.where(
      candidate =>
```

```

        candidate == co.int ||
        candidate == co.string
    );

    sortableNumberType co.predicateType =
        co.type.where(
            candidate =>
                candidate <: co.number &&    // candidate is subtype of
number
                candidate.implements(co.Comparable) &&
                !candidate.isAbstract
        );
}

```

Usage: Eg1 :

// SomeGenericStruct.fol

```

@co.dap.generic(
    types=[
        {
            name=T,
            variance=covariant,
            constraints=[someType]
        }
    ]
)
_ co.struct = {
    ...
}

```

Eg2:

//someotherUnit.unit.fol

```

_ co.unit= {

    inspect(t someType)->() = {
        (t == co.int).then({
            // Handle the int type object.
        }).otherwise(t == co.string).then({
            // Handle the string type object.
        });
    };
}

```

```
}
```

```
}
```

Note: `<:` subtype , `>:` supertype and `==` for same type and `!=` different types

Dependent Types

Type-Valued Functions

A function may accept ordinary values or type values and may return a type object. It remains an ordinary function declaration: the parser does not create a separate function category by inspecting its parameters or results. The declared result kind describes the returned value. `co.type` identifies a type object, while `co.dependentType` identifies a type object whose identity depends on one or more values.

When a returned type depends on an argument, the function computes or selects a dependent type. This differs from a parameterized `co.type` declaration such as `Option(T)`, which directly defines a family of types.

A value of a concrete dependent type cannot contain `co.const.none`. A dependent-typed declaration without an initializer is definitely uninitialized; access before assignment produces the compile-time diagnostic `Uninitialized`, and `.isNone()` is invalid for that value. This non-none rule applies to the resolved dependent value type such as `Vector(3)` or `Matrix(r, c)`. See [Non-None Refinement and Dependent Values](#).

```
// sometypes4.unit.fol
```

```
_ co.unit = {  
    // Vector – ordinary function returning a value-indexed type object  
    // takes → co.int (size)  
    // returns → co.dependentType (a type)  
    Vector(n co.int)->(co.dependentType) =  
        co.int->([n]);  
  
    // calling Vector(3) returns a TYPE at compile time  
    // that type is co.int->([3])
```

```

v3 Vector(3) = Vector(3){1, 2, 3};    // type is Vector(3)
v4 Vector(4) = Vector(4){1, 2, 3, 4}; // type is Vector(4) –
different type!

// Vector(3) ≠ Vector(4) – completely different types
// size is part of the type – compiler knows at compile time
}

```

More About Type

```

Name(T) co.data = variants;
  → concrete parameterized ADT definition
  → right-hand-side definition is mandatory

T co.associatedType;
  → associated-type requirement
  → declared by a signature and concretely bound by each matching module

Stack co.type;
  → required type-alias component
  → permitted only inside a signature

T co.associatedType = ExistingType;
  → matching-module binding for the associated generic type component

Stack co.type = GenericType(T);
  → matching-module definition of the required alias after T is resolved

Name co.type = ExistingType;
  → concrete type alias in an ordinary type context, or a fixed/manifest
  type component in a signature

```

An Ordinary Function May Return a Type

```

Vector      → ordinary function
Vector(3)   → function call → returns type co.int->([3])
Vector(4)   → function call → returns type co.int->([4])

just like:
  add(1, 2) → returns a value  (3)
  Vector(3) → returns a type   (int[3])

```

Compiler Enforced Size Safety

//sometypes5.unit.fol

```
_ co.unit = {  
  // dot product – only valid for same size vectors  
  // compiler enforces this via dependent types  
  dotProduct(a Vector(n), b Vector(n))->(co.int) = {  
    // n is same for both – compiler verified  
  }  
  
  v3 Vector(3) = Vector(3){1, 2, 3};  
  v4 Vector(4) = Vector(4){1, 2, 3, 4};  
  
  dotProduct(v3, v3);    // ✓ same type Vector(3)  
  dotProduct(v3, v4);    // ✗ compiler error – Vector(3) ≠ Vector(4)  
}
```

Matrix — Two-Parameter Type-Valued Function

//sOMEMATRIX.unit.fol

```
_ co.unit = {  
  // Matrix – takes rows and cols, returns dependent type  
  Matrix(r co.int, c co.int)->(co.dependentType) =  
    co.int->([r, c]);  
  
  m34 Matrix(3, 4) = Matrix(3, 4){1,2,3,4,5,6,7,8,9,10,11,12};  
  m45 Matrix(4, 5) = ...;  
  
  // matrix multiply – cols of A must equal rows of B  
  // n must match – compiler verified  
  multiply(a Matrix(r, n), b Matrix(n, c))->(Matrix(r, c)) = {  
    // compiler ensures dimensions are compatible  
  }  
  
  multiply(m34, m45);    // ✓ Matrix(3,4) × Matrix(4,5) = Matrix(3,5)  
  multiply(m34, m34);    // ✗ compiler error – 4 ≠ 3  
}
```

Stack — Value and Type Parameter

//somestack.unit.fol

```
_ co.unit = {  
  // Stack – takes size and element type  
  Stack(n co.int, T co.type)->(co.dependentType) =  
    T->([n]);  
  
  s Stack(10, co.int)    = ...; // stack of max 10 ints  
  t Stack(5,  co.string) = ...; // stack of max 5 strings  
}
```

Type Is Value + Kind Combined

```
Vector(3):  
  kind  = Vector    (what it is)  
  value = 3         (how many)  
  type  = Vector(3) (both together – the dependent type)  
  
Vector(3) ≠ Vector(4)  ← different types entirely  
Vector(3) = Vector(3)  ← same type
```

Parameterized Types and Type-Valued Functions

```
// option.unit.fol  
_ co.unit = {  
  // Parameterized type declaration: Option accepts one type parameter.  
  Option(T) co.type =  
    co.variants(Some(T), None);  
  
  // Value-indexed ordinary function: Vector computes a dependent type  
  // object.  
  Vector(n co.int)->(co.dependentType) =  
    co.int->([n]);  
}
```

`Option` and `Vector` both operate at the type level, but they are different declaration categories:

```
Option(T) co.type
  -> parameterized type declaration
  -> substitution produces Option(T)

Vector(n)->(co.dependentType)
  -> ordinary function returning a type object
  -> computation produces a type
```

Simple Dependent Type

//someiden.unit.fol

```
_ co.unit = {
  identity(x co.int)->(x.type) = { this => x; }
}
```

Compile-Time and Runtime Values in Type-Related Computation

FoLang distinguishes value-indexed dependent types, compile-time type computation, runtime type descriptors, and runtime values whose concrete types may differ. These mechanisms are related, but they are not interchangeable.

1. Value-Indexed Dependent Types

A dependent type may contain a value as part of its type identity. The value index may be a compile-time constant, a symbolic type-level value, or a runtime function parameter whose relationship to the result is tracked by the compiler.

//someIndex.unit.fol

```
_ co.unit = {
  readVector(n co.int)->(Vector(n)) = {
    ...
  }
}
```

The compiler does not need to know the concrete value of `n` while compiling `readVector`. It records the relationship:

```
input index = n
```

```
result type = Vector(n)
```

Likewise: // sometypes6.unit.fol

```
_ co.unit = {  
  dotProduct(  
    left  Vector(n),  
    right Vector(n)  
  )->(co.int) = {  
    ...  
  }  
}
```

requires both vectors to have the same value index. Dependent typing means that a type contains or is constrained by a value; it does not mean that an arbitrary runtime branch can silently change the static type of an already compiled variable.

2. Compile-Time Type Computation

A function may compute and return a type when it is guaranteed to execute during compilation. // someFuncomp.unit.fol

```
_ co.unit = {  
  @co.dap.comptime  
  @co.dap.eager  
  chooseType(value co.int)->(co.type) = {  
    (value < 100)  
      .then(co.string)  
      .default(co.bool);  
  }  
}
```

The arguments must be compile-time evaluable when the result is used in a static type position: //someTest1.unit.fol

```
_ co.unit = {  
  someFun()->()={  
    Selected co.type = chooseType(10);  
    value Selected = "Hello";  
  }  
}
```

Invalid: `//someTest2.unit.fol`

```
_ co.unit = {  
  someFun()->()={  
    input := co.in.readInt();  
    Selected co.type = chooseType(input);  
    // compiler error: `input` is not compile-time evaluable  
  }  
}
```

Conceptually:

```
compile-time value  
-> compiler executes the type function  
-> one concrete static type is available before ordinary type checking  
completes
```

3. Built-in compile-time type computation

A function may compute and return a type when it is guaranteed to execute during compilation.

`decltype` is a built-in method.

The arguments must be compile-time evaluable when the result is used in a static type position: `// someFun5.unit.fol`

```
_ co.unit = {  
  someFun()->()={  
    someIntVar co.int ;  
    someVar co.hokrlt.type.decltype(someIntVar) = 200;  
  }  
}
```

4. Runtime Type Descriptors

An ordinary function returning `co.type` produces a runtime type descriptor when it is not executed at compile time. `//someruntime2.unit.fol`

```

_ co.unit = {
  selectType(value co.int)->(co.type) = {
    (value < 100)
      .then(co.string)
      .default(co.bool);
  }

  selectedType co.type = selectType(input);
}

```

Here, `selectedType` is a runtime object that describes a type. It may be used for reflection, runtime validation, dynamic loading, metadata inspection, or dynamic object creation where those capabilities are permitted.

A runtime type descriptor cannot ordinarily be used as the static type of a declaration: `//someruntime3.unit.fol`

```

_ co.unit = {
  someFun()->()={
    value selectType(input);

    // compiler error: runtime type descriptor used in a static type
    position
  }
}

```

Conceptually:

```

runtime value
  -> runtime type-selection function
  -> co.type descriptor object

```

A value that represents a type is not automatically a compile-time-resolved static type. ***

5. `@co.dap.typefromvalue`

`@co.dap.typefromvalue` derives a type from the type of a returned compile-time value. In the initial FoLang specification, it is permitted only for compile-time evaluation. `//sometypeval.unit.fol`

```

_ co.unit = {
  @co.dap.comptime
  @co.dap.typefromvalue
  inferType(value co.int)->(co.type) = {
    (value < 100)
      .then("Hello")
      .default(co.const.true);
  }
}

```

The compiler derives:

```

"Hello"      -> co.string
co.const.true -> co.bool

```

Every argument must be compile-time evaluable when the result is used in a type position. Runtime value-based type selection must instead use an explicit runtime representation.

6. Runtime Values with Different Concrete Types

When a runtime branch may produce unrelated concrete value types, a local binding declared with `::=` may hold those values at different times. A function result nevertheless requires one declared stable outer type; use a tagged value, an ADT, or another explicitly packaged representation when unrelated concrete result types must cross the callable boundary.

Using `co.tag`: `//someruntime1.unit.fol`

```

_ co.unit = {
  selectValue(value co.int)->(co.tag) = {
    (value < 100)
      .then(co.tag(co.string, "Hello"))
      .default(
        co.tag(co.bool, co.const.true)
      );
  }
}

```

The outer static type is always `co.tag`:

```
co.tag
├ runtime type descriptor
└ value compatible with that descriptor
```

An ADT provides a more strongly typed alternative:

```
SelectedValue co.data =
  StringValue(co.string)
| BoolValue(co.bool);

selectValue(value co.int)->(SelectedValue) = {
  (value < 100)
    .then(StringValue("Hello"))
    .default(BoolValue(co.const.true));
}
```

Summary

```
Vector(n)
  -> value-indexed dependent type

@co.dap.comptime function returning co.type
  -> compile-time type computation

ordinary runtime function returning co.type
  -> runtime type descriptor

runtime branch returning unrelated value types
  -> tag, ADT, dynamic value, or another explicit package
```

A runtime type descriptor is a value that represents a type. It must not be confused with a statically resolved dependent type.

Parameterized Type Declarations and Type-Valued Functions

Two declaration families produce types from parameters. The spelling depends on whether the declaration directly defines a type family or computes a type through a function body. `//someEg7.unit.fol`

```
_ co.unit = {
  // all parameters are types -> parameterized co.type declaration
```



```

Option(T) co.type = co.variants(Some(T), None);
someAlias(F) co.type = Functor(F);

// ordinary functions returning dependent type objects
Vector(n co.int)->(co.dependentType) = co.int->([n]);
Stack(n co.int, T co.type)->(co.dependentType) = T->([n]);
}

```

A parameterized `co.type` declaration defines a parameterized type. Its type parameters appear directly in the declaration head and it does not use `@co.dap.generic`.

A function that accepts values or type values and returns `co.dependentType` remains an ordinary function. `Stack` can mix value parameters and type-valued parameters and compute the resulting type object.

`co.dependentType` is both a result kind and a direct type-declaration kind. A function uses it when a value parameter determines the produced type. A direct declaration may use it when no parameter list is required:

```

LengthBound co.dependentType = co.int;

```

The kind is also usable in a declarator. If a function returns `co.dependentType`, a binding receiving that result may therefore be declared `co.dependentType`.

A type-valued function has the same result-list rules as every other function. It may have zero, one, or multiple result positions, and each declared result kind describes the value occupying that position. Result positions are unnamed and ordered exactly as for every other FoLang function. No `isType` flag is needed because `co.type` and `co.dependentType` already identify type-object values.

Parameterized aliases are transparent

An alias declared with `co.type` names the same type, not a new one.

```

//someParameg1.unit.fol

```

```

_ co.unit = {
  someAlias(F) co.type = Functor(F);

  someFun()->()={
    someAlias(co.List); // the same type as Functor(co.List)
    someAlias(Option); // the same type as Functor(Option)
  }
}

```

```
}
}
```

Because the alias adds no identity, it creates no separate instance slot: an instance cannot be declared for `someAlias` as distinct from one for `Functor`. Declaring one would be a duplicate.

Named parameters also allow reordering and partial application, which a positional placeholder could not express. `//someTypeClz1.unit.fol`

```
_ co.unit = {
  Pair(F, G) co.type = Transformer(F, G);
  Flip(F, G) co.type = Transformer(G, F);
  Fixed(F)   co.type = Transformer(F, Set);
}
```

Use `co.newtype` instead when a **distinct** identity is wanted, such as the wrapper described in [Where an Instance Is Declared](#).

Dependent Type Index Rules

An **index** is an argument to a dependent type, such as the `n` in `Vector(n)`, or a dimension in an array derivation, such as the `n` in `co.int->([n])`. Both positions obey the same rules.

Examples in this section use the following dependent-type-producing function when an array type appears in an ordinary declarator. Because `N` is a value that determines the array shape, `Buffer` is not a parameterized `co.type` alias:

```
Buffer(N co.int)->(co.dependentType) = co.int->([N]);
```

An index is a literal or a name

An index is an integer literal or a name. Arithmetic, function calls, indexing and every other operator are rejected. `// someIdxEG1.unit.fol`

```
_ co.unit = {
  Buffer(N co.int)->(co.dependentType) = co.int->([N]);

  someFun()->()={
    @co.dap.const SIZE co.int = 1024;
```

```

    v Vector(3);                // ✓ literal
    v Vector(SIZE);             // ✓ @co.dap.const name
    buf Buffer(SIZE);           // ✓ named array type with the
same index rule

    v Vector(n + 1);            // ✗ arithmetic is not permitted
in an index
    v Vector(computeSize());    // ✗ a call is not permitted in an
index
    buf Buffer(n * 2);          // ✗ same rule for array sizes
}
}

```

This restriction applies only to the **size** of an array, never to element access. Indexing an array is an ordinary expression and arithmetic is fine.

```

buf Buffer(SIZE);                // dependent type application; size is
restricted
buf[i + 1] = 42;                // access – unrestricted
buf[compute(x)] = 7;           // access – unrestricted

```

What a named index may resolve to

A name used as an index resolves in exactly one of two ways.

A parameter bound by the enclosing signature. A dependent-type-producing function signature introduces the name, and every use of it inside that signature and its body refers to the bound parameter. The name is not a constant; it stands for whatever value the caller supplies. `//someEG2.unit.fol`

```

_ co.unit = {
  // n is introduced here, and bound for the whole declaration
  Vector(n co.int)->(co.dependentType) = co.int->([n]);

  // n is introduced by this signature, and both parameters must share it
  dotProduct(a Vector(n), b Vector(n))->(co.int) = {
    // ...
  }

  // n is introduced as a value parameter and reused in the return type
  readVector(n co.int)->(Vector(n)) = {
    // ...
  }
}

```

```
}  
}
```

A `@co.dap.const` **compile-time constant**. Outside a signature that binds it, a name has nothing to bind to, so it must be a constant the compiler can substitute.

```
_ co.object->(for=VectorClient) = {  
  @co.dap.const SIZE co.int = 1024;  
  
  allocate()->>() = {  
    buf Buffer(SIZE); // ✓ SIZE substitutes to 1024  
    v Vector(SIZE);   // ✓ same rule for dependent types  
  }  
}
```

Nothing else qualifies. `@co.dap.final` marks an immutable binding, and an immutable value need not be known while compiling, so it cannot be substituted.

```
_ co.object->(for=VectorClient) = {  
  @co.dap.final n co.int = readInput();  
  
  invalidIndices()->>() = {  
    bad Vector(n);           // ✗ immutable, but not known at compile  
time                           time  
  
    m co.int = 10;  
    alsoBad Vector(m);       // ✗ an ordinary variable is not an index  
  }  
}
```

So in a plain variable declaration, where no signature is binding anything, the only legal names are `@co.dap.const` constants.

An index is non-negative

Zero is permitted; a negative index is not.

```
empty Buffer(0);           // ✓ zero-length array  
  
buf Buffer(-1);            // ✗ rejected while parsing  
v Vector(-1);             // ✗ rejected while parsing  
  
_ co.object->(for=VectorClient) = {
```

```

@co.dap.const OFFSET co.int = -1;

invalidOffset()->() = {
    buf Buffer(OFFSET); // ❌ rejected after substitution
}

```

A negative literal cannot be written at all, because no prefix operator is reachable in an index position. A negative constant is rejected when the compiler substitutes it. Both are compile-time errors.

When two dependent types are equal

Two dependent types are equal when their constructors are the same and their indices are pairwise equal. An index comparison has exactly three cases.

Index form	Compared by
integer literal	value
<code>@co.dap.const</code> name	substituted literal value
parameter	name identity

```

Vector(3)    vs Vector(3)    // equal
Vector(n)    vs Vector(n)    // equal
Vector(n)    vs Vector(m)    // NOT equal – rejected
Vector(SIZE) vs Vector(1024) // equal when @co.dap.const SIZE = 1024

```

Rejecting `Vector(n)` against `Vector(m)` is the point of the feature. It is what lets the compiler catch a size mismatch without the developer writing a single check.

What FoLang deliberately does not do

FoLang does not decide index equality up to arithmetic. `Vector(n+1)` and `Vector(1+n)` are not merely unequal — they cannot be written.

Accepting them would require symbolic reasoning, and there is no partial version of it. Once `n+1 == 1+n` is accepted, the next reasonable request is `2*n == n+n`, and the type checker becomes a theorem prover by accretion. That is the complexity FoLang is built to avoid.

The cost is narrow. Length-arithmetic signatures such as `concat(Vector(n), Vector(m)) -> Vector(n+m)` are out of scope; return a dynamically sized type and check at run time instead. Everything that needs only same-parameter identity still works, and that covers the common cases.

```
multiply(a Matrix(r, n), b Matrix(n, c)) -> (Matrix(r, c))
dotProduct(a Vector(n), b Vector(n))      -> (co.int)
zip(a Vector(n), b Vector(n))             -> (Vector(n))
```

Matrix multiplication, the usual demonstration of dependent types, needs only that the shared `n` matches.

Dependent types are checked, never inferred

Every dependent type appears in a written signature. FoLang never infers one.

This is what keeps checking decidable without a constraint solver, and it is why FoLang does not adopt Hindley-Milner style whole-program inference. Inferring a dependent type would mean inferring the index **value**, not merely the type, which is the step that makes checking undecidable in general.

Indexer

Indexer functions for a struct are associated functions and must be declared inside

```
<StructName>.comp.unit.fol.
```

```
// MyListTypes.unit.fol
_ co.unit = {
  VariableInts co.type = co.int->([...]);
}

// MyList.fol
_ co.struct = {
  eles VariableInts;
}

// MyList.comp.unit.fol
_ co.unit = {

  @co.dap.indexer(symbol="[]")
  (g MyList) get(index co.int)->(co.int) = {
```

```

        this => g.eles[index];
    }

    @co.dap.indexer(symbol="[]=")
    (g MyList) set(index co.int, value co.int)->() ={
        g.eles[index] = value;
    }
}

lst MyList;
co.out.println(lst[0]);
lst[1] = 22;

```

Generics

```

@co.dap.generic(
    at=callsite,
    types=[
        {name=U, variance=invariant, bound=Number, inference=param},
        {name=T, variance=invariant, bound=Number}
    ],
    mapping=[
        {U=co.int, T=co.int},
        {U=co.float, T=co.float}
    ],
    impredicative=false,
    resolution=compiletime
)
add(a U, b U)->(T) = { this => a + b; }

```

Generic annotation fields:

Attribute	Values
types	list of generic-parameter records such as <code>[{name=T}, {name=R}]</code>
aliases	declaration-local derived type aliases such as <code>[{name=Result, type=F(B)}]</code> ; alias expressions may reference markers declared by <code>types=</code>
requires	

mapping	compile-time relationships among declared generic parameters; valid only for compile-time generic resolution
resolution	<code>runtime</code> , <code>compiletime</code>
reified	<code>true</code> or <code>false</code>
at	<code>usesite</code> or <code>callsite</code>
specializable	<code>true</code> or <code>false</code>
impredicative	<code>true</code> or <code>false</code>
lifecycle	<code>true</code> or <code>false</code> ; interpreted only when the generic declaration target is <code>co.class</code> ; ignored for lifecycle semantics on generic structs/functions/methods

types attributes

Attribute	Values
name	
constraints	
upper-bound	
lower-bound	
bound	not needed when <code>upper-bound</code> and/or <code>lower-bound</code> is specified
default	
variance	<code>covariant</code> , <code>invariant</code> , <code>contravariant</code>
nullable	
inference	<code>param</code> , <code>arg</code> , <code>var</code>
capabilities	
isAKind	an interface, class, or trait; e.g. <code>SomeInterface</code>
typekind	<code>type</code> , <code>class</code> , <code>function</code> , <code>struct</code> , <code>typecon</code> <code>structor</code>
inclusive	

The entries above are fields of each generic-parameter record inside the `types=[...]` list.

Example: `@co.dap.generic(types=[{name=T, variance=..., bound=...}])`

These are not independent attributes; they describe each declared generic type entry.

`lifecycle` is different from the per-type records above: it is a field of the outer `@co.dap.generic(...)` application. Every class already has compiler-owned inherited lifecycle implementations. For a generic class, `lifecycle=true` grants the developer permission to override or overload `@@new` and `@@init`; absent or `false` forbids developer lifecycle customization. The field does not itself expose a lifecycle call. A developer-defined lifecycle implementation participates in `::new(...)` or `::init(...)` lookup according to its own accessibility. On a generic struct, function, or method, the field is accepted but not considered for lifecycle semantics.

Generic Marker Classification

Names listed by the immediately associated `@co.dap.generic(types=[...])` are **generic markers** for that declaration. In applicable type positions the parser/frontend records those names as generic-marker references; they are not looked up as ordinary type symbols.

A type-position identifier that is **not** listed in that declaration's `types=[...]` list is an ordinary type reference. It is resolved through the normal symbol/type environment and is a compiler error when no such type exists.

```
@co.dap.generic(types=[{name=U}])  
f(x U, y T)->(U) = { ... }
```

Here `U` is a generic marker. `T` is not declared as a marker by the annotation, so `T` must resolve as an actual type symbol. If no type named `T` is visible, the declaration is invalid.

By contrast:

```
@co.dap.generic(types=[{name=U}, {name=T}])  
f(x U, y T)->(U) = { ... }
```

both `U` and `T` are generic markers.

Generic-marker spelling does not manufacture a distinct generic signature. These two declarations have the same generic parameter structure and the same parameter signature, so they conflict as duplicates in one callable identity:

```
@co.dap.generic(types=[{name=T}])  
f(x T)->(T) = { ... }  
  
@co.dap.generic(types=[{name=U}])  
f(x U)->(U) = { ... } // compiler error: duplicate generic signature
```

The compiler compares declared generic-marker roles/positions rather than treating `T` and `U` as ordinary type names. This normalization applies **only to names already classified as generic markers**. An undeclared type-position name is never converted into a generic marker merely because another declaration uses a marker with a similar role.

For example, if an actual type named `T` exists, these parameter structures are distinct:

```
@co.dap.generic(types=[{name=U}])  
f(x U, y T)->(U) = { ... } // second parameter is actual type T  
  
@co.dap.generic(types=[{name=U}])  
f(x U, y U)->(U) = { ... } // both parameters use generic marker U
```

Return types still do not participate in ordinary overload selection; the example uses the parameter signature deliberately because that is where the structural distinction matters.

Generic-Context Derived Aliases

The built-in `aliases=` field of `@co.dap.generic` declares derived type names owned by that one generic declaration. Each record has exactly two fields: `name`, the local alias name, and `type`, the type expression derived from the markers

declared by the same annotation's `types=` list.

```
@co.dap.generic(  
  types=[{name=F}, {name=A}, {name=B}],  
  aliases=[  
    {name=FunctorOf,      type=Functor(F)},  
    {name=MapFunction,    type=(A)->(B)},  
    {name=InputContainer, type=F(A)},  
    {name=ResultContainer, type=F(B)}  
  ]  
)  
mapAll(  
  inst FunctorOf,  
  value InputContainer,  
  fn MapFunction  
)->(ResultContainer) = {  
  this => inst.map(value, fn);  
}
```

These aliases are not `forall` types and do not introduce another generic parameter list, a new type, nominal identity, or a separate specialization mechanism. The `aliases=` field only gives another name to a type representation within the annotated generic declaration. Each alias resolves exactly like a `co.type` alias, using the generic parameters introduced by the same annotation. Ordinary generic substitution applies to the alias's underlying representation. For a substitution with `F=co.List`, `A=co.int`, and `B=co.string`, the effective representations are:

```
FunctorOf      = Functor(co.List)  
MapFunction    = (co.int)->(co.string)  
InputContainer = co.List(co.int)  
ResultContainer = co.List(co.string)
```

All names referenced by an alias expression must be either declared generic markers or ordinary visible type names. Alias names are available only in the associated declaration's signature and body; they are not package members and cannot be imported or used by another declaration. An alias name must not duplicate a generic marker, parameter, sibling alias, or another name in the declaration's signature scope. References between aliases use the ordinary `co.type` alias-resolution and cycle-detection rules; `aliases=` does not introduce separate ordering, expansion, or cycle semantics.

The `type=` member in this field is a deliberate built-in-metadata exception to the ordinary annotation-value restriction. The parser reads it with the type expression grammar because `@co.dap.generic` is compiler-defined and already classifies the following declaration. Custom annotations cannot define an equivalent field, introduce aliases, or affect parsing decisions.

Frontend Handling of Generic Metadata

The frontend must parse and preserve the complete `@co.dap.generic` metadata application. Fields needed to establish frontend syntax, symbol identity, generic-marker classification, type resolution, callable selection, or a concrete result contract are interpreted where required. In particular, `types=` establishes generic markers, `aliases=` establishes declaration-local derived type names after those markers are known, and `mapping=` participates in the compile-time result-resolution rules defined below.

Other generic fields and attributes may be backend- or later-stage-oriented. The frontend records and serializes them in the Final AST/backend interchange and does **not** fail frontend generation merely because it has no semantic handler for such a field. A later compiler/backend stage may interpret, validate, specialize, reify, or reject those preserved values according to the applicable feature contract. Malformed metadata syntax remains a parser error.

When `mapping=` is present, the mapping relation itself must be resolved by the frontend wherever it is necessary to produce a concrete callable/result contract. A backend-oriented field such as `resolution`, `reified`, `at`, `specializable`, or `impredicative` does not by itself block frontend artifact generation merely because the frontend does not otherwise act on it.

Generic Mapping, Result Resolution, and Class-Inheritance Augmentation

`mapping=` defines a finite compile-time relationship among the generic parameters declared by the same `@co.dap.generic` annotation. It is **not required merely because a generic parameter appears in the return signature**. If every generic needed by the return signature is already resolvable from callable parameter-position generic information (or from explicit generic arguments), the return signature uses those already-resolved generic values directly and `mapping=` is

unnecessary. `mapping=` is required only when a generic needed by the result contract would otherwise remain unresolved and is intended to be derived from already-resolved generic inputs.

```
@co.dap.generic(  
  types=[  
    {name=U, inference=param},  
    {name=T}  
  ],  
  mapping=[  
    {U=co.int, T=co.int},  
    {U=co.float, T=co.float}  
  ]  
)  
f(x U, y U)->(T) = {  
  ...  
}
```

For `f(10, 20)`, ordinary parameter analysis first establishes `U=co.int`; the mapping then establishes `T=co.int`. For arguments statically typed `co.float`, the second row establishes `T=co.float`. The return type did not select the callable and the expected destination type was not consulted.

When `mapping=` Is Not Required

A generic result needs no mapping when every generic variable occurring in the return signature is already determined directly from the callable's parameter types or explicit generic arguments.

```
@co.dap.generic(types=[{name=T}])  
identity(x T)->(T) = {  
  this => x;  
}  
  
@co.dap.generic(types=[{name=T}])  
choose(x T, fallback T)->(T) = {  
  ...  
}
```

For these declarations, parameter-position inference establishes all generic values needed by the result contract before result typing begins. For example, passing `co.int` values to `identity` or `choose` establishes `T=co.int`, so the result type is

`co.int` without a mapping row.

By contrast, this declaration is incomplete without another explicit resolution source for `T`:

```
@co.dap.generic(types=[{name=U}, {name=T}])
f(x U, y U)->(T) = {
    ...
}
```

A call such as `f(10, 20)` can establish `U=co.int`, but nothing in the parameter signature establishes `T`. FoLang does not infer `T` from the assignment target, expected destination type, or another return context. Therefore `T` must be supplied explicitly, resolved by an applicable `mapping=` row, or resolved by another mechanism explicitly defined by this specification.

The generic-result rule is therefore:

```
infer generic values from parameter positions and explicit generic
arguments
-> if every generic needed by the return signature is resolved
    mapping is unnecessary
-> otherwise
    resolve the remaining generic through mapping= or another
explicit mechanism
-> expected/destination return type never participates
```

Generic candidate preparation and resolution proceed without consulting return context:

```
resolve canonical callable identity and candidate declarations
-> for each generic candidate, infer generic values available from
    ordinary call-parameter positions and explicit generic arguments
-> instantiate enough of each candidate's parameter signature to test
    ordinary static overload applicability/specificity
-> select the unique parameter overload
-> apply mapping rows to the selected generic's already-resolved inputs
-> resolve any remaining mapped generic parameters
-> complete generic instantiation
-> type-check the concrete return/result contract
```

A `mapping=` row is therefore never used to make a generic function win ordinary overload selection. It resolves mapped generic parameters only after the callable has been selected from parameter information.

A generic parameter that occurs only in the return signature is **never inferred from the expected destination or return context**. It must be resolved by an explicit generic argument, by an applicable `mapping=` row, or by another generic-resolution mechanism explicitly defined by this specification. Result-context generic inference is not part of FoLang.

When `mapping=` is present, its rows define the permitted mapped relationships for the generic parameters they mention. Mapping matches use canonical type identity; overload-style subtype widening is not performed while choosing a mapping row. Once the determining generic values are known, the mapping must yield one consistent assignment for every unresolved mapped generic. No applicable row, multiple conflicting applicable rows, a cyclic unresolved dependency, or a mapping that leaves a required generic unresolved is a compile-time error.

Mapping does not create sibling function overloads. The declaration above remains one generic callable with one body and one declared return-signature structure `-> (T)` even though different valid instantiations may make `T` concrete as different types. This is generic instantiation, not return-type overloading.

Generic Mapping Augmentation Through Class Inheritance

Generic mapping augmentation is intentionally restricted to **class inheritance**. Package-level/free functions do not need cross-package augmentation because package ownership is part of callable identity: two same-named functions in different packages are different callables rather than one overload family. An application therefore cannot reopen another package merely to add `mapping=` rows to a package function.

When a class inherits a visible generic method, additional mapping entries may be associated with that **already inherited generic method** in the derived-class context. The thing being augmented is the inherited method's effective **mapping set**. The augmentation does not declare another callable, does not provide another body, and is not a bodyless/forward generic method declaration.

A bodyless generic method remains an ordinary forward declaration and **cannot contain mapping=**. Mapping metadata intended as an inheritance augmentation must resolve to exactly one inherited generic method and must match that method's callable/receiver category, declared generic-marker structure, ordinary parameter-signature structure, and declared return-signature structure. If no such inherited generic exists, the mapping contribution is a compiler error.

This can apply to classes defined by the application itself or classes from package contexts explicitly exported by a packaged component or standalone packaged library and therefore present in the executable application's open graph. It does not penetrate projected `application`, `native`, or `dynamicvmrt` boundaries because their internal classes and methods remain hidden behind surface APIs.

Conceptually:

```
BaseProcessor.convert
  generic markers: U, T
  implementation body: owned by BaseProcessor
  mapping set:
    {U=co.int, T=co.int}

DerivedProcessor inherits BaseProcessor.convert
  augmentation mapping contribution:
    {U=abc.Employee, T=abc.SuperEmployee}

DerivedProcessor effective inherited convert mapping set:
  {U=co.int, T=co.int}
  {U=abc.Employee, T=abc.SuperEmployee}
```

The compiler merges inherited and derived-context mapping entries as a set:

```
identical row + identical row
  -> one logical row

same already-resolved input assignment -> same derived assignment
  -> duplicate; one logical row

same already-resolved input assignment -> different derived assignment
  -> conflict; compiler error

different input assignment
  -> additional valid mapping row
```


The augmentation affects the effective inherited generic in that derived-class context; it does not rewrite the base declaration globally and does not alter sibling derived classes. Generic mapping remains a compile-time frontend mechanism where it is required to establish the concrete callable/result contract, and it is not re-evaluated from runtime argument types by `@co.ddap.dynamicdispatch(true)`.

Generic Functions — Parameters and Return Values

Rank-1: Outer function is generic; parameter uses the same type variable

`T` is fixed at the call site before the function parameter is used. The passed function is already monomorphic inside the body.

Invalid inline signature

The inline form is a compiler error because `(T, T)->(T)` is a derived type used directly in a parameter. It must be named by the generic declaration.

Required named type alias

//somGen2.unit.fol

```
_ co.unit = {
  @co.dap.generic(
    types=[{name=T}],
    aliases=[{name=SomeFArg, type=(T, T)->(T)}]
  )
  someFunction(f SomeFArg, a T, b T)->(T) = {}
}
```

`SomeFArg` belongs to the generic declaration context established by `@co.dap.generic`; it is not a free-standing parameterized alias.

Rank-2: The function parameter is itself polymorphic (higher-rank)

The passed function stays generic **inside the callee**. The `forall` binder belongs to a named `co.type`; the consuming function does not own that binder and is not made generic merely by accepting the named polymorphic type.

Named polymorphic type //someGen4.unit.fol

```

_ co.unit = {
  SomeFArg co.type = forall(T).(T, T)->(T);

  // T belongs to SomeFArg; this consumer is not itself generic.
  someFunction(f SomeFArg)->(co.int) = {}
}

```

Returning Generic Functions

Rank-1 return //someGen6.unit.fol

```

_ co.unit = {
  @co.dap.generic(
    types=[{name=T}],
    aliases=[{name=Adder, type=(T)->(T)}]
  )
  makeAdder(a T)->(Adder) = {
    adder Adder = (b T)->(T) {
      this => a + b;
    };
    this => adder;
  }
}

```

Rank-2 return — returning a polymorphic function //somGen7.unit.fol

```

_ co.unit = {
  polyIdentity co.type = forall(T).(T)->(T);

  @co.dap.generic(types=[{name=T}])
  identity(x T)->(T) = {
    this => x;
  }

  makeIdentity()->(polyIdentity) = {
    this => identity;
  }
}

```

`identity` is the named generic implementation. `makeIdentity` merely returns that already-polymorphic callable and therefore does not declare generic parameters. An anonymous function may use a generic name owned by an enclosing

`@co.dap.generic` declaration, but it cannot introduce a `forall` binder of its own.

Rank-3: A Parameter is Itself a Rank-2 Function

Rank-3 uses named `co.type` layers. Each `forall` binder remains inside the type declaration that owns it; consuming and returning functions use the resulting named types.

Named type layers `//someGen9.unit.fol`

```
_ co.unit = {  
  Rank2FnType co.type = forall(T).(T, T)->(T);  
  Rank3ArgType co.type = (Rank2FnType)->(co.int);  
  
  applyRank2(f Rank3ArgType, value Rank2FnType)->(co.int) = {  
    this => f(value);  
  }  
}
```

Rank-3 return `//somGen10.unit.fol`

```
_ co.unit = {  
  Rank2FnType co.type = forall(T).(T)->(T);  
  Rank3ConsumerType co.type = (Rank2FnType)->(co.int);  
  
  consumeRank2(f Rank2FnType)->(co.int) = {  
    this => f(42);  
  }  
  
  makeRank2Consumer()->(Rank3ConsumerType) = {  
    this => consumeRank2;  
  }  
}
```

Passing a Polymorphic Type Object to a Generic Function

FoLang makes no value/object distinction that excludes types: every type is an object. A marker introduced by `@co.dap.generic(types=[{name=T}])` may therefore resolve to `co.type`, and the corresponding parameter value may be a type object such as `co.int`, `Employee`, or `PolyId`.

In this subsection, `Box(T)` denotes a parameterized `co.type` constructor; `Box` is not an annotation-declared generic struct or class. The polymorphic shape is first named by the required `co.type` declaration, and that named type object is passed through the ordinary call syntax:

```
//somGen11.unit.fol
```

```
_ co.unit = {
  Box(T) co.type = co.variants(Boxed(T));
  PolyId co.type = forall(U).(U)->(U);

  @co.dap.generic(
    types=[{name=T}],
    aliases=[{name=BoxOfT, type=Box(T)}]
  )
  box(x T)->(BoxOfT) = {}

  someFun()->() = {
    result := box(PolyId);
    // T resolves to co.type; x contains the PolyId type object.
  }
}
```

This is ordinary generic inference, not impredicative instantiation. The call does not bind `T` to the anonymous representation `forall(U).(U)->(U)`; it binds `T` to `co.type` and passes `PolyId` as the object value. Consequently no `impredicative=true` option and no special call-argument grammar are required.

True Impredicative Instantiation

True impredicativity remains a separate explicitly modelled generic rule. It occurs when the argument is a callable value whose own type is the named polymorphic type. In that case inference attempts to bind `T` to `PolyId`, rather than to `co.type`:

```
_ co.unit = {
  Box(T) co.type = co.variants(Boxed(T));
  PolyId co.type = forall(U).(U)->(U);

  @co.dap.generic(types=[{name=U}])
  identity(value U)->(U) = {
    this => value;
  }
}
```

```

polymorphicIdentity PolyId = identity;

@co.dap.generic(
  types=[{name=T}],
  aliases=[{name=BoxOfT, type=Box(T)}]
)
box(x T)->(BoxOfT) = {}

someFun()->() = {
  result := box(polymorphicIdentity); // ❌ T = PolyId needs opt-in
}

```

The generic declaration explicitly enables that binding with `impredicative=true`. The frontend preserves this option; full impredicative instantiation is a 1.0 compiler/backend capability:

```

@co.dap.generic(
  types=[{name=T}],
  aliases=[{name=BoxOfT, type=Box(T)}],
  impredicative=true
)
box(x T)->(BoxOfT) = {}

someFun()->() = {
  result := box(polymorphicIdentity); // ✅ T = PolyId
}

```

The named-type rule still applies. `impredicative=true` permits a generic marker to resolve to a named polymorphic type; it does not permit an anonymous `forall` expression in an ordinary call argument.

The inline spelling below remains invalid because complex polymorphic types must be named before use:

```

result := box(forall(U).(U)->(U)); // ❌ use PolyId

```

Generic Function Rank Support Matrix

Scenario	Allow?	Notes
----------	--------	-------

Rank-1 generic parameter	✓ Yes	Declare markers and derived aliases in <code>@co.dap.generic</code>
Rank-1 generic return	✓ Yes	Parameters and results use named aliases when their type is derived
Rank-2 param via named <code>co.type</code>	✓ Yes	<code>forall</code> belongs to the named type declaration; the consumer uses that type
Rank-2 param via a <code>co.function</code> value declaration	✗ Compiler error	Function objects are concrete values; use <code>co.type = forall(T).(T)->(T)</code> instead
Rank-2 return via named <code>co.type</code>	✓ Yes	Return a named generic callable matching the polymorphic type
Rank-3 via named <code>co.type</code> layers	✓ Yes	Higher-rank structure is expressed by composing named types
Rank-3 return	✓ Yes	Return a named callable matching the named Rank-3 type
Rank-3 via a <code>co.function</code> value declaration	✗ Compiler error	Same rule as Rank-2; function objects are concrete
Pass a named polymorphic type object to a generic parameter	✓ Yes	<code>T</code> resolves to <code>co.type</code> ; the parameter value is the named type object
Pass a value whose type is a named polymorphic type	➡ SOON 1.0 with explicit opt-in	<code>T</code> resolves to the polymorphic type itself; declare <code>impredicative=true</code>
Inline <code>forall(...)</code> call argument	✗ Compiler error	Name the polymorphic type with <code>co.type</code> and pass that name

`@co.dap.generic(types=[...])` declares generic markers that belong to a named struct, class, function, or method declaration and carries that declaration's generic metadata. Derived parameter and result types are named in that declaration's

`aliases=[...]` metadata. This is separate from `forall(...)`, which binds names only inside the value of a `co.type` declaration. Functions express higher-rank parameters and returns by using those named polymorphic types. See [forall](#) and [Generic Declarations and Parameterized Types](#).

Generics Inheritances and Types

This is in conceptual stage not supported.

- A) Abstract vs concrete type members
- B) Path-dependent types
 - 1. Type-level projection
 - 2. Path-dependent In folang how it would be

forall

What `forall` Is — and Is Not

`forall` is **not** a general-purpose generic declaration keyword and is **not globally hard-reserved**. It is a **contextual keyword** recognized only in the value type expression of a `co.type` declaration. It introduces the complete polymorphic type written `forall(...) . ...`.

Outside that contextual polymorphic-type form, the spelling `forall` is an ordinary identifier and follows the normal declaration and name-resolution rules for the position in which it occurs. Recognizing `forall` contextually therefore does not consume the spelling globally.

Named generic structs, classes, functions, and methods use `@co.dap.generic` as their sole generic-parameter declaration mechanism. `forall` is not a declaration mechanism. A declaration-head form that attempts to use `forall(T)` as a generic declaration prefix is invalid because declaration grammar does not define such a prefix; the error does not arise from `forall` being globally reserved.

A `forall(...)` type expression may appear only as the value of a `co.type` declaration. Structs, classes, named functions, and methods declare their own generic names exclusively through `@co.dap.generic`; they may also use a named polymorphic `co.type` in a field, parameter, or result position without acquiring or redeclaring that type's internal binder. Enums, unions, modules, objects, instances, matchers, signatures, interfaces, delegates, operators, templates, macros,

decorators, execution-model declarations, and other construct categories cannot introduce `forall` binders. An anonymous function likewise cannot introduce a `forall` binder; it may use generic names already owned by its enclosing annotated generic declaration, or accept and return named polymorphic types.

Where `forall` Is Allowed — `co.type` Value Only

The contextual form is `forall(T).` followed by an anonymous type body. The parser recognizes it only while parsing the value of a `co.type` declaration. The `.` after the binder list confirms the polymorphic type body; no function, class, struct, anonymous function, or other declaration can introduce this binder directly.

Pattern:

```
forall(T). <anonymous type body>
```

Contextual-recognition rule:

```
`co.type` value context
      +
identifier spelling "forall"
      +
valid binder list `( ... )`
      +
`. `
      ↓
polymorphic forall type expression
```

For example, `polyFunction co.type = forall(T).(T)->(T);` declares a named polymorphic function type. By contrast, an occurrence of the identifier `forall` outside a `co.type` value does not enter polymorphic-type parsing.

```
// co.type alias – naming a polymorphic type for reuse

someFArg co.type = forall(T).(T, T)->(T);

// Rank-2 parameter – the function uses the named polymorphic type
someFunction(f someFArg) -> (co.int) = {}

// Named generic implementation of that type
@co.dap.generic(types=[{name=T}])
```



```
identity(x T)->(T) = { this => x; }
```

```
// Rank-2 return — returns an already-polymorphic named callable  
makeIdentity() -> (someFArg) = { this => identity; }
```

Where `forall` Is Banned — Use `@co.dap.generic` Instead

```
// ❌ compiler error — invalid generic declaration-head form; use  
@co.dap.generic  
forall(T) identity(x T)->(T) = {}
```

```
// ✅ correct  
@co.dap.generic(types=[{name=T,variance=invariant}])  
identity(x T)->(T) = {}
```

```
// ❌ compiler error  
// LinkedList.fol  
forall(T)  
_(T) co.struct = { value T; next LinkedList; }
```

```
// ✅ correct  
// LinkedList.fol  
@co.dap.generic(types=[{name=T}])  
_ co.struct = { value T; next LinkedList; }
```

```
// ❌ compiler error — invalid generic declaration-head form; Rank-1  
generics belong to @co.dap.generic  
forall(T) someFunction(f (T,T)->(T), a T)->(T) = {}
```

```
// ✅ correct  
@co.dap.generic(types=[{name=T,variance=invariant}])  
someFunction(f (T,T)->(T), a T)->(T) = {}
```

Quick Reference

Form	Status	Context
<code>forall(T) name ...</code>	❌ Compiler error	Not a defined declaration-head generic form — use <code>@co.dap.generic</code> instead

<code>name co.type = forall(T).(T)->(T);</code>	✓ Allowed	<code>co.type</code> value owns the binder
<code>function(f forall(T).(T)->(T))</code>	✗ Compiler error	Declare a named polymorphic <code>co.type</code> and use that parameter type
<code>this => forall(T).(x T)->(T) { ... };</code>	✗ Compiler error	Anonymous functions cannot introduce generic binders

The rule in one sentence: `forall(T).` binds `T` only in the value of a `co.type` declaration; a call passes that named type object, and no function or anonymous function introduces the binder directly.

Generic declarations are supported only for structs, classes, ordinary functions, and ordinary methods. Their type parameters are introduced exclusively by `@co.dap.generic`.

`OperatorOverloadDecl` is deliberately excluded even though an operator implementation has a callable shape. A declaration carrying `@co.dap.operator` must not also carry `@co.dap.generic`. A generic class or struct may own an operator, but the operator itself remains non-generic and is associated with the canonical owner declaration rather than with operator-level type parameters.

The following declaration-head generic forms are invalid:

```
// Cache.fol
_(T) co.module = {}           // compiler error
// operations.unit.fol
_(F(_)) co.unit = {}         // compiler error
Callback(T) co.delegate = (T)->(T); // compiler error
```

A parameterized `co.type` declaration is a separate parameterized-type form and does not use `@co.dap.generic`:

```
// option.unit.fol
_ co.unit = {
  Option(T) co.type =
    co.variants(Some(T), None);
}
```

Generic structs and classes remain file-backed primary declarations. Their names come from filenames:

```
// LinkedList.fol
@co.dap.generic(types=[{name=T}])
_ co.struct = {
    value T;
    next  LinkedList;
    prev  LinkedList;
}
```

```
// linked_list_values.unit.fol
_ co.unit = {
    IntLinkedList co.type = LinkedList(co.int);
    someFun()->()={
        myIntList IntLinkedList;
    }
}
```

```
// Employee.fol
@co.dap.generic(types=[{name=T}, {name=R}])
_ co.class = {
    id    T;
    name  R;
}
```

```
// employee_types.unit.fol
_ co.unit = {
    EmployeeIntString co.type =
        Employee(co.int, co.string);
    emp EmployeeIntString;
}
```

Generic functions use the same annotation but are declared inside a legal function-owning context such as an ordinary unit, class, or companion unit:

//sommGen1.unit.fol

```
_ co.unit = {

    @co.dap.generic(types=[{name=T}, {name=R}])
    add(a T, b T)->(R) = {
        ...
    }
}
```

```

}
someFun()->()={
    add_int_int := add.withTypes(co.int,co.int);

    //    or

    add_int_int co.function = add.withTypes(co.int,co.int);

    k := add_int_int(12,10);
}
}

```

Specialization

`@co.dap.specialize` to specialize generics for specific types upfront
 //sommGen2.unit.fol

```

_ co.unit = {

    @co.dap.generic(
        types=[
            {name=T}
        ],
        requires=[
            co.Add(left=T, right=T, result=T)
        ]
    )
    add(a T, b T)->(T) = {
        this => a + b;
    }
}

```

for the above generic want to specialize for `co.int` //sommGen5.unit.fol

```

_ co.unit = {
    @co.dap.specialize(
        target=add,
        types=[
            {name=T, type=co.int}
        ]
    )
    addInt(a co.int, b co.int)->(co.int) = {
        this => co.intrinsic.intAdd(a, b);
    }
}

```

```
}  
}
```

`folang` provides partial specialization below is the example for partial specializationn `//sommGen7.unit.fol`

```
_ co.unit = {  
  @co.dap.generic(  
    types=[  
      {name=T},  
      {name=R}  
    ]  
  )  
  transform(value T)->(R) = {  
    ...  
  }  
}
```

`//sommGen8.unit.fol`

```
_ co.unit = {  
  @co.dap.specialize(  
    target=transform,  
    types=[  
      {name=T, type=co.string},  
      {name=R}  
    ]  
  )  
  transformString(value co.string)->(R) = {  
    ...  
  }  
}
```

fields of specialize

Attribute	Values
target	the generic fully qualified name includes package name if omitted it is current package
types	resoulution types
priority	
strategy	intrinsic

Type Application and Arrow Tails

`->` is a structural spelling, not an expression operator. In type position, an arrow tail describes a derived type after a base type or introduces the result portion of a function type after a parameter list. Every type-level application uses parentheses directly; generic declarations do not use an arrow tail for instantiation.

FoLang deliberately distinguishes **declaration-level generics** from **parameterized `co.type` declarations**:

```
@co.dap.generic declaration
  -> apply with (...)

parameterized co.type
  -> apply with (...)

typeclass or abstract typeclass parameter
  -> apply with (...)
```

Examples:

```
// annotation-based generic declaration
@co.dap.generic(types=[{name=T}])
_ co.struct = {
  value T;
}
```

```
// box_types.unit.fol
_ co.unit = {
  IntList co.type = co.List(co.int);
  somFun ()->()={
    value IntList;
  }

  // parameterized co.type
  Option(T) co.type = co.variants(Some(T), None);
  someOtherFun()->()={
    value Option(co.int);
  }
}
```

The declarations are different: `List` receives its marker from built-in generic metadata, while `Option(T)` declares a parameterized `co.type` constructor in its head. Both are applied with the uniform `Name(...)` syntax. Because a complete application is a type expression, ordinary declarator positions use a named `co.type` alias such as `IntBox`.

An arrow tail may also denote existing derivation/function-type forms, for example:

```
co.int->([5])
co.int->(&, meta={type=out})
(co.int)->(co.int)
co.Map(co.string, co.int)
```

Type arguments occur directly inside the applied name's parentheses. The same syntax applies to annotation-declared and built-in generics, parameterized `co.type` declarations, typeclasses, and abstract type parameters. Symbol resolution determines the category and validates its arguments.

Generic type arguments are positional only and bind in declaration order. For an annotation-declared generic, that order is the order of entries in `@co.dap.generic(types=[...])`. Named and space-bound applications such as `Employee(T=co.int, R=co.string)` and `Map(key co.string, val co.int)` are compiler errors. Expected/destination return type is never used to infer an otherwise unresolved generic marker.

A fully instantiated annotation-based generic declaration is a type expression, not an ordinary declarator type. Name it with `co.type`; the subsequent typed declaration uses that alias and introduces no additional statement form.

Generic Declarations and Parameterized Types

FoLang distinguishes annotation-based generic declarations from parameterized `co.type` declarations.

Generic Structs, Classes, Functions, and Methods

`@co.dap.generic` is the sole mechanism for declaring generic structs, classes, ordinary functions, and ordinary methods. Generic parameters for these declaration kinds must not appear in the declaration head. Operator overload declarations are

not generic declarations and must not carry `@co.dap.generic`, even when their owning class, struct, or extension target is generic.

```
// Box.fol
@co.dap.generic(types=[{name=T}])
_ co.struct = {
    value T;
}
```

A generic struct can be used to achieve type-level-function behavior. Because a struct is a pure data declaration without methods, inheritance, or other object-oriented behavior, its generic application can be modelled as a mapping from type arguments to a specialized pure-data type:

```
Pair : (Type, Type) -> Type
Pair(co.int, co.string) -> specialized Pair type
```

This is a semantic model, not a separate FoLang declaration category. `Pair` remains an ordinary `@co.dap.generic` struct, and its concrete application must be named through `co.type` before use:

```
IntStringPair co.type = Pair(co.int, co.string);
```

Generic structs are therefore not pure type-level functions: they introduce ordinary struct types and participate in the normal struct type system. They merely provide the same useful type-to-type mapping behavior during generic specialization. No `co.typeLevelFunction` kind, parser production, or runtime callable is introduced.

```
// conversion.unit.fol
_ co.unit = {
    @co.dap.generic(types=[{name=T}, {name=R}])
    convert(value T)->(R) = {
        ...
    }
}
```

These forms are invalid:

```
// Box.fol
_(T) co.struct = { ... }           // compiler error
```



```
// Container.fol
_(T) co.class = { ... }           // compiler error
```

Parameterized `co.type` Declarations

A parameterized `co.type` declaration does not use `@co.dap.generic`. Its type parameters appear directly in the type declaration head, and the declaration must be inside an ordinary unit or another explicitly legal type-declaration context.

```
// option.unit.fol
_ co.unit = {
  Option(T) co.type =
    co.variants(Some(T), None);
}
```

`Option` denotes a unary parameterized type:

```
Option : Type -> Type
```

Terminology note: FoLang calls this a **parameterized type**. In type-theory literature, the same `Type -> Type` behavior is often described as a *type constructor*. FoLang does not introduce a separate constructor declaration category for it.

The alternatives declared by `co.variants(...)` are likewise not called constructors in normative FoLang terminology. A parameterized alternative is a **state function** operating at the value level, while a zero-parameter alternative is a **state value**. This keeps the type-level mapping of `Option(T)` distinct from the value-level mapping of `Some(value)`:

```
Option : Type -> Type
Some    : T -> Option(T)
None    : Option(T)
```

A zero-parameter state is referenced without `()`. Therefore `None` is the state value; `None()` is invalid.

When the RHS is `co.variants(...)`, the enclosing `co.type` is a closed variant-based type definition. Each item inside `co.variants(...)` is a declaration, not a lookup of an already-existing symbol:

```
Option(T) co.type =
  co.variants(
    Some(T),
    None
  );
```

This declaration creates the following symbols and relationships:

Option	-> unary parameterized type
Some	-> variant state function: T -> Option(T)
None	-> variant state value: Option(T)

The state names are ordinary user-defined identifiers; spellings such as `Some`, `None`, `Just`, `Nothing`, `Success`, and `Failure` are not required by the language unless supplied by the standard library. `co.variants(...)` is valid only as the variant-definition RHS of a `co.type` declaration.

During parsing/frontend construction of this RHS, the head identifier of each variant entry is introduced as a new state symbol owned by the enclosing type declaration; it is not resolved as an existing type or ordinary callable. An entry with parameters defines a compiler-provided state function; a bare entry defines a zero-parameter state value. Payload entries inside a state-function parameter list are type expressions and are resolved normally, including against type parameters from the enclosing `co.type` declaration. Duplicate state names within the same enclosing type are a compiler error.

Applying it produces a type: `//applyingEg1.unit.fol`

```
_ co.unit = {
  someFun()->()={
    value Option(co.int);
    employeeOption Option(Employee);
  }
}
```

`co.variants(...)` is a declaration-producing RHS form. Each entry declares a state owned by the enclosing `co.type`. In the example, `Some(T)` declares a one-payload state function whose result is `Option(T)`, while `None` declares a zero-parameter state value of `Option(T)`. State functions are compiler-provided and do not require separate function implementations.

`@co.dap.generic` is invalid on `co.type`, and declaration-head type parameters are invalid on structs, classes, functions, methods, signatures, interfaces, modules, enums, unions, cstructs, units, and other declaration kinds unless a later specification version explicitly adds support.

No Dedicated Parameterized-Type or Type-Function Annotation

FoLang requires no dedicated annotation for parameterized `co.type` declarations and no `@co.dap.typefunction` annotation.

```
Option(T) co.type = ...  
    -> recognized syntactically as a parameterized type declaration  
  
ElementType(container co.type)->(co.type) = ...  
    -> parsed as an ordinary function; its result kind denotes a type  
    object
```

The declaration form already determines the category unambiguously.

Templates

Typed

```
// mytypedtemplate.unit.fol  
  
_ co.unit = {  
    @co.dap.template  
    add(a co.int, b co.int)->(co.int) = {  
        this => a + b;  
    }  
}
```

Untyped

```
// MyTemplate.unit.fol  
  
_ co.unit = {  
  
    @co.dap.template  
    add(a, b)->(co.untyped) = {  
        this => a + b;  
    }  
}
```

```
}  
}
```

Annotations and Decorators

```
// Annotation implementation – named singleton object, can carry data  
  
// myAnnotation.fol  
  
_ co.object->(for=someAnnotation) = {  
    value    co.string;  
    enabled  co.bool;  
}  
  
// Decorator – function, transforms target, returns  
// Decorator.unit.fol  
  
_ co.unit = {  
  
    @co.dap.decorator  
    myDecorator(target co.function)->(co.function) = { }  
}
```

User-defined directives and pragmas cannot be declared. They are language-internal metadata categories. FoLang provides declaration constructs for user-defined annotations and decorators only.

User-defined annotation/object liveness is defined in [Unused Symbols, Liveness, and Reachability](#).

Built-in Metadata Parsing

Built-in directives, annotations, pragmas, and decorators under `@co.*` use one generic metadata application grammar:

```
annotation = "@", qualified-name,  
            [ "(", [ annotation-argument-list ], ")" ] ;
```

Every named field or attribute in a directive, annotation, pragma, or decorator uses `=`. The same rule applies recursively to records/maps nested inside a metadata application. `:` is not a metadata binder and its use in any metadata field or nested metadata record is a syntax error.

```
@co.dap.generic(types=[{name=T, variance=covariant}]) // valid
@co.dap.generic(types=[{name=U, bound=co.number}]) // valid nested fields
@co.dap.implementation(
  kind=co.dap.implementationKind.runtime,
  operation=co.runtime.operation.out.println
) // valid backend-neutral runtime-operation marker
```

For `@co.dap.implementation`, `kind` classifies how a bodyless standard declaration is implemented and `operation` identifies the compiler-owned backend-neutral runtime operation. The `operation` value is resolved as a qualified operation symbol and preserved in `.foIenc`/HIR; it is not target-language source text. The annotation is valid only on a declaration kind for which this specification permits a runtime-operation marker.

The reference intentionally contains no colon-bound `@co.*` metadata example; all such spellings are rejected by the grammar.

This metadata rule does not change ordinary value syntax. Object field initializers and runtime map entries continue to use `:` according to their own grammar:

```
employee := Employee{name: "Rao"};
StringMap co.type = co.Map(co.string, co.string);
map := StringMap{"name": "Rao"};
```

The compiler maintains a predefined built-in metadata registry for language-owned `@co.*` forms. After reading the qualified metadata name, the parser must match the **complete name** against that registry. A registered enabled form is parsed according to the common metadata grammar and its applicable known frontend rules. A registered reserved/future form may be recognized and diagnosed as unsupported according to its registry entry. An `@co.*` metadata name that is not present in the predefined registry is a **parse error**; an unknown language-owned metadata name is never silently accepted.

```
@co.* metadata name
-> lookup complete name in predefined built-in metadata registry
```

-> registered and enabled application	-> parse/collect metadata
-> registered but unsupported	-> unsupported-feature diagnostic
-> not registered	-> parse error

Recognition of the metadata **name** is strict; knowledge of every field is not. Once a built-in form has been recognized, the parser must collect and preserve the complete metadata application, including every supplied positional argument, named argument, field, attribute, and argument expression.

For each collected field or argument:

- when the frontend already has defined knowledge of that field, it may validate the applicable value shape, structural requirements, defaults, or other frontend rules;
- when the frontend has no defined knowledge or semantic handling for that field, the field is still accepted, collected, and preserved as parsed; lack of frontend field knowledge alone is not an error and does not block frontend artifact generation;
- a malformed argument expression or malformed metadata argument-list structure remains a syntax error under the common metadata grammar; and
- later semantic/backend stages may interpret preserved fields according to the applicable feature contract.

Accordingly, an unknown built-in **form name** and an unknown/unhandled **field of a known form** are deliberately different cases:

```
@co.dap.unknownForm(...)
  -> name absent from built-in registry
  -> parse error

@co.dap.generic(knownField=..., backendField=...)
  -> co.dap.generic is registered
  -> collect both fields
  -> validate fields the frontend understands
  -> preserve fields the frontend does not understand
```

User-defined metadata outside `co.*` is limited to annotations and decorators. Their qualified names are not looked up in the built-in registry; they are resolved through the ordinary imported/package symbol table and must resolve to a valid

user-defined annotation or decorator declaration. An unresolved custom metadata name is a name-resolution/compiler error. FoLang provides no user declaration construct for directives or pragmas, so custom directives and custom pragmas are not available.

Directive Placement

Directives are **source-file-level compiler metadata**. Every language-owned metadata form registered in the `DIRECTIVE` category, including every `@co.ddap.*` directive, must occur in the top-level metadata region of a source file. A directive cannot occur inside the body of a component, unit, class, struct, module, function, method, typeclass, instance, extension, matcher, annotation declaration, block, or any other declaration or nested lexical context.

The placement rule is structural and category-wide:

source-file top-level metadata region	-> directive permitted
inside <code>`_ co.component = { ... }`</code>	-> compiler error
inside <code>`_ co.unit = { ... }`</code>	-> compiler error
inside class/struct/module/typeclass/instance/etc. body	-> compiler error
inside function/method/extension/matcher body	-> compiler error
inside ordinary/nested block	-> compiler error

For file-backed declaration sources, directives appear before the file's primary declaration. For `src/component.fo1` and `components/<kind>/component.fo1`, directives therefore appear before the `_ co.component = { ... }` declaration, never inside its body. For the executable entry file `src/app1.fo1`, directives belong to the entry-file metadata preamble before the first non-metadata declaration or executable statement.

A directive's **semantic scope** is defined by the individual directive, but its **syntactic placement** is always file-level. For example, `@co.ddap.import` and `@co.ddap.alias` establish file-local bindings; `@co.ddap.use` establishes file-scoped activation; `@co.ddap.dynamicdispatch` is application-wide but is written only in the application entry-file preamble; and `@co.ddap.dynamicruntime` is valid only in a permitted `dynamicvmrt` capability source while still being written at that source file's top level.

A directive immediately preceding a primary declaration is not an annotation on that declaration. The compiler classifies the metadata name through the built-in registry first; entries classified as `DIRECTIVE` are attached to the current source-file/top-level semantic context rather than to an inner declaration AST node. Encountering a directive after entering a declaration/body context is a compile-time **metadata-placement error**.

For an ordinary package source file, the file has one primary top-level declaration. A directive in that file's metadata preamble may therefore configure or otherwise affect that primary declaration when the directive's own semantic contract says so. This does **not** make the directive a lexical declaration or a member of the primary declaration. The directive remains file-level compiler metadata.

Directives do not introduce names, lexical scopes, or symbol-table entries. The frontend records them separately with the source-file/top-level semantic context and consults them while validating or compiling the primary declaration. By contrast, symbols and symbol tables model name-bearing declarations and lexical visibility and may therefore exist recursively for classes, functions, methods, blocks, nested declarations, and other scoped constructs.

The common `@qualified.name(...)` surface syntax does not weaken this rule. After parsing the common metadata shape, the frontend classifies the built-in name by registry category. A form classified as `DIRECTIVE` or `PRAGMA` is accepted only through the file-preamble path; declaration/member/block metadata positions accept annotations or decorators instead. Thus a directive cannot be smuggled into a nested scope merely because directives and annotations share the same lexical shape.

```
SourceFileContext
├─ directives / pragmas / import metadata    // compiler metadata, not
symbols
├─ primary top-level declaration
│   └─ lexical/semantic contexts
│       └─ symbol tables
│           └─ nested symbol tables as required
```

This restriction applies automatically to future entries added to the language-owned `DIRECTIVE` registry unless the specification explicitly changes the category-wide rule.

Pragma Placement

Pragmas are **executable-application-owned configuration metadata**. Every language-owned pragma whose complete name is registered in the `PRAGMA` metadata category, including every `@co.pdap.*` form, is valid only in the executable application's fixed entry source, `src/app1.fol`. This is a source-role/metadata-placement rule, not a distinct grammar production.

The following placement invariant applies to the entire pragma category:

executable application: <code>src/app1.fol</code>	-> pragma permitted
application package source under <code>src/<package>/</code>	-> compiler error
project-local components/ <code><kind>/...</code>	-> compiler error
standalone projected application library	-> compiler error
standalone native library	-> compiler error
standalone dynamicvmrt library	-> compiler error
standalone packaged library	-> compiler error
exported/packaged package contexts	-> compiler error

A component, package, or library may document operational assumptions or recommended settings, but it cannot publish, export, inherit, or impose a pragma on its consumer. The executable application owns final application-wide policy. A pragma found outside `src/app1.fol` is therefore a compile-time **metadata-placement error** after the built-in metadata name has been recognized.

This restriction applies automatically to future entries added to the language-owned `PRAGMA` registry unless the language specification explicitly changes the category-wide rule.

Macros

```
// a. Basic macro
//macro1.unit.fol

_ co.unit = {

    @co.dap.macro
    say()->()={ this => co.macro.quote({ println("Line 1"); println("Line
2"); }); }

    // b. Escape assign
```

```

@co.dap.macro
yes_esc_assign()->(co.untyped)={
  this => co.macro.quote({
    co.macro.esc(y) = 42;
    co.out.println("Inside macro: y = ", y);
  });
}

}

// c. Debug macro with gensym

// macro2.unit.fol

_ co.unit = {
  @co.dap.macro
  debug(expr)->(co.untyped)={
    tmp := co.macro.gensym(co.var, "tmp");
    this => co.macro.quote({
      tmp = co.macro.esc(expr);
      co.out.println("Result: ", tmp);
      tmp;
    });
  }

  // d. if/else condition macro
  @co.dap.macro(
    group={items=["if","else"], chain=true},
    sugarform={forms=["if expr block"]},
    bind={vars=["x"]},
    isolate={vars=["temp", "index"]},
    gensym={prefix="tmp_"},
    hygienic=true,
    argtransform={param="body", wrap="lambda", whentype="block"},
    desugar={exprs=["if($cond) { $block }" => "if($cond,$block)"]},
    mode="inject"
  )
  if(condition expr, body block)->()={}

  blockormacro co.kind = block | macro

  @co.dap.macro(
    group={items=["if","else"], chain=true},
    sugarform={forms=["else block","else if"]},
    chainswith={macro="if", position="immediate", required=true},

```

```

    argtransform={param="body", wrap="lambda", whentype="block"},
    standalone=false,
    desugar={exprs=[
        "else if($cond) { $block }" => "else(if($cond, $block))",
        "else { $elseblock }" => "else($elseblock)"
    ]},
)
else(body blockormacro)->()={}
}

```

Other macro utilities:

1. `@co.dap.compose(using=["base_if", "blockify"])`
2. `@co.dap.guard(expr="is_bool_expr(expr)")`
3. Quasiquote macros use `co.macro.quote` and `co.macro.unquote`

Macro Member Access Rules

The macro declaration itself follows [Package Declaration Access Rules](#). After the macro is accessible, its owned members follow this table:

Annotation	Inside the Macro Ownership Boundary	Outside the Macro Ownership Boundary
<code>@co.dap.public</code>	✓	✓
<code>@co.dap.private</code>	✓	✗

A public macro member does not widen access to an inaccessible macro.

Collections

Built-in generic collection parameters also have a fixed positional order:

```

co.List(Element)
co.Set(Element)
co.Map(Key, Value)
co.Array(Dimensions, Element, Sizes)
co.Matrix(Rows, Columns, Element)

```

For `co.Map`, the first type is always the key type and the second is always the value type. A completed application must be named by a concrete `co.type` alias before values are declared or constructed.

```
// appl.fol
StringList  co.type = co.List(co.string);
IntSet      co.type = co.Set(co.int);
StringIntMap co.type =
    co.Map(co.string, co.int);
IntMatrix2x4 co.type =
    co.Array(2, co.int, [2,4]);
FloatMatrix2x4 co.type =
    co.Matrix(2, 4, co.float);

x StringList = StringList{"A","B","C"};
y IntSet = IntSet{1,2,3};
map StringIntMap = StringIntMap{"A":1, "B":2, "C":3};
arr IntMatrix2x4;
matrix FloatMatrix2x4;

// Variables with inferred declaration types still use explicit named
construction.
y2 := IntSet{1,2,3};
x2 := StringList{"A","B","C"};
map2 := StringIntMap{"A":1, "B":2, "C":3};
```

Every runtime composite value uses the explicit `Type{...}` representation. Lists, arrays, sets, maps, records, and objects differ through the resolved type, not through their delimiters. Maps and object-like values use `:` between key/value or field/value entries. `Type[...]` is indexing and `Type(...)` is invocation; neither is collection construction.

Dynamic Multi Dispatch

FoLang distinguishes **static overload dispatch** from **dynamic multiple dispatch**. Ordinary overload resolution is static by default. `@co.ddap.dynamicdispatch(true)` is an application-wide directive that changes eligible overload-family selection in the application's **open compilation graph** to use runtime argument types.

This feature is distinct from ordinary class/interface virtual dispatch. Virtual dispatch selects a receiver implementation from the runtime receiver type; dynamic multiple dispatch selects among an overload family using the runtime type tuple of all participating arguments. Operator expressions are not reclassified into this named-callable dynamic-dispatch model: operator implementations continue to use the exact normalized operand-type resolution defined in [Operators](#), and

`@co.ddap.dynamicdispatch` does not introduce subtype widening for operator selection.

Dynamic multiple dispatch does not alter FoLang's ordinary overload-family signature contract: eligible instantiated overloads are distinguished only by applicable parameter signatures, return types never participate in selection, and sibling overloads in the family must have the identical declared return signature. Generic mappings have already been resolved at compile time and are not re-evaluated from runtime types. Dynamic selection may change the implementation that executes, but it cannot change the call's compile-time result contract.

Example Type Hierarchy

```
// Animal.fol
_ co.class = {
}
```

```
// Dog.fol
@co.dap.oops(
    classes=[Animal]
)
_ co.class = {
}
```

```
// Cat.fol
@co.dap.oops(
    classes=[Animal]
)
_ co.class = {
}
```

```
// Human.fol
@co.dap.oops(
    classes=[Animal]
```

```
)
_ co.class = {
}
```

```
// collisions.unit.fol
_ co.unit = {
  collide(a Animal, b Animal)->(co.bool) = {
    ...
  }

  collide(a Dog, b Cat)->(co.bool) = {
    ...
  }

  collide(a Dog, b Human)->(co.bool) = {
    ...
  }

  collide(a Human, b Cat)->(co.bool) = {
    ...
  }

  checkOverload()->>() = {
    some1 Animal = Dog{};
    some2 Animal = Cat{};
    some3 Animal = Human{};

    collide(some1, some2);
    collide(some1, some3);
    collide(some3, some2);
  }
}
```

Without dynamic dispatch, the static argument tuple at each of the three calls is `(Animal, Animal)`, so static overload resolution selects `collide(Animal, Animal)` even though the runtime objects may be `Dog`, `Cat`, or `Human`.

FoLang uses the ordinary postfix conversion call `value.to(TargetType)` as its explicit conversion/cast form; it does not introduce a separate C-style cast grammar. When the conversion is valid, the resulting expression presents `TargetType` as its static type to subsequent type checking and overload resolution.

An explicit cast therefore changes the static type presented to ordinary overload resolution:

```
collide(some1.to(Dog), some2);
```

The static tuple is then `(Dog, Animal)`. If there is no exact `collide(Dog, Animal)`, ordinary applicability/widening rules search supertypes and may select `collide(Animal, Animal)` as the unique most-specific applicable overload.

Enabling Dynamic Multiple Dispatch

`@co.ddap.dynamicdispatch(...)` is an **executable-application-only** semantic directive. It may be declared only in the top-level metadata preamble of the application rooted at `src/app1.fo1`, in accordance with [Directive Placement](#). It cannot appear inside any declaration or block. Using it in `src/component.fo1`, in any standalone library, or in any `components/<kind>/component.fo1` is a compiler error. Components are not libraries, but they are equally forbidden from enabling this application-wide semantic mode.

```
// src/app1.fo1
@co.ddap.dynamicdispatch(true)

// imports and application code
```

No standalone library or project-local component may independently enable dynamic multiple dispatch. **Projected** libraries/components retain ordinary static overload bindings internally because their implementation remains hidden behind surface APIs. Packaged producers also compile without independently enabling dynamic dispatch, but their explicitly exported package contexts are different: when those contexts are later merged into an executable application's open graph, eligible packaged call sites inherit that application's dynamic-dispatch mode. This restriction is independent of ordinary class/interface virtual receiver dispatch, which retains its separately defined semantics.

When the executable application enables dynamic multiple dispatch, an eligible call in the application's open semantic graph uses the actual runtime type tuple instead of the static tuple. For example, if the runtime tuple is `(Dog, Cat)`, `collide(Dog, Cat)` is selected when that overload exists.

Applicability and Widening

Dynamic multiple dispatch uses the **same applicability, nominal widening, specificity, and ambiguity rules as ordinary static overload resolution**. The difference is only the source of the selection tuple: static overload resolution uses compile-time argument types, while dynamic multiple dispatch uses actual runtime argument types.

For a runtime tuple $(R_1, R_2, \dots, R_n)$, an overload with parameter tuple $(P_1, P_2, \dots, P_n)$ is **applicable** when every runtime type R_i is the same as or a subtype of P_i according to FoLang's applicable nominal subtype relation.

An exact runtime-type match is therefore the most specific possible match. When no exact overload exists, dispatch widens one or more positions through the type hierarchy and considers applicable parent-type signatures.

Define signature A as more specific than signature B when:

1. every parameter type in A is the same as or a subtype of the corresponding parameter type in B ; and
2. at least one parameter type in A is a proper subtype of the corresponding parameter type in B .

Dynamic dispatch selects the **unique most-specific applicable overload**. As with static overload resolution, return types, arbitrary conversions, declaration order, and left-parameter priority do not participate in candidate selection.

Compile-Time Ambiguity Detection

Consider:

```
f(a Dog,    b Animal)->(...) = { ... }  
f(a Animal, b Cat)->(...)    = { ... }  
f(a Animal, b Animal)->(...) = { ... }
```

For runtime tuple (Dog, Cat) :

```
f(Dog, Animal)  applicable  
f(Animal, Cat)  applicable  
f(Animal, Animal) applicable
```


The first two are each more specific than `f(Animal, Animal)`, but neither is more specific than the other. The tuple is therefore ambiguous.

FoLang does not use left-parameter priority, declaration order, or a runtime guess to resolve this case. When `@co.ddap.dynamicdispatch(true)` is enabled, the frontend constructs the application's closed dynamic-dispatch type universe from the application-owned types plus types in package contexts explicitly exported from packaged components and packaged `.fo1enc` artifacts. Projected-library/component internal types are excluded because those domains remain black boxes.

For each dynamically dispatched call, the frontend uses that prepared open type hierarchy and the applicable overload family to validate that every runtime-capable type combination admitted by the call's static parameter bounds has one unique most-specific overload. Multiple incomparable best candidates are a compiler ambiguity error. If no applicable overload exists for an admitted runtime type combination, compilation likewise fails unless the applicable language rule for that call form defines another explicit fallback.

The frontend is responsible for establishing and validating this semantic type/overload set. It does not prescribe the runtime lookup representation.

Binary Generation

After the frontend has validated the dynamic-dispatch universe and emitted the required semantic information in the Final AST/backend interchange, the backend may implement dispatch with tables, type-ID matrices, decision trees, inline caches, direct-call specializations, or another equivalent representation.

```
statically proven concrete tuple
  -> direct call is permitted

runtime-polymorphic tuple
  -> precomputed dynamic dispatch selection
```

The implementation strategy is not observable language semantics provided the selected overload is the one required by these rules.

Open Application Graph

The application-wide directive applies to code participating directly in the application's open semantic graph:

```
ordinary src/ application packages
+
packages explicitly selected by components/packaged/component.fol
+
explicitly exported packaged/open package contexts imported from
lib/*.folenc
```

Exported packaged code is intentionally not protected by a projected API boundary. Only explicitly exported packaged package contexts join the application graph; unexported packaged packages remain private to their producer/component. The exported contexts' classes, type hierarchy, overload families, and applicable AST/HIR participate in the application graph. Consequently, when the application enables dynamic multiple dispatch, eligible overload calls **inside participating exported packaged code** are also resolved under dynamic-dispatch semantics.

A packaged library authored under assumptions of static overload dispatch can therefore change behavior or become ambiguous when consumed by a dynamic-dispatch application. This is an intentional consequence of choosing packaged/open integration rather than a projected boundary.

A packaged `.folenc` must retain enough canonical type hierarchy, overload-family, call-site, symbol/context, and applicable AST/HIR information for the final consuming application to perform this integration and validation.

Library and Component Dispatch Boundaries

The following rules apply uniformly to standalone libraries and project-local components:

1. no library or component may declare `@co.ddap.dynamicdispatch(...)`; doing so is a compiler error;
2. projected-library/component implementation call sites retain static overload bindings; packaged producer source cannot independently enable dynamic dispatch, but explicitly exported packaged call sites may be re-resolved under

the consuming executable application's dynamic-dispatch mode after open-graph integration;

3. only the executable application may enable dynamic multiple dispatch;
4. an application's dynamic-dispatch mode does not penetrate projected boundaries; projected `application`, `native`, and `dynamicvmrt` implementations remain black boxes whose internal overload call sites retain their statically compiled bindings;
5. packaged producers are the exposure-model exception only for package contexts explicitly selected by `@co.dap.export(...)`: those exported contexts join the consuming application's open semantic graph and therefore follow the consuming application's dynamic-dispatch mode, while the packaged producer itself still cannot enable dynamic dispatch;
6. `components/operators/` is parser/operator infrastructure rather than a library and does not create a dynamic-dispatch domain; and
7. the same isolation/exposure rules apply whether the producer is standalone or project-local.

In particular, `dynamicvmrt` remains an isolated projected domain. Runtime-created or runtime-loaded types may inherit compiled types that are legitimately visible **inside that `dynamicvmrt` domain**, but those private runtime types do not become new members of the consuming application's compiled dynamic-multidispatch type universe and do not extend application overload families implicitly. Internal `dynamicvmrt` function overload binding remains static; the runtime executes the target established by those static overload rules, apart from any separately defined ordinary OOP virtual receiver dispatch.

Projected standalone libraries and projected project-local components (`application`, `native`, `dynamicvmrt`) therefore remain isolated behind their surface APIs. Their private packages, private types, internal overload families, and internal call sites are not reinterpreted by the owning/consuming application's `@co.ddap.dynamicdispatch(true)`.

Execution Models and Control Abstractions

FoLang executes ordinary code **sequentially by default**. A normal function or method declaration therefore requires no execution-model decorator merely to be called sequentially.

The built-in decorator `@co.dap.executionmodel(...)` is used only when a declaration requires non-default execution semantics that must remain observable across conforming FoLang implementations. The language exposes an execution-model choice only when that choice changes required FoLang behaviour. A distinction that changes only a backend's internal implementation strategy is not a separate FoLang execution model.

This gives the following separation:

```
source-level execution semantics
  -> @co.dap.executionmodel(...)

concurrent / parallel / asynchronous submission and communication
  -> co.cPCA

continuation / CPS control operations
  -> co.control

backend/runtime implementation
  -> threads, pools, virtual/green threads, event loops, processes,
      work-stealing schedulers, or another conforming mechanism
```

A backend may choose any internal mechanism that preserves the semantics required by the declaration. For example, a logical FoLang task may be implemented using an operating-system thread, a thread pool, a virtual thread, a fiber-like runtime mechanism, an event loop, or another conforming scheduler. Such implementation choices do not become FoLang source-level execution kinds merely because a backend uses them.

Execution-model declarations use ordinary function/method declaration syntax and expose a callable signature, but they are classified as `ExecutionModelFunctionDecl` under [Function-Shaped Declaration Classification](#), rather than as ordinary `FunctionDecl` nodes. The specialized declaration preserves the execution semantics explicitly for semantic analysis and backend interchange without changing its callable function/method interface at the language level.

Effect Boundary

Every `@co.dap.executionmodel` declaration is a recoverable-effect boundary. Effects may arise from calls made while its implementation executes, and that implementation may locally use ordinary `@co.dap.onEffect` policies. However, an unhandled recoverable effect cannot unwind or propagate into the submitting caller. The runtime captures the first observed effect object and completes the execution-model invocation through its ordinary `co.error`-compatible result.

Consequently, every execution-model declaration must contain exactly one result position compatible with `co.error`. It does not advertise outward effects with `@co.dap.effects`, because its public invocation has no outward effect channel. Known internal effects may still be retained as implementation metadata for diagnostics and backend lowering.

At the external invocation, including a canonical submission call statically bound to the execution-model target, `@co.dap.onEffect` remains legal for registering ordered handlers. Each matching effect record must contain a non-empty `handlers=[...]` list and omit `resolution`; the language supplies the fixed `return_with_error` resolution. Any explicitly written resolution is a compile-time error. A call inside the execution-model implementation is an ordinary call unless it independently crosses another execution-model boundary, so its own local `@co.dap.onEffect` record follows the applicable ordinary or boundary-specific rule.

Default Sequential Execution

Sequential execution is the default. FoLang therefore does **not** define `@co.dap.executionmodel(type=sequential)` as a standard execution-model form; an ordinary undecorated function or method already has sequential semantics.

```
calculate(a co.int)->(co.int) = {  
    this => a + 1;  
}  
  
value := calculate(10);
```

The call proceeds as an ordinary FoLang function call according to the normal call, evaluation-order, error, and return-value rules.

Concurrent Execution

`concurrent` describes work whose execution may overlap with other work. The standard source-level kinds are deliberately small:

- `task` — a logical schedulable unit of work. This is the preferred general concurrent abstraction and leaves the backend substantial scheduling freedom.
- `thread` — requests thread-semantic execution when thread identity, affinity, thread-local behaviour, or another specification-defined thread property is intentionally observable.

Backend-specific names such as `goroutine` or `greenlet` are not FoLang execution kinds. `subroutine` is ordinary callable code, while `generator` and `coroutine` primarily describe suspension/resumption behaviour rather than an independent concurrent execution carrier. `fiber` likewise is not exposed merely because a backend may use a fiber-like mechanism internally. Such abstractions may exist as ordinary package/runtime facilities when separately defined, but they are not synonyms for `concurrent.kind`.

```
SubmitParams  co.type = co.List(co.any);
SubmitResults co.type = co.List(co.any);

// concurrent.unit.fol
_ co.unit = {
  @co.dap.executionmodel(type=concurrent, kind=task)
  someConcurrent(a co.int)->(co.int,co.error) = {
    ...
  }
}

co.cpcp.submit(someConcurrent, params=SubmitParams{10},
results=SubmitResults{val, errors});
(errors.isEmpty).then(
  co.out.println(val)
).default(
  co.out.println(errors)
);
```

A declaration may request thread semantics explicitly when those semantics are required:

```

_ co.unit = {
    @co.dap.executionmodel(type=concurrent, kind=thread)
    threadBoundWork(a co.int)->(co.int,co.error) = {
        ...
    }
}

co.cpcp.submit(threadBoundWork, params=SubmitParams{10},
results=SubmitResults{val, errors});

```

Scheduling is a separate dimension from execution kind. Where supported by the selected kind, an optional scheduling policy may be declared:

```

@co.dap.executionmodel(
    type=concurrent,
    kind=task,
    scheduling=cooperative
)

```

The standard scheduling values are `cooperative` and `preemptive`. When the field is omitted, the backend/runtime may choose any conforming scheduling policy. A structural policy such as `fork-join`, when requested, is likewise separate from both the execution kind and the scheduling policy; omitting it provides no fork-join guarantee.

Parallel Execution

`parallel` states that the declaration has parallel-execution semantics. FoLang does not currently require a `parallel.kind` merely to expose an implementation mechanism:

```

@co.dap.executionmodel(type=parallel)
parallelWork(data SomeData)->(Result, co.error) = {
    ...
}

```

`process`, `fork`, `spawn`, and `exec` are not parallel execution kinds. A process is an execution/isolation container, while `fork`, `spawn`, and `exec` are process/runtime operations. They remain ordinary facilities of the applicable `co.cpcp`, `co.os`, or system API rather than values of `@co.dap.executionmodel(type=parallel, kind=...)`.

A conforming backend may implement parallel semantics with threads, worker pools, processes, accelerators, or another mechanism when the program cannot observe a forbidden difference.

Asynchronous Execution

`async` describes execution whose completion is not required to block the submitting context. Completion representation is separate from the execution model:

```
@co.dap.executionmodel(type=async, completion=future)
loadData(request Request)->(Data, co.error) = {
    ...
}
```

The standard completion forms are:

- `future` — the caller receives or is associated with a future completion value;
- `callback` — completion is delivered through a compatible callback mechanism.

`promise` is not a separate execution-model value. If a producer-side completion object is required by a particular runtime/API, it is an implementation or package abstraction rather than another `async` execution kind. `await` is a consumption or control operation supplied by `co.cPCA`; it is not an execution-model kind.

An actor is a concurrency abstraction with state/message semantics, not an `async` completion kind. Channels are communication abstractions, events are notification abstractions, and distributed scaling is a deployment/runtime policy. These may be provided through `co.cPCA`, but none is a top-level `@co.dap.executionmodel` type merely because it participates in concurrent or asynchronous programs.

Continuations and Control

Continuation semantics are deliberately separated from concurrent, parallel, and asynchronous execution facilities. A declaration may use:

- `kind=full` — full/undelimited continuation semantics;
- `kind=delimited` — continuation capture is bounded by a delimiter.

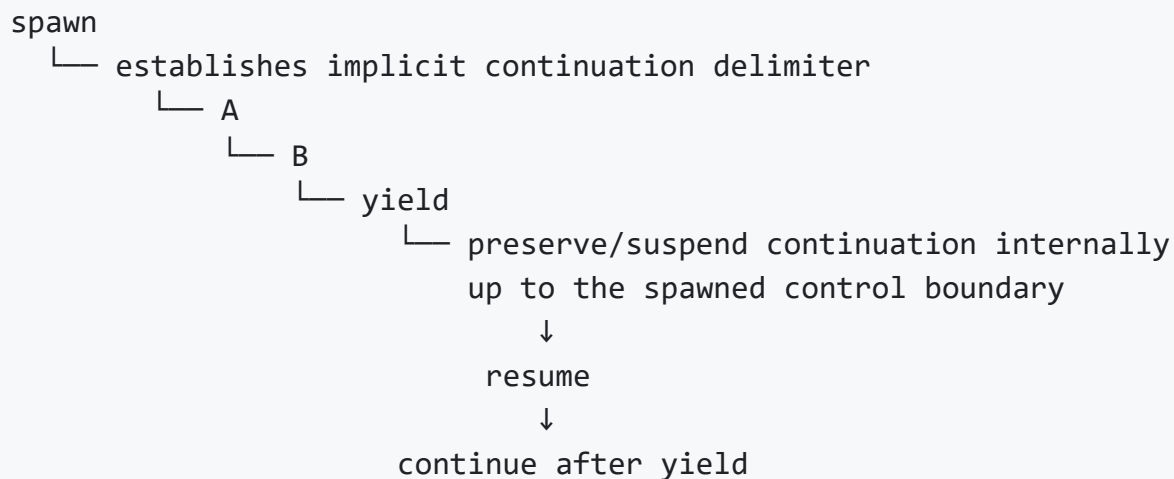
Continuation and CPS operations are supplied by `co.control`, not `co.cPCA`.


```
@co.dap.executionmodel(type=continuation, kind=delimited, control="shift-
reset")
continuationWork(...)->(..., co.error) = {
    ...
}
```

For delimited continuations the currently documented control families are `shift-reset`, `prompt-control`, and `spawn-yield`. `csp` is not a continuation kind; Communicating Sequential Processes belong to the concurrency/communication domain.

`spawn-yield` has a specific continuation meaning in FoLang and must not be confused with an ordinary process/task spawn operation or with a scheduler-only yield. In the `spawn-yield` control family, `spawn` establishes an **implicit delimited-control boundary**. When execution reaches `yield`, the continuation from that suspension point back to the spawned control boundary is preserved internally by the control runtime. The suspended execution can later resume from the yield point.

Conceptually:



Unlike `shift-reset` and `prompt-control`, `spawn-yield` does not require the captured continuation to be exposed directly to application code as an explicit continuation value such as `k`. The continuation may remain an internal control-runtime object. The observable contract is suspension at `yield`, preservation of the delimited execution state, and later resumption from that suspension point according to the applicable `co.control` operation.

This distinction is important:

ordinary `co.cpc` / process spawn

-> creates or submits concurrent execution

ordinary scheduler yield

-> voluntarily gives execution opportunity to another schedulable unit

`co.control` spawn-yield

-> establishes an implicit delimited continuation boundary and suspends/resumes execution by preserving the continuation internally

Accordingly, the word `spawn` may occur in more than one conceptual domain, but the package and operation determine its semantics. Concurrent/process creation remains under the applicable `co.cpc`, `co.os`, or system API. Continuation-oriented `spawn-yield` operations belong to `co.control`.

Continuation kind	Control family	Delimiter model	Continuation exposure	Meaning
<code>full</code>	supplied by the applicable <code>co.control</code> full-continuation API	no delimited boundary	according to the full-continuation API	captures the continuation according to the full-continuation contract
<code>delimited</code>	<code>shift-reset</code>	explicit nearest/applicable <code>reset</code> boundary	exposed according to Shift/Reset semantics	captures only the continuation bounded by <code>reset</code>
<code>delimited</code>	<code>prompt-control</code>	explicit/applicable prompt boundary	exposed according to Prompt/Control semantics	captures only the continuation bounded by the applicable prompt
<code>delimited</code>	<code>spawn-yield</code>	implicit spawned control boundary	internal by default	suspends at <code>yield</code> , preserves the bounded continuation internally, and later resumes from the yield point

Parameter table for delimited continuations

Operator Pair	Delimiter Targeting	Captures Delimiter into \$k\$?	Keeps Delimiter on Stack During \$expr\$?	Primary Modern Use Case / Paradigm
Shift / Reset	Nameless (Closest)	Yes (Implicitly)	Yes	Standard algebraic effects, functional stream processing
Prompt / Control	Nameless (Closest)	No	Yes	Dynamic context modification, legacy Scheme systems
Spawn / Yield	Implicit (Fiber Bound)	Internal	No	Cooperative multitasking, async/await engines, generators

Example declaration:

```
@co.dap.executionmodel(  
  type=continuation,  
  kind=delimited,  
  control="spawn-yield"  
)  
resumableWork(...)->(..., co.error) = {  
  ...  
}
```

The decorator identifies the required continuation family; the actual suspension, yield, resume, and continuation-control operations are supplied by `co.control1`.

Canonical Submission and Placement

`co.cpa.submit(...)` is the canonical submission operation for declarations whose execution is governed by `co.cpa`. The execution-model decorator states the required semantics; the submission call requests execution of that declaration.

A specialized operation such as `submitToPool`, `submitThread`, or `submitToEventLoop` is justified only when the standard `co.cpc` API explicitly defines an additional observable placement/resource constraint. Such a function must not exist merely as another spelling of `submit(...)`. If no additional semantic guarantee is defined, `submit(...)` is used and the backend/runtime is free to select the underlying mechanism.

Application Pool Resource Pragmas

Application pragmas may configure execution-resource limits without changing the execution semantics of the submitted declaration:

```
@co.pdap.threadpool(min=10,max=40,increment=10,queue=1000)
@co.pdap.schedularpool(min=2,max=10,increment=2,queue=5000)
```

Both runtime resource domains exist even when the corresponding pragma is omitted. `@co.pdap.threadpool` configures the application-wide resource domain used for ordinary concurrent/thread-oriented submissions.

`@co.pdap.schedularpool` configures a separate application-wide resource domain used by `co.cpc.schedule(...)` and the applicable delayed, periodic, calendar-based, cron-like, or otherwise scheduled execution facilities. A scheduled submission is therefore serviced by the schedular resource domain rather than consuming the ordinary thread-pool resource domain.

The pragmas allocate and bound resources; they do not themselves define when a particular operation runs. The timing, delay, recurrence, calendar, or cron semantics of scheduled work are supplied by the applicable `co.cpc` scheduling operation.

The fields have the following resource meaning:

- `min` is the runtime's configured minimum worker capacity for that resource domain;
- `max` is a **hard upper bound** on workers/resources in that domain;
- `increment` is the amount by which the runtime may grow worker capacity while still remaining at or below `max`;
- `queue` is a **hard upper bound** on pending work accepted by that resource domain.

For the scheduler resource domain, pending work includes accepted scheduled work that has not yet completed, including delayed work awaiting its trigger and runnable work waiting for execution. A recurring schedule should be represented as a recurring registration rather than by eagerly materializing every future occurrence merely to populate the pending-work queue.

When a pragma is present, neither `max` nor `queue` may be exceeded. If the permitted worker capacity is fully occupied but the queue still has capacity, the submission is accepted and queued. If worker capacity has reached `max` and the pending-work queue has reached `queue`, FoLang rejects the new submission immediately.

A saturated pool must not:

- create workers or equivalent runtime resources beyond `max`;
- enlarge or bypass the configured `queue` bound;
- block the submitting execution context waiting for queue capacity;
- silently redirect the work to an unbounded fallback pool.

Instead, the applicable `co.cpc` submission or scheduling operation reports a **pool-capacity rejection notification/error** through its defined result/error contract. The application developer is responsible for handling that rejection, for example by reporting overload, retrying according to application policy, dropping non-critical work, degrading functionality, or taking another application-specific action. FoLang enforces the resource boundary; it does not choose the business-level overload response.

Pool rejection is an operational capacity signal. Persistent or repeated rejection normally indicates that the configured capacity, workload characteristics, downstream latency, retry behaviour, or application work-generation rate should be investigated. Applications are expected to establish appropriate `max` and `queue` limits through capacity planning, workload analysis, load testing, and production observation rather than relying on unlimited growth.

These pragmas follow the category-wide [Pragma Placement](#) rule: they may appear only in the executable application's `src/app1.fo1`. They are invalid in application package source, every project-local component, every standalone library form, and

every packaged/exported context. A component or library may document resource assumptions or recommendations, but it cannot force application pool limits on the executable that consumes it.

When either pragma is absent, the corresponding FoLang runtime pool still exists and operates using runtime-managed defaults. In that case FoLang defines no application hard upper bound for that pool's worker growth or pending-work capacity. This default preserves ease of use but can permit resource exhaustion under excessive or erroneous workloads. Production applications that require bounded execution resources should therefore configure explicit limits.

These application-level limits reduce the risk that one execution category exhausts resources needed by another part of the application, but they do not replace host, process, container, cgroup, service-manager, or operating-system resource controls used to isolate the machine from a misbehaving application.

Execution-Model Summary

Type	Source-level kind/model	Optional semantic dimension	Package responsible for operations
default sequential	—	—	ordinary function/method call
<code>concurrent</code>	<code>task</code> , <code>thread</code>	<code>scheduling=cooperative preemptive</code> ; structural policies such as <code>fork-join</code> when separately defined	<code>co.cPCA</code>
<code>parallel</code>	—	parallel-specific policy only when separately specified	<code>co.cPCA</code>
<code>async</code>	—	<code>completion=future callback</code>	<code>co.cPCA</code>
<code>continuation</code>	<code>full</code> , <code>delimited</code>	delimited <code>control="shift-reset" "prompt-control" "spawn-yield"</code>	<code>co.control</code>

The following are intentionally **not** execution-model types or kinds:

backend/runtime mechanisms:

goroutine, greenlet, virtual thread, event-loop implementation,
work-stealing implementation

ordinary process/runtime operations:

process, fork, spawn, exec

completion/control operations:

await

communication/notification abstractions:

channel, event

higher-level concurrency abstractions:

actor, CSP

control/suspension abstractions unless separately specified:

coroutine, generator, fiber, yield/resume

deployment/runtime policy:

distributed scaling, scale up/down/out/in

The governing rule is:

Expose an execution-model choice in `@co.dap.executionmodel` only when the choice changes externally observable FoLang semantics. If two choices differ only in how a backend implements the same required behaviour, they are not separate FoLang execution-model choices.

Native Code and Foreign Interop (native capability)

`@co.dap.native` marks a function or method declaration as a **native implementation declaration**. It does not grant native capability merely because the annotation is present. The annotation is valid only inside a `native` library/component domain when the installation permits the native capability.

A native library/component may use the `co.native` package to express low-level implementation such as assembly or machine-level operations through facilities including `co.native.asm` and `co.native.inline`. The same native domain also owns foreign-function interoperability: extern declarations, foreign symbols, C/native ABI-compatible types, calling conventions, symbol linkage, pointer/address forms, and permitted FoLang-side marshalling or invocation code.

These facilities intentionally share one capability boundary. A foreign call may produce an ABI value, address, or pointer that native memory, platform, assembly, or runtime code consumes directly, and native code may prepare values for a foreign call without routing them through a second projected API. `ffi` therefore names an interoperability feature area/API family where useful; it is **not** a separate FoLang capability, library, or component kind.

The frontend preserves native and foreign-interoperability declarations and their metadata through the backend interchange contract. The reference backend demonstrates one implementation of that contract. A conforming third-party backend is not required to reproduce the reference backend's internal native-code lowering, ABI lowering, instruction representation, allocation strategy, marshalling implementation, or code-generation mechanism, but the externally observable behavior required by the specification must be preserved.

Native Functions

```
// native.unit.fol
```

```
_ co.unit = {  
  @co.dap.native  
  nativeMethod(a co.int, b co.int)->(co.int) = {  
    // native implementation  
  }  
}
```

Dynamic Runtime (dynamicvmrt capability)

The `@co.ddap.dynamicruntime` directive enables full access to the `co.meta` package. It is valid **only for source files in a `dynamicvmrt` capability domain**: a standalone `@co.dap.library(type=dynamicvmrt)` project or the project-local

`components/dynamicvmrt/` component. In every permitted source file it must obey the category-wide [Directive Placement](#) rule and appear at file top level, never inside the `_ co.component` declaration or any nested declaration/body. Using `@co.ddap.dynamicruntime` in an executable application, packaged code, an application projected library/component, a `native` domain, or any other source context is a compiler error.

Within a valid `dynamicvmrt` domain, the directive enables dynamic class and type loading, monkey patching, runtime reflection, instrumentation, eval-based code execution, and other defined dynamic-runtime/metaprogramming capabilities through `co.meta`. These capabilities remain inside that projected dynamic-runtime boundary and do not automatically escape into ordinary application, packaged, or other library/component code.

1. runtime type creation from strings, streams, files, and other supported dynamic-runtime inputs
2. complete reflection and introspection
3. Runtime code modification add/remove/update methods etc.

When a library is marked `dynamicvmrt` and enables `@co.ddap.dynamicruntime`, the final binary includes the runtime support required to create and manage dynamic types, methods, and objects.

Dynamically created objects may interact with compiled types according to the dynamic-runtime boundary rules; ordinary compiled code does not gain unrestricted reverse access to dynamic-runtime facilities.

Through surface API's they connect to runtime and through runtime apis invoke the dynamic type and the results will be returned are compiled types only.

Runtime can directly provide handle to compiled types to dynamic code running inside it.

Loaders are the dynamic-runtime containers used to manage these runtime-

created objects and types.

FoLang provides `BasicLoader`. A user-defined loader extends `co.meta.BaseLoader` using the ordinary loader declaration rules, for example:

```
//MySpecLoader.fol
```

```
@co.dap.extends(co.meta.BaseLoader)
_ co.loader={

}
```

The basic loader provides the operations required to create, update, delete, and manage runtime-created objects.

Loaders form a hierarchy. When a referenced runtime type is not found in the current loader realm, lookup proceeds through the base-loader chain and finally to the compiled-type environment.

Variable Kinds Support

Kind	Where
Normal	All
Dynamic (<code>::=</code>)	contexts that permit an initialized variable declaration; forbidden where a fixed static/ABI type is required
Pointers	<code>native</code> capability domain only
Arrays	All
References Heap, Lvalue, Rvalue	<code>native</code> capability domain only
Addresses	<code>native</code> capability domain only
Thunks	application-domain code and any other context whose capability rules explicitly permit thunks

Ranges	All
Slices	All

Builtin Data Types

Type	Kind
<code>co.string</code>	
<code>co.int</code>	
<code>co.bit</code>	
<code>co.double</code>	
<code>co.float</code>	
<code>co.long</code>	
<code>co.byte</code>	
<code>co.char</code>	
<code>co.any</code>	
<code>co.bool</code>	
<code>co.void</code>	
<code>co.value</code>	value types stores values when take snapshot
<code>co.typed</code>	
<code>co.untyped</code>	
<code>co.word</code>	
<code>co.MatchBindings</code>	
<code>co.tag</code>	
<code>co.hokrlt</code>	
<code>co.number</code>	
<code>co.uninit</code>	
<code>co.error</code>	standard recoverable-error interface and first-error result contract; every emitted effect object is a class instance satisfying this interface

<code>co.AbstractError</code>	standard mixin supplying common <code>co.error</code> state and behavior; custom recoverable-error classes normally compose it while declaring the <code>co.error</code> interface
<code>co.literal</code>	literal representation for simple and compound literal objects
<code>co.operator</code>	declaration kind valid only in the <code>components/operators/component.fol</code> component context; parsed by the common FoLang parser and invalid in all other source contexts
<code>co.variants</code>	Built-in variadic type used to define a closed variant-based type. Its arguments declare states owned by the enclosing <code>co.type</code> ; parameterized entries are state functions and bare entries are zero-parameter state values.
<code>co.newtype</code>	
<code>co.opaquetype</code>	
<code>co.subtype</code>	
<code>co.supertype</code>	
<code>co.dependentType</code>	result kind of a value-indexed function returning a dependent type object
<code>co.refinementType</code>	base type restricted by a Boolean predicate over the candidate value
<code>co.associatedType</code>	type parameter associated with another generic or parameterized signature component; a matching module supplies its concrete <code>co.associatedType</code> binding
<code>co.predicateType</code>	works on types unlike refinement type like type constraints
<code>co.data</code>	
<code>co.type</code>	
<code>co.shape</code>	type expressions on right side can be shapes (A)->(B) or function type expression
<code>co.delegate</code>	

A name appearing in this registry is not necessarily an enabled source-language feature. A built-in kind is usable only when this specification defines its declaration syntax and semantics. An undocumented or explicitly reserved kind remains unavailable and must produce an unsupported-feature diagnostic when used.

Built-in Directives

The entries in this language-defined inventory form the current built-in metadata registry used for `@co.*` name recognition. The parser must recognize a language-owned metadata name through this predefined registry before accepting the metadata application. Field/argument preservation and partial frontend field validation follow [Built-in Metadata Parsing](#). Every entry classified as `DIRECTIVE` follows the category-wide [Directive Placement](#) rule and is file-level only. Every entry classified as `PRAGMA` additionally follows the category-wide [Pragma Placement](#) rule and is valid only in an executable application's `src/app1.fo1`.

Kind		
PRAGMA	"@co.pdap.threadpool", "@co.pdap.schedularpool"	
DIRECTIVE	"@co.ddap.import", "@co.ddap.dynamicruntime", "@co.ddap.use", "@co.ddap.alias", "@co.ddap.dyn amicdispatch", "@co.ddap.o verload"	@co.ddap.overload is different from @co.dap.overload it has takes whether paramtypes or paramandreturntypes as attributevalue of strategy
ANNOTATION	"@co.dap.extend", "@co.dap.t emplate", "@co.dap.macro", "@co.dap.o perator", "@co.dap.annotation", "@co.dap.library", "@co.dap.native", "@co.dap.class", "@co.dap.static", "@co.dap.obj ect", "@co.dap.inline", "@co.dap.ctf e", "@co.dap.friend",	//mop => meta object programming

	"@co.dap.sealed", "@co.dap.extension","@co.dap.override","@co.dap.implement", "@co.dap.virtual", "@co.dap.abstract", "@co.dap.delegate", "@co.dap.dynamicscope","@co.dap.mixedscope", "@co.dap.typeclass","@co.dap.matcher", "@co.dap.constructor", "@co.dap.oops","@co.dap.extends","@co.dap.hokrlt", "@co.dap.indexer", "@co.dap.generic", "@co.dap.comptime", "@co.dap.typefromvalue", "@co.dap.local", "@co.dap.private","@co.dap.public","@co.dap.compose", "@co.dap.guard","@co.dap.package","@co.dap.protected","@co.dap.internal","@co.dap.export","@co.dap.eager", "@co.dap.lazy", "@co.dap.packed", "@co.dap.declare","@co.dap.implementation","@co.dap.simd", "@co.dap.reflection", "@co.dap.mop","@co.dap.nested","@co.dap.inner","@co.dap.final","@co.dap.const","@co.dap.decorator","@co.dap.specialize","@co.dap.symbol"	
DECORATOR	"@co.dap.before", "@co.dap.after","@co.dap.around", "@co.dap.effects", "@co.dap.onEffect", "@co.dap.defer","@co.dap.callable", "@co.dap.executionmodel"	

Builtin Kinds

Kind	Purpose
<code>co.struct</code>	
<code>co.cstruct</code>	
<code>co.class</code>	struct-like ordinary mutable per-instance storage plus inheritance, abstraction, polymorphism, encapsulation, trait/mixin composition, behaviour extension/modification, and dynamic dispatch; field-level constant/immutable/shared/COW/locking policies are forbidden
<code>co.interface</code>	all abstract methods
<code>co.union</code>	
<code>co.object</code>	one named singleton used for annotation implementation or as an explicit non-owning support object associated through <code>for=</code> with one or more classes; may own constants, immutable bindings, globals, shared state, and locks
<code>co.instance</code>	
<code>co.matcher</code>	
<code>co.loader</code>	
<code>co.trait</code>	interfaces with default implementations
<code>co.mixin</code>	abstract classes alias
<code>co.extension</code>	reusable implemented functions that can be composed with supported classes without inheritance
<code>co.typeclass</code>	
<code>co.module</code>	
<code>co.unit</code>	stateless file-level container; ordinary units merge into the package namespace and <code>*.comp.unit.fold</code> attaches to a struct
<code>co.block</code>	

<code>co.signature</code>	
<code>co.function</code>	
<code>co.enum</code>	Closed tagged ADT. Its members are enum states; parameterized states are compiler-provided state functions returning the enclosing enum type. State-function payload parameters and calls are always named (<code>name=value</code>) and never positional; zero-parameter states are referenced directly without <code>()</code> .
<code>co.symbol</code>	Used by AST
<code>co.expression</code>	Used by AST
<code>co.statement</code>	Used by AST
<code>co.component</code>	structural surface/container valid only in <code>src/component.fol</code> and standardized <code>components/<kind>/component.fol</code> ; source context determines projected, packaged, or operator semantics

Builtin Collections

Name	Purpose
<code>co.List</code>	
<code>co.Set</code>	
<code>co.Map</code>	
<code>co.Tree</code>	
<code>co.Trie</code>	
<code>co.Array</code>	
<code>co.Tuple</code>	
<code>co.Comparable</code>	
<code>co.Stack</code>	
<code>co.Queue</code>	
<code>co.StructObject</code>	

co.ClassObject	
co.ModuleObject	
co.InstanceObject	
co.ObjectObject	
co.Matrix	

Builtin Operators

Arithmetic operators

`+`, `-`, `*`, `/`, `%`, `**`

Logical operators

`&&`, `||`, `!`

Bitwise operators

`&`, `|`, `^`

Comparison operators

`==`, `!=`, `<`, `>`, `<=`, `>=`

Standard operator examples

```

left  co.int = 10;
right co.int = 3;

notEqual co.bool = left != right;  // true

bitsAnd co.int = 6 & 3;             // 2
bitsOr  co.int = 6 | 3;             // 7
bitsXor co.int = 6 ^ 3;             // 5

mulAssign co.int = 6;
mulAssign *= 3;                     // 18

divAssign co.int = 18;
divAssign /= 3;                     // 6

```

```

modAssign co.int = 17;
modAssign %= 5;                                // 2

powAssign co.int = 2;
powAssign **= 3;                                // 8

andAssign co.int = 6;
andAssign &= 3;                                // 2

xorAssign co.int = 6;
xorAssign ^= 3;                                // 5

orAssign co.int = 6;
orAssign |= 3;                                // 7

```

For a compound assignment `lhs op= rhs`, FoLang resolves the corresponding binary operator `op`, evaluates the left-hand location only once, and stores the resulting value back through that same location. The ordinary target-type conversion rules apply to the stored result.

Other operator and language-token spellings

@, #, !, ~, \$, ^, (,), _, ` , ? , { , [,] , } , \ , : , ; , " , ' , = , . , :: , ? ,
 = , := , ::= , , , .. , ... , <.. , ..< , <..
 <..< , => , => , -> , <- , -> , <-> , @@ , += ,
 -= , *= , /= , %= , **= , &= , ^= , |= , <: , :> , ^=> , ->|

Contiguous symbolic spellings that are absent from this inventory and from the active custom-operator table are unrecognized symbolic tokens; the lexer does not split them into shorter operators.

This inventory includes punctuation and reserved token spellings. Presence in this list does not make a spelling declarable as a `co.operator`, nor usable with `mode=overload`, `mode=implements`, `mode=extends`, `mode=inherits` or `mode=override`.

Pre-Declared Operator Glyphs

U, n

Every glyph in this list is language-owned and already has fixed parse properties. It cannot be redeclared with `co.operator`, but it can receive implementations through `mode=overload` in a class, struct companion unit, or applicable extension

contribution to an existing type. Until a matching implementation is visible, use of the glyph fails during resolution.

An operator contributed to an existing type may combine `@co.dap.operator` with `@co.dap.extension`. The operator declaration itself does not carry `@co.dap.generic`.

For a generic existing type such as `co.Set`, the extension identifies the canonical owner declaration rather than introducing an operator-level type parameter. The abbreviated form below intentionally omits the ordinary callable signature so that the ownership metadata is clear; the ellipsis is explanatory notation, not FoLang source syntax:

```
@co.dap.operator(symbol='U', mode=overload)
@co.dap.extension(fortype=co.Set, what=extends)
union(...)

@co.dap.operator(symbol='n', mode=overload)
@co.dap.extension(fortype=co.Set, what=extends)
intersection(...)
```

In this operator-extension form, `fortype=co.Set` identifies the `Set` declaration itself. A spelling such as `fortype=co.Set(T)` does not introduce `T` for the operator, and `@co.dap.generic` must not be added to the operator declaration.

Uses of the pre-declared glyphs remain ordinary operator expressions once matching implementations are visible:

```
IntSet co.type = co.Set(co.int);
v := IntSet{1, 2, 3};
p := IntSet{4, 5, 2};
w := IntSet{7, 8};

u := v U p;
i := v n p;
combined := v n p U w; // same precedence and left associativity: (v n p)
                        U w

z := p + v U w;        // parses as p + (v U w)
x := p * v U w;        // parses as (p * v) U w
```

The generic arguments of the owning type do not create generic operator candidates:

```
co.Set(co.int)
  -> canonical operator owner: co.Set

co.Set(co.string)
  -> canonical operator owner: co.Set

operator lookup
  -> locate operator family owned by co.Set
  -> no operator-generic inference
  -> no operator-generic instantiation
```

This rule concerns operator **ownership**. It does not make arbitrary non-owner operand types interchangeable, does not change assignment or subtype compatibility, and does not enable numeric promotion or `to` / `from` conversion during operator candidate selection.

The following combinations summarize the metadata rule:

```
@co.dap.operator
  -> valid operator declaration

@co.dap.operator + @co.dap.extension
  -> valid operator contributed to an existing type

@co.dap.operator + @co.dap.generic
  -> compile-time error

@co.dap.operator + @co.dap.extension + @co.dap.generic
  -> compile-time error
```

For a user-defined owner such as a class or a struct with a companion unit, `@co.dap.extension` is not required merely to define that owner's own operators; the operator can be declared directly in the legal owner location. The extension annotation is used when contributing an operator to an existing type through the extension mechanism.

See [Pre-Declared Operator Glyphs](#).

Reserved words

`co`, `let`, `this`, `for`, and `fϕλ` are hard-reserved words. `forall` is a contextual keyword.

`forall` has its language-defined meaning only when it begins the polymorphic type-expression form `forall(...).<type-body>` as the value of a `co.type` declaration; outside that contextual position it is an ordinary identifier.

`fϕλ` (`f` = U+0066, `ϕ` = U+03A6, `λ` = U+03BB) is the permanently reserved language mark and the compiler-owned **private standard-package root**. Although ordinary identifiers are ASCII-only, the lexer recognizes this exact case-sensitive code-point sequence as one indivisible hard-reserved token before ordinary identifier recognition. Ordinary project, component, and third-party library source cannot use `fϕλ` as a variable, declaration, package segment, field, parameter, import target, alias, or other user-defined name. The only source context in which `fϕλ.<package>` is admitted is the privileged standard-package bootstrap compilation that builds the installed `co.fo1enc`; there it identifies canonical internal package contexts before export projection. Visually similar Unicode sequences are not equivalent. The former spelling `fo` is not reserved.

`this` Receiver

Outside a symbolic `this` control production, `this` is an ordinary receiver expression. It denotes the enclosing receiver made available by the callable context; the callable category fixes the receiver kind:

Callable context	Meaning of ordinary <code>this</code> receiver expression
Instance method	current class instance
Class method, including <code>@@new</code>	current class/type
Instance lifecycle method, including <code>@@init</code>	current class instance
Target-bound extension instance method	instance of <code>fortype</code>
Target-bound extension class method	class/type identified by <code>fortype</code>
Singleton-object function	current singleton object
Static method	unavailable; use the explicit class/type name
Free, module, or unit function	unavailable as a receiver

Receiver expressions and symbolic `this` control forms are distinct grammar categories. In `this.member`, relationship selectors such as `this->parent`, and an otherwise valid bare `this` expression, `this` is evaluated as the ordinary receiver described above. In `this =>`, `this ^=>`, `this ->>`, and `this ->|`, the leading hard-reserved `this` token is instead part of a complete executable-control production and is not evaluated as that receiver.

Consequently, a symbolic control form can be valid where an ordinary receiver expression is unavailable. For example, `this => value;` is valid in a free function even though `this.member` is invalid there. Conversely, using a symbolic control form does not create or imply an instance, class, type, or object receiver.

In a class method, relationship selectors such as `this->parent` and `this->classes[Type]` denote type contexts. In an instance method, the same selectors denote views or branches of the current instance.

Symbolic `this` Control Forms

FoLang defines four dedicated symbolic control productions headed by the hard-reserved `this` token:

```
this => value;           // produce callable result and exit the current
callable
this ^=> value;          // from an anonymous argument block, exit its
enclosing callable
this ->>;                // advance to the next iteration of the nearest
applicable loop
this ->|;                // exit the nearest applicable structured control
region
```

In these productions, `this` identifies the current executable context selected by the complete form rather than the ordinary receiver expression:

Control form	Selected executable context	Effect
<code>this => values;</code>	current function or method invocation	produce the declared result values and terminate that invocation
<code>this ^=> values;</code>	function or method lexically enclosing the current	terminate the block and enclosing invocation,

	anonymous call-argument block	producing the invocation's declared results
<code>this ->>;</code>	nearest applicable enclosing loop execution	advance to its next iteration
<code>this -> ;</code>	nearest applicable enclosing structured-control execution	terminate that control execution

An anonymous call-argument block introduces no callable result target, but its boundary must be crossed explicitly. `this ^=>` is the only outward callable return form admitted there; ordinary `this =>` cannot cross that boundary. The target is the nearest **lexically enclosing** function or method, not an implementation frame belonging to `then`, `default`, `loop`, `each`, or another operation that executes the block. A nested function-shaped expression creates a new callable boundary and cannot inherit the surrounding block's permission. No repeated-caret or arbitrary runtime-stack traversal form exists.

The distinction remains local even when both roles of `this` occur in one statement:

```
this => this.leftBindingPower(this.current()) >= minimumBindingPower;
```

The first `this` belongs to the callable-result control production. The later `this.leftBindingPower(...)` and `this.current()` occurrences are ordinary receiver expressions and therefore refer to the enclosing receiver.

For a callable with multiple result positions, `this =>` produces values in declared result order:

```
this => firstValue, secondValue;
```

The zero-result form is `this ==>;`. It terminates a callable that declares zero result positions without producing result values. A value-producing `this =>` statement must produce the required number of values, and each value must satisfy the corresponding declared result type. Result positions remain unnamed and do not introduce callee-local bindings.

`this ->>;` is valid only where an enclosing iteration permits an advance to its next iteration. `this ->|;` exits the nearest applicable structured-control target. Their labeled forms are `this ->> 'label;` and `this ->| 'label;`; the former requires

the resolved label to denote an enclosing loop.

These are complete contextual control constructs. They are not member access, and `=>`, `^=>`, `->>`, and `->|` are not methods, properties, or operators resolved on the receiver represented by ordinary `this`. The glyphs do not acquire these control meanings by themselves. Existing `=>` lambda and match/case syntax and existing `->` type/function-signature syntax retain their ordinary meanings outside the corresponding `this`-headed control production.

Reserved Words properties/methods

Reserved Word	Property/Method
<code>let</code>	"in"
<code>forall</code>	
<code>co</code>	The implicitly available public standard-package projection root. Its frozen subpackage paths include <code>dynamic</code> , <code>macro</code> , <code>hokrlt</code> , <code>encoding</code> , <code>crypto</code> , <code>dap</code> , <code>ddap</code> , <code>pdap</code> , <code>out</code> , <code>const</code> , <code>native</code> , <code>meta</code> , <code>sys</code> , <code>os</code> , <code>in</code> , <code>pattern</code> , <code>control</code> , <code>runtime</code> , <code>compiletime</code> , <code>cpc</code> , <code>utils</code> , <code>operator</code> , <code>regex</code> , <code>hw</code> , and <code>stex</code> . Declarations projected directly into <code>co</code> , including data types, kinds, network declarations, and core collections, are resolved from <code>co.folenc</code> and are not a closed lexical member list.
<code>this</code>	"object", "class", "module", "kind", "type", "struct", "instance", "callee", "args", "params", "results", "associatedtype", "owner", "caller", "fallthrough", "yield", "parent", "super", "parents", "classes", "mixins", "traits", "interfaces", "builtins" (all these accessed using <code>-></code> on this unlike dot in case of others)
<code>f$\Phi$$\lambda$</code>	
<code>for</code>	
<code>@co</code>	is not exactly a reserved word but @ before reserved word

`this` has special access mechanism to reserved attributes normally dot `.` is used to access but when referring these attributes on this we need to use arrow `->` to avoid conflict between user defined member (fields/methods/...) with reserved attributes.

Special lifecycle members

Every `co.class` has the compiler-owned lifecycle members `@@new` and `@@init` as inherited class lifecycle machinery. Those inherited compiler implementations are not automatically exposed as ordinary source-callable lifecycle APIs.

A generic class with `@co.dap.generic(..., lifecycle=true)` may override or overload these existing lifecycle members. `lifecycle=true` enables **developer lifecycle customization**; it does not create lifecycle members and does not itself change their accessibility.

Developer lifecycle declaration	Invocation	Responsibility
<code>@@new</code> override/overload	<code>Type::new(...)</code>	Customized class allocation / uninitialized-instance lifecycle operation
<code>@@init</code> override/overload	<code>object::init(...)</code>	Customized class instance initialization lifecycle operation

A developer-defined lifecycle implementation participates in lifecycle lookup according to its declared accessibility. Public implementations are externally accessible. An implementation using any other valid accessibility classifier follows the ordinary accessibility rules for that classifier.

Ordinary methods, including ordinary methods named `new` or `init`, continue to use `.`. Lifecycle customization declarations use `@@`, and source-visible lifecycle invocation uses `::` only for matching developer-defined lifecycle implementations that are accessible to the caller. The inherited compiler lifecycle implementation remains internal/protected lifecycle machinery rather than an automatically exposed `::` candidate.

Builtin Packages

`co` — root (reserved word)

`co` is the reserved **public projection root** of the FoLang standard package. The standard source itself is canonically organized under private `fϕλ.*` package contexts and exported into `co.*` through the bootstrap projection rules above. The resulting public tree is distributed as `<install-root>/stdlib/co.folenc`. At compilation startup, the running compiler resolves its real executable path, derives its installation root, and loads the artifact directly before parsing project source. The frontend makes only the projected `co` root implicitly available without a source-level import; ordinary source cannot address the private `fϕλ.*` names. Project source must not copy the standard artifact into the project, install it as a third-party dependency, or declare an import for it. Projected declarations retain their canonical declaration identity.

The table below describes the current standard package hierarchy and API responsibilities. After version 1.0, the package/subpackage paths in this hierarchy are fixed, but the declarations contained inside an existing package are not frozen. Later standard-package artifact versions may add or update ordinary types, unit-level functions, methods, data structures, algorithms, modules, and other declarations without creating new FoLang grammar or a new package path.

Public projection shape

The public `co.*` hierarchy is a **curated projection**, not a mechanical mirror of the private `fϕλ.*` package tree. FoLang deliberately flattens only the internal packages whose package qualifier adds little value at ordinary use sites. The current flattened roots are `fϕλ.lang`, `fϕλ.core`, and `fϕλ.net`; their exported declarations are projected directly into `co`. Other standard-package domains normally retain their meaningful package segment under `co`.

```
@co.dap.export(  
    packages={fϕλ.lang, fϕλ.core, fϕλ.net},  
    as=co  
)  
  
@co.dap.export(packages={fϕλ.out},      as=co.out)  
@co.dap.export(packages={fϕλ.in},      as=co.in)
```

```
@co.dap.export(packages={f0λ.cpa}, as=co.cpa)
@co.dap.export(packages={f0λ.operator}, as=co.operator)
```

Representative projections are therefore:

Private canonical path	Public source path	Projection policy
f0λ.lang.string	co.string	flattened
f0λ.core.List	co.List	flattened
f0λ.net.http	co.http	flattened
f0λ.out.println	co.out.println	package retained
f0λ.in.readLine	co.in.readLine	package retained
f0λ.cpa.*	co.cpa.*	package retained
f0λ.operator.*	co.operator.*	package retained
metadata namespaces	co.dap.*, co.pdap.*, co.ddap.*	public namespace retained; not flattened

A package is **not** flattened merely because it is implemented below `f0λ.*`. Flattening is part of the explicit public projection contract. Unless this specification defines a package as projected directly `as=co`, its meaningful public package segment is retained. This prevents the `co` root from becoming an undifferentiated symbol bucket while keeping the most frequently used language, core, and network declarations concise.

Standard-library implementation source **SHALL** resolve standard-library declarations through their canonical `f0λ.*` names. Exported `co.*` projections are consumer-facing names and **SHALL NOT** be used to establish dependencies among standard-library declarations. A standard-library package therefore self-references, and refers to declarations from another standard-library package, through `f0λ.*`; this keeps standard-library compilation independent of the public projection that is produced from those canonical declarations.

Ordinary project, application, component, and third-party library source **SHALL** resolve exported standard-library facilities through `co.*`. Canonical `f0λ.*` names are private to privileged standard-library/bootstrap compilation and **SHALL NOT** participate in ordinary source name resolution, imports, aliases, or dependencies. Consequently, `f0λ.*` defines canonical standard-library identity and internal organization, while `co.*` defines the curated developer-facing surface.

Availability of an ordinary declaration inside `co.*` is determined by the applicable `co.folenc` artifact, not by whether that declaration has an example in this document. After the artifact is loaded, standard-package declarations participate in the ordinary package, symbol, type, member, visibility, extension, and overload-resolution model. A standard declaration classified by `@co.dap.implementation` may be bodyless and carry a backend-neutral `co.runtime.operation.*` marker; that marker and its signature are serialized in `co.folenc`. Package API evolution cannot manufacture new special syntax; any new grammar-level form requires an explicit specification revision to the core language grammar.

Public path	Responsibility
<code>co</code>	language data types and kinds projected from <code>f0λ.lang</code> ; network declarations such as <code>http</code> , <code>tcp</code> , and <code>udp</code> projected from <code>f0λ.net</code> ; and core collections/types such as <code>List</code> , <code>Map</code> , <code>Set</code> , and <code>Comparable</code> projected from <code>f0λ.core</code>
<code>co.sys</code>	file, concurrent, parallel, goto, invoke, bind, call, apply, setTimeout, setInterval, scheduler, cron, event
<code>co.os</code>	signal, cmd, execute, run, env, getenv, setenv, sleep, exit, cwd, chdir, fork, wait, pipe, dup, dup2, close, readfd, writefd, random
<code>co.meta</code>	ast, instrument, transform, augment, reflect, introspect, patch, inject, create, runtime(eval,etc), realm
<code>co.native</code>	load, register, asm, inline, emit, ffi, spawnnon[gpu,cpu,npu,apu,fpga,asic,tpu,mki,mcu],arch[x86,x86-64,risc,arm,vliw]
<code>co.in</code>	read, readln
<code>co.out</code>	println, print
<code>co.regex</code>	pattern, match, search
<code>co.crypto</code>	rsa, aes, hash, md5, rand, uuid, ssl, tls
<code>co.dap</code>	built-in decorators and annotations, including backend-neutral standard runtime-operation implementation markers

<code>co.ddap</code>	built-in directives
<code>co.pdap</code>	built-in pragmas
<code>co.const</code>	<code>true</code> , <code>false</code> , <code>none</code>
<code>co.encoding</code>	base64Encode, base64Decode, json, yml, bson
<code>co.utils</code>	makeImmutable, makeShared, copyOnWrite, toSnapshot — object behaviour policies
<code>co.dynamic</code>	dynamic capabilities
<code>co.runtime</code>	compiler-owned backend-neutral runtime-operation identifiers and semantic contracts; <code>co.runtime.operation.*</code> markers are implemented by the selected backend/runtime
<code>co.compiletime</code>	
<code>co.macro</code>	
<code>co.pattern</code>	
<code>co.control</code>	continuation, CPS, full/delimited continuation control, shift/reset, prompt/control, and continuation-oriented control abstractions
<code>co.cpa</code>	concurrent/parallel/async submission, task/thread execution facilities, future/callback completion, await, pools, channels, events, actors, process/distributed facilities, scheduling, fiber/coroutine facilities, defer, lazy, and related execution APIs
<code>co.hokrlt</code>	
<code>co.operator</code>	
<code>co.hw</code>	cpu, memory
<code>co.stex</code>	

Uniform Object Model

Uniform Object Principle

In FoLang, everything is an object.

Unless explicitly stated otherwise, the reference and mutation rules in this section apply to **managed FoLang objects**. `co.cstruct` remains an object in the language model, but it is an explicitly value-semantic ABI representation: assignment and parameter passing copy its value rather than a managed object reference.

This includes:

- scalar objects
- collection objects
- user-defined objects
- ADT objects
- function objects

Functions are objects in the same sense as all other values.

This is true for both **named functions** and **anonymous functions**.

Functions, scalars, UDTs, and ADTs participate in the same language-level object model except where this specification defines an explicit exception. The following core rules apply:

- assignment of managed objects copies references
- assignment of `co.cstruct` copies its value
- `==` compares values
- mutation is applied through the object
- behaviour policies such as immutability, shared, copy-on-write, and literal conversion apply uniformly where meaningful

Conceptual Prototype Delegation for Non-Class Object Kinds

FoLang uses **prototype delegation** only as a **conceptual member-resolution model** for specifying how language-defined members are available on runtime object kinds that do not use class inheritance. It defines the required member-resolution contract without prescribing a physical runtime representation.

The idea is loosely analogous to JavaScript prototype lookup only in one narrow sense: when a member is not supplied at the most specific level, lookup may be understood as continuing through successively more general language-defined

member sources. FoLang does **not** adopt JavaScript's prototype object model, mutable prototype chains, `instanceof` semantics, prototype reflection, or prototype-based `is-a` relationships.

For an applicable non-class runtime object, the specification may be understood conceptually as:

```
runtime value
  ↓ conceptual member lookup
<concrete declaration/type members>
  ↓
<language-kind members>
  ↓
<zero or more common language-defined member sources>
  ↓
end
```

For example, a struct value may be reasoned about as:

```
employee
  ↓
<Employee-specific members>
  ↓
<co.struct kind members>
  ↓
<common FoLang object members, where applicable>
  ↓
end
```

The angle-bracketed levels above are **explanatory notation only**. They do not introduce source expressions, runtime values, named parent types, or a required object named `prototype`. In particular, the common member source shown above is not `co.object`: `co.object` remains its own distinct FoLang declaration kind and is not the universal parent of other kinds.

The source-language contract exposes only the resulting member availability and behavior. A FoLang program cannot name, access, inspect, enumerate, reflect upon, traverse, create, attach, detach, replace, reorder, extend, remove, or otherwise observe a conceptual prototype object or prototype link. No source forms such as `Employee.prototype`, `value.prototype`, `[[Prototype]]`, `getPrototypeOf`, or `setPrototypeOf` exist as a consequence of this model.

Conceptual prototype delegation does **not** establish any type relationship. Reaching a kind-level or common member source does not mean that the receiver is a subtype, subclass, implementation, instance, child, or descendant of that source. For example, a struct value obtaining language-defined members through the `co.struct` kind contract does not imply `Employee <: co.struct`, nor does it make `co.struct` a parent type. Class inheritance, interface implementation, trait/mixin composition, typeclass relationships, and other type-system relationships remain governed exclusively by their own rules.

The normative semantic requirement is the **effective member contract**. An applicable object must expose all language-defined members required by its concrete declaration, its language kind, and any common FoLang object contract that applies to that kind. Member resolution must behave **as if** lookup proceeds from the most specific applicable member source toward successively more general language-defined sources until the applicable member is resolved or lookup fails under the ordinary member-resolution rules.

Common operations such as conversion, identity, or other built-in object operations are therefore available wherever the applicable FoLang kind contract defines them. The conceptual model does not make an otherwise invalid operation valid; for example, `co.cstruct` remains outside the managed-reference identity model and does not gain `sameRef()` merely because common member resolution exists for other object kinds.

Conceptual prototype/member-source relationships are not managed-object graph edges. They are not traversed for deep equality, object-policy propagation, snapshots, serialization, copying, reachability ownership, or `sameRef()` identity.

`co.class` is deliberately outside this non-class conceptual prototype model. Every class receives the language-defined operations required by the `co.class` kind, while class-to-class reuse, parent lookup, overriding, polymorphism, and dynamic dispatch follow the class inheritance rules.

Contracts and non-value declarations that do not denote runtime object values, such as interfaces, signatures, units, and packages, do not acquire a per-value conceptual prototype chain merely by being declaration kinds.

Backend Freedom and FoLang Semantic Contract

FoLang specifies **language semantics and externally observable contracts**, not a mandatory backend object layout or implementation algorithm. A backend generator has freedom to choose, optimize, and evolve its internal representation provided that generated programs preserve the behavior, member availability, type relationships, access rules, and other semantic requirements defined by this specification.

This freedom applies to the conceptual prototype/member-resolution model described above. A backend is not required to materialize prototype objects, maintain pointer links, perform runtime traversal, or provide one universal top-level member object. It may implement the required effective member contract using any semantically equivalent technique, including but not limited to:

- composition or embedding into the generated representation;
- flattened or duplicated member tables;
- delegation or prototype-like internal links;
- vtables, runtime descriptors, or dispatch tables;
- compiler intrinsics or backend-neutral operation identifiers;
- statically pre-resolved or directly emitted calls;
- cached, specialized, or otherwise optimized lookup structures;
- an internal host-language inheritance hierarchy; or
- another backend-specific mechanism.

An implementation technique does **not** create a FoLang language relationship that this specification does not define. Backend-internal inheritance, embedding, delegation, table linkage, or composition must not be exposed or described as FoLang inheritance, subtyping, parentage, implementation, or an `is-a` relationship unless the FoLang language rules independently define that relationship.

For example, a declaration of kind `co.struct` does not inherit from a parent struct merely because a backend chooses an inheritance-based host representation. A backend may accurately state that it implements the FoLang struct member contract by composing or embedding common behavior, flattening the effective member set, using prototype-like lookup, or even using host-language inheritance internally. It must not state that FoLang structs themselves inherit from an internal base struct, because FoLang defines no such struct-inheritance relationship.

Language-defined members likewise do not appear as unexplained or magical operations. Their availability follows from the effective member contract defined by the receiver's concrete declaration, its applicable FoLang kind, and any common language-defined member contract that applies to that kind. The conceptual prototype model gives one language-level way to reason about that contract; the backend remains free to realize the same result by another implementation technique.

A backend may optimize away every conceptual level, merge several levels, duplicate required operations, or lower them directly to native/runtime operations. Such optimization is conforming only when it preserves the same observable FoLang behavior. Backend implementation details therefore remain backend-defined, while FoLang semantic relationships and contracts remain specification-defined.

1. Default Object Model

Every value in FoLang is an object and is **mutable by default**.

This includes:

```
co.int, co.float, co.string    → built-in scalar types
user-defined types (structs, classes, ADTs) → user-defined objects
co.List, co.Map, co.Array      → built-in collections
```

Example:

```
positive_int co.int = 10;
```

`positive_int` denotes an object.

Its current value is `10`.

By default, it is mutable.

2. Assignment, Aliasing, and Mutation

FoLang distinguishes three related ideas:

- **assignment**
- **aliasing**

- mutation

2.1 Assignment copies references

When one object variable is assigned to another, FoLang copies the **reference**, not the internal contents.

```
b := a;
```

After this, `a` and `b` refer to the same object.

2.2 Mutation changes object contents

If two names refer to the same object, mutating through one name is visible through the other.

```
// Employee.fol
_ co.class = {
    Name co.string;
}
```

```
// appl.fol
a Employee = Employee{Name: "Kamesh"};
b := a;
c Employee = Employee{Name: "Kamesh"};

a == b;    // true
a == c;    // true
b == c;    // true

b.Name = "Ramesh";    // changes a.Name too
c.Name = "Ramesh";    // does not change a.Name
```

Why:

- `a` and `b` are the **same object**
- `c` is a **different object** with the same earlier value

2.3 `==` means value equality

In FoLang, `==` compares **values**, not object identity.

That rule is uniform across all object kinds, including built-in types and user-defined types.

So:

```
a == b;
```

means:

- compare contents deeply
- return `true` when the values match
- even if `a` and `b` are not the same object

2.4 Mutation accessors

The accessor used for mutation depends on the object type:

scalar (int, float, bool, string, char, byte)	→ <code>.value</code>
struct field	→ <code>.fieldname</code>
class member	→ <code>.membername</code>
map entry	→ <code>.key</code>
array / list element	→ <code>.index</code>

Examples:

```
count.value = 30;  
emp.name = "Rao";  
marks.math = 95;  
nums[0] = 42;
```

2.5 Function-call behaviour

Function parameters introduce a **new local binding**.

So:

- rebinding a parameter does **not** affect the caller
- mutating the object through the parameter **does** affect the caller if both still refer to the same object

```
checkPositive(a co.int)->(co.bool) = {  
  a = 20;           // local rebinding only – caller unchanged
```

```
a.value = 30;    // object mutation – caller sees 30
}
```

If `positive_int` is passed to `checkPositive(a)`:

```
a = 20          → local rebinding only
a.value = 30     → mutates the passed object
```

3. Literal Objects

All literals in FoLang create **Literal Objects**.

```
"Kumar"    → Literal Object – string
42          → Literal Object – int
true       → Literal Object – bool
```

A Literal Object is an **anonymous object created from a literal expression**.

Literal Objects participate in value equality just like every other object in FoLang:

```
10 == 10;    // true
```

In FoLang, `==` compares **values**, not object identity.

So two literal expressions with the same value compare equal, but that does not mean they are the same object.

Binding a Literal Object

When a literal is assigned to a named object, the name refers to the object created from that literal expression.

```
a co.int = 10;
```

Here, `a` refers to the anonymous integer object created from literal `10`.

So:

```
a.value = 30;
```

mutates that same object, and `a` now has value `30`.

The important points are:

- literal expressions create anonymous objects
- literal-created objects are still ordinary objects unless another policy is applied
- `==` compares values, not identity
- once bound to a name, a literal-created object behaves like any other normal mutable object
- immutability applies only when explicitly requested, for example through `makeImmutable(...)`

Why Literal Objects Cannot Be Mutated Directly

Mutation can occur in two ways:

1. Through rebinding

```
a co.int = 10; a = 20;
```

The above statement is valid because `a` is valid identifier and named handle to `co.int` type.

```
10 = 20; // ❌ invalid
```

The above statement is invalid because `10` is not a valid identifier and if `10` in literal object type there is no named handle.

The rebinding happens through named handles which are valid identifiers of `foLang`

2. Through a property or method

An object may also be mutated through one of its mutable properties or methods. `a co.int = 10; a.value = 20;`

A literal object cannot be mutated directly because it does not provide properties/methods that can be accessed.

```
10.value = 20; // ❌ invalid
```

```
a co.int = 10;  
a.value = 30;
```

is valid after binding, a bare literal such as `10` cannot be mutated directly because there is no name or handle through which to perform mutation.

Not the Same as `toSnapshot(...)`

A literal object created by a literal expression is **not** the same thing as the internal snapshot representation produced by `co.utils.toSnapshot(...)`.

- literal expression → anonymous object
- `toSnapshot(x)` → internal snapshot representation used to reconstruct a fresh local object later

These are related concepts, but they are not the same mechanism.

Types of Literal objects:

1. Simple types
2. Complex or compound types (UDTS)

Simple literal forms include values such as ``10``, ``'A'``, and ``"A"``.

```
k co.int=10;
```

Composite runtime values use explicit typed braced construction. The resolved type determines the initializer shape.

```
// Employee.fol
_ co.class = {
```

```
    id co.string;
    name co.string;
}
```

```
k Employee = Employee{ id: "10", name: "ABC" };
```

> A typed braced construction is an expression, not a block; when used as a statement or initializer it follows the ordinary semicolon rule.

Summary

- literal objects are real objects
- literal expressions create anonymous objects

- identical literals compare equal by value
 - identical literals are not automatically the same object
 - literal-created objects are mutable by default once bound to a handle
 - only `makeImmutable(...)` makes an object immutable
-

4. Object Behaviour Policies

Any managed object can be given a behaviour policy using `co.utils.*`.

For a `co.class` instance, the policy root must be the complete instance, not one selected field. A call such as `makeImmutable(customer.id)`, `makeShared(customer.queue)`, or `copyOnWrite(customer.cache)` is rejected when the selected value is class-owned instance storage. Applying the policy to `customer` already includes all of those fields through the deep object-graph rule. Independently managed constant, immutable, global, shared, or locked state belongs to a named `co.object` instead.

The policy operations are **in-place transformations of the object graph**:

- the object itself changes behaviour kind;
- there is no wrapper object;
- there is no alternate binding to capture;
- aliases continue to refer to the same transformed object graph.

Object policy belongs to the **object graph**, not to a variable name or alias. If two or more bindings refer to the same managed object, applying Immutable, Shared, or CopyOnWrite behaviour through one alias is observed through every alias that still refers to that object. Rebinding one alias later does not change the policy of the original object graph.

All policies are **deep by default**. The reachable object graph is the transitive closure obtained by starting at the policy root and recursively following managed-object references through members, nested objects, collection elements, and other managed references. Repeated references and cycles identify the same reachable object rather than creating additional logical objects. The applicable policy therefore governs the complete reachable graph, not only the root object.

Immutable, Shared, and CopyOnWrite are mutually exclusive object policies. Once an existing object graph enters one of these policy states, that policy is permanent for the lifetime of that graph; it cannot later be changed into either of the other policy states. `makeValueImmutable(x)` and `makeImmutable(x)` both make the current object graph Immutable; `makeImmutable(x)` additionally prevents rebinding of the binding supplied as `x`. Binding immutability is distinct from object policy and does not make other aliases non-rebindable.

4.1 Immutable

```
co.utils.makeImmutable(positive_int);
```

After this call the object itself is immutable.

This is an in-place transformation, not a wrapper.

Any attempt to mutate the object or any reachable object beneath it is a **compiler error** where detectable statically, and a **runtime error** otherwise.

```
positvie_int co.int = 10;
positive_int.value = 30    // ✗ compiler error
positive_int = 30         // ✗ compiler error
```

For nested objects:

```
// Employee.fol
_ co.struct = { address Address; }
Address co.Address = {city co.string; state co.string; lane co.string; pin
co.string;}

emp Employee = Employee{
    address: Address{
        city: "Pune"
    }
};
co.utils.makeImmutable(emp);

emp = Employee{
    address: Address{
```

```

        city: "ABC"

    }
}; // ❌ compiler error

emp.address = newAddress // ❌ compiler error
emp.address.city.value = "Mumbai" // ❌ compiler error

```

Immutability is deep and total.

4.2 Value Immutable

```

positive_int co.int = 20;

co.utils.makeValueImmutable(positive_int);

positive_int.value = 30; // ❌ current object is immutable
positive_int = 30; // ✅ binding may reference a new object

co.utils.makeValueImmutable(emp);

emp.address.city = "ABC"; // ❌
emp.address.city.value = "ABC"; // ❌

emp = Employee{
    address: Address{
        city: "ABC"
    }
}; // ✅

```

Difference between Immutable and Immutable Value

```

makeValueImmutable(x)
└─ current object graph cannot change

makeImmutable(x)
└─ current object graph cannot change
└─ binding cannot be reassigned

```

Table

Operation	Binding	Current value/object graph
<code>makeValueImmutable(x)</code>	Mutable	Immutable
<code>makeImmutable(x)</code>	Immutable	Immutable

4.3 Shared

```
co.utils.makeShared(positive_int);
```

A Shared Object is safe for concurrent and multi-threaded use.

Shared policy does not change the object's public type or ordinary member-access syntax; it changes the required mutation and concurrency semantics of the managed graph.

Shared behaviour is deep:

- all reachable objects within the shared object are also shared

4.4 CopyOnWrite

```
co.utils.copyOnWrite(positive_int);
```

A CopyOnWrite object passes by reference like a normal managed object until mutation is attempted. Merely passing or reading the object does not change the source state.

Normative CopyOnWrite Semantics

CopyOnWrite in FoLang has **whole-reachable-object-graph isolation semantics**. If any member or reachable object beneath a CopyOnWrite root is mutated in a context that must not affect the source object, the mutating context must observe an independent modified state for the complete logical graph rooted at that CopyOnWrite object, while external aliases that continue to refer to the source graph must continue to observe the unchanged source state.

For example:

```
a Employee = Employee{
    dept: Dept{ id: 10 },
    address: Address{ city: "Pune" }
};

co.utils.copyOnWrite(a);

process(emp Employee)->>() = {
    emp.dept.id = 20;
}

process(a);
```

After `process(a)`, the caller's `a` must still observe the original `Employee` graph with `dept.id == 10`, while the mutating context must observe an `Employee` state in which `dept.id == 20`. The isolation applies to the complete logical graph rooted at `a`, including `dept`, `address`, and all other reachable managed objects.

The specification defines this **observable isolation contract**, not a required physical copying algorithm. A conforming backend is not required to allocate a complete duplicate graph if it can preserve the same observable semantics by another implementation technique.

Internal alias topology and cycles are part of the logical graph semantics. If multiple members logically refer to the same object, that relationship must remain correct in the isolated COW state. Cyclic relationships must likewise remain semantically intact. An implementation must not allow a mutation in the isolated COW state to become visible through an external alias that still denotes the original source state.

Reference Backend Implementation

The default FoLang backend is the **reference implementation** for these semantics. Its straightforward reference strategy is whole-graph materialization: on the first mutation requiring isolation, it clones the entire reachable managed-object graph rooted at the `CopyOnWrite` object, preserves cycles and repeated internal references, redirects the mutating context to the cloned root, and applies the mutation to the clone. External aliases remain attached to the original graph.

Conceptually, the reference implementation behaves as follows:

caller before mutation

```
a → Employee A
    ├── Dept B      (id = 10)
    ├── Address C
    └── ... other reachable objects
```

reference-backend COW materialization

caller	mutating context
<pre>a → Employee A ├── Dept B (id = 10) ├── Address C └── ...</pre>	<pre>emp → Employee A' ├── Dept B' (id = 20) ├── Address C' └── ...</pre>

This reference algorithm is deliberately simple and is intended to make the required semantics easy to understand, implement, inspect, and test. It is not a requirement that production backends use the same physical representation or copying strategy.

Reference Backend Whole-Graph Cost

Because the reference backend materializes the complete reachable graph, the cost is determined by the size of the COW root's reachable graph, not by the size of the individual member being modified.

For example:

```
EmployeeList co.type = co.List(Employee);
employees EmployeeList = ...;
co.utils.copyOnWrite(employees);

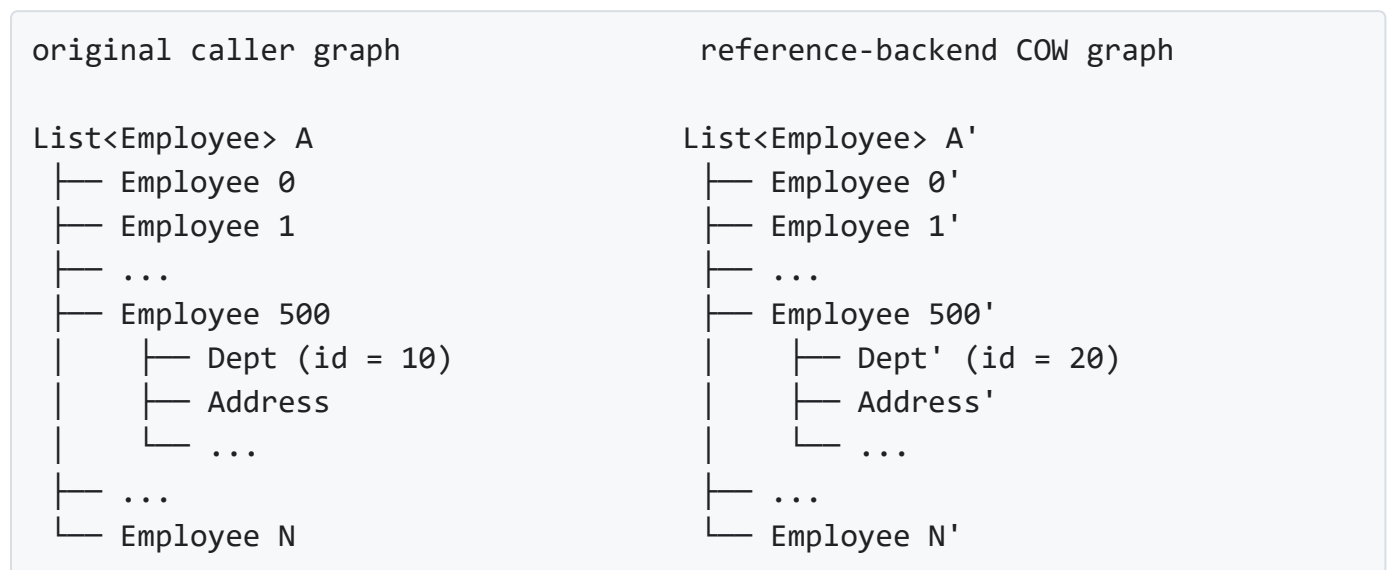
process(emps EmployeeList)->>() = {
    emps[500].dept.id = 20;
}

process(employees);
```

Although the logical change is only to `emps[500].dept.id`, the COW root is the employee list. The reference backend therefore materializes an isolated copy of the entire reachable `co.List(Employee)` graph before applying the mutation. This

includes the list, all reachable `Employee` objects, their departments, addresses, and other reachable managed objects. The caller's original list graph remains intact.

Conceptually:



This simple reference strategy may be unsuitable for some production workloads. A tiny nested mutation can require work proportional to a very large reachable graph, may temporarily keep both source and isolated graphs live, and can increase allocation pressure, peak memory use, memory-bandwidth consumption, and allocator fragmentation. These are characteristics of the **reference backend implementation**, not additional FoLang language semantics.

Conforming Alternative Backends

A third-party or specialized backend may implement the same CopyOnWrite semantics using structural sharing, path copying, persistent data structures, lazy materialization, reference counting, arenas, page-based techniques, virtual-memory mechanisms, a garbage-collected runtime, a custom memory manager, or another strategy. Such implementation choices are permitted provided that all externally observable FoLang semantics are preserved.

In particular, a backend may physically share unchanged storage between the source and isolated COW states as long as later mutations cannot leak across those logical states. FoLang source code must not be able to distinguish the optimized backend from the reference semantics except through non-semantic characteristics such as performance, memory consumption, allocation behaviour, or diagnostics that do not alter program meaning.

Backend implementers should use the written specification as the normative contract and may test their implementation against the reference backend and the FoLang conformance suite. Matching the reference backend's internal whole-graph allocation algorithm is **not** a conformance requirement; matching the required observable behavior is.

4.5 toSnapshot

```
co.utils.toSnapshot(positive_int);
```

`toSnapshot` converts an object into a **snapshot representation** — a value descriptor, not a live object. The snapshot representation itself cannot be mutated.

When passed to a function, the compiler/runtime uses the snapshot representation to construct a fresh independent local variable bound to the parameter name. That local is a normal mutable object with no shared identity with the original. The conversion and reconstruction steps are part of the language semantics and require no additional source-level reconstruction operation.

```
process(a co.int)->>() = {  
    a.value = 99;    // mutates the fresh local – positive_int completely  
    unaffected  
}  
  
process(co.utils.toSnapshot(positive_int))  
  
or  
  
k co.value = co.utils.toSnapshot(positive_int);  
  
// positive_int unchanged
```

The flow:

```
positive_int  
  ↓ co.utils.toSnapshot(positive_int)  
  snapshot representation ← value descriptor – immutable by  
  nature, not by policy  
  ↓ passed to function  
  compiler constructs fresh local 'a' from the snapshot representation
```

'a' is a new independent normal object – mutations never reach `positive_int`

`toSnapshot` is deep — the snapshot representation covers the entire reachable object graph.

5. Policy Summary

Object Kind	Mutation allowed	Caller sees mutation	Thread safe	Copy on write	Deep
Literal expression object	✗ directly in source — no handle	—	value-like use	—	✓
Normal (default)	✓ via accessor	✓	✗	✗	—
Immutable	✗	—	✓	—	✓
Shared	✓ via accessor	✓	✓	✗	✓
CopyOnWrite	✓ on own copy	✗	✓	✓	✓
toSnapshot result	✓ on reconstructed local object	✗	independent snapshot	—	✓

6. Literal vs Value/snapshot

`co.value` vs `co.literal`

Literal (`co.literal`) is literal representation of objects literals are object.

Literal representations use `to` conversion methods to produce typed objects. When no suitable conversion exists for a custom type, the required overload must be supplied through the supported extension mechanism. A literal representation by itself does not carry the complete reconstruction metadata stored by a snapshot value.

A `co.value` snapshot carries more information than a literal representation: it records the type information, literal/value representation, and reconstruction information required to create the corresponding object according to its declaration kind.

7. Formal Object-Policy and Identity Rules

The following rules define the previously open object-model precision points.

7.1 Aliases and Policy Ownership

Managed-object assignment copies a reference. Therefore multiple bindings may refer to the same object. Immutable, Shared, and CopyOnWrite state belongs to that object graph rather than to an individual alias. Applying one of these policies through any alias changes the policy observed through every alias that still refers to the same graph.

```
a Employee = Employee{...};  
b := a;  
  
co.utils.makeShared(a);  
  
// a and b still refer to the same object graph.  
// The graph is Shared through both aliases.
```

7.2 Rebinding Is Independent of Object Policy

Rebinding an alias changes only which object that binding refers to. It does not remove, transfer, or alter the policy of the previously referenced object.

```
a Employee = Employee{...};  
b := a;  
  
co.utils.makeShared(a);  
b = Employee{...};
```

After the rebinding, `a` still refers to the original Shared graph. `b` refers to a different newly created object graph. The original graph remains Shared for its lifetime.

`makeImmutable(x)` is special only in that it also freezes the supplied binding `x`; the Immutable object policy itself still belongs to the object graph. Another alias to the same immutable graph cannot mutate that graph, although that other alias may itself be rebound unless its own binding has separately been made non-rebindable.

7.3 Deep Policy Scope

A deep policy covers the entire managed-object graph reachable from its root. Reachability is transitive: members, nested objects, collection elements, and other managed-object references are followed recursively until no new object identity is encountered. Cycles terminate by identity rather than by traversal depth, and repeated references to the same object remain repeated references to one object.

Thus applying a deep policy to an `Employee` also applies that policy to its reachable `dept`, `address`, nested collections, collection elements, and other managed objects in that graph. The policy invariant is maintained for the graph while that policy is active.

A `co.object->(for=...)` association is metadata, not a managed-reference edge. Consequently, an Immutable, Shared, or CopyOnWrite policy applied to a particular class instance never incorporates or propagates to an associated object. Class inheritance likewise does not place the associated object in the child instance graph. The associated object retains its independent singleton lifetime and policy state.

Conceptual prototype/member-source relationships for non-class object kinds are likewise member-resolution semantics rather than managed-reference edges. Deep object policies never traverse into or through those conceptual member sources.

7.4 Mutation Visibility Across Calls

Function arguments use the managed reference model. Mutation visibility therefore follows the object's policy:

- **Normal** — mutation through the parameter mutates the same object graph and is visible to the caller and other aliases.
- **Shared** — mutation is visible to aliases of the same graph, with the concurrency-safety guarantees of Shared behaviour.
- **Immutable** — mutation is prohibited.

- **CopyOnWrite** — a mutation requiring isolation gives the mutating context an independent logical state for the complete reachable graph while the caller's original graph remains unchanged. The reference backend realizes this by materializing a complete private graph copy; other conforming backends may use different internal techniques.

The policy of an existing graph is not temporary or call-local. Once that graph becomes Immutable, Shared, or CopyOnWrite, it remains in that state for its lifetime.

7.5 Value Equality and Reference Identity

`==` performs deep value equality. It answers whether two values/object graphs are equal by value; it does not answer whether two bindings refer to the same managed object.

Reference identity is exposed through `sameRef()` for managed reference-semantic objects:

```
a Employee = Employee{name: "Rao"};
b := a;
c Employee = Employee{name: "Rao"};

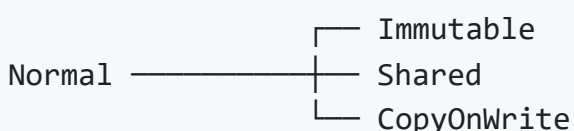
a == b;           // true
a == c;           // true

a.sameRef(b);     // true  - same managed object
a.sameRef(c);     // false - equal value, different managed object
```

`co.cstruct` remains value-semantic and is outside the managed-object reference-identity model.

7.6 Policy Lifetime and Non-Stacking

Immutable, Shared, and CopyOnWrite are mutually exclusive permanent states of an existing managed object graph. A graph may transition from its normal mutable state into one of these states, but it cannot subsequently transition from one policy state into another.



Immutable	-X-> Shared
Immutable	-X-> CopyOnWrite
Shared	-X-> Immutable
Shared	-X-> CopyOnWrite
CopyOnWrite	-X-> Immutable
CopyOnWrite	-X-> Shared

`makeValueImmutable(x)` and `makeImmutable(x)` both place the current object graph into the Immutable state. Their difference concerns the supplied binding:

`makeValueImmutable(x)` permits that binding to be rebound later, whereas `makeImmutable(x)` also prevents rebinding of that binding. Rebinding to a different object does not transfer the old object's policy to the new object.

8. Normative Object-Model Summary

All managed FoLang objects use reference semantics by default. `co.cstruct` is an explicitly value-semantic ABI representation and is an exception to managed-object reference semantics.

In FoLang, everything is an object and managed objects are mutable by default. Assignment of managed objects copies references, `co.cstruct` assignment copies values, `==` compares values deeply, and `sameRef()` exposes managed-object reference identity.

Immutable, Shared, and CopyOnWrite policies do not change the public type of an object. These policies belong to the object graph, are deep, are observed through all aliases to that graph, are mutually exclusive, and are permanent for that graph's lifetime.

Rebinding an alias changes the object referenced by that binding but does not modify or transfer the policy of the previously referenced object.

CopyOnWrite has whole-graph isolation semantics: a mutation anywhere in the reachable graph that requires isolation must leave the source logical graph intact while the mutating context observes an independent modified logical graph. The reference backend demonstrates this by cloning the complete reachable graph, but conforming backends may use any internal mechanism that preserves the same observable semantics.

FoLang Definition and Documentation License

Unless otherwise stated, the copyrightable material contained in the FoLang language definition and documentation is licensed under the [Creative Commons Attribution 4.0 International License](#).

The CC BY 4.0 licence applies to the copyrightable expression, organization, and presentation of the FoLang language definition and documentation, including:

- the FoLang language specification;
- original FoLang-specific syntax forms;
- grammar productions and grammar notation;
- rules governing how FoLang syntax forms may be combined;
- semantic rules and behavioural descriptions;
- name-resolution and scope-resolution rules;
- type-system definitions and constraints;
- execution-model and control-abstraction descriptions;
- compiler-behaviour descriptions;
- source-code examples demonstrating FoLang syntax and semantics;
- diagrams, tables, illustrations, and formal notation;
- reference documentation;
- explanatory and instructional material.

This includes the documented expression and presentation of FoLang-specific constructs such as:

```
(booleanExpression).then({  
    ...  
}).default({  
    ...  
});
```

and other original FoLang syntax, grammar, and semantic-rule descriptions contained in this document.

Under CC BY 4.0, this material may be:

- copied and redistributed in any medium or format;
- adapted, modified, translated, and extended;

- used for commercial or non-commercial purposes.

A person exercising these permissions must:

- give appropriate attribution;
- provide a link to the CC BY 4.0 licence;
- indicate whether modifications were made;
- retain notices supplied with the licensed material where required;
- not imply endorsement by the FoLang project or its contributors.

Independent Use and Implementation

This licence applies to the copyrightable expression contained in the FoLang language definition and documentation.

It does not restrict the independent use or implementation of programming-language ideas, concepts, features, procedures, systems, or methods of operation.

For example, this licence does not claim exclusive rights over general programming-language concepts such as:

- continuations;
- modules;
- structs and classes;
- dynamic, lexical, or mixed scope;
- functions and closures;
- pattern matching;
- algebraic data types;
- dependent types;
- concurrency and parallelism;
- coroutines and asynchronous execution. etc.,

Another project may independently implement such concepts without copying the copyrightable expression of the FoLang documentation.

The FoLang-specific documentation describing how these concepts are expressed through FoLang syntax, grammar, semantic rules, examples, diagrams, and explanatory text remains subject to this licence when that material is copied or adapted.

Software Licences

This licence does not change or replace the licences applied to:

- the FoLang frontend source code;
- the default backend source code;
- other FoLang software components;
- generated compiler binaries;
- third-party software;
- third-party documentation or assets.

The FoLang frontend and backend continue to be governed by their separately stated software licences.

Other Rights

CC BY 4.0 does not grant rights relating to:

- trademarks;
- project names and branding;
- logos;
- patents;
- third-party material;
- privacy, publicity, or personality rights.

Such rights, where applicable, remain governed separately.

Suggested Attribution

FoLang Language Definition and Documentation, including its syntax, grammar, and semantic-rule descriptions, by [Kemeswara Rao Mithipati](#) and FoLang contributors, licensed under [CC BY 4.0](#).

When modifications are made, the attribution should also indicate that the material was modified.

Example:

Based on the FoLang Language Definition and Documentation by [Kemeswara Rao Mithipati](#) and FoLang contributors, licensed under [CC BY 4.0](#). Modified from the original. ***

Appendix A - Complete FoLang EBNF Grammar

The standalone consolidated EBNF referenced below is the normative lexical and syntactic grammar for FoLang. The prose sections of this reference define semantics and parser-validity constraints without maintaining a second embedded copy of the grammar.

```
(*
  FoLang EBNF Grammar – current alpha profile

  Consolidated from:
    docs/language-ref.md
    SHA-256:
c9a9b6dadfb61b04325b1c2f4169a99ef1444b9856b23ca70ee592b8ffda85ef
    Bytes: 711896
    Lines: 16269

  Authority:
    The language reference is normative. This grammar is a syntactic
    consolidation of the source forms defined there.

  Notation:
    =          defines a production
    ;          ends a production
    |          alternative
    [ ... ]    optional
    { ... }    zero or more
    ( ... )    grouping
    "..."    terminal text
    ? ... ?    lexical or context-sensitive condition described in prose

  Parser/lexer layering:
    - Source text is UTF-8.
    - U+FEFF is permitted only as the first source code point.
    - white-space/comments are discarded between tokens.
    - newlines never terminate statements.
    - the parser consumes the token stream produced by the lexer.
```


Filesystem-selected parser roles:

src/appl.fol

-> application-entry-file

src/component.fol

components/application/component.fol

components/native/component.fol

components/dynamicvmrt/component.fol

components/packaged/component.fol

components/operators/component.fol

-> component-surface-file

ordinary package files

<Name>.fol

<Fragment>.unit.fol

<StructName>.comp.unit.fol

-> the matching package-source-file alternative selected from the externally validated filename class

All FoLang source under src/ and components/ uses this common grammar.

Filesystem placement supplies the structural role/component kind and

the

semantic validation context. lib/*.folenc is deserialized rather than source-parsed.

Current-alpha rules that are deliberately explicit here:

- A named function block body requires "=" before the body.

- An anonymous function literal does not use "=" between

signature/body.

- Direct declaration/function/block bodies end at "}" and cannot take

";".

- Braced expressions/literals still require the enclosing statement

";".

- Object construction is Type{field: value, ...}.

- A type's arrow tail "T->(...)" carries two structural meanings

that

the base and tail shape separate: a derivation applied to T or the

result

list of a function type. Generic and parameterized type applications

use

the uniform Type(...) postfix form. Type arguments are positional

and

bind in the generic declaration's parameter order:

co.lang.int->([5])

derivation

co.lang.int->(&, meta={type=out})

derivation,

attributes

<code>(co.lang.int)->(co.lang.int)</code>	function type
<code>co.core.List(co.lang.string)</code>	positional type

args

<code>co.core.Map(co.lang.string, co.lang.int)</code>	key, value
<code>co.core.Array(2, co.lang.int, [2,4])</code>	bound values

An argument's value may be a type, an integer, or a bracketed list, which is what the Array and Matrix declarations bind.

- Generic arguments are positional; ``name=value`` remains exclusive to metadata and derivation attributes.
- A generic collection application is first named by a concrete type

alias.

That alias is followed directly by its literal body:

```
StringList{"A","B","C"}
StringIntMap{"A": 1, "B": 2}
IntSet{1,2,3}
```

The resolved type determines the entry interpretation.

`builtin-collection-type-name` records the reference's Builtin

Collections

registry, which is the closed set of built-in names admissible in

that

prefix position.

- There is no untyped composite literal. Every runtime list, array,

set,

map, struct, or class value names its concrete type and uses ``{ ... }``.

Bare ``{ ... }`` and ``[ ... ]`` are not primary expressions. Bracketed

lists

remain available only where a type production explicitly admits a dependent value such as array dimensions. See "Canonical Object and Collection Construction" in the language reference.

- `.match` and `.match()` mean automatic matcher selection; `.match(expr)` selects an explicit matcher.
- Control-chain vocabulary uses `.then(X)` for one-shot conditional selection, `.loop(block)` for single-condition repeated execution, `.otherwise(condition)` only for another selection condition, and `.default(X)` only for a terminal selection fallback. A `.loop(block)`

form

has exactly one condition and cannot be followed by

`.otherwise(condition)`

or `.default(X)`. Element iteration is owned by `.each(...)` itself: its explicit-binding form takes index/key, value, and the per-element

action

directly, while its single-argument form takes a callable callback.

An

`.each(...)` call cannot be followed by `.loop(...)`. There is no conditionless

`.otherwise` form and no `.do` control verb in the current alpha profile.

- Built-in precedence follows "Built-In Operator Parse Table" in the language reference.
- The built-in type-relation operators `<:` and `:>` are binary infix relational operators at precedence 350. Relational and equality expressions are non-associative and therefore admit at most one operator at their respective precedence levels.
- A `co.lang.predicateType` declaration has a dedicated named predicate binder inside `co.lang.type.where(name => expression)`. This binder

form

is not a general lambda-expression and does not relax the collection-only

placement rule for `|...| => ... lambdas`.

- The only current-alpha pre-declared operator glyphs are `U` and `n`.

They

are active binary infix operators at precedence 500, left associative;

missing overload implementations fail during resolution, not parsing.

- reserved-future-operator-fixity is a diagnostic-recognition production:

encountering an explicitly specification-reserved future fixity in the

current alpha profile produces an unsupported-feature parser error.

- Examples never enable, disable, or reserve syntax. Active syntax comes

from this grammar; reserved-future treatment applies only to spellings or

forms the language reference explicitly identifies as future/reserved.

- All built-in `@co.*` directives, annotations, pragmas and decorators use

the single generic annotation application shape, but the complete `@co.*`

name must first match the compiler's predefined built-in metadata registry. An unregistered `@co.*` name is a parser error. Once a known form is recognized, every supplied field/argument is collected and preserved; fields the frontend already understands may be validated, while unknown/unhandled fields of that known form do not block

parsing

or frontend artifact generation.

- Every named metadata field/attribute and every `co.lang.operator`

declaration attribute uses "=". The same binder applies at every metadata nesting level. The ":" token remains the entry separator

for

ordinary object values and runtime map literals; it is never a declarative attribute binder.

- For a class with ordered direct class parents, ``this->parent`` and

its

``this->super`` alias select the primary parent. The type-named plural

form

``this->parents[SomeParent]`` aliases ``this->classes[SomeParent]``.

- ``classes[Type]``, ``mixins[Type]``, ``traits[Type]``, and

``interfaces[Type]``

are contextual compile-time relationship selectors available through

``this``. In a class method ``this`` denotes the current class/type; in

an

instance method it denotes the current instance. The key is a

complete

type name or import-alias-based type name that resolves to a direct

entry

of the matching ordered `@co.dap.oops` field. Numeric, string,

computed,

and runtime type-object keys are invalid. The selector categories

are

not runtime collections.

- Accessible inherited class/mixin fields are grouped by identifier. Matching canonical types merge into one child instance-state slot; mismatching canonical types are a semantic error. Field-level

constant,

immutable, Shared, locking, and CopyOnWrite policies are rejected on

class and mixin storage before merging. The broader accessible level

is

selected as public > protected > internal, while private fields do

not

enter the child-visible merge.

- A `co.lang.class` contains ordinary mutable per-instance fields. An Immutable, Shared, or CopyOnWrite policy may target the complete

class

instance graph, but a selected class-owned field cannot be an

independent policy root. Class-associated constant, immutable,

global,

shared, or locked state belongs to a named `co.lang.object`.

- A `co.lang.object` is one named singleton declaration and requires

`->(for=Target)` or `->(for=[Target, ...])`. An annotation target

denotes an

annotation implementation; a homogeneous non-empty class-target list

denotes an explicitly associated support object. The association is non-owning, non-structural, non-transitive, and never becomes a managed reference edge merely through `for=`.

- For methods, repeated paths to one canonical declaration deduplicate. Distinct matching concrete methods within one relationship category use the first source in that ordered list. Matching concrete methods across categories require an explicit child resolution. Matching abstract obligations may share one implementation, and matching mandatory virtual slots may share one override.
- ``forall`` is a contextual spelling recognized only in the value of a `co.lang.type` declaration. Higher-rank parameters and results use the resulting named polymorphic type; functions and anonymous functions do not introduce `forall` binders directly.
- A `co.lang.type` declaration may have a parameterized declaration head, as in `Option(T) co.lang.type`. Concrete specializations and derived types are named by a non-parameterized alias; value-indexed families are ordinary functions returning `co.lang.dependentType`.
- In a signature, ``Name co.lang.type;`` is an ordinary abstract type slot. ``T co.lang.associatedType;`` is narrower: `T` must parameterize at least one required `co.lang.type` alias whose matching-module definition instantiates a generic container. The module repeats `co.lang.associatedType` when binding `T`. An unused associated type is invalid.
- Typeclass `shape=(F(_)) metadata` and `F(A)/F(B)` contract applications remain a context-bounded higher-kinded exception and are not general type-alias syntax.

Context-sensitive/semantic requirements intentionally remain outside EBNF:

- filename canonicalization and package ownership; project layout and reachability; component/project-kind placement; import target uniqueness/mutual exclusion; projected-library capability restrictions;
- interface/signature/typeclass conformance;
- instance placement/liveness; overload resolution; dependent-index name resolution; generic-argument binding by declared positional order;

whether a type prefix on a
collection body names a collection type; built-in metadata field
schemas
beyond frontend-known checks; custom annotation/decorator symbol
resolution; annotation target/type checks; operator metadata
uniqueness,
ranges and arity/fixity compatibility; operator ownership
normalization,
where an operator belonging to or extending a generic type is
associated
with that type's canonical declaration identity rather than with
operator-level
generic parameters; rejection of `co.dap.operator` + `co.dap.generic` on
the same
function-shaped declaration; and all type/runtime semantics.
Predicate-type binder scope and immutability, the requirement that its
body resolve to `co.lang.bool`, type-object membership, canonical type
identity, and the subtype/supertype meaning of `<:` and `:>` are likewise
semantic checks outside the context-free grammar.
An omitted initializer ordinarily places that storage location in the
universal `co.const.none` state. Refinement-type and concrete dependent-
type
values are non-`none` exceptions: they cannot store, receive, pass,
return,
or otherwise admit `co.const.none`. Their declarations may omit an
initializer, but then denote definitely-uninitialized storage with no
readable language-level value. Flow-sensitive definite-initialization
analysis must prove a valid assignment on every reachable path before
every value-producing access; otherwise the compiler reports
`Uninitialized`. `Uninitialized` is the canonical case-sensitive name in
the
language's named diagnostic registry; messages, locations, and
internal
numeric identifiers may vary without changing that name. The complete
registry is normative in the language reference and groups lexical,
parser, project, symbol, type, generic, OOP, metadata, operator,
effect,
execution-model, usage, and backend rejection families. Diagnostic-
name
selection is semantic metadata rather than a context-free production.
Parser recovery uses `UnexpectedToken`, `ExpectedToken`, or fallback
`InvalidSyntax`; once a valid AST construct exists, later phases select
the
most specific applicable registered diagnostic and suppress derivative
cascades. A simple assignment target is not a read. The intrinsic

isNone() operation is semantically invalid for statically resolved refinement-type and dependent-type values. A construct whose missing-result behavior would substitute `co.const.none` into such a position is invalid, including applicable `continue`, `return`, `return_with_error`, and execution-model failure paths.

Aggregate construction is checked per path. Every refinement/dependent struct field must be initialized before the struct value is returned

or

`read`. Class `@@new` may yield `co.lang.uninit`, but every successful

`@@init`

path must initialize each refinement/dependent field before the

instance

becomes readable. Each `@@init` overload is analysed independently;

multiple `@@new` overloads do not merge initialization proofs.

For ordinary types that admit `none`, `none` is not a singleton and has no stable observable identity: a runtime may reuse or rematerialize its representation between accesses. Only intrinsic `value.isNone()`

reliably

observes the state and returns `co.const.true` or `co.const.false`.

Equality,

inequality, identity, ordering, hashing, and other observations

involving

`none` are semantically unspecified unless a construct defines explicit `none` behavior. The backend must not expose raw host-memory contents or backend undefined behavior while implementing this indeterminate language-level state. Optional-argument omission remains a separate

state

and is not `co.const.none`.

Every `co.lang.class` declaration has class construction semantics.

`FoLang`

has no abstract or unconstructable class category. `co.dap.sealed` on a class prevents its selection as a parent but does not prevent

constructing

that class; on a method it prevents overriding but not invocation. A `co.lang.mixin` is a distinct composition declaration kind with no independent class-instance construction operation, not an

unconstructable

class.

A closed `co.lang.variants(...)` result is the recommended, non-mandatory

representation for expected domain alternatives such as

`Found/NotFound`.

Such a declared `None` variant is an ordinary initialized variant value and is distinct from `co.const.none`. No general rule requires a variant destination merely because `resolution=continue` may substitute

co.const.none; tooling may issue only an advisory diagnostic for ordinary
none-admitting types. Refinement/dependent result positions are the strict
exception and reject the none substitution. Continue never manufactures
a user-declared variant constructor.
Object-association target resolution, homogeneous target-kind checking,
duplicate-target rejection, singleton lifetime, access rules, and the prohibition against propagating class inheritance or class-instance object policies through a for= association are semantic checks outside the context-free grammar. Rejection of field-level policy metadata on class/mixin storage and rejection of a class-owned selected field as an
independent makeImmutable/makeShared/copyOnWrite policy root are likewise
semantic checks.
Every recoverable effect is an instance of a class satisfying the standard
co.lang.error interface. co.lang.AbstractError is the standard co.lang.mixin supplying common error state and concrete behavior;
custom
error classes normally compose that mixin while declaring co.lang.error as
an interface. The mixin is not a base class and has no independent class-instance construction semantics. Exact matching composed methods may satisfy interface slots; explicit source mappings use co.dap.implement on the consuming
class. co.dap.implementation remains the distinct runtime-operation marker. co.runtime.operation.effect.emit is the backend-neutral emission operation and accepts only co.lang.error. Recoverable backend failures must be translated to such class instances. Fatal failures do not
satisfy co.lang.error and cannot participate in effect metadata, handling,
resolution, or propagation.
Effect declaration and handling use separate placements.
co.dap.effects
is legal only on ordinary callable declarations/definitions and requires
one emits list advertising known effects that may leave that callable.
It
is invalid on co.dap.executionmodel declarations. Every emits item must be

a class satisfying `co.lang.error`. The list is open and non-exhaustive: absence of `co.dap.effects` advertises no explicit effects but does not prohibit undeclared effects from propagating.

`co.dap.onEffect` is illegal as declaration metadata and is parsed only by

effect-handled-call-expression immediately before one call expression. Each direct qualified-name field of `co.dap.onEffect` must resolve to a

valid

effect type; it need not appear in the callee's open advertised set.

Each

ordinary-call record must contain exactly one singular resolution

equal to

`continue`, `retry`, `return`, `return_with_error`, or `propagate`. The optional `handlers` field must be a non-empty ordered list of accessible caller-

side

module values matching `co.core.EffectHandler`. For a call whose

resolved

callee is a `co.dap.executionmodel` declaration, each record instead

requires

a non-empty `handlers` list and forbids resolution; `return_with_error` is implicit and fixed by the language. Any explicit resolution is

invalid.

That signature constrains only `handle(error co.lang.error)->()`; each concrete `handle` implementation may independently carry `co.dap.effects`, and both its advertised and unexpected effects may escape the handler. `co.dap.onEffect` is forbidden throughout an effect-handler module, including call sites in private handler functions, so a handler cannot start a nested local effect-handling pipeline.

Their `handle(error co.lang.error)->()` functions execute synchronously

in

list order before the resolution. Ordinary calls use the receiving

caller

context; execution-model calls use the context observing the effect

before

completion is published.

If handler `H` emits recoverable effect `E1` while handling original

effect

`E0`, remaining handlers are skipped and `E0`'s selected resolution is cancelled. The standard library creates a `co.lang.GenericError` class instance satisfying `co.lang.error` and wrapping `E0`, `E1`, `H`, and

available

causal/source/runtime diagnostic context. At an ordinary call

this wrapper propagates from the enclosing caller; at an execution-

model

boundary it is returned through the ordinary error-result position and

otherwise unproduced none-admitting results receive `co.const.none`.
 Such a
 missing-result path is invalid for refinement/dependent positions. The
 wrapper is not
 reprocessed by the same handler list. `GenericError` is not created for
 normal propagate or `return_with_error` paths, which preserve `E0`, and a
 fatal handler failure terminates immediately without wrapping.
 retry requires `retry={max_attempts=PositiveCompileTimeInteger,`
`on_exhausted=TerminalResolution}`; `TerminalResolution` is `continue`,
`return`,
`return_with_error`, or `propagate`. Receiver and argument expressions are
 evaluated once and their values are reused by `retry`. `continue`
 substitutes
`co.const.none` for each missing call result. `return` and
`return_with_error`
 likewise use `co.const.none` for otherwise unproduced caller result
 slots;
`return_with_error` is checked against the enclosing caller's error
 result.
 Propagate and unmatched effects leave the caller whether or not they
 were
 explicitly advertised; statically known escaping effects are included
 in
 computed outgoing symbol metadata. Every `co.dap.executionmodel`
 declaration
 must expose exactly one `co.lang.error-compatible` result and forms an
 effect
 boundary: its first observed unhandled recoverable effect is converted
 to
 that ordinary result, unmatched effects receive the same conversion
 without
 handlers, and no recoverable effect propagates to the submitting
 caller.
 Known internal effects may remain implementation metadata but are not
 advertised outward. For concurrent/parallel execution, first observed
 may
 depend on scheduling unless a model defines deterministic priority.
 There is no effects wrapper, resolutions list, or `invoke resolution` in
`co.dap.onEffect`.
`co.dap.defer` provides no injected error or completion-status binding;
 it
 registers explicitly parameterized completion work for LIFO execution
 on
 every enclosing-function exit after registration is reached.
 *)

```

(* =====
*)

(* 1. Source roots and externally selected file forms *)

(* =====
*)

compilation-unit = package-source-file
                  | application-entry-file
                  | component-surface-file ;

source-filename = companion-unit-filename
                  | ordinary-unit-filename
                  | application-entry-filename
                  | component-surface-filename
                  | ordinary-source-filename ;

companion-unit-filename = filename-identifier, ".comp.unit.fol" ;

ordinary-unit-filename = filename-identifier, ".unit.fol" ;

application-entry-filename = "appl.fol" ;

component-surface-filename = "component.fol" ;

ordinary-source-filename = filename-identifier, ".fol" ;

filename-identifier = identifier-head, { "_", identifier-segment } ;

package-source-file = package-primary-source-file
                     | ordinary-unit-source-file
                     | companion-unit-source-file ;

package-primary-source-file = file-preamble, primary-declaration ;

ordinary-unit-source-file = file-preamble, unit-declaration ;

companion-unit-source-file = file-preamble, unit-declaration ;

application-entry-file = file-preamble, { entry-item } ;

component-surface-file = file-preamble, component-declaration ;

```

```
file-preamble = { file-directive } ;
```

(* The file preamble is the only syntactic position for language-owned DIRECTIVE/PRAGMA metadata. It is parsed before the primary top-level declaration (or before the first non-metadata entry item in src/appl.fol).

Directives are recorded on the source-file/top-level semantic context; they

do not introduce lexical names, scopes, or symbol-table entries. A directive

may affect the file's primary declaration when its semantic contract says so, without becoming a declaration annotation or lexical member.

File directives and pragmas use the generic metadata application shape. The zero-width category guard consults the predefined built-in metadata registry. Unknown @co.* names fail during parsing; unknown fields of a known

form remain collectable/preservable. *)

```
file-directive = import-directive
                | alias-directive
                | use-directive
                | dynamic-runtime-directive
                | dynamic-dispatch-directive
                | other-file-metadata-directive ;
```

```
other-file-metadata-directive =
    annotation, file-directive-category-guard ;
```

```
file-directive-category-guard =
```

? zero-width parser condition: the just-parsed metadata name is registered

as a built-in DIRECTIVE or PRAGMA, is not one of the specifically guarded

directive forms above, and is permitted in this file context; custom annotation/decorator applications are not standalone file directives

? ;

```
entry-item = type-declaration
            | bare-function-pattern-clause
            | capturing-function-pattern-clause
            | entry-statement ;
```

```
entry-statement = variable-declaration
```

```
| inferred-variable-declaration  
| grouped-variable-declaration  
| multiple-assignment-statement  
| expression-statement  
| empty-statement ;
```

```
primary-declaration = struct-declaration  
                    | cstruct-declaration  
                    | enum-declaration  
                    | union-declaration  
                    | class-declaration  
                    | trait-declaration  
                    | mixin-declaration  
                    | interface-declaration  
                    | signature-declaration  
                    | module-declaration  
                    | typeclass-declaration  
                    | object-declaration  
                    | instance-declaration  
                    | matcher-instance-declaration  
                    | extension-declaration ;
```

```
(* =====  
*)
```

```
(* 2. Directives, annotations, and metadata *)
```

```
(* =====  
*)
```

```
annotations = { declaration-metadata } ;
```

```
declaration-metadata = annotation, declaration-metadata-category-guard ;
```

```
declaration-metadata-category-guard =
```

```
  ? zero-width parser condition: for a built-in co.* metadata name, the  
    registry category is ANNOTATION or DECORATOR; DIRECTIVE and PRAGMA  
are  
    rejected in declaration/member/block annotation positions.  
co.dap.onEffect is also rejected here because its only legal parser  
path  
is  
    is on-effect-call-metadata before a call expression. co.dap.effects  
    accepted here only when the annotated node is callable; that target
```

```

check
    is semantic. A non-co.* metadata name is accepted here as custom
    annotation/decorator syntax and is resolved later through the
ordinary
    symbol table ? ;

(* Generic application shape. Sharing this surface syntax does not grant
    placement permission: the surrounding category guard decides whether the
    parsed metadata application is legal in the current syntactic position.
    The complete qualified name is classified
    immediately after it is read:
    - co.* -> must match builtin-metadata-name; otherwise parser error.
    - non-co.* -> syntactically accepted only as a custom
annotation/decorator
    application and later resolved through the ordinary symbol table.
    Field names are intentionally NOT closed by this registry. Every field
of a
    recognized form is parsed and preserved; frontend-known fields may
    receive
    additional validation, while unknown/unhandled fields remain available
to
    later stages. *)
annotation = "@", qualified-name,
    [ "(", [ annotation-argument-list ], ")" ] ;

builtin-metadata-name-check =
    ? parser validation applied immediately after annotation qualified-
name:
    when the name begins with co.*, the complete name must match
    builtin-metadata-name (or a registered reserved/future entry) and an
    unregistered co.* name is a parse error; a non-co.* name is collected
as
    custom metadata and must later resolve through the symbol table to a
    user-defined annotation or decorator ? ;

builtin-metadata-name = builtin-pragma-name
    | builtin-directive-name
    | builtin-annotation-name
    | builtin-decorator-name ;

builtin-pragma-name = "co.pdap.threadpool"
    | "co.pdap.schedularpool" ;

builtin-directive-name = "co.ddap.import"
    | "co.ddap.dynamicruntime"

```

```
| "co.ddap.use"  
| "co.ddap.alias"  
| "co.ddap.dynamicdispatch"  
| "co.ddap.overload" ;
```

```
builtin-annotation-name = "co.dap.template"  
| "co.dap.macro"  
| "co.dap.operator"  
| "co.dap.annotation"  
| "co.dap.library"  
| "co.dap.module"  
| "co.dap.native"  
| "co.dap.class"  
| "co.dap.static"  
| "co.dap.instance"  
| "co.dap.object"  
| "co.dap.inline"  
| "co.dap.ctfe"  
| "co.dap.friend"  
| "co.dap.sealed"  
| "co.dap.extension"  
| "co.dap.override"  
| "co.dap.implement"  
| "co.dap.virtual"  
| "co.dap.abstract"  
| "co.dap.delegate"  
| "co.dap.dynamicscope"  
| "co.dap.lexicalscope"  
| "co.dap.staticscope"  
| "co.dap.mixedscope"  
| "co.dap.typeclass"  
| "co.dap.matcher"  
| "co.dap.constructor"  
| "co.dap.oops"  
| "co.dap.extends"  
| "co.dap.hokrlt"  
| "co.dap.indexer"  
| "co.dap.generic"  
| "co.dap.comptime"  
| "co.dap.typefromvalue"  
| "co.dap.local"  
| "co.dap.private"  
| "co.dap.public"  
| "co.dap.compose"  
| "co.dap.guard"
```

```
| "co.dap.package"  
| "co.dap.protected"  
| "co.dap.internal"  
| "co.dap.export"  
| "co.dap.eager"  
| "co.dap.lazy"  
| "co.dap.packed"  
| "co.dap.declare"  
| "co.dap.implementation"  
| "co.dap.simd"  
| "co.dap.reflection"  
| "co.dap.mop"  
| "co.dap.nested"  
| "co.dap.inner"  
| "co.dap.final"  
| "co.dap.const"  
| "co.dap.decorator"  
| "co.dap.specialize"  
| "co.dap.symbol" ;
```

```
builtin-decorator-name = "co.dap.before"  
| "co.dap.after"  
| "co.dap.around"  
| "co.dap.effects"  
| "co.dap.onEffect"  
| "co.dap.defer"  
| "co.dap.callable"  
| "co.dap.executionmodel" ;
```

```
annotation-argument-list = annotation-argument,  
                           { ",", annotation-argument }, [ ",", ] ;
```

```
annotation-argument = [ annotation-key, annotation-binder ], annotation-  
value ;
```

```
annotation-binder = declarative-attribute-binder ;
```

```
declarative-attribute-binder = "=" ;
```

```
annotation-key = annotation-key-segment,  
                { "-", annotation-key-segment }  
                | qualified-name ;
```

```
annotation-key-segment = identifier | "for" ;
```



```

annotation-value = literal
                  | qualified-name
                  | annotation-declaration-reference
                  | annotation-list
                  | annotation-map
                  | annotation-arrow-pair ;

annotation-list = "[" , [ annotation-value ,
                          { "," , annotation-value } , [ "," ] ] , "]" ;

annotation-map = "{", [ annotation-map-entry ,
                        { ",", annotation-map-entry } , [ "," ] ] , "}" ;

annotation-map-entry = annotation-map-key , annotation-binder ,
                      annotation-value ;

annotation-map-key = annotation-key | qualified-name ;

annotation-arrow-pair = string-literal , "=>" , string-literal ;

annotation-declaration-reference = qualified-name , "(",
                                  [ annotation-reference-type ,
                                    { ",", annotation-reference-type } ] ,
                                  ")",
                                  "->" , "(",
                                  [ annotation-reference-type ,
                                    { ",", annotation-reference-type } ] ,
                                  ")" ;

annotation-reference-type = qualified-name ;

(* These productions are contextual views over annotation. They
   deliberately do
       not close the field list: recognized fields may be validated by the
   frontend
       while additional fields of the same known form remain parsed/preserved.
   *)
import-directive = annotation , import-directive-guard ;

import-directive-guard =
    ? zero-width parser condition: metadata name is exactly co.ddap.import;
    known target fields are package, library, component and as; semantic
    validation requires exactly one of package/library/component, but
    additional unhandled fields remain preserved ? ;

```

```

alias-directive = annotation, alias-directive-guard ;

alias-directive-guard =
    ? zero-width parser condition: metadata name is exactly co.ddap.alias ?
;

use-directive = annotation, use-directive-guard ;

use-directive-guard =
    ? zero-width parser condition: metadata name is exactly co.ddap.use ? ;

dynamic-runtime-directive = annotation, dynamic-runtime-directive-guard ;

dynamic-runtime-directive-guard =
    ? zero-width parser condition: metadata name is exactly
        co.ddap.dynamicruntime ? ;

dynamic-dispatch-directive = annotation, dynamic-dispatch-directive-guard ;

dynamic-dispatch-directive-guard =
    ? zero-width parser condition: metadata name is exactly
        co.ddap.dynamicdispatch ? ;

(* =====
*)

(* 3. Names and references *)

(* =====
*)

filename-derived-name = "_" ;

qualified-name = ( identifier | "co" ), { ".", identifier } ;

declaration-reference = qualified-function-reference | qualified-name ;

qualified-function-reference = qualified-name, "(", [ type-list ], ")",
    declaration-return-type-clause ;

lifecycle-declaration-name = "@@new" | "@@init" ;

lifecycle-invocation-name = "new" | "init" ;

```

```

special-binding = result-binding | recursive-binding ;

this-receiver-expression = "this", this-receiver-context-guard,
                           ordinary-relationship-selector-exclusion-guard ;

relationship-selector-expression =
    "this", this-receiver-context-guard,
    "->", relationship-arrow-guard, relationship-category,
    "[", relationship-type-name, "]",
    relationship-selector-guard,
    compiler-owned-property-call-exclusion-guard ;

relationship-category = "classes"
                      | "mixins"
                      | "traits"
                      | "interfaces" ;

relationship-arrow-guard =
    ? zero-width contextual condition: "->" has relationship-selection
    meaning
        only when its left operand is the hard-reserved this token and its
    right
        token is a language-defined relationship name; value->member is
    invalid,
        while this.member remains ordinary member access ? ;

compiler-owned-this-selector-expression =
    "this", "->", compiler-owned-this-selector-name,
    compiler-owned-this-selector-guard,
    compiler-owned-property-call-exclusion-guard ;

compiler-owned-this-selector-name =
    "object" | "class" | "module" | "kind" | "type" | "struct"
    | "instance" | "callee" | "args" | "params" | "results" |
    "associatedtype"
    | "owner" | "caller" | "fallthrough" | "yield" | "builtins" ;

compiler-owned-this-selector-guard =
    ? zero-width contextual and semantic condition: the selected compiler-
    owned
        property or operation is available in the enclosing executable
    context;
        these names are contextual after this-> rather than globally
    reserved;

```

this->args cannot implicitly capture an enclosing invocation's arguments
 when the nearest enclosing callable is annotated @co.dap.defer;
 values
 needed by that deferred callable must be passed explicitly at
 registration
 and received through its declared parameters ? ;

compiler-owned-property-call-exclusion-guard =
 ? zero-width syntactic condition: a compiler-owned this-> attribute is
 a
 property and cannot be followed immediately by call-suffix. Indexing
 that
 belongs to a keyed relationship property is part of its primary form;
 ordinary member-suffix, lifecycle-call-suffix where otherwise
 permitted,
 and a later call of an ordinary member remain available ? ;

relationship-type-name = qualified-name ;

relationship-selector-guard =
 ? zero-width contextual and semantic condition:

- the occurrence is inside the applicable class context;
- the selected category maps exactly to the ordered direct

 relationship

- list with the same name in @co.dap.oops;
- only directly declared relationships are present; transitive relationships are not appended;
- relationship-type-name is a compile-time declaration reference

 written

- as a complete type name or through an applicable import alias;
- the written name resolves to one canonical declaration of the kind required by the selected category and that declaration occurs in

 the

- corresponding direct relationship list;
- a numeric, string, runtime expression, computed expression, co.lang.type value, unresolved name, ambiguous name, wrong-kind

 name,

- absent direct relationship, or missing key is invalid;
- classes, mixins, traits, and interfaces are not standalone runtime values or collections;
- only classes selects a class-parent lifecycle branch;
- mixins and traits may qualify a composed implementation branch;
- interfaces produces an interface type/view and ordinary interface dispatch, not an implementation-branch selection;

- whenever a complete `this->classes`, `this->mixins`, `this->traits`, or `this->interfaces` prefix occurs in an applicable class context, this specialized production is selected before ordinary `qualified-name/member/index postfix parsing` ? ;

`parent-selector-expression =`

```
"this", this-receiver-context-guard,
( "->", relationship-arrow-guard, ( "parent" | "super" )
| "->", relationship-arrow-guard, "parents", "[", relationship-type-
name, "]" ),
direct-parent-selector-guard,
compiler-owned-property-call-exclusion-guard ;
```

`direct-parent-selector-guard =`

? zero-width contextual and semantic condition:

- the occurrence is inside the applicable class context;
- direct class parents are selected in their declared `classes=[...]` order;
- singular `this->parent` and `this->super` select the first direct class parent;
- plural `this->parents[Type]` is identical to `this->classes[Type]`;
- only plural `this->parents` is keyed, and it always requires a

`compile-time`

`relationship-type-name`;

- the type name must resolve canonically to a direct class parent;
- `this->parent[Type]`, `this->super[Type]`, numeric keys, string keys,

`computed keys, runtime`

`co.lang.type` values, and unkeyed `this->parents` are invalid;

- this is a compile-time branch selector, not runtime indexing;
- repeated canonical ancestors use one virtual-base identity;
- whenever a complete `this->parent`, `this->super`, or `this->parents`

prefix occurs in an

```
applicable class context, this
specialized production is selected before ordinary
qualified-name/member/index postfix parsing ? ;
```

`ordinary-relationship-selector-exclusion-guard =`

? zero-width parser lookahead condition: the immediately following tokens

```
are none of the complete contextual prefixes "->", "classes", "->",
"mixins",
"->", "traits", "->", "interfaces", "->", "parent", "->", "super", or
"->", "parents"
in a context where relationship-selector-expression or
parent-selector-expression applies ? ;
```

```

this-receiver-context-guard =
    ? zero-width contextual condition: this occurrence denotes a receiver
in
    one of the following contexts:
    - an instance method of co.lang.class, where it denotes the current
      instance;
    - a co.dap.class method of co.lang.class, where it denotes the
current
      class/type;
    - a developer lifecycle declaration admitted by the lifecycle
      customization guard, where @@new supplies the class/type receiver
and
      @@init supplies the instance receiver;
    - an instance or co.dap.class method of a target-bound
co.lang.extension,
      where it denotes respectively an instance of fortype or the fortype
      class/type; or
    - a concrete/default method of co.lang.trait or co.lang.mixin, where
it
      denotes the current composed instance; or
    - a function of co.lang.object, where it denotes the current
singleton
      object.
    A co.dap.static method and an ordinary free, module, signature,
interface,
    typeclass, instance, trait, mixin, or unit function do not acquire a
    receiver through this production. The separately parsed this =>,
    this ^=>, this ->>, and this ->| control forms do not require a
    receiver ? ;

refinement-candidate = "_", refinement-candidate-guard ;

refinement-candidate-guard =
    ? zero-width contextual condition: this occurrence is inside the
predicate
    of the co.lang.refinementType declaration currently being parsed ? ;

recursive-binding = "$" ;

result-binding = "$", digit, { digit } ;

wildcard = "_" ;

```

```

(* =====
*)

(* 4. Type syntax *)

(* =====
*)

type-expression = union-type-expression ;

polymorphic-type-expression = "forall", "(", type-parameter-list, ")", ".",
                             type-expression,
                             forall-context-guard ;

forall-context-guard =
    ? zero-width contextual condition: this occurrence is the value of a
      co.lang.type declaration; every other direct use of forall as a type
      binder is invalid ? ;

type-declaration-value = polymorphic-type-expression | type-expression ;

type-parameter-list = identifier, { ",", identifier } ;

union-type-expression = arrow-type-expression,
                       { "|", arrow-type-expression } ;

arrow-type-expression = type-postfix-expression,
                       [ "->", arrow-type-tail ]
                       | "(", [ function-type-parameter,
                               { ",", function-type-parameter } ],
                               ")", "->", arrow-type-tail ;

arrow-type-tail = type-derivation
                 | parenthesized-type-list
                 | type-expression ;

type-postfix-expression = type-atom, { type-argument-list } ;

type-atom = qualified-name
           | "(", type-expression, ")" ;

type-argument-list = "(", [ type-or-value-argument,
                           { ",", type-or-value-argument } ], ")" ;

type-or-value-argument = type-expression | dependent-index | annotation-

```

list ;

dependent-index = integer-literal | qualified-name ;

type-list = type-expression, { ",", type-expression } ;

(* A parenthesized postfix on a type atom is the uniform type-application form.

Symbol resolution classifies its target as a built-in or annotation-declared

generic, a parameterized `co.lang.type`, a typeclass, or an abstract type parameter. Arguments bind positionally in declaration order, and non-

type

values use the same argument slots:

```
co.core.Map(co.lang.string, co.lang.int)
co.core.Array(2, co.lang.int, [2,4])
LinkedList(co.lang.int)
```

Named bindings are deliberately absent. Parentheses after a generic declaration bind that declaration's parameters by position, while an arrow

tail may carry the derivation-attribute-list described by "Type Application

and Arrow Tails". *)

parenthesized-type-list = "(", [return-item-list], ")" ;

type-derivation = "(", derivation-specification, ")" ;

derivation-specification = pointer-specification
| array-specification
| reference-specification
| range-type-specification
| slice-type-specification
| thunk-type-specification
| address-type-specification
| derivation-attribute-list ;

pointer-specification = pointer-stars,
[",", derivation-attribute-list] ;

pointer-stars = ? one contiguous symbolic run consisting only of one or more "*" characters; its length is the pointer degree ? ;

reference-specification = ("&" | "&&" | "~"),


```

[ ",", derivation-attribute-list ] ;

address-type-specification = "@", [ ",", derivation-attribute-list ] ;

thunk-type-specification = "^", [ ",", derivation-attribute-list ] ;

slice-type-specification = "[:]", [ ",", derivation-attribute-list ] ;

range-type-specification = "..", [ ",", derivation-attribute-list ] ;

array-specification = array-dimension-group, { array-dimension-group },
    [ ",", derivation-attribute-list ] ;

array-dimension-group = "[", array-dimension-content, "]" ;

array-dimension-content = [ array-dimension ], { ",", [ array-dimension ] }
;

array-dimension = "... " | "." | dependent-index ;

(* Attributes carried by a type derivation. Generic type applications
instead
    use the positional type-argument-list production. *)
derivation-attribute-list = derivation-attribute,
    { ",", derivation-attribute } ;

derivation-attribute = annotation-key, "=", annotation-value ;

return-type-clause = "->", "(", [ return-item-list ], ")" ;

return-item-list = return-item, { ",", return-item } ;

return-item = type-expression ;

type-use = qualified-name, { type-argument-list } ;

type-use-argument = type-use
    | dependent-index
    | identifier, "=", ( type-use | dependent-index ) ;

declaration-return-type-clause =
    "->", "(", [ declaration-return-item,
        { ",", declaration-return-item } ], ")" ;

declaration-return-item = type-use ;

```

```

(* =====
*)

(* 5. Common declaration components *)

(* =====
*)

generic-parameter-clause = "(", generic-parameter,
                           { ",", generic-parameter }, ")" ;

generic-parameter = identifier, [ generic-arity-clause ] ;

generic-arity-clause = "(", generic-arity-slot,
                      { ",", generic-arity-slot }, ")" ;

generic-arity-slot = "_" | identifier ;

kind-options = "->", "(", [ annotation-argument-list ], ")" ;

field-declaration = annotations, identifier, type-use,
                   [ "=", expression ], statement-end ;

embedded-field-declaration = annotations, type-use, statement-end ;

value-specification = annotations, identifier, type-use, statement-end ;

variable-declaration = annotations, typed-variable-declarator,
                       { ",", typed-variable-declarator }, statement-end ;

typed-variable-declarator = identifier, type-use,
                           [ "=", expression ] ;

inferred-variable-declaration = annotations, inferred-variable-declarator,
                                { ",", inferred-variable-declarator },
                                statement-end ;

inferred-variable-declarator = identifier, definition-operator, expression
;

definition-operator = ( ":@" | "?=" ),
                     multi-symbol-infix-operator-boundary-guard ;

```



```

        | "co.lang.dependentType"
        | "co.lang.kind" ;

refinement-type-declaration =
    annotations, identifier, "co.lang.refinementType", "=",
    refinement-type-expression, statement-end ;

refinement-type-expression =
    "(", type-expression, ")", ".where", "(", expression, ")" ;

predicate-type-declaration =
    annotations, identifier, "co.lang.predicateType", "=",
    predicate-type-expression, statement-end ;

predicate-type-expression =
    "co.lang.type", ".where", "(", predicate-type-binder, "=>",
    expression, ")" ;

predicate-type-binder = identifier ;

(* =====
*)

(* 7. Containers, contracts, instances, and component surfaces *)

(* =====
*)

unit-declaration = annotations, filename-derived-name,
    "co.lang.unit", "=", unit-body ;

unit-body = "{", { unit-member }, body-close ;

unit-member = function-declaration
    | closure-declaration
    | data-declaration
    | type-declaration
    | function-object-declaration
    | delegate-declaration
    | extern-variable-declaration ;

extern-variable-declaration =
    "@co.dap.declare", "(", "type", "=", "extern", ")",
    identifier, type-expression,
    statement-end ;

```

```

class-declaration = annotations, filename-derived-name,
                    "co.lang.class", [ kind-options ], "=", class-body,
                    class-lifecycle-capability-guard ;

class-lifecycle-capability-guard =
    ? zero-width semantic condition:
        - every co.lang.class has compiler-owned inherited lifecycle
implementations
        for @@new and @@init as non-public/protected lifecycle machinery;
        - lifecycle is an optional field of the enclosing class's
co.dap.generic
        metadata application, not a separate annotation;
        - for a generic co.lang.class, lifecycle=true grants source
permission to
        override or overload the existing compiler-owned @@new / @@init
family;
        lifecycle absent or lifecycle=false forbids developer lifecycle
        customization but does not remove the inherited compiler lifecycle;
        - if lifecycle is present on a generic struct, generic function, or
generic
        method, that field is accepted but is not considered for lifecycle
        semantics;
        - if any lifecycle-method-declaration occurs in the class body, the
class
        must carry valid co.dap.generic(types=[...], lifecycle=true)
metadata;
        - lifecycle=true does not itself expose ::new(...) or ::init(...);
        - no filesystem/package/component placement condition is imposed by
the
        lifecycle facility itself ? ;

class-body = "{", { class-member }, body-close ;

class-member = class-instance-field-declaration
               | function-declaration
               | type-declaration
               | lifecycle-method-declaration ;

class-instance-field-declaration =
    pure-field-declaration, class-instance-field-policy-guard ;

class-instance-field-policy-guard =
    ? zero-width semantic condition:
        - the declaration is ordinary mutable per-instance storage;

```

- co.dap.const and co.dap.final are forbidden on the field;
- field-level Shared, locking, CopyOnWrite, static/class/global storage, ownership, atomicity, and inline initialization are forbidden;
- an object-graph policy may later target the complete class instance,

never this selected field as an independent class-storage root ? ;

trait-declaration = annotations, filename-derived-name,
 "co.lang.trait", "=", trait-body ;

trait-body = "{", { trait-member }, body-close ;

trait-member = function-declaration, trait-member-guard ;

trait-member-guard =
 ? zero-width semantic condition:
 - a trait carries no instance state, so fields and other value-bearing members are not permitted;

- a function may be abstract/bodyless, may provide a default implementation, or may be declared virtual when it has the implementation required by the virtual-method rules ? ;

mixin-declaration = annotations, filename-derived-name,
 "co.lang.mixin", "=", mixin-body ;

mixin-body = "{", { mixin-member }, body-close ;

mixin-member = class-instance-field-declaration
 | function-declaration
 | type-declaration ;

lifecycle-method-declaration = annotations, lifecycle-declaration-name,
 parameter-list, [declaration-return-type-clause],
 function-definition,
 lifecycle-declaration-context-guard ;

lifecycle-declaration-context-guard =
 ? zero-width semantic condition:
 - the enclosing declaration is a co.lang.class carrying valid co.dap.generic metadata with an explicit types=[...] list and lifecycle=true;

- the source declaration is an override of an existing compiler

lifecycle

signature or an overload that adds another signature to the same language-owned lifecycle name; it never creates a new lifecycle

name;

- a non-generic class, a generic class with lifecycle absent/false, or any

non-class declaration cannot source-declare @@new or @@init;

- each admitted lifecycle implementation retains its ordinary declared

accessibility and other applicable method metadata;

- lifecycle=true on a generic struct, function, or method does not change

this class-only customization rule ? ;

interface-declaration = annotations, filename-derived-name,

"co.lang.interface", "=", interface-body ;

interface-body = "{", { function-specification | type-declaration }, body-close ;

signature-declaration = annotations, filename-derived-name,

"co.lang.signature", "=", signature-body ;

signature-body = "{", { signature-member }, body-close ;

signature-member = value-specification

| function-specification

| signature-type-component

| associated-type-requirement

| type-declaration ;

signature-type-component = annotations, identifier,

"co.lang.type",

["=", type-expression], statement-end ;

(* `Name co.lang.type;` is an ordinary abstract type component and does not imply genericity. Every co.lang.associatedType component must name a type

argument used by at least one bodyless co.lang.type component of the same

signature. A matching module must fulfill that alias by instantiating a generic container with the associated type; otherwise the associated-type

declaration is invalid. *)

associated-type-requirement = annotations, identifier,


```

        "co.lang.associatedType", statement-end ;

module-declaration = annotations, filename-derived-name,
                    "co.lang.module", [ kind-options ], "=", module-body ;

module-body = "{", { module-member }, body-close ;

module-member = variable-declaration
               | inferred-variable-declaration
               | function-declaration
               | type-declaration
               | associated-type-binding ;

associated-type-binding = annotations, identifier,
                        "co.lang.associatedType", "=", type-expression,
                        statement-end ;

(* Each module co.lang.associatedType binding corresponds by name to a
   requirement in the matched signature. The module must also contain the
   corresponding required co.lang.type alias, and that alias must
   instantiate
   a generic container using the associated type. Missing, unrelated, or
   unused
   bindings are compiler errors. Typeclass and instance higher-kinded forms
   are
   unaffected. *)

component-declaration = annotations, filename-derived-name,
                        "co.lang.component", "=", component-body,
                        component-surface-context-guard ;

component-body = "{", { component-member }, body-close ;

component-surface-context-guard =
    ? zero-width structural validation supplied by filesystem role:
      src/component.fol is either a projected standalone library annotated
with
      co.dap.library (application when type is omitted; otherwise
application,
      native, or dynamicvmrt) or a packaged standalone component containing
the
      co.dap.export package selector, never both; components/application,
      components/native, and components/dynamicvmrt expose only their
permitted
      projected boundary declarations and carry no co.dap.library

```

```

annotation;
    components/packaged uses the export selector; components/operators
    contains operator declarations only; the operators component is
permitted
    only for an executable application or a projected application library
? ;

component-member = import-directive
                  | surface-struct-declaration
                  | surface-cstruct-declaration
                  | type-declaration
                  | function-declaration
                  | component-export-selector
                  | operator-declaration ;

component-export-selector = annotation, component-export-selector-guard ;

component-export-selector-guard =
    ? zero-width structural/context condition: metadata name is exactly
      co.dap.export and this standalone component-body entry is used only
for
    packaged src/component.fol or components/packaged/component.fol; the
    parser collects the complete field payload, while semantic validation
    checks the packages selector and project/component context ? ;

extension-declaration = annotations, filename-derived-name,
                        "co.lang.extension", extension-target-options, "=",
                        extension-body ;

extension-target-options =
    "->", "(", "fortype", "=", type-expression, ")" ;

extension-body = "{", { extension-member }, body-close ;

extension-member = field-declaration | function-declaration ;

object-declaration = annotations, filename-derived-name,
                    "co.lang.object", object-association-options,
                    "=", object-body, object-association-guard ;

object-association-options =
    "->", "(", "for", "=", object-association-targets, ")" ;

object-association-targets =

```

```
qualified-name  
| "[" , qualified-name , { " , " , qualified-name } , [ " , " ] , "]" ;
```

object-association-guard =

? zero-width semantic condition:

- the scalar target resolves to exactly one annotation declaration or exactly one co.lang.class declaration;
- the list form is non-empty, contains only co.lang.class declaration references, contains no duplicate canonical target, and cannot mix annotation and class declaration kinds;
- the object name follows the ordinary filename-derived primary-name rule and is independent of every target name; no companion filename

or

companion-owner relationship is created;

- the declaration denotes one singleton object whose association is explicit, non-owning, non-structural, and non-transitive;
- association does not promote fields or methods, participate in

class

inheritance/dynamic dispatch, add a managed-reference graph edge,

or

propagate construction, copying, freezing, sharing, locking,
CopyOnWrite behavior, or destruction between either side ? ;

```
object-body = "{", { field-declaration | function-declaration }, body-close  
;
```

```
instance-declaration = annotations, filename-derived-name,  
                        "co.lang.instance", [ kind-options ], "=",  
                        instance-body ;
```

```
instance-body = "{", { function-declaration | variable-declaration }, body-  
close ;
```

```
typeclass-declaration = typeclass-annotations, filename-derived-name,  
                        "co.lang.typeclass", "=", contract-body ;
```

```
typeclass-annotations = { annotation }, typeclass-annotation, { annotation  
} ;
```

```
typeclass-annotation = "@co.dap.typeclass", "(",  
                        typeclass-option, { " , " , typeclass-option } , ")" ;
```

```
typeclass-option = typeclass-kind-option  
                  | typeclass-shape-option  
                  | typeclass-aliases-option ;
```

```

typeclass-kind-option = "kind", annotation-binder, identifier ;
typeclass-shape-option = "shape", annotation-binder, typeclass-shape ;
typeclass-aliases-option = "aliases", annotation-binder,
    "[" , typeclass-alias,
    { " , " , typeclass-alias }, "]" ;
typeclass-alias = "{", "name", annotation-binder, identifier, " , " ,
    "type", annotation-binder, type-expression, "}" ;

(* Exactly one typeclass-annotation is required. Kind and shape are
required;
    aliases is optional, option order is immaterial, and duplicate options
are
    invalid. Its shape is metadata-owned; no parameter clause may occur
after
    filename-derived-name. *)

typeclass-shape = generic-parameter-clause ;

matcher-instance-declaration = annotations, filename-derived-name,
    "co.lang.matcher", matcher-options, "=",
    matcher-body ;

matcher-options = "->", "(", "type", annotation-binder, type-expression,
    ")" ;

matcher-body = "{", { field-declaration | function-declaration }, body-
close ;

contract-body = "{", { function-specification | value-specification },
body-close ;

named-block-declaration = annotations, identifier,
    "co.lang.block", "=", block, body-closure-guard ;

(* =====
*)

(* 8. Functions and callable declarations *)

(* =====
*)

delegate-declaration = annotations, identifier,

```

```

        "co.lang.delegate", "=", function-type, statement-
end ;

function-object-declaration = annotations, identifier,
                             "co.lang.function", "=",
                             function-object-binding ;

function-object-binding = expression, statement-end ;

function-declaration = annotations,
                      [ receiver-clause, receiver-clause-context-guard ],
                      function-name,
                      function-parameter-lists,
                      [ declaration-return-type-clause ], function-
binding,
                      function-shaped-declaration-classification-guard ;

function-parameter-lists = parameter-list
                          | nonempty-parameter-list, nonempty-parameter-
list,
                          { nonempty-parameter-list } ;

function-shaped-declaration-classification-guard =
    ? zero-width semantic classification:
        when attached directly to this function-shaped declaration, the
following
    classifying metadata forms select specialized AST kinds:
        co.dap.generic          -> GenericFunctionDecl
        co.dap.decorator        -> DecoratorDecl
        co.dap.extension        -> ExtensionMethodDecl
        co.dap.macro            -> MacroDecl
        co.dap.template         -> TemplateDecl
        co.dap.native           -> NativeFunctionDecl
        co.dap.executionmodel   -> ExecutionModelFunctionDecl
        co.dap.operator         -> OperatorOverloadDecl
        co.dap.indexer          -> IndexerDecl
    these classifying forms are mutually exclusive on one function-shaped
declaration except for the deliberate co.dap.operator +
co.dap.extension
composition. In that combination the declaration remains an
OperatorOverloadDecl and co.dap.extension supplies the existing
target/owner;
    it does not create a second AST declaration kind. co.dap.operator and
co.dap.generic are incompatible on the same declaration, with or
without

```

co.dap.extension: an operator declaration never introduces operator-level generic parameters. A generic enclosing class, struct, or extension target does not make the operator declaration generic; operator ownership is associated with the canonical owner declaration identity. Every other combination of two classifying forms is a compiler error. When none is present the declaration is an ordinary FunctionDecl; non-classifying metadata does not change its AST declaration kind. An ExecutionModelFunctionDecl must have exactly one co.lang.error-compatible result and must not carry co.dap.effects ? ;

function-name = identifier ;

receiver-clause = "(", (type-use
 | identifier, type-use), ")" ;

receiver-clause-context-guard =
 ? zero-width condition: an explicit receiver occurs only on a function declared directly in a validated <StructName>.comp.unit.fol companion;
 ordinary units and nested class, module, signature, interface, function,
 or executable-block contexts do not inherit receiver permission ? ;

parameter-list = "(", [parameter,
 { ",", parameter }], ")" ;

nonempty-parameter-list = "(", typed-parameter,
 { ",", typed-parameter }, ")" ;

parameter = typed-parameter | untyped-template-parameter ;

typed-parameter = ["..."], ["~"], identifier, ["?"],
 type-use, ["=", expression] ;

untyped-template-parameter = ["..."], ["~"], identifier, ["?"],
 ["=", expression],
 template-parameter-context-guard ;

template-parameter-context-guard =
 ? zero-width condition: the enclosing function-shaped declaration carries

built-in @co.dap.template metadata; ordinary functions and every other

callable category require an explicit parameter type ? ;

```
function-binding = function-definition
                  | function-delegation
                  | function-type-value-binding
                  | function-alias-binding
                  | statement-end ;
```

```
function-definition = "=", block, body-closure-guard ;
```

```
function-delegation = "=>>", expression,
                      { "=>>", expression }, statement-end ;
```

```
function-type-value-binding = "=", type-expression, statement-end ;
```

```
function-alias-binding = "=", non-block-expression, statement-end ;
```

```
function-specification = annotations, function-name,
                          parameter-list, { parameter-list },
                          [ declaration-return-type-clause ], statement-end
;

```

```
function-type = "(", [ function-type-parameter,
                      { ",", function-type-parameter } ], ")",
                return-type-clause ;
```

```
function-type-parameter = type-expression
                          | identifier, type-expression ;
```

```
anonymous-function-expression = parameter-list, declaration-return-type-
clause, block,
                                anonymous-function-binding-context-guard,
                                anonymous-function-generic-context-guard ;
```

```
anonymous-function-binding-context-guard =
    ? zero-width condition: the anonymous-function expression is the root
value
    of a typed, inferred, let, field or co.lang.function binding
initializer;
    an optional postfix invocation may immediately follow it, but the
literal
    cannot be a standalone statement, argument, return value or nested
operand ? ;
```

```

anonymous-function-generic-context-guard =
    ? zero-width condition: the anonymous function declares no generic
names;
    any generic names in its signature belong to an enclosing
    @co.dap.generic struct, class, function or method ? ;

lambda-expression = "|", [ lambda-parameter,
                        { ",", lambda-parameter } ], "|", "=>",
                        ( expression | block ) ;

lambda-parameter = identifier, [ type-use ] ;

closure-declaration = annotations, identifier, "=", parameter-list,
                      { parameter-list }, "=>>", expression,
                      statement-end ;

local-function-declaration = annotations, function-name, parameter-list,
                              { parameter-list }, declaration-return-type-
clause,
                              function-definition,
                              local-function-nongeneric-guard ;

local-function-nongeneric-guard =
    ? zero-width condition: an inner/local function must not carry
    co.dap.generic and cannot introduce generic parameters; it may use
types
    already visible from an enclosing generic class or function ? ;

(* =====
*)

(* 9. Function-pattern clauses and patterns *)

(* =====
*)

bare-function-pattern-clause = annotations, identifier, pattern-parameter-
list,
                              [ where-clause ], "=>", pattern-result ;

capturing-function-pattern-clause = annotations, "let", identifier,
                                    pattern-parameter-list,
                                    [ where-clause ], "=", pattern-result ;

```



```

pattern-parameter-list = "(" , [ pattern ,
                                { "," , pattern } ] , ")" ;

where-clause = ".where" , "(" , expression , ")" ;

pattern-result = block , body-closure-guard
                | non-block-expression , statement-end ;

pattern = wildcard
        | literal-pattern
        | binding-pattern
        | constructor-pattern
        | record-pattern
        | tuple-pattern
        | qualified-name ;

literal-pattern = literal
                | ( "+" | "-" ) ,
                  ( integer-literal | floating-literal ) ;

binding-pattern = identifier ;

constructor-pattern = qualified-name , "(" , constructor-pattern-item ,
                        { "," , constructor-pattern-item } , ")" ,
                        enum-state-pattern-guard ;

constructor-pattern-item = [ identifier , "=" ] , pattern ;

enum-state-pattern-guard =
    ? zero-width semantic condition: when qualified-name resolves to a
      parameterized co.lang.enum state, every item has identifier "=",
every
    declared state parameter occurs exactly once, and no unknown or
duplicate
    label occurs; when it resolves to a zero-parameter enum state the
bare
    qualified-name form is required instead; co.lang.variants and
co.lang.data
    constructors retain their separately specified positional patterns ?
;

record-pattern = qualified-name , "{" , [ record-pattern-field ,
                                         { "," , record-pattern-field } ] ,
                "}" ;

```

```

record-pattern-field = identifier, [ ":", pattern ] ;

tuple-pattern = "(", pattern, ",", pattern,
                { ",", pattern }, ")" ;

match-case = ".case", "(", match-case-body, ")" ;

match-case-body = pattern, [ ":", expression ], "=>",
                    ( expression | block ) ;

match-default = ".default", "(", ( expression | block ), ")" ;

(* =====
*)

(* 10. Statements, blocks, and termination *)

(* =====
*)

block = "{", { block-item }, [ block-tail-expression ], "}" ;

block-item = statement ;

block-tail-expression = expression ;

statement = named-block-declaration
            | type-declaration
            | variable-declaration
            | inferred-variable-declaration
            | grouped-variable-declaration
            | let-value-declaration
            | local-function-declaration
            | closure-declaration
            | multiple-assignment-statement
            | enclosing-callable-return-statement
            | return-statement
            | break-statement
            | continue-statement
            | lock-statement
            | labeled-block
            | labeled-loop-statement
            | expression-statement

```

```
| block-statement  
| empty-statement ;
```

```
grouped-variable-declaration = "(", typed-variable-declarator,  
                                { ",", typed-variable-declarator }, ")",  
                                statement-end ;
```

```
let-value-declaration = "let", identifier, [ type-expression ], "=",  
                        expression, statement-end ;
```

```
multiple-assignment-statement = assignment-target, ",", assignment-target,  
                                { ",", assignment-target }, "=",  
                                expression-list, statement-end ;
```

```
assignment-target = postfix-expression  
                  | tuple-assignment-target ;
```

```
tuple-assignment-target = "(", assignment-target, ",", assignment-target,  
                          { ",", assignment-target }, ")" ;
```

```
return-statement = "this", "=>", [ expression-list ], statement-end ;
```

```
enclosing-callable-return-statement =  
    "this", "^=>", [ expression-list ], statement-end,  
    enclosing-callable-return-guard ;
```

```
enclosing-callable-return-guard =  
    ? zero-width contextual and semantic condition:  
        - the occurrence is inside an anonymous block supplied as a call  
argument;  
        - no named function, method, anonymous function, function expression,  
or  
        lambda boundary occurs between the statement and that argument  
block;  
        - the statement terminates the anonymous block and its nearest  
lexically  
        enclosing callable, producing values for that callable;  
        - the values satisfy that callable's declared result positions;  
        - a direct callable-body occurrence, bare or named block occurrence,  
function-expression occurrence, and every ^^=> or multi-level  
spelling  
        is invalid ? ;
```

```
expression-statement = non-block-expression, statement-end ;
```

```

break-statement =
    "this", "->|", [ label-reference ], statement-end,
    break-target-guard ;

break-target-guard =
    ? zero-width semantic condition:
        - without label-reference, this ->| targets the ordinary nearest
          breakable control region;
        - with label-reference, the reference resolves only to an active
          enclosing
            labeled-block or labeled-loop-statement;
        - resolution is lexical and selects the innermost enclosing matching
          label;
        - the target is exited structurally; no arbitrary goto/address jump
          exists ? ;

continue-statement =
    "this", "->>", [ label-reference ], statement-end,
    continue-target-guard ;

continue-target-guard =
    ? zero-width semantic condition:
        - without label-reference, this ->> targets the ordinary nearest
          loop;
        - with label-reference, the reference must resolve to an active
          enclosing
            labeled-loop-statement;
        - a plain labeled-block is not a valid continue target;
        - resolution is lexical and selects the innermost enclosing matching
          label ? ;

labeled-block =
    label-declaration, ":", block, body-closure-guard ;

labeled-loop-statement =
    label-declaration, ":", expression-statement,
    labeled-loop-statement-guard ;

labeled-loop-statement-guard =
    ? zero-width semantic condition: the labeled expression-statement is a
      current-profile loop form whose outer control operation is
      .loop(...);
    labeling an arbitrary expression statement does not turn it into a
    loop ? ;

```

```

label-declaration = label-identifier ;

label-reference = label-identifier ;

empty-statement = ";" ;

expression-list = expression, { ",", expression } ;

statement-end = ";" ;

body-close = "}", body-closure-guard ;

body-closure-guard =
    ? zero-width condition: the next significant token is not ";", or there
    is no next token ? ;

block-statement = block, body-closure-guard ;

lock-statement = "lock", "(", expression, ")", block,
                body-closure-guard ;

non-block-expression = expression, non-block-expression-guard ;

non-block-expression-guard =
    ? zero-width condition: the complete source span is not admissible as
    an
        unparenthesized block production; when both block and another braced
        expression are possible, the block reading has priority ? ;

(* =====
*)

(* 11. Expressions and built-in operator precedence *)

(* =====
*)

expression = assignment-expression
            | extended-operator-expression ;

assignment-expression = logical-or-expression,
                        [ runtime-assignment-operator,
                          assignment-expression ] ;

```

```

runtime-assignment-operator = "="
                                | compound-assignment-operator ;

compound-assignment-operator = ( "+=" | "-=" | "*=" | "/=" | "%="
                                | "**=" | "&=" | "^=" | "|=" ),
                                multi-symbol-infix-operator-boundary-guard ;

constant-expression = ( logical-or-expression
                        | extended-operator-expression ),
                        ? zero-width condition: no runtime-assignment-
operator
                        occurs anywhere in this constant-expression subtree
                        ? ;

logical-or-expression = logical-and-expression,
                        { logical-or-operator, logical-and-expression } ;

logical-or-operator = "||", multi-symbol-infix-operator-boundary-guard ;

logical-and-expression = bitwise-or-expression,
                        { logical-and-operator, bitwise-or-expression } ;

logical-and-operator = "&&", multi-symbol-infix-operator-boundary-guard ;

bitwise-or-expression = bitwise-xor-expression,
                        { "|", bitwise-xor-expression } ;

bitwise-xor-expression = bitwise-and-expression,
                        { "^", bitwise-and-expression } ;

bitwise-and-expression = equality-expression,
                        { "&", equality-expression } ;

equality-expression = relational-expression,
                        [ equality-operator, relational-expression ] ;

equality-operator = ( "==" | "!=" ),
                    multi-symbol-infix-operator-boundary-guard ;

relational-expression = range-expression,
                        [ relational-operator, range-expression ] ;

relational-operator = "<" | ">" | multi-symbol-relational-operator ;

multi-symbol-relational-operator = ( "<=" | ">=" | "<:" | ">:" ),

```

```

                                multi-symbol-infix-operator-boundary-
guard ;

range-expression = additive-expression,
                    [ range-operator, [ additive-expression ] ]
                    | range-operator, additive-expression ;

range-operator = ( ".." | "<.." | "..<" | "<..<" ),
                  multi-symbol-range-operator-boundary-guard ;

additive-expression = predeclared-glyph-expression,
                      { additive-operator, predeclared-glyph-expression } ;

additive-operator = "+" | "-" ;

predeclared-glyph-expression = multiplicative-expression,
                                { predeclared-operator-glyph,
                                  multiplicative-expression } ;

multiplicative-expression = unary-expression,
                            { multiplicative-operator, unary-expression } ;

multiplicative-operator = "*" | "/" | "%" ;

unary-expression = { prefix-operator }, power-expression ;

prefix-operator = "+" | "-" | "!" ;

power-expression = ( effect-handled-call-expression | postfix-expression ),
                   [ power-operator, unary-expression ] ;

effect-handled-call-expression = on-effect-call-metadata,
                                 postfix-expression,
                                 effect-handled-call-context-guard ;

on-effect-call-metadata = annotation, on-effect-call-metadata-guard ;

on-effect-call-metadata-guard =
    ? zero-width parser condition: the complete metadata name just parsed
    is
        exactly co.dap.onEffect; co.dap.effects and every other metadata name
    are
        invalid in this expression-prefix position ? ;

effect-handled-call-context-guard =

```

? zero-width semantic condition: the decorated postfix-expression's outermost operation is an ordinary or lifecycle invocation. The metadata belongs to that one call AST node, not to calls nested in its receiver or argument expressions. Each effect key must resolve to a class satisfying co.lang.error and need not be present in the callee's open advertised emits set. For an ordinary callee every record requires exactly one resolution and may contain a non-empty handlers list. For a callee classified as ExecutionModelFunctionDecl, or a canonical submission call statically bound to such a target, every record requires a non-empty handlers list, forbids an explicit resolution, and receives implicit return_with_error. Retry captures the evaluated receiver and argument values once and reinvokes only an ordinary decorated call node ? ;

power-operator = "**", multi-symbol-infix-operator-boundary-guard ;

postfix-expression = primary-expression,
 { postfix-suffix | postfix-operator } ;

postfix-operator = "!" ;

postfix-suffix = call-suffix
 | index-suffix
 | member-suffix
 | lifecycle-call-suffix
 | match-suffix ;

call-suffix = "(", [argument-list], ")", enum-state-call-guard ;

enum-state-call-guard =
 ? zero-width semantic condition: when the call target resolves to a parameterized co.lang.enum state, every argument has identifier "=", every declared state parameter occurs exactly once, and no unknown or duplicate label occurs; a zero-parameter enum state is a bare value and cannot take a call suffix; ordinary functions and co.lang.variants or co.lang.data state functions retain their own argument rules ? ;

argument-list = argument, { ",", argument } ;

argument = ([identifier, "="], expression)
 | block


```
| lambda-expression  
| wildcard ;
```

```
index-suffix = "[", [ expression-list ], "]" ;
```

```
member-suffix = ".", ( member-identifier | "for" ) ;
```

(* Lifecycle members are invoked through a dedicated call form rather than ordinary member lookup. Every class has compiler-owned inherited `@@new` and

`@@init` lifecycle machinery, but those inherited implementations are not automatically source-callable through `::``. A generic class with `co.dap.generic(..., lifecycle=true)` may source-define lifecycle overrides or

overloads. Only matching developer-defined lifecycle candidates participate

in ordinary source `::`` lookup, and each candidate retains its declared accessibility. The lifecycle field is ignored for lifecycle semantics on generic structs/functions/methods. The call parentheses are part of the production so lifecycle members cannot be selected as first-class

ordinary

member values.

Current invocation names map to declaration names as follows:

```
new   -> @@new  
init  -> @@init
```

Future language-defined lifecycle names extend both the declaration-name and invocation-name sets; every lifecycle invocation still uses `::``.

Semantic validation checks receiver kind, developer customization, accessibility, overload resolution, and construction state. *)

```
lifecycle-call-suffix = lifecycle-invocation-marker, lifecycle-invocation-name,
```

```
    "(", [ argument-list ], ")",  
    lifecycle-call-context-guard ;
```

```
lifecycle-call-context-guard =
```

```
    ? zero-width semantic condition checked after receiver/type resolution:  
    - lifecycle invocation performs lookup only in source-defined
```

```
lifecycle
```

```
    override/overload candidates admitted for the resolved class;
```

```
    - such candidates can exist only on a generic co.lang.class carrying
```

```
valid
```

```
    co.dap.generic(types=[...], lifecycle=true) metadata;
```

```
    - the inherited compiler-provided lifecycle implementation is not an  
    automatically exposed ordinary-source ::` candidate;
```

```
    - overload resolution selects a matching developer-defined lifecycle
```

```

signature and ordinary accessibility rules must permit the caller
to
    reach that candidate;
- no matching accessible candidate is a compile-time error;
- ordinary methods named new/init remain unrelated and continue to
use
    ordinary '.' member lookup ? ;

(* value.to(TargetType) is therefore parsed by the existing member-suffix
plus
    call-suffix path; explicit conversion/cast syntax introduces no
dedicated
    cast production. Target-type validity is resolved semantically. *)

member-identifier =
    ? an identifier token other than "match"; "match" is routed through
    match-suffix so matcher-chain syntax has one parser path ? ;

(* No unprefixed composite literal exists. Lists, arrays, sets, maps,
records,
    and objects all use the explicit Type{...} construction production.
Blocks occur
    only in grammar positions that explicitly request a block; a bare block
is not
    a general primary expression. *)
primary-expression = literal
    | special-binding
    | relationship-selector-expression
    | parent-selector-expression
    | compiler-owned-this-selector-expression
    | this-receiver-expression
    | refinement-candidate
    | qualified-name
    | grouped-expression
    | tuple-expression
    | composite-construction
    | anonymous-class-expression
    | anonymous-function-expression
    | let-expression
    | comprehension-expression ;

grouped-expression = "(" , expression , ")" ;

tuple-expression = "(" , expression , ",", expression ,
    { ",", expression }, ")" ;

```

```

composite-construction = type-postfix-expression, "{",
                        [ construction-element,
                          { ",", construction-element }, [ ",", " ] ], "}",
                        composite-construction-guard ;

construction-element = expression, [ ":", expression ] ;

composite-construction-guard =
    ? zero-width semantic condition: the prefix resolves to a constructible
      concrete type; list, array and set values admit element expressions,
maps
      admit key:value entries, and object/record values admit
identifier:value
      field initializers; "=" is forbidden as an entry binder because it
belongs
      to metadata records, not typed JSON-like runtime representations; a
generic
      collection declaration itself must first be specialized and named by
a
      co.lang.type alias ? ;

(* A built-in generic collection application must first be named by a
concrete
co.lang.type alias. Every constructed value then uses Type{...}:

    StringList co.lang.type = co.core.List(co.lang.string)
    StringIntMap co.lang.type = co.core.Map(co.lang.string, co.lang.int)
    IntSet co.lang.type = co.core.Set(co.lang.int)
    StringList{"A","B","C"}
    StringIntMap{"A": 1}
    IntSet{1,2,3}

    Type[...] remains indexing and Type(...) remains invocation. An
unspecialized
    generic collection symbol is not a value constructor. *)

(* The reference's Builtin Collections registry. These names identify
generic
declarations and cannot directly stand in a value-construction prefix.
Their
specialized co.lang.type aliases carry the applicable body form. *)
builtin-collection-type-name = "co.core.List"
                              | "co.core.Set"
                              | "co.core.Map"

```

```

        | "co.core.Tree"
        | "co.core.Trie"
        | "co.core.Array"
        | "co.core.Tuple" ;

anonymous-class-expression = "co.lang.class", "{",
                              { class-member }, "}" ;

let-expression = "let", "(", "{", let-binding,
                 { ",", let-binding }, "}", ")",
                 ".in", "(", "{", expression, "}", ")" ;

let-binding = ( identifier | special-binding ), "=", expression ;

comprehension-expression = "for", "(", comprehension-binding, ")",
                           ".yield", "(", expression-list, ")" ;

comprehension-binding = pattern, "<-", expression ;

match-suffix = ".match", [ "(", [ expression ], ")" ],
               match-case, { match-case }, [ match-default ] ;

multi-symbol-infix-operator-boundary-guard =
    ? zero-width condition: a multi-symbol infix operator has an explicit
      boundary on both operand-facing sides; a boundary is whitespace, a
      comment, or an applicable delimiter, checked before separators are
      discarded ? ;

multi-symbol-range-operator-boundary-guard =
    ? zero-width condition: a multi-symbol range operator has an explicit
      boundary on every side for which an operand is present ? ;

extended-operator-expression =
    ? expression containing a registered project-local custom operator,
      parsed by precedence climbing from its declared alpha-supported
fixity,
    precedence, associativity, and arity; U and n are not handled by this
    extension path because they are language-owned operators in the
ordinary
    precedence grammar; explicitly reserved future fixities remain
    unsupported; every registered multi-symbol operator satisfies the
    operand-facing boundary rule for its fixity ? ;

(* =====

```

```

*)

(* 12. Informative control-chain shapes (not parser entry paths) *)

(* =====
*)

(* Control vocabulary in the current alpha profile:
    - .then(X) conditionally evaluates X once; X may be a block or value.
    - .then(...) may participate in .otherwise(condition) selection
chains.
    - .loop(X) is a single-condition repetition form and cannot
participate in
        .otherwise(condition) chains or take a .default(X) fallback.
    - .otherwise(condition) always introduces another selection condition;
there
        is no conditionless .otherwise form.
    - .default(X) is the optional terminal fallback of a selection chain;
it is
        not part of loop syntax.
    - .each(...) is itself element iteration and cannot be followed by
.loop.
        The explicit-binding form is .each(index-or-key, value, action),
where
        action may be any expression (including a block or anonymous
function) or
        a lambda-expression. The single-argument form accepts a callable
callback;
        semantic analysis requires it to match the receiver's iteration
tuple.
    These informative productions document recognizable shapes; ordinary
parsing
        still proceeds through the general postfix/member/call grammar above. *)

informative-condition-chain =
    "(", expression, ")", ".then", "(", block, ")",
    { ".otherwise", "(", expression, ")", ".then", "(", block, ")" },
    [ ".default", "(", block, ")" ] ;

informative-loop-chain =
    "(", expression, ")", ".loop", "(", block, ")" ;

informative-labeled-loop =
    label-declaration, ":", "(", expression, ")", ".loop", "(", block, ")"
;

```

```

informative-ternary-chain =
  "(", expression, ")", ".then", "(", expression, ")",
  { ".otherwise", "(", expression, ")", ".then", "(", expression, ")" },
  ".default", "(", expression, ")" ;

informative-each-chain =
  postfix-expression, ".each", "(",
  ( ( identifier | "_" ), ",", identifier, ",",
    ( expression | lambda-expression )
  | ( expression | lambda-expression ) ), ")" ;

informative-contains-chain =
  postfix-expression, ".contains", "(", expression, ")",
  ".then", "(", block, ")",
  [ ".default", "(", block, ")" ] ;

informative-pipeline-chain =
  postfix-expression,
  { ( ".filter" | ".map" | ".reduce" | ".forEach"
    | ".sortBy" | ".groupBy" | ".fold" ),
    "(", argument-list, ")" } ;

(* =====
*)

(* 13. Literals and lexical grammar *)

(* =====
*)

literal = builtin-literal ;

builtin-literal = integer-literal
                  | floating-literal
                  | string-literal
                  | character-literal
                  | boolean-literal
                  | none-literal ;

integer-literal = ( binary-integer-literal
                  | octal-integer-literal
                  | decimal-integer-literal
                  | hexadecimal-integer-literal ),

```

```

[ integer-suffix ] ;

binary-integer-literal = ( "0b" | "0B" ), binary-digit-sequence ;

octal-integer-literal = "0", [ octal-digit-sequence ] ;

decimal-integer-literal = nonzero-digit, { decimal-digit } ;

hexadecimal-integer-literal = hexadecimal-prefix,
                             hexadecimal-digit-sequence ;

hexadecimal-prefix = "0x" | "0X" ;

binary-digit-sequence = binary-digit, { binary-digit } ;

octal-digit-sequence = octal-digit, { octal-digit } ;

decimal-digit-sequence = decimal-digit, { decimal-digit } ;

hexadecimal-digit-sequence = hexadecimal-digit, { hexadecimal-digit } ;

integer-suffix = unsigned-suffix,
                [ long-suffix | long-long-suffix | size-suffix ]
                | long-suffix, [ unsigned-suffix ]
                | long-long-suffix, [ unsigned-suffix ]
                | size-suffix, [ unsigned-suffix ] ;

unsigned-suffix = "u" | "U" ;

long-suffix = "l" | "L" ;

long-long-suffix = "ll" | "LL" ;

size-suffix = "z" | "Z" ;

floating-literal = decimal-floating-literal
                  | hexadecimal-floating-literal ;

decimal-floating-literal = fractional-constant,
                          [ exponent-part ],
                          [ floating-point-suffix ]
                          | decimal-digit-sequence,
                          exponent-part,
                          [ floating-point-suffix ] ;

```

```

fractional-constant = decimal-digit-sequence, ".",
                        decimal-digit-sequence ;

hexadecimal-floating-literal = hexadecimal-prefix,
                                ( hexadecimal-fractional-constant
                                  | hexadecimal-digit-sequence ),
                                binary-exponent-part,
                                [ floating-point-suffix ] ;

hexadecimal-fractional-constant = hexadecimal-digit-sequence, ".",
                                hexadecimal-digit-sequence ;

exponent-part = ( "e" | "E" ), [ sign ], decimal-digit-sequence ;

binary-exponent-part = ( "p" | "P" ), [ sign ], decimal-digit-sequence ;

sign = "+" | "-" ;

floating-point-suffix = "f" | "F" | "l" | "L"
                        | "f16" | "F16"
                        | "f32" | "F32"
                        | "f64" | "F64"
                        | "f128" | "F128"
                        | "bf16" | "BF16" ;

character-literal = single-quote, alpha-basic-c-character, single-quote ;

alpha-basic-c-character =
    ? any translation character except apostrophe, backslash,
      carriage return, or line feed ? ;

string-literal = double-quote, { alpha-basic-s-character }, double-quote ;

alpha-basic-s-character =
    ? any translation character except double quote, backslash,
      carriage return, or line feed ? ;

double-quote = ? Unicode scalar value U+0022 ? ;

single-quote = ? Unicode scalar value U+0027 ? ;

backslash = ? Unicode scalar value U+005C ? ;

boolean-literal = "co.const.true" | "co.const.false" ;

```



```

none-literal = "co.const.none" ;

byte-order-mark =
    ? U+FEFF, permitted only as the first code point of a source file ? ;

(* Structured control labels use a dedicated apostrophe-prefixed lexical
form.
    The same token shape is used in declarations and references:
        'outer:      declaration prefix
        'outer      reference
    A character literal remains distinct because it has a closing
apostrophe,
    e.g. 'c'. Labels are not ordinary identifiers and are not operators. *)
label-identifier =
    single-quote, identifier, label-identifier-guard ;

label-identifier-guard =
    ? a zero-width assertion that the next character is not single-quote;
      this prevents a complete character literal such as 'c' from being
      tokenized as a label identifier plus a trailing apostrophe ? ;

identifier = identifier-head, { "_", identifier-segment },
            identifier-trailing-guard ;

identifier-trailing-guard =
    ? a zero-width assertion that the next character is not "_" ? ;

identifier-head = ascii-letter, { ascii-alphanumeric } ;

identifier-segment = ascii-alphanumeric, { ascii-alphanumeric } ;

ascii-alphanumeric = ascii-letter | decimal-digit ;

ascii-letter = "A" | "B" | "C" | "D" | "E" | "F" | "G" | "H"
              | "I" | "J" | "K" | "L" | "M" | "N" | "O" | "P"
              | "Q" | "R" | "S" | "T" | "U" | "V" | "W" | "X"
              | "Y" | "Z"
              | "a" | "b" | "c" | "d" | "e" | "f" | "g" | "h"
              | "i" | "j" | "k" | "l" | "m" | "n" | "o" | "p"
              | "q" | "r" | "s" | "t" | "u" | "v" | "w" | "x"
              | "y" | "z" ;

binary-digit = "0" | "1" ;

octal-digit = "0" | "1" | "2" | "3" | "4" | "5" | "6" | "7" ;

```

```

hexadecimal-digit = decimal-digit
                    | "a" | "b" | "c" | "d" | "e" | "f"
                    | "A" | "B" | "C" | "D" | "E" | "F" ;

digit = decimal-digit ;

decimal-digit = "0" | nonzero-digit ;

nonzero-digit = "1" | "2" | "3" | "4" | "5" | "6" | "7" | "8" | "9" ;

hard-reserved-word = "co" | "let" | "this" | "for" | "fφλ" ;

contextual-keyword = "forall" ;

(* `::`, `->|`, and `^=>` are active structural markers consumed
respectively
    by lifecycle-call-suffix, break-statement, and
    enclosing-callable-return-statement. They are not ordinary infix
operators
    and are therefore intentionally absent from reserved-operator. *)
lifecycle-invocation-marker = "::" ;

reserved-operator = "::~" | "->>" | "<->" | "`" | backslash | "#" ;

predeclared-operator-glyph = "U" | "n" ;

reserved-future-operator =
    ? a complete documented reserved/future symbolic spelling that is not a
      current built-in operator and not a predeclared-operator-glyph; the
lexer
    recognizes the complete spelling and the parser reports unsupported ?
;

token = label-identifier
        | identifier
        | keyword-token
        | literal
        | delimiter-token
        | predeclared-operator-glyph
        | reserved-operator
        | symbolic-token
        | reserved-future-operator ;

keyword-token = hard-reserved-word ;

```

```
delimiter-token = "(" | ")" | "{" | "}" | "[" | "]"  
                | ",", ";" | double-quote | single-quote ;
```

symbolic-token = ? the complete maximal contiguous run of one or more
symbol

characters after comments, literals, and closed

composite

spellings are recognized; the run is preserved whole for
contextual classification and is never split as a
fallback ? ;

```
token-separator = white-space ;
```

```
line-comment = "//", { ? any Unicode scalar value except CR or LF ? } ;
```

```
block-comment = "/*", { block-comment-character }, "*/" ;
```

block-comment-character = ? any Unicode scalar value that does not begin
the

two-character sequence */ ? ;

```
line-break = "\r\n" | "\n" | "\r" ;
```

```
horizontal-white-space = " " | "\t" | "\f" ;
```

```
white-space = horizontal-white-space | line-break | line-comment | block-  
comment ;
```

```
(* =====  
*)
```

```
(* 14. Operator declarations inside components/operators/component.fol *)
```

```
(* =====  
*)
```

(* The operator component uses the same component-surface-file parser root
as

every other component.fol. Filesystem context restricts co.lang.operator
to

components/operators/component.fol and forbids all non-operator members
there. The scanner still recognizes each declaration head as one maximal
symbolic run while this component builds the owning

```
ProjectOperatorTable. *)
```

```
operator-declaration = operator-symbol, "co.lang.operator", "=",  
                        operator-body, statement-end,  
                        operator-declaration-context-guard ;
```

```
operator-declaration-context-guard =
```

```
    ? zero-width structural condition: declaration occurs only inside the  
      _ co.lang.component body of components/operators/component.fol; that  
      component body contains only operator declarations and its project
```

```
kind
```

```
    is either executable application or projected application library ? ;
```

```
operator-body = "{", operator-property,  
                { ",", operator-property }, "}" ;
```

```
operator-property = "fixity", declarative-attribute-binder, operator-fixity  
                  | "precedence", declarative-attribute-binder,  
                    ( "0" | decimal-integer-literal )  
                  | "associativity", declarative-attribute-binder,  
                    operator-associativity  
                  | "arity", declarative-attribute-binder, operator-arity  
                  | "commutative", declarative-attribute-binder,  
                    boolean-literal  
                  | "idempotent", declarative-attribute-binder,  
                    boolean-literal  
                  | "identity", declarative-attribute-binder,  
                    operator-identity-value  
                  | "foldable", declarative-attribute-binder, boolean-
```

```
literal
```

```
                  | "vectorizable", declarative-attribute-binder,  
                    boolean-literal  
                  | "distributes_over", declarative-attribute-binder,  
                    operator-symbol-list  
                  | "desugar", declarative-attribute-binder, string-literal
```

```
;
```

```
operator-fixity = "co.operator.fixity.infix"  
                 | "co.operator.fixity.prefix"  
                 | "co.operator.fixity.postfix"  
                 | reserved-future-operator-fixity ;
```

```
reserved-future-operator-fixity =
```

```
    "co.operator.fixity.circumfix"  
    | "co.operator.fixity.postcircumfix"
```

```

| "co.operator.fixity.precumfix"
| "co.operator.fixity.mixedfix"
| "co.operator.fixity.ternary"
| "co.operator.fixity.distfix" ;

operator-associativity = "co.operator.associativity.left"
                        | "co.operator.associativity.right"
                        | "co.operator.associativity.none" ;

operator-arity = "co.operator.arity.unary"
                 | "co.operator.arity.binary"
                 | "co.operator.arity.ternary"
                 | decimal-integer-literal ;

operator-identity-value = literal ;

operator-symbol-list = "[", [ operator-symbol-reference,
                             { ",", operator-symbol-reference } ], "]" ;

operator-symbol-reference = character-literal | string-literal ;

operator-symbol = ? a maximal run of one or more symbol characters, where a
                  symbol character is any character that is not an ASCII
                  letter, digit, underscore, whitespace, or one of the
                  delimiters ( ) { } [ ] , ; " ' ; the run must not be a
                  language-owned or hard-reserved symbol and must not
contain
                  // or /* ? ;

```

Appendix B - Frontend Context and Symbol-Table Model

Status: informative implementation model. This appendix documents the context and symbol-table model used by the FoLang reference frontend. It does not require other conforming FoLang implementations to use the same internal data structures. Source-level name-resolution behavior remains governed by the normative language rules in this specification.

The frontend keeps **contexts** and **symbol tables** as related but distinct structures:

- a **Context** represents a semantic/lexical region such as the application or library root, a package, a declaration body, a function-shaped scope, or a nested executable block;
- a **SymbolTable** represents one declaration-order visibility segment within a context;
- one context may therefore own more than one symbol-table segment;
- a child context records the exact symbol-table segment of its parent from which it branched, so name resolution begins from the visibility state that existed at that branch point;
- relationships are stored by stable IDs rather than object pointers, which keeps the model map-based and serialization-friendly.

B.1 Example

```
// some.unit.fol
_ co.unit = {
  firstfun()->() = {
    k co.int = 10;
    v := 20;

    co.out.println(k + v);

    j ?= 30;

    {
      j co.char = 'A';
      co.out.println(j);
    }

    co.out.println(j);
  }

  secondfun()->() = {
    k co.int = 10;
    v := 20;

    co.out.println(k + v);

    j ?= 30;

    {
```

```

        j co.char = 'A';
        co.out.println(j);
    }

    co.out.println(j);
}

```

In this example, `j ?= 30` creates `j` in the function context because no visible `j` has been defined there yet. The nested block then declares a distinct block-local `j` of type `co.char`. Inside the block, lookup resolves to the block-local symbol. After the block ends, lookup resumes in the function context and resolves to the outer `j`.

The frontend model can be visualized as follows. The symbolic table IDs shown here are illustrative; real IDs are implementation-generated and globally unique within the frontend symbol model.

```

app_or_lib_context
├─ package_context
│   └─ firstfun_context
│       ├── ST-F1-1
│       │   └─ symbols: k : co.int, v : inferred co.int
│       ├── ST-F1-2
│       │   ├── ParentId: ST-F1-1
│       │   └─ symbols: j : inferred co.int
│       └─ block_context
│           ├── ParentId: firstfun_context
│           ├── ParentCtxSymbolTableId: ST-F1-2
│           └─ ST-F1-B1
│               └─ symbols: j : co.char
└─ secondfun_context
    ├── ST-F2-1
    │   └─ symbols: k : co.int, v : inferred co.int
    ├── ST-F2-2
    │   ├── ParentId: ST-F2-1
    │   └─ symbols: j : inferred co.int
    └─ block_context
        ├── ParentId: secondfun_context
        ├── ParentCtxSymbolTableId: ST-F2-2
        └─ ST-F2-B1
            └─ symbols: j : co.char

```

`some_unit_context` is an internal frontend context. It does not imply that a `co.unit` introduces a user-visible namespace; unit declarations continue to follow the package/unit semantics defined elsewhere in this specification.

B.2 Why a Context Can Have Multiple Symbol Tables

A context is a semantic region; a symbol table is a **visibility segment** inside that region. These are intentionally not the same thing.

For example, in `firstfun`, `k` and `v` are visible before the first call to `co.out.println`. The later `j ?= 30` introduces another binding after that earlier source position. The frontend may represent this new visibility frontier by creating a second symbol-table segment rather than creating another function context.

This preserves lexical declaration order without turning every statement boundary into a new Context. The symbol-table chain records the declaration-order history **within the same context**.

A nested block is different: it is a new lexical context. Its `ParentCtxSymbolTableId` points to the precise parent symbol-table segment that was active when the block was entered. This distinction is important because declarations added later to the parent context must not be treated as though they had been visible at an earlier branch point when the applicable resolution policy is declaration-order-sensitive.

B.3 Top-Level Symbol Model

The frontend keeps contexts and symbol tables in ID-addressable maps:

```
FolangSymbols {
  RootContextId:  <context-id>,
  SymboltableMap: { <symbol-table-id>: SymbolTable },
  ContextMap:     { <context-id>: ContextInfo },
  SymbolsById:    { <symbol-id>: SymbolRecord }
}
```

`FolangSymbols` is the complete project-owned symbol graph and the resolution boundary for every ID stored in the AST, Contexts, SymbolTables, and surface projection. It has the following responsibilities:

- `RootContextId` identifies the project's `FoIContext` entry. It must be empty only for an intentionally incomplete graph.
- `SymboltableMap` owns every complete visibility segment, keyed by its stable `SymbolTable.Id`.
- `ContextMap` owns the `FoIContext` and every lexical or semantic `Context`, keyed by their stable IDs. Its read-only `ContextInfo` values discriminate the concrete kind.
- `SymbolsById` is the one canonical symbol-record registry. No AST node or `SymbolTable` embeds another complete copy of a registered symbol.
- `FoIContext.SymbolTable_` selects the `appl.fol` or `component.fol` surface table chain, while `FoIContext.Context_` selects the independent operational root `Context` used for project imports and facilities. This transparent link is not lexical ancestry.

The maps are the ownership/indexing layer. Cross-structure relationships use IDs, so the serialized representation does not require cyclic parent/child object references. A live frontend may hold concrete implementations behind its `SymbolInfo` interface. JSON, protobuf, and other interchange formats must write the concrete backend-neutral `SymbolRecord` representation instead of an implementation-language interface value.

B.4 SymbolTable Structure

```
SymbolTable {
  Id:          string,    // unique symbol-table ID
  ParentId:    string,    // previous symbol-table segment in the same
context; empty for the first segment
  ContextId:   string,    // ID of the Context that owns this table
  Prefix:      string,    // frontend qualification/debug prefix associated
with this table

  SymbolIds:   [ <symbol-id> ],
  SymbolsByName: { <declaration-key>: [ <symbol-id> ] }
}
```

`SymbolTable.Id` is the stable key under which the segment appears in `FolangSymbols.SymboltableMap`. `Prefix` is the frontend qualification/debug prefix associated with the segment; it is not a lookup or ownership edge.

ParentId

`ParentId` is the reverse link to the preceding visibility segment in the **same context**. Name lookup can walk this chain from the active/latest table toward earlier declarations.

The earlier `Prev` name is better expressed as `ParentId`, because the relationship is structural rather than merely positional. A serialized `Next` field is not required for ordinary name resolution when the Context records its active/terminal symbol table: lookup naturally proceeds from newer visibility to older visibility through `ParentId`.

If a compiler tool needs forward traversal for diagnostics or visualization, it may derive or maintain a separate auxiliary index. That auxiliary index is not required by the core symbol-resolution model.

ContextId

`ContextId` identifies the Context that owns the table. Multiple symbol tables may therefore have the same `ContextId`.

SymbolIds and SymbolsByName

`SymbolIds` preserves declaration order without embedding complete symbol records in each table. Every entry must resolve through the enclosing `FolangSymbols.SymbolsById` registry. The sequence is normative for consumers that need source/declaration order; map iteration order is not.

`SymbolsByName` is the lookup index. It maps the frontend declaration key—which includes symbol category and, for callable declarations, receiver category and overload signature where applicable—to one or more IDs in the same canonical registry. The list form permits an indexed name or family to designate multiple declarations without duplicating their records.

The following invariants apply:

- an ID occurs at most once in `SymbolIds` for one table segment;
- every ID referenced by `SymbolIds` or `SymbolsByName` exists in `FolangSymbols.SymbolsById`;

- every symbol bound by the table identifies that table as its source-position visibility anchor;
- `SymbolsByName` is an index, not an ownership container; and
- removing a speculative declaration removes its table indexes without changing the identity of unrelated declarations.

Lookup and declaration operations therefore require the enclosing `FolangSymbols` registry. A `SymbolTable` does not retain a runtime back-pointer to that registry merely to resolve its IDs.

B.5 Context Structure

```
Context {
    ParentId:                string,                // parent Context ID;
empty for the root context
    ParentCtxSymbolTableId:  string,                // parent-context table
active at the branch point
    Id:                      string,                // unique Context ID

    RestrictedSymbolNameReuse: [string],            // names whose reuse is
restricted in this context
    ImportedContextIds:      { <alias>: <context-id> },

    Prefix:                  string,                // frontend
qualification/debug prefix
    ContextType_:            string,                // context-kind tag
    SymbolTable_:            string,                // active/terminal
symbol-table ID for this context
    ChildCtxIds:             [string],              // direct child Context
IDs
    ResolutionPolicy:        string,                // resolver-policy tag
for this context
    OwnerSymbolId:          string                  // ID of the symbol
owning this context; empty for structural roots
}
```

`Context.Id` is the stable key under which the `Context` appears in `FolangSymbols.ContextMap`. `Prefix` is the frontend qualification/debug prefix associated with that scope. Neither field replaces lexical ownership: parent, branch-point, active-table, and owning-symbol relationships remain explicit IDs.

ParentId

`ParentId` records the structural parent Context. It answers **which context contains this context**.

OwnerSymbolId

`OwnerSymbolId` is the inverse of the owning symbol's `OwnedContextId`. Functions, methods, classes, modules, local and anonymous functions, lambdas, and blocks are symbols in their parent context and own the child context containing their members, parameters, or body. Both IDs must be present together and resolve within the same serialized symbol/context model. File and project structural roots have no owner.

For every nonempty `Context.OwnerSymbolId`:

1. `FolangSymbols.SymbolsById[OwnerSymbolId]` must exist;
2. that symbol's `OwnedContextId` must equal this Context's `Id`; and
3. no different symbol may claim the same owned Context unless a future language rule explicitly defines shared scope ownership.

Symbol, Context, and SymbolTable IDs are deterministic content-derived identities, not process counters or native hashes. The frontend hashes length-delimited canonical identity components with SHA-256: project/source identity establishes the root domain; child contexts add parent, scope kind, and structural occurrence; visibility segments add their owning Context and segment occurrence; symbols add source domain, owning SymbolTable, symbol kind/name, and declaration occurrence. Identical source at the same canonical project location must produce identical IDs in independent compiler runs. Distinct source identities and distinct declarations must not share an ID.

ParentCtxSymbolTableId

`ParentCtxSymbolTableId` records the exact symbol-table segment in the parent Context that was visible when the child Context branched. It answers a different question from `ParentId`: **from which parent visibility point should lexical lookup continue?**

This distinction is essential when the parent Context has multiple declaration-order symbol-table segments.

SymbolTable_

SymbolTable_ identifies the active/latest symbol-table segment owned by the Context. As new visibility segments are created in that same Context, this field advances to the newest segment. Earlier segments remain reachable through **SymbolTable.ParentId**.

This definition is what makes a serialized **SymbolTable.Next** link unnecessary for name lookup.

SymbolTable_ is **not** by itself a sufficient visibility anchor for every reference after parsing. Because it advances as declarations are encountered, an AST/reference node that may be resolved in a later pass must retain the symbol-table ID that was active at that node's source position, or retain an equivalent visibility snapshot. Deferred resolution must start from that use-site anchor rather than from the Context's final **SymbolTable_**; otherwise a reference could incorrectly see declarations introduced later in the same context.

ImportedContextIds

ImportedContextIds maps the import key visible in the current Context to the imported Context ID. The key is the explicit local alias when **as=** is supplied; otherwise the resolver may use the canonical imported package/library name or path defined by the import model. Import edges are not lexical-parent edges: they are consulted according to the applicable import and resolution rules rather than inserted into the **ParentId** context chain.

RestrictedSymbolNameReuse

RestrictedSymbolNameReuse records names that the frontend must not redeclare/reuse in positions governed by this Context's reuse restrictions. The exact population of this set follows the applicable FoLang semantic rules.

ChildCtxIds

ChildCtxIds records direct child contexts for traversal, diagnostics, analysis, and serialization. Lexical lookup does not search child contexts when resolving a name in the parent.

ContextType_

`ContextType_` identifies the semantic category of the Context. In the Go implementation this should preferably be represented by a constrained enum-like type rather than an unconstrained string, even if the serialized form uses a string value.

ResolutionPolicy

`ResolutionPolicy` selects the resolver strategy applicable to the Context. The current implementation vocabulary includes:

```
lexical_ordered  
lexical_complete_container  
late_lexical_call_site  
late_lexical_formation_site  
macro_definition_site  
macro_expansion_site  
runtime_bound  
dynamic_call_site  
lexical_call_site  
mixed_call_site
```

These are frontend policy identifiers, not FoLang source syntax. Their exact resolver behavior belongs to the frontend implementation contract unless a corresponding behavior is separately defined as normative language semantics. Keeping them as a constrained enum-like set is preferable to accepting arbitrary strings.

B.6 Lexical Lookup Through the Model

For ordinary lexical lookup, the model supports the following traversal:

1. Start with the symbol-table segment active at the use site.
2. Search that table's `SymbolsByName` index and resolve matching IDs through `SymbolsById`.
3. If not found, follow `SymbolTable.ParentId` through earlier segments belonging to the same Context.
4. When the current Context's table chain is exhausted:
 - a. read `Context.ParentId` to identify the parent Context;
 - b. continue from `Context.ParentCtxSymbolTableId`, which is the exact parent visibility point from which this Context branched.
5. Consult imported contexts and any non-ordinary lookup domains according to the Context's applicable `ResolutionPolicy` and the language's `import`,

macro, dynamic-runtime, or other semantic rules.

6. Stop when a valid declaration is resolved or when every permitted lookup path has been exhausted.

For a reference resolved during a later compiler pass, step 1 uses the symbol-table ID captured for that **use site**, not automatically the Context's final `SymbolTable_`.

This keeps three different relationships explicit:

`Context.ParentId`

-> structural context containment

`Context.ParentCtxSymbolTableId`

-> lexical branch/visibility point in the parent context

`SymbolTable.ParentId`

-> earlier declaration-order visibility segment in the same context

Conflating these relationships would make declaration-order lookup, nested scopes, and serialization harder to reason about.

B.7 Structural Invariants

The reference frontend should maintain the following invariants:

1. Every `Context.Id` is unique within `ContextMap`.
2. Every `SymbolTable.Id` is unique within `SymboltableMap`.
3. Every `SymbolTable.ContextId` resolves to an existing Context.
4. Every non-empty `SymbolTable.ParentId` resolves to a symbol table owned by the same Context.
5. Every non-empty `Context.ParentId` resolves either to an existing parent Context or to the enclosing lightweight `FolContext.Id` surface boundary.
6. Every non-empty `Context.ParentCtxSymbolTableId` resolves to a symbol table owned by `Context.ParentId`.
7. Every non-empty `Context.SymbolTable_` resolves to a symbol table whose `ContextId` equals that Context's `Id`.
8. Every ID in `Context.ChildCtxIds` resolves to a Context whose `ParentId` equals the containing Context's `Id`; every ID in `FolContext.ChildCtxIds` resolves to a Context whose `ParentId` equals `FolContext.Id`.

9. Every context ID stored in `ImportedContextIds` resolves to an existing imported Context visible under the associated alias.
10. A symbol-table chain is acyclic.
11. A Context parent chain is acyclic.
12. Ordinary lookup never searches a child Context to resolve a name in its parent.
13. Every expression and statement that participates in semantic resolution records the `SymbolTable.Id` that was current at that source occurrence, or retains an equivalent visibility anchor.
14. A recorded source-occurrence symbol-table ID resolves to the Context associated with that occurrence unless the resolver record explicitly represents a permitted cross-context lookup mode.
15. Every newly created Context receives an initial SymbolTable whose `ParentId` is empty.
16. In a `lexical_ordered` Context, consecutive variable declarations at the same context level share the current SymbolTable segment until a non-variable declaration, statement, expression, or child-block construct occurs at that level.
17. After such an intervening non-variable item, the next variable declaration creates a new SymbolTable segment in the same Context; the new segment's `ParentId` identifies the previously active segment, and `Context.SymbolTable_` advances to the new segment.
18. A child Context branches from the exact parent SymbolTable active when the child begins. Leaving the child returns parsing to that same parent Context and parent SymbolTable; processing the child does not itself advance the parent's SymbolTable chain.
19. A unit may use a temporary parse-time Context, but that root unit Context does not survive as an independent final semantic scope: a non-companion unit is merged into its package Context, and a companion unit is merged into the corresponding struct Context.
20. Unit source separation does not introduce a `UnitStmt` or `CompanionUnitStmt` into the final AST.
21. A complete graph's `RootContextId` resolves to exactly one `FoIContext` entry in `ContextMap`. The operational Context named by its `Context_` has no structural parent.
22. Every key in `SymbolsById` equals the contained `SymbolRecord.symbolId`.

23. Every symbol ID in a SymbolTable's `SymbolIds` and `SymbolsByName` resolves through the same enclosing `FolangSymbols.SymbolsById` registry.
24. Every nonempty `Context.OwnerSymbolId` and symbol `OwnedContextId` form a bidirectional pair.
25. `FolContext.SymbolTable_` resolves to the published surface table in the complete model and retains its stable ID and contents.
26. A surface symbol ID resolves through the complete model's `SymbolsById`; `FolContext` never introduces an independent symbol object with the same ID.
27. The serialized artifact contains one canonical `FolangSymbols` graph; root Context and symbol registries are not duplicated at the artifact top level.

These invariants are particularly useful when validating the frontend's serialized symbol/context state before later semantic passes or backend-facing artifact generation.

B.7.1 Project Hierarchy

The frontend AST for a complete project is rooted at `ProjectStatement`. The project-level frontend entry point returns a `Stmt`; at the project root, the concrete statement represented by that return value is always a `ProjectStatement`.

```
ProjectStatement {  
    Span  
  
    EntryStmt: Stmt  
        primary project entry; application or standalone-library entry  
  
    PackageStmts: {  
        <package-name>: Stmt  
    }  
        each value represents a PackageStatement  
  
    LibraryStmt: {  
        <library-name>: Stmt  
    }  
        each value represents the ProjectStatement reconstructed for  
        that library  
  
    ComponentStmt: {  
        <component-name>: Stmt  
    }
```

each value represents a `ComponentDeclarationStmt`

`FolangSymbols: FolangSymbols`

complete symbol-table and Context model for this project

`IsLibrary: bool`

true when this `ProjectStatement` represents a standalone library

`Kind: string`

effective project/library kind according to the project model

`SDapst: Stmt`

`Symb: ComponentSymbol`

}

`PackageStmts` contains the project's immediate top-level packages.

`LibraryStmt` contains libraries reconstructed from compiled artifacts under `lib/`. Each library is itself represented by a complete `ProjectStatement`, with its own project hierarchy and complete `FolangSymbols`; a separate library-root AST type is therefore unnecessary.

`ComponentStmt` contains the project-local component ASTs parsed and assembled from the applicable `components/<kind>/` source domains.

`Kind` identifies the effective project or library kind defined by the project model. Component-only classifications, such as the `operators` component kind, do not classify an entire `ProjectStatement` merely because `ComponentDeclarationStmt` uses that component kind.

Project surface and operational root

`FolContext` is the lightweight structural context returned for a complete project. It implements the common read-only `ContextInfo` contract but is not a full lexical `Context`:

```
FolContext {  
  Id:                <project structural id>  
  SymbolTable_:      <appl.fol/component.fol table id>  
  Context_:          <private operational root Context id>  
  Kind:              application | library | packaged-library  
  ChildCtxIds:       [<published surface Context ids>]
```

```
ExportedPackages: { <package name>: <Context id> }  
}
```

Both concrete context forms implement a read-only frontend contract:

```
ContextInfo {  
    GetId() -> string  
    GetSymbolTableId() -> string  
    GetContextKind() -> context | fol-context  
}
```

Normal consumers resolve entries through this contract. Code that needs lexical-parent or import fields narrows an entry to `Context`; code that needs the published surface, operational root, or project kind narrows it to `FoIContext`. A serialized `ContextMap` record carries `ContextKind` so the concrete value is reconstructed without inference. Mutation is performed by `FolangSymbols` graph operations rather than writable interface setters.

`SymbolTable_` selects the project surface directly from the canonical `FolangSymbols.SymboltableMap`; no second publication table map exists. `Context_` selects the independent private root `Context` holding the complete project graph, import bindings, and project-wide facilities. `ChildCtxIds` publishes the surface `Contexts`, whose `ParentId` names `FoIContext.Id`; ordinary nested scopes below those surface contexts retain normal `Context-to-Context` parentage. The operational root is not their lexical parent. Surface code reaches its imports through the transparent `FoIContext.Context_` link only for an explicitly qualified name.

`ChildCtxIds` and `ExportedPackages` are mutually exclusive publication forms. An application or projected library publishes its surface through `ChildCtxIds`; a packaged library, including `stdlib/co.folenc`, publishes selected package entry points through `ExportedPackages`. Private contexts may remain reachable from the owning project's `Context_`, but a consumer must enter a dependency only through its declared publication form. It must never use the dependency's `Context_` as an import entry point.

Lookup first exhausts the current symbol-table and lexical-parent hierarchy. If the name is composite and its first qualifier was not found lexically, lookup walks `ImportedContextIds` from the current `Context` through lexical parents and then consults the operational root through `FoIContext.Context_`. The selected imported

Context's own tables and ownership parents may be searched, but its `ImportedContextIds` are never traversed and lookup cannot escape into the consumer project's operational root.

`FolangSymbols` represents the complete symbol-table and Context model belonging to a project:

```
FolangSymbols {
  RootContextId: <context-id>

  SymboltableMap: {
    <symbol-table-id>: SymbolTable
  }

  ContextMap: {
    <context-id>: ContextInfo // FolContext or Context
  }

  SymbolsById: {
    <symbol-id>: SymbolRecord
  }
}
```

`RootContextId` always identifies the `FolContext` entry. Its `Context_` field identifies the independent operational root used by lookup. `SymbolsById` is the single canonical symbol registry; AST nodes and symbol tables carry IDs rather than repeated symbol objects. The serialized registry uses the concrete, discriminated `SymbolRecord` representation rather than an implementation-language interface. `FolContext.SymbolTable_` identifies the published surface through the same canonical table and symbol registries.

The frontend interchange artifact has one AST root and one complete symbol graph:

```
FrontendArtifact {
  symbolFormatVersion: integer,
  AST:                AST root,
  FolangSymbols:      FolangSymbols
}
```

The `FoIContext` is not duplicated beside `FolangSymbols`; it is obtained from `FolangSymbols.ContextMap[FolangSymbols.RootContextId]`. Likewise, a second top-level `SymbolsById` is forbidden: the canonical registry inside `FolangSymbols` is sufficient for AST, table, Context-owner, and surface references. JSON and protobuf encodings must preserve this logical shape even when their field naming or map representation differs.

The distinction is that `FolangSymbols` owns the complete semantic graph while `FoIContext` identifies its published-surface and operational-root entry points.

Every compiled library or component graph has its own `FoIContext`. When such a graph is decoded into a consuming project, that dependency `FoIContext`, its ordinary Contexts, its SymbolTables, and its canonical SymbolRecords retain their artifact-owned IDs and are inserted into the consumer's maps. The consumer's `RootContextId` continues to identify only the consumer's `FoIContext`; importing a dependency must not replace it. Any ID collision with an entry already present in the consumer is an artifact/link error rather than permission to rewrite IDs, because AST and graph records already refer to those IDs.

A compiled artifact's exported package Context is separate from its project root. `FolangSymbols.RootContextId` identifies the artifact's `FoIContext`, whereas the artifact export-context ID identifies the ordinary Context exposed under an import qualifier such as `co`. These IDs are not required to be equal. For the installed standard artifact, the consuming operational root binds `co` to that exported Context; the standard-library AST is not inserted into the consumer's `PackageStmts`.

The live symbol objects stored canonically in `FolangSymbols.SymbolsById` conform to the `SymbolInfo` contract:

```
SymbolInfo {
    GetSymbolType() -> string
    GetSymbolID() -> string
    GetContextID() -> string // context owned by this symbol, or empty
    GetType() -> string
    GetName() -> string
    IsInternal() -> bool
    ImplementationKind() -> ImplementationKind?
    RuntimeOperationId() -> RuntimeOperationId?
    Clone() -> SymbolInfo
}
```

Different concrete symbol kinds may implement this contract according to the declaration they represent. The two implementation-related queries return no value for declarations that do not carry a runtime-operation marker. A serialized `.fo1enc` preserves the resolved operation identity rather than backend-specific source text.

The serialized form is a concrete `SymbolRecord`, not a serialized Go interface:

```
SymbolRecord {
    symbolFormatVersion: integer,
    symbolId:            string,
    symbolType:          string,
    name:                string,
    type:                string | absent,
    symbolTableId:       string | absent,
    ownedContextId:     string | absent,
    symbolFlags:         hexadecimal bytes,
    fields:               { <field-name>: <portable-value> } | absent
}
```

Resolution progress is not canonical symbol metadata and is therefore absent from both `SymbolInfo` and `SymbolRecord`. A declaration may have multiple AST occurrences whose resolution progresses differently; storing one state on the shared symbol would lose that distinction and would not round-trip faithfully.

The fixed fields carry universal symbol identity and ownership. `symbolFlags` uses the versioned compact Boolean layout; `fields` carries the remaining kind-specific attributes. Symbol-valued relationships use IDs when they refer to canonical records, and any nested value record must preserve its own attributes and Boolean state. Readers must reject an unsupported symbol-format version and must not infer semantics from implementation-language concrete type names.

Package hierarchy

Every package or subpackage directory is represented by a `PackageStatement`.

```
PackageStatement {
    Span

    Name: string

    Body: [Stmt]
        effective top-level declarations directly owned by this package
```

```

SubPackage: {
    <package-name>: Stmt
}
    each value represents an immediate child PackageStatement

SDapst: Stmt
Symb: ComponentSymbol
}

```

`PackageStatement.Body` contains only the effective top-level declarations owned directly by that package. A child package does not appear in `Body`; child packages are represented only through `SubPackage`.

The recursive package hierarchy therefore follows the filesystem package hierarchy:

```

PackageStatement
├── Body
│   └── top-level declarations owned directly by this package
├── SubPackage
│   └── PackageStatement
│       ├── Body
│       └── SubPackage

```

Unit source contribution

Unit files are source-separation forms. They do not introduce a `UnitStmt` or `CompanionUnitStmt` into the resulting project AST.

For a non-companion unit:

```

*.unit.fol
-> contained top-level declarations
-> owning PackageStatement.Body

```

For a companion unit:

```

<StructName>.comp.unit.fol
-> contained declarations
-> corresponding StructStmt.Body

```

The physical unit file determines where declarations are written, while the project AST records their semantic owner. Appendix B's Context model may use a temporary parse-time unit Context, but that temporary Context is merged into the owning package or struct Context and does not introduce an additional final AST container.

Component hierarchy

A project-local component is represented by `ComponentDeclarationStmt`:

```
ComponentDeclarationStmt {
    Span

    Name: string

    SurfaceFile: Stmt
        applicable component surface declaration, including exports,
        operators, or another valid surface form

    Kind: string
        filesystem-selected component kind:
            application
            native
            dynamicvmrt
            packaged
            operators
            ""

    "" is used for the standalone src/component.fol surface whose
    role is determined by its own members

    SubPackage: {
        <package-name>: Stmt
    }
        each value represents an immediate child PackageStatement

    Projected: bool
        true when the standalone surface carries @co.dap.library and
        therefore represents a projected library rather than a packaged
        component; the two forms are mutually exclusive

    LibraryType: string
        projected library type:
            application
```



```

        native
        dynamicvmrt

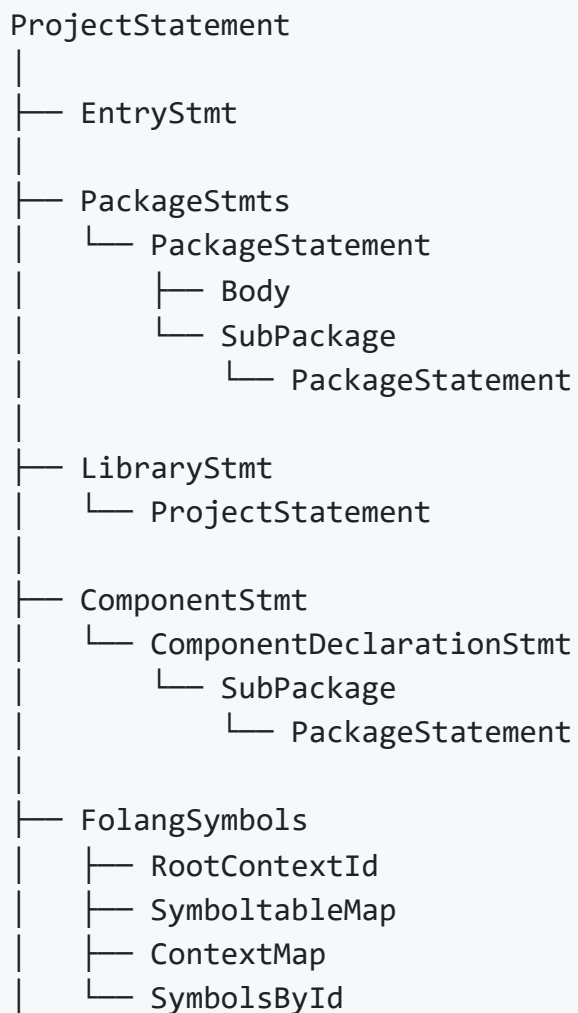
        application is the default when omitted

SDapst: Stmt
Symb: ComponentSymbol
}

```

The remaining declaration, statement, and expression AST forms are represented by their corresponding frontend statement types and are placed under the applicable `ProjectStatement`, `PackageStatement`, `ComponentDeclarationStmt`, or declaration body according to their semantic ownership.

The resulting project hierarchy is:



B.7.2 Source-Occurrence Symbol-Table Anchors

Every expression and statement records the `SymbolTable` that is current when that source occurrence is parsed. The recorded ID is the starting visibility segment for later semantic resolution of that node.

The per-node anchor is distinct from `Context.SymbolTable_`.

`Context.SymbolTable_` names the `Context`'s current/latest segment and may advance as later source is parsed; an earlier AST node continues to carry the `SymbolTable` ID that was current at its own source position.

For example:

```
run()->() = {  
  a := 10;  
  b := 20;  
  
  co.out.println(a + b);  
  
  c := 30;  
  co.out.println(c);  
}
```

may carry anchors conceptually as follows:

a declaration	-> ST-R-1
b declaration	-> ST-R-1
println(a + b)	-> ST-R-1
c declaration	-> ST-R-2
println(c)	-> ST-R-2

Nested expressions inside a variable declaration do not by themselves terminate the surrounding consecutive-variable-declaration run. Segmentation is determined by the sequence of context-level items: consecutive variable-declaration items share the current segment; an intervening context-level item that is not a variable declaration closes that run.

The symbol-table ID is a visibility **starting point**. Any finer rule about declaration order among names that intentionally share one segment is determined by the applicable `ResolutionPolicy` and declaration semantics; the anchor itself does not add an additional ordering rule inside that segment.

B.8 When the Frontend Creates a Context

Every semantic block/body start that is not merely a literal/value/pattern/metadata brace creates a new Context, except where the enclosing construct must establish that Context earlier. Function-shaped constructs are the principal exception because their parameters must belong to the function Context before the body brace is reached.

The parser knows from the active production whether a brace starts a semantic block/body or merely forms a value. Context creation therefore follows the source construct rather than the brace character in isolation.

Project and package contexts

Some Contexts arise from project structure rather than from a source brace:

ProjectStatement root	application or library project Context
PackageStatement	one package Context for each
package/subpackage	

A library deserialized from `lib/` is itself represented by a `ProjectStatement` and owns its own complete project context/symbol model. A package subfolder is represented by a `PackageStatement` and owns the package Context through which its package declarations are resolved.

Ordinary block/body contexts

A semantic block or declaration body normally creates a new Context. Examples include:

block-statement	{ ... }
labeled-block	'outer: { ... }
named-block-declaration	name co.block = { ... }
block-expression	a scoped braced block in operand position
block-argument	a scoped block passed as an argument

class	interface	signature	struct	cstruct
union	enum	module	object	
instance	matcher	contract	trait	mixin
extension	component	anonymous	class expression	

When such a child Context begins, it records:

```
Context.ParentId
    -> the containing Context

Context.ParentCtxSymbolTableId
    -> the exact SymbolTable currently active in that parent Context
```

The child then receives its own initial SymbolTable.

When parsing leaves the child, the frontend returns to the same parent Context and the same parent SymbolTable from which the child branched. The child Context's own symbol-table activity does not advance the parent's symbol-table chain.

The child construct is nevertheless a non-variable context-level item in the parent. Therefore, if a variable declaration appears after the child block, that next variable declaration begins a new parent SymbolTable segment according to B.9.

Function-shaped constructs: Context starts before the body brace

A function-shaped construct must establish its Context before parsing the body because its parameters belong to that function Context and must be entered into its first/top SymbolTable:

```
function-declaration      top-level, member, and local forms
anonymous-function-expression
lambda-expression
```

Conceptually:

```
function declaration
    -> parameter list begins
        -> create function Context
        -> create function's first SymbolTable
        -> parameters belong to that first SymbolTable
    -> function body begins
        -> reuse the already-created function Context
        -> do NOT create a second body Context
```

The initial consecutive run of local variable declarations may continue in that same first SymbolTable until a non-variable context-level item closes the run.

```

apply(base co.int)->(co.int) = {
    step := 1;
    scale := 2;

    co.out.println(step);

    result := base * step * scale;
    this => result;
}

```

Conceptually:

```

apply_context
├─ ST-A-1
│   └─ symbols: base, step, scale
└─ ST-A-2
    ├── ParentId: ST-A-1
    └─ symbols: result

```

The function body brace belongs to `apply_context`; it does not introduce another Context between the parameters and the function locals.

Unit source separation and temporary parse Contexts

A FoLang unit is source separation, not a semantic AST container. No `UnitStmt` or `CompanionUnitStmt` is created.

The frontend may nevertheless create a temporary Context while parsing a unit file so the file can be processed independently. That Context is a parse-time mechanism only and is merged into the unit's semantic owner before the final project context graph is produced.

```

ordinary *.unit.fol
  -> optional temporary unit parse Context
  -> declarations contribute to PackageStatement.Body
  -> temporary root unit Context is merged into the owning package
  Context

<StructName>.comp.unit.fol
  -> optional temporary companion-unit parse Context
  -> declarations contribute to the corresponding StructStmt.Body

```

```
-> temporary root unit Context is merged into the corresponding struct Context
```

Real nested semantic Contexts created while parsing the unit — function Contexts, nested blocks, lambdas, and similar scopes — remain real Contexts. When the temporary root unit Context is merged, those children are re-associated with the package or struct Context as required by the implementation's ID-based context graph.

A conforming frontend may avoid allocating the temporary unit Context and parse directly against the destination package/struct Context. What matters to this reference model is the resulting semantic ownership: the final Context graph must not contain an independent unit-wrapper scope merely because the source was physically separated into a unit file.

Braces that create no Context

A brace used only to construct a value, pattern, or metadata payload creates no Context. Examples include:

```
composite construction  Type{...}
object construction     Employee{id: 1}
annotation map          the braced argument of an annotation
record pattern          the braced pattern of a match case
```

The distinction is therefore:

```
semantic block/body brace
  -> new Context

function body brace
  -> function Context already exists from the parameter-list boundary

literal/value/pattern/metadata brace
  -> no Context

unit wrapper
  -> may use a temporary parse Context
  -> final semantic ownership is package or struct Context
```

B.9 When the Frontend Creates a Symbol-Table Segment

Every Context starts with one SymbolTable. In a `lexical_ordered` Context, consecutive variable declarations at the same context level share the current SymbolTable. Once a non-variable context-level item occurs, the next variable declaration creates a new SymbolTable segment chained to the previous one.

The initial SymbolTable is created together with the Context:

```
new Context
  -> create initial SymbolTable
  -> initial SymbolTable.ParentId = ""
  -> Context.SymbolTable_ = initial SymbolTable.Id
```

The first variable declaration in that Context uses the initial table; the frontend does not create an otherwise-empty preliminary table and then create another table for the first declaration.

Consecutive variable declarations share one segment

```
run(a co.int)->() = {
  x := 10;
  y := 20;
  z co.int = 30;
}
```

The function parameters are already in the function's first SymbolTable, and the initial consecutive local-variable run continues there:

```
run_context
├─ ST-R-1
│   └─ symbols: a, x, y, z
```

The rule is based on the sequence of items at the current Context level:

```
previous context-level item is a variable declaration
current context-level item is a variable declaration
  -> reuse current SymbolTable
```

Typed, inferred, grouped, and other variable-declaration forms that introduce variable bindings participate in the same rule according to their normal declaration semantics.

A non-variable item closes the declaration run

A non-variable declaration, statement, expression, or child-block construct at the same Context level closes the current consecutive-variable-declaration run. It does not immediately create a SymbolTable by itself. Instead, the **next variable declaration** creates the new segment.

```
total()->(co.int) = {  
  k co.int = 10;  
  v := 20;  
  
  co.out.println(k + v);  
  
  j ?= 30;  
  m := 40;  
  
  this => k + v + j + m;  
}
```

```
total_context  
├─ ST-T-1  
│   └─ symbols: k, v  
└─ ST-T-2  
    └─ ParentId: ST-T-1  
       symbols: j, m
```

`k` and `v` are consecutive variable declarations and therefore share `ST-T-1`. The call to `co.out.println` closes that run. `j` is the next variable declaration, so it creates `ST-T-2`; `m` immediately follows another variable declaration and therefore remains in `ST-T-2`.

The same rule applies when the intervening item is a non-variable declaration. For example:

```
run()->() = {  
  seed := 1;  
  
  helper()->() = {
```



```

        co.out.println("helper");
    }

    scale := 3;
}

```

`helper` has its own declaration/function Context, but as an item in `run_context` it is not a variable declaration. The variable-declaration run containing `seed` is therefore closed, and `scale` begins the next SymbolTable segment in `run_context`.

Child blocks branch from the current segment and return to it

A block, named block, labeled block, or other child Context branches from the SymbolTable currently active in its parent:

```

run()->() = {
    seed := 1;

    {
        local := seed;
        co.out.println(local);
    }

    scale := 3;
}

```

At the block entry, `ST-R-1` is current:

```

run_context
├── ST-R-1
│   └── symbols: seed
├── block_context
│   ├── ParentId: run_context
│   ├── ParentCtxSymbolTableId: ST-R-1
│   └── ST-R-B1
│       └── symbols: local
└── ST-R-2
    ├── ParentId: ST-R-1
    └── symbols: scale

```

When the block finishes, parsing returns to `run_context` with `ST-R-1` again current. The block has not created or advanced a parent segment. However, the block itself is an intervening non-variable context-level item, so the declaration run in the parent is closed. `scale`, being the next variable declaration, creates `ST-R-2` whose `ParentId` is `ST-R-1`.

This gives two independent relationships:

```
child Context branching
-> Context.ParentId
-> Context.ParentCtxSymbolTableId

same-Context declaration history
-> SymbolTable.ParentId
```

Expression/statement anchors use the current segment

Every expression and statement records the currently active SymbolTable ID at its own source occurrence. Therefore, in the previous example:

```
seed declaration          -> ST-R-1
child-block occurrence    -> ST-R-1
scale declaration        -> ST-R-2
```

Expressions/statements inside `block_context` record that block Context's current SymbolTable IDs, not the parent's `ST-R-1` directly. Lexical lookup can reach the parent through `block_context.ParentCtxSymbolTableId` when required.

Complete-container Contexts

`ResolutionPolicy` still determines whether ordinary declaration-order segmentation is applicable. A `lexical_complete_container` Context may treat its member declarations as one complete visibility region according to that container's semantics. The consecutive-variable-declaration segmentation rule above is specifically the reference model for declaration-order-sensitive (`lexical_ordered`) Contexts.

Package and struct contexts that receive declarations from unit files use the policy appropriate to those semantic owners. A temporary unit parse Context, when used, is not an additional final lookup scope after it has been merged.

B.10 What Parsing Establishes and What It Leaves Open

Creating Contexts and SymbolTable segments is not the same as completing every use-site resolution. The reference frontend creates declaration entries in `SymbolsByName` and canonical declaration information in `SymbolsById` while parsing. It also resolves deterministic lexical references and locally determinable inferred types. Later passes complete import, overload, generic, dynamic-dispatch, and other deferred occurrence resolution.

Every AST occurrence for which identity or type resolution is meaningful may carry `ResolutionState_` with one of `UNRESOLVED`, `RESOLVING`, `PARTIALLY_RESOLVED`, `RESOLVED`, or `ERROR`. The field describes that AST occurrence, never its referenced declaration record. Built-in type and literal occurrences are resolved immediately. A named user-type occurrence is at least partially resolved when its spelling and lookup domain are known but canonical type lookup remains. Parameters and returns derive their state from their type or value occurrence. Structural AST nodes for which resolution is not meaningful omit the field.

What parsing establishes immediately is structural ownership and the source-position visibility anchor:

```
AST expression/statement
-> Context identity as applicable
-> current SymbolTable.Id at that source occurrence
```

A later resolver starts ordinary lookup from the node's recorded SymbolTable ID rather than automatically from the Context's final `SymbolTable_`. The final `SymbolTable_` may point to a later segment created after additional source was parsed.

For example:

```
run()->() = {
  a := 1;
  b := 2;

  co.out.println(a + b);

  c := 3;
}
```

has the structural shape:

```
run_context
├── ST-R-1
│   └── a, b
└── ST-R-2
    ├── ParentId: ST-R-1
    └── c
```

and the call `co.out.println(a + b)` records `ST-R-1`, while `c` records `ST-R-2`.

Unit-context merge and stored anchors

If the implementation uses a temporary unit parse Context, merging that Context into its package or struct owner must preserve the validity of all stored IDs and relationships. An implementation may preserve temporary SymbolTable IDs while changing their owning `ContextId`, or it may remap IDs; either approach is valid provided every AST anchor, child `ParentId`, `ParentCtxSymbolTableId`, `Context.SymbolTable_`, and map entry is updated consistently.

After the merge:

```
non-companion unit root Context
  -> no independent final Context
  -> declarations belong to package Context

companion-unit root Context
  -> no independent final Context
  -> declarations belong to corresponding struct Context

nested real scopes parsed inside either unit
  -> remain as real Contexts
  -> are attached to the resulting semantic owner/context graph
```

Speculative parsing

A frontend that performs speculative parsing may temporarily read the same span more than once. If a speculative branch creates a Context, SymbolTable segment, child-context edge, or per-node symbol-table anchor and that branch is later abandoned, those structural effects must be rolled back with the parser cursor. The accepted AST and Context/SymbolTable graph must describe only the parse branch that actually survives.

What is allowed in each Folang construct

1. Class
 - member fields, methods and named non-UDT type declarations
2. structs
 - member fields, embedded structs
3. cstructs
 - member fields
4. unions
 - member fields
5. enums
 - members
6. indexers
 - methods shape
7. modules
 - member fields, functions, named non-UDT type declarations and associated-type bindings
8. signatures
 - member fields, function signatures without bodies, named non-UDT type declarations, and abstract type requirements
9. interfaces
 - method signatures without bodies and named non-UDT type declarations
10. traits
 - methods, method signatures no body
11. mixins
 - methods, method signatures without bodies, member fields, and named non-UDT type declarations
12. typeclasses
 - method signatures no body
13. instances
 - methods, and fields
14. objects/Annotations
 - methods, fields
15. Decorators
 - methods shape
16. Matcher
 - methods, fields
17. execution model
 - method shape
18. macro
 - method shape
19. native
 - method shape

- 20. template
 - method shape
- 21. extensions
 - methods and fields/members
- 22 extension methods
 - methods /functions like things
- 23. operators
 - a. definition like variable
 - b. methods
- 24. Generics
 - structs/classes/functions/methods with ``@co.dap.generic``; polymorphic ``co.type`` values with ``forall``
- 25 Anonymous
 - classes, blocks and methods
- 27. effects
 - is annotation and handlers
- 28. units and/or companion units
 - Direct unit members are named type declarations and named function-shaped declarations permitted by the unit grammar. The executable forms below occur inside those functions; they are not loose direct unit members.
 - a. named blocks
 - similar to functions except parameters and results
 - b. different types of functions
 - Generic
 - curried
 - named
 - optional
 - variadic args
 - default
 - b. functions
 - a variable declaration
 - b. inner functions
 - c. anonymous functions
 - d. anonymous blocks
 - e. label blocks
 - f. let bindings
 - g. let function-pattern groups
 - h. pattern matching
 - i. comprehensions
 - j. loops
 - k. conditions
 - l. ternary operators
 - m. named non-UDT type definitions, with lexical function/block scope

- n. capturing anonymous/local functions and ordinary curried functions
- o. contains
- q. each
- r. lambda expression in each and comprehensions, not outside
- s. calls to functions
- t. expressions and other statements
- u. delegates
- c. association with struct type if companion unit

29. components

based on folder name

component definition

named non-UDT type definitions except in the operators component

project-root/components/<kind>

kind is

1. operators
2. packaged
3. native
4. application

30. libraries

project-root/src/component.fol

component definition

1. @co.dap.library
 - a. application
 - b. native
 - c. dynamicvmrt
2. package export

@co.dap.export

31. application

project-root/src/appl.fol contains

- variable definitions (limited to simple and arrays)
- imports
- pragmas/directives/decorators/annotations
- call to functions
- named non-UDT type definitions
- loops
- conditions
- contains
- each
- ternary operators
- expressions and statements

32. packages

packages are nothing but folder names under src

with fol files

Parsing rules

1. Every folder under `src` is package and/or subpackage
2. Every ordinary package file is a type, unit or companion unit;
``src/appl.fol``
and ``src/component.fol`` are the two fixed root-surface exceptions.
3. Parser should do following as pre parsing
 - a. pull the symbol tables and AST from ``<install-root>/stdlib/co.folenc``
 - b. fetch symbol tables and ASTs from libraries under ``<project-root>/lib``
 - c. process ``components/operators`` as the independent operator bootstrap
 - d. parse components in dependency order: ``native``, ``application``, ``dynamicvmrt``, then ``packaged``
 - e. traverse through each folder under `project-root/src` collect all filenames.
 - f. check filenames with `comp.unit`
 - g. check if `<type>.fol` exists for every `<type>.comp.unit.fol`
 - h. start parsing `<type>.fol`
 - i. collect pragmas/directives/annotations/decorators if any
 - ii. if imports are there parse imports first
 - iii. use them to identify/resolve ambiguities like function shapes or types
 - iv. start parsing
 - v. any User defined type or type alias resolve then and there as these should be already imported and available or they are built in.
 - vi. any variable usage or function call already available in symbol table or it is built in.
 - vii. dynamic dispatch and overload resolution can be kept as unresolved other simple things are parse errors/compiler errors
 - i. parse ``*.unit.fol``
 - j. start parsing ``component.fol`/`appl.fol`` under ``<project-root>/src``
 - k. after the permitted resolution fixed point, report unused imports and
unreachable packages according to "Unused Symbols, Liveness, and Reachability"

The pre-parsing header/index work is not a second full syntax parse. It establishes file identity, metadata classification, imports, package contexts, and declaration headers. The following full parse therefore resolves ordinary lexical, package, imported, and built-in names without backtracking. Only static overload selection, receiver/type-directed dispatch, explicitly dynamic dispatch, and resolution policies that are explicitly late-bound may remain for the semantic completion pass.

Context creation

FoLang creates semantic contexts for the following scoped constructs:

- ordinary units (`<Fragment>.unit.fol`) and companion units (`<Type>.comp.unit.fol`);
- component surfaces (`component.fol`) and the application entry surface (`appl.fol`);
- classes, structs, cstructs, enums, unions, modules, signatures, interfaces, traits, mixins, typeclasses, instances, objects/annotations, matchers, and extensions;
- functions and specialized function-shaped declarations, including methods, extension methods, indexers, macros, templates, decorators, native functions, execution-model functions, and operators;
- blocks, lambdas, anonymous functions and classes, `let` function-pattern groups, and ordinary `let` bindings where their declarations introduce lexical bindings.

A generic declaration does not create a separate generic context category. It remains a class, struct, or function context whose built-in generic metadata introduces type parameters and aliases into that declaration's scope.

The serialized context representation is:

```
Context {
  ParentId:          string,
  ParentCtxSymbolTableId: string,
  Id:                string,
  RestrictedSymbolNameReuse: [string],
  ImportedContextIds:  { <alias>: <context-id> },
  Prefix:            string,
  ContextType_:       string,
  SymbolTable_:       string,
  ChildCtxIds:        [string],
  ResolutionPolicy:   string,
  OwnerSymbolId:      string
}
```

`ParentId` identifies the parent Context. `ParentCtxSymbolTableId` identifies the parent visibility segment active where this context branches. `OwnerSymbolId` identifies the declaration that owns the context and is empty only for structural roots.

The semantic `ContextType_` and the parser's immediate syntactic position serve different purposes. `ContextType_` classifies the symbol-resolution region. The parser must additionally retain whether it is reading, for example, a direct unit member, companion-unit member, class member, or executable block. Immediate grammar permissions use that syntactic context; walking parent contexts is reserved for inherited semantic information such as enclosing generic parameters.

Symbol-table representation

```
SymbolTable {  
    Id:          string,  
    ParentId:    string,  
    ContextId:   string,  
    Prefix:      string,  
  
    SymbolIds:    [ <symbol-id> ],  
    SymbolsByName: { <declaration-key>: [ <symbol-id> ] }  
}
```

`SymbolsByName` contains the named declarations introduced in that visibility segment. It does not contain ordinary identifier uses or accesses. Named bindings include types, classes and other type declarations; functions, methods, function patterns and named closures; variables, fields and parameters; generic type parameters and aliases; enum/variant states and state functions; labels; and other explicitly named bindings defined by their constructs.

An anonymous expression, literal, operator occurrence, wildcard, function result position, or ordinary identifier reference does not introduce a `SymbolsByName` entry.

The canonical symbol registry is broader than a symbol table's name index. AST nodes, including identifier references and synthetic expression nodes, may carry symbol records so later stages have stable identities and source-position lookup anchors. Such records are not declarations and are not bound into `SymbolsByName`.

Expressions and bindings

An expression is not a symbol-table declaration merely because it occurs in source. When an expression is assigned or bound to a name, the declared name is the binding:

```
offset := 100;
```

Here `offset` is entered in `SymbolsByName`; the literal `100` is not.

An anonymous function assigned to a variable likewise introduces the variable binding, not a separately visible function declaration:

```
// bindings.unit.fol
_ co.unit = {
  example()->>() = {
    add := (a co.int, b co.int)->(co.int) {
      this => a + b;
    };
  }
}
```

Here `add` is a variable whose value is a function object. The anonymous function has no independent source-level name.

A named `let` function-pattern family introduces its function name, and each clause may introduce its own local pattern bindings:

```
offset := 100;
let adjust(0) = offset;
let adjust(n) = n + offset;
```

`adjust` is the named local function-pattern family. `n` is local to its clause, while `offset` resolves to the already initialized enclosing runtime binding and is therefore captured. Merely referencing `offset` does not introduce another local binding.

The same construct may have no captures:

```
let f(Some(x)) = x + 1;
let f(None) = 0;
```

`f` is the declared `let` function-pattern family and `x` is a clause-local pattern binding. `Some` is a use of an already declared state function and `None` is a use of an already declared state value; they are not new declarations at this location. The function-pattern family's own name is available to its clauses for recursive calls and is not treated as a lexical capture.

Context-first parsing decisions

The parser selects productions in this order:

1. immediate syntactic context and source-file classification;
2. built-in annotations, directives, and other language-owned markers already attached to the declaration;
3. declarations already classified in the visible symbol tables;
4. the current token or explicit grammar delimiter;
5. bounded lookahead only when the preceding information still leaves two or more legal productions.

Parent contexts supply inherited semantic information but do not grant their direct member grammar to nested contexts. For example, a function inside a companion unit may use the companion's visible types, but its inner functions cannot acquire receiver-function declaration permission from the companion ancestor. At direct companion-unit member level, `(` therefore commits immediately to a receiver-function declaration; calls, casts, grouped expressions, and anonymous functions are not legal competing unit members.